# APEX

полная документация проекта

**mc-apex.ru**

механики · архитектура · код

31.10.2026

# Оглавление


## Часть I. Проект целиком

1. Что такое APEX
2. Технологический стек
3. Карта репозитория

## Часть II. Бэкенд

4. Архитектура бэкенда
5. Роли и доступ
6. Аутентификация

## Часть III. Фронтенд

7. Устройство фронтенда
8. Маршрутизация
9. Стили и дизайн-система
10. Слой запросов к API
11. Состояние: Pinia
12. Вебсокеты на клиенте
13. Ошибки и тесты на фронтенде
14. Сборка

## Часть IV. Справочник механик

Домены Players, Tiers, Seasons, Achievements, Games

Кланы, форум и друзья: социальное ядро APEX

1. Кланы
2. Клановые войны (КВ)
3. Форум
4. Друзья
5. Эндпоинты
6. Каскады и целостность
7. Все политики
8. Чего в коде нет

Экономика ApexCoin и управление функциями: домены Shop, Wallet, Control

Глава. Турниры, модерация, поддержка, новости и ядро сайта

Реальное время: чат, каналы и мост к Minecraft-серверу

Мои питомцы: полная механика

**Часть V. Разработка**

21. Локальный запуск

22. Тесты

23. Как добавить новую механику

24. Грабли, проверенные на практике

**Приложения**

Приложение А. Схема базы данных

Приложение Б. Остальные домены

Приложение В. Шпаргалка

Приложение Г. Все числа проекта

**Веб-платформа: сайт mc-apex.ru**

# Часть I. Проект целиком

## 1. Что такое APEX

APEX — это игровое сообщество вокруг Minecraft с полноценным веб-сайтом: `mc-apex.ru`. Сайт не является «визиткой» — это работающая платформа, в которой живёт вся жизнь сообщества: рейтинги игроков, тир-тесты, кланы, турниры, магазин с внутренней валютой, форум, чат реального времени и мост с игровыми серверами.

Ключевая идея — **одно место, где живёт вся жизнь сообщества**. Игрок заходит на сайт, видит свой тир и рейтинг, записывается на тир-тест, находит клан, следит за турнирами. Всё остальное — чат, форум, магазин, достижения, уведомления — обвязка вокруг этой оси.

### 1.1. Что умеет проект

| Блок | Что делает |
|---|---|
| **Игроки и рейтинг** | профили, два «аспекта» (PvP и BedWars), расчёт рейтинга, место в топе, сравнение игроков, респекты |
| **Тиры** | тир-тесты от тестеров, тиры от E до S+, очередь с приоритетом, награды монетами за тир |
| **Бридж** | отдельная дисциплина с загрузкой видео и проверкой кураторами |
| **Кланы** | создание, роли внутри клана, сила клана, войны кланов, события |
| **Турниры** | сетки на выбывание, регистрация, матчи, автоматическое продвижение по сетке, BYE |
| **Экономика** | валюта ApexCoin, кошелёк с леджером, магазин, кейсы, питомцы, аукцион, подарки, промокоды |
| **Общение** | личный и групповой чат через вебсокеты, форум, друзья, уведомления, лента активности |
| **Достижения и сезоны** | ачивки с наградами, сезонные сбросы, награды за сезон |
| **Служебное** | модерация с журналом, поддержка, новости, объявления, панель управления функциями сайта |
| **Игра** | мини-игры на сайте (например «Шахтёр»), питомцы |

### 1.2. Из чего состоит кодовая база

Проект лежит в одном репозитории и состоит из двух приложений:

1. **Бэкенд** — `backend/` (Laravel 13, PHP 8.4): API, бизнес-логика, база данных.
2. **Фронтенд** — `frontend/` (Vue 3.5): то, что открывается в браузере на `mc-apex.ru`.

У каждого своя сборка, свои тесты и свой цикл правок — но домены на бэкенде и папки на фронтенде называются одинаково, поэтому переход от механики к её экрану занимает секунды, а не поиск по репозиторию.

### 1.3. Кому адресована эта книга

Это техническая документация, а не руководство игрока. Она рассчитана на разработчика, который:

- только что открыл репозиторий и хочет понять, где что лежит;
- собирается добавить новую механику и должен знать конвенции проекта;
- будет эксплуатировать сервер и разбирать инциденты;
- пишет плагин, который должен общаться с сайтом.

Книга описывает **реальное состояние кода**, а не задумку. Там, где механика в коде отсутствует, это сказано прямо.

# 2. Технологический стек

## 2.1. Бэкенд

| Технология | Версия | Зачем |
|---|---|---|
| PHP | ^8.3 (на проде 8.4.26) | язык |
| Laravel | ^13.17 | фреймворк |
| MySQL | 8.x | основная БД |
| Redis | 7.x | кеш, очереди, сессии |
| Laravel Sanctum | ^4.0 | аутентификация SPA по cookie |
| Laravel Reverb | * (встроен) | сервер вебсокетов |
| Laravel Tinker | ^3.0 | отладка через консоль |
| Sentry (`sentry/sentry-laravel`) | ^4.29 | сбор ошибок |

Особенность стека: **аутентификация сессионная, а не токенная**. Sanctum в режиме SPA выдаёт CSRF-cookie и держит обычную сессию Laravel. Поэтому фронтенд ходит на `/sanctum/csrf-cookie`, а не хранит токены в localStorage.

## 2.2. Фронтенд

| Технология | Версия | Зачем |
|---|---|---|
| Vue | ^3.5.42 | фреймворк (Composition API + `<script setup>`) |
| Pinia | ^4.0.3 | состояние |
| vue-router | ^5.3.1 | маршрутизация, история браузера |
| Vite | 8.x | сборка и dev-сервер |
| Chart.js + vue-chartjs | ^4.5 / ^5.3 | графики рейтинга и статистики |
| skinview3d | ^3.4.2 | 3D-просмотр скина игрока |
| @sentry/vue | ^11.9.0 | ошибки фронтенда |
| Vitest + jsdom | — | тесты |
| Golos Text, Unbounded | 5.3 | шрифты (локально, `@fontsource`) |

Сборка — Vite 8 с роллапом внутри. Код-сплиттинг агрессивный: стартовый бандл содержит только главную страницу, всё остальное подгружается чанками по маршрутам.

## 2.3. Как это работает вместе

```
Браузер ──► Vue (frontend/dist, отдаётся статикой)
              │
              ├── /api/...        JSON, Sanctum по cookie
              ├── /sanctum/...    выдача CSRF-cookie
              ├── /broadcasting/  авторизация каналов
              └── /app/           WebSocket к Reverb
                    │
                Laravel ──► MySQL (данные)
                        └──► Redis (кеш, очереди)
```

Разделение простое: всё, что меняет данные, идёт через `/api/`; всё, что меняется само, приходит по WebSocket из `/app/`.

# 3. Карта репозитория

```
apex/
├── backend/                    Laravel-приложение
│   ├── app/
│   │   ├── Domains/            ★ вся бизнес-логика, по доменам
│   │   ├── Http/               middleware, общие ответы
│   │   ├── Support/            трейты, общие классы
│   │   └── Providers/
│   ├── config/
│   │   ├── apex.php            экономика, награды, приоритеты
│   │   ├── bridge.php          лимиты загрузки видео бриджа
│   │   ├── broadcasting.php    Reverb
│   │   └── ...                 стандартные конфиги Laravel
│   ├── database/
│   │   ├── migrations/         130 миграций
│   │   └── seeders/            7 сидеров (ачивки, товары магазина, ...)
│   ├── routes/
│   │   ├── api.php             1017 строк, ~390 маршрутов
│   │   ├── channels.php        каналы вебсокетов
│   │   ├── console.php         команды планировщика
│   │   └── web.php             только служебные
│   └── tests/                  127 тестовых файлов
│
├── frontend/                   Vue-приложение
│   ├── src/
│   │   ├── assets/             9 глобальных CSS + логотип
│   │   ├── components/         171 компонент, по папкам-разделам
│   │   ├── composables/        переиспользуемая логика Vue
│   │   ├── layouts/            AppLayout и др.
│   │   ├── router/index.js     все маршруты
│   │   ├── services/           29 модулей HTTP-запросов
│   │   ├── stores/core/        Pinia: auth, counter, guide
│   │   ├── utils/              seo, format, sound и др.
│   │   ├── views/              45 экранов
│   │   ├── App.vue
│   │   ├── main.js             точка входа
│   │   ├── echo.js             подключение к Reverb
│   │   └── sentry.js
│   └── package.json
│
├── docs/book/                  ★ эта книга
└── .gitignore
```

Три правила, которые объясняют почти всё в структуре:

1. **Бэкенд разложен по доменам, а не по типам файлов.** Нет папок `app/Models` и `app/Services` — всё лежит в `Domains/<Домен>/`.
2. **Фронтенд повторяет домены бэкенда.** Папки `views/clan`, `services/clan`, `components/clan` соответствуют `Domains/Clan`.
3. **У каждого компонента стили лежат рядом** — см. главу 6.

# Часть II. Бэкенд

## 4. Архитектура бэкенда

### 4.1. Домены вместо «слоёв»

В обычном Laravel-проекте код раскладывают по типам: `Models/`, `Services/`, `Http/Controllers/`. Здесь сделано иначе — **по предметным областям**. Один домен = одна папка = всё, что относится к его механике.

```
backend/app/Domains/
├── Achievements/     награды и ачивки
├── Activity/         лента событий
├── Auth/             вход, регистрация, сброс пароля
├── Bridge/           бридж-дисциплина и видео
├── Chat/             чат реального времени
├── Clan/             кланы и войны (60 файлов — самый крупный)
├── Control/          панель управления функциями сайта
├── Core/             главная страница, настройки
├── Forum/            форум
├── Friends/          друзья
├── Games/            мини-игры
├── Minecraft/        привязка ника к аккаунту
├── Moderation/       жалобы и журнал модерации
├── News/             новости
├── Notifications/    уведомления
├── Og/               OG-картинки для превью ссылок
├── Players/          профили, рейтинг, респекты
├── Project/          служебные страницы проекта (roadmap, changelog)
├── Search/           поиск
├── Seasons/          сезоны
├── Shop/             магазин, кейсы, инвентарь
├── Support/          техподдержка
├── Tiers/            тир-тесты
├── Tournaments/      турниры и сетки
├── Users/            пользователь, баны
└── Wallet/           кошелёк, транзакции, награды
```

Всего **26 доменов, 64 модели, 130 миграций**.

### 4.2. Виды файлов внутри домена

Не каждый домен содержит все виды — только те, что нужны.

| Папка | Что внутри | Пример |
|---|---|---|
| `Controllers/Api/` | HTTP-контроллеры | `Clan/Controllers/Api/ClanController.php` |
| `Controllers/Api/Admin/` | контроллеры админки | `Tournaments/Controllers/Api/Admin/TournamentController.php` |
| `Models/` | Eloquent-модели | `Clan/Models/Clan.php` |
| `Services/` | бизнес-логика | `Players/Services/RatingService.php` |
| `Requests/` | валидация входящих данных | `Auth/Requests/Auth/LoginRequest.php` |
| `Resources/` | преобразование в JSON | `Players/Resources/PlayerResource.php` |
| `Policies/` | правила доступа к сущности | `Clan/Policies/ClanPolicy.php` |
| `Notifications/` | классы уведомлений | `Clan/Notifications/` |

| | | |
|---|---|---|
| | | `ClanInviteNotification.php` |
| `Events/` | события для вебсокетов | `Chat/Events/MessageSent.php` |
| `Support/` | внутренние помощники домена | `Clan/Support/ClanRankCalculator.php` |
| `Observers/` | реакции на изменения модели | `Activity/Observers/UserObserver.php` |
| `Exceptions/` | исключения домена | `Shop/Exceptions/InsufficientFundsException.php` |
| `Mail/` | письма | `Auth/Mail/EmailVerificationCode.php` |

**Правило проекта:** контроллер не содержит бизнес-логики. Он принимает `Request`, дёргает `Service` (или модель) и отдаёт `Resource`. Вся логика — в `Services/`.

### 4.3. Контроллер: типовой вид

```
// backend/app/Domains/.../Controllers/Api/...Controller.php
public function store(StoreThingRequest $request): ThingResource
{
    $thing = $this->service->create($request->user(), $request->validated());

    return new ThingResource($thing);
}
```

Три обязательных признака стиля:

1. **Валидация — в `Request`-классе**, а не в контроллере.
2. **Ответ — через `Resource`**, а не `->toArray()` модели. Так наружу не утекают лишние поля (`password`, `remember_token`).
3. **Ошибки — через трейт `AbortsWithMessage`** (см. ниже), а не `abort(400)`.

### 4.4. Единый формат ошибок

В проекте есть трейт `App\Support\Concerns\AbortsWithMessage`, который бросает исключение с текстом для конкретного поля:

```
// backend/app/Domains/Auth/Services/AuthService.php
if (! Auth::attempt([$field => $login, 'password' => $password], $remember)) {
    $this->invalid('login', 'Неверный логин или пароль.');
}
```

Наружу это приходит как ошибка валидации (HTTP 422) с ключом поля — фронтенд умеет показать сообщение рядом с нужным полем формы. Именно поэтому в формах на сайте ошибка появляется под полем, а не общим тостом.

### 4.5. Middleware

| Middleware | Что делает |
|---|---|
| `auth:sanctum` | требует авторизации (сессия Sanctum) |
| `EnsureUserHasRole` | пускает только перечисленные роли (`role:admin`, `role:tester,admin`) |
| `EnsureUserIsNotBanned` | блокирует забаненных на любом запросе |
| `EnsureFeatureEnabled` | закрывает маршруты выключенной функции сайта |

| | |
|---|---|
| | (`feature:registration`) |
| `EnsureClanMember` | пускает только участников клана |
| `EnsureMinecraftServer` | аутентификация игровых серверов по ключу |
| `ThrottlePerRoute` | ограничение частоты запросов на конкретный маршрут |
| `TouchLastSeen` | обновляет `last_seen_at` у пользователя |

### 4.6. Группировка маршрутов

`backend/routes/api.php` — один файл на 1017 строк и около 390 маршрутов. Он построен «матрёшкой»: сначала публичные маршруты, потом группа `auth:sanctum`, внутри неё — группы по префиксам, поверх — группы по ролям.

```
Route::middleware('auth:sanctum')->group(function () {

    Route::prefix('clans')->group(function () { /* ... */ });

    // Доступ по ролям
    Route::middleware('role:moderator,admin')->prefix('admin')->group(function () { /* ... */ });
    Route::middleware('role:admin')->prefix('admin')->group(function () { /* ... */ });
});
```

Методов по файлу: `GET` 137, `POST` 188, `PUT` 25, `DELETE` 37 — всего ~387.

**Важная деталь порядка:** точные пути объявляются раньше параметрических. Laravel берёт первый совпавший маршрут, поэтому `/shop/priority-candidates` объявлен до `/shop/{shopItem}`, иначе он ушёл бы в параметрический.

# 5. Роли и доступ

## 5.1. Роли пользователя

Поле `users.role` — enum:

| Роль | Кто это | Доступ |
|---|---|---|
| `user` | обычный игрок | всё, что доступно авторизованному |
| `tester` | тестер | пульт тестера `/tester`, выставление тиров |
| `bridge_tester` | проверяющий бриджа | очередь проверки видео бриджа |
| `bridge_curator` | куратор бриджа | управление видами бриджа + проверка |
| `moderator` | модератор | админка: жалобы, журнал, баны |
| `admin` | администратор | вся админка и панель управления |

Роли — плоская последовательность, а не иерархия: `admin` не «наследует» права модератора автоматически, поэтому в маршрутах он всегда перечисляется явно — `role:moderator,admin`.

## 5.2. Три уровня проверки

Доступ в проекте проверяется в трёх местах, и это не дублирование, а разные вопросы:

1. **Middleware `role:`** — «пускать ли роль на этот раздел вообще».
2. **Policy** — «может ли этот конкретный пользователь трогать этот конкретный объект» (например, редактировать этот клан).
3. **Фронтенд `meta.role`** — «показывать ли пункт меню и страницу». Это только удобство: настоящая проверка всегда на бэкенде.

```
// frontend/src/router/index.js
{ path: '/admin', component: () => import('@/views/admin/AdminView.vue'),
  meta: { auth: true, role: ['moderator', 'admin'] } },
```

## 5.3. Баны

Бан живёт в двух полях: `is_banned` (флаг) и `banned_until` (срок, может быть `null` — навсегда), плюс `ban_reason` и `banned_by`. Проверка идёт дважды: в `EnsureUserIsNotBanned` на каждом запросе и в `AuthService::login` — при входе.

Второй проверки можно было бы избежать, но она нужна: без неё забаненный получал рабочую сессию, и блокировка срабатывала только на следующем запросе.

```
// backend/app/Domains/Auth/Services/AuthService.php
if ($user->isBanned()) {
    Auth::guard('web')->logout();
    $this->invalid('login', 'Ваш аккаунт забанен.');
}
```

# 6. Аутентификация

## 6.1. Схема

Sanctum в режиме SPA: не токены в заголовке, а **cookie сессии + CSRF-токен**. Поэтому:

- фронтенд перед первым POST запрашивает `/sanctum/csrf-cookie`;
- запросы идут с `withCredentials: true`;
- на сервере нет «выдачи токена», есть обычная сессия Laravel.

## 6.2. Регистрация в три шага

Самое нестандартное место во всём проекте. Регистрация разбита на шаги, и аккаунт нельзя создать без подтверждённой почты:

```
Шаг 1. POST /api/auth/register/send-code
        { email }
        → на почту уходит код, создаётся запись в email_verifications
        → ограничение: 3 запроса в минуту
        → выключенная функция registration останавливает уже здесь

Шаг 2. POST /api/auth/register/verify-code
        { email, code }
        → код сверяется, взамен выдаётся одноразовый verificationToken
        → ограничение: 10 запросов в минуту

Шаг 3. POST /api/auth/register
        { username, password, verificationToken, referralCode?, profileMode? }
        → токен «сжигается» и отдаёт email
        → создаётся пользователь
```

Смысл разделения: пока почта не подтверждена, в базе не появляется пользователь. Иначе таблица заполнялась бы мусорными аккаунтами от любого, кто отправил форму.

```
// backend/app/Domains/Auth/Services/AuthService.php
public function register(
    string $username, string $password, string $verificationToken,
    ?string $referralCode = null, string $profileMode = 'pvp',
): User {
    $email = $this->verification->consumeToken($verificationToken);

    if (User::where('email', $email)->exists()) {
        $this->invalid('email', 'Этот email уже занят.');
    }
    ...
}
```

## 6.3. Что происходит при регистрации

1. Генерируется `referral_code` — 8 символов в верхнем регистре, с проверкой уникальности в цикле.
2. Если пришёл чужой реферальный код — находится пригласивший, записывается `referred_by`.
3. Пригласившему начисляется награда, новичку — приветственный бонус (`RewardService`).
4. Почта сразу помечается подтверждённой (`markEmailAsVerified()`), потому что она уже подтверждена кодом.
5. Выбирается `profile_mode`: `pvp` или `bridge` — от этого зависит, какой профиль показывать.

### 6.4. Вход

Логином служит **и почта, и ник** — определяется по формату строки:

```
$field = filter_var($login, FILTER_VALIDATE_EMAIL) ? 'email' : 'username';
```

После успешной проверки пароля выполняются две проверки — бан и подтверждённая почта. Обе при неудаче разлогинивают сессию сразу, а не «потом».

### 6.5. Сброс пароля

Отдельный контроллер `PasswordResetController`, кодовый (не ссылочный): запрос кода на почту → проверка кода → смена пароля. Ограничения: 3 запроса в минуту на отправку, 5 в минуту на сброс.

### 6.6. Что отдаёт `/api/auth/me`

Эндпоинт профиля — самый «тяжёлый» ответ в API: он собирает всё, что нужно для отрисовки профиля одним запросом.

```
$user->load([
    'aspectPvp', 'aspectBedwars',
    'clanMember.clan' => fn ($q) => $q->withCount('members'),
    'achievements', 'tierTests',
    'friendsList', 'friendsOf',
    'bridgeRank',
]);

return [
    'user'   => [...$user->toArray(), 'aspects' => [...], 'bridge' => [...]],
    'rank'   => ['position' => $position, 'total' => $total],
    'rating' => $user->isUnranked() ? null : $rating->breakdown($user),
];
```

Отсюда видно устройство профиля игрока:

- **два аспекта** — `aspectPvp` и `aspectBedwars`: игрок имеет отдельный рейтинг в PvP и в BedWars;
- **место в топе** считается на лету (`PlayerProfileService::rankOf`);
- **рейтинг** отдаётся разложенным по составляющим (`RatingService::breakdown`) — именно это рисует график в профиле;
- **бридж** — сводка и отдельный ранг.

# Часть III. Фронтенд

## 7. Устройство фронтенда

### 7.1. Точка входа

`frontend/src/main.js` собирает приложение из четырёх частей: шрифты, глобальные стили, роутер, Pinia — плюс подключает вебсокеты и Sentry.

```
// frontend/src/main.js
import '@fontsource/golos-text/400.css'   // 5 начертаний
import '@fontsource/unbounded/500.css'    // 4 начертания
import './assets/base.css'
import './assets/main.css'
import './assets/transitions.css'
import './assets/backdrops.css'
import './assets/nameEffects.css'
import './assets/chatBubbles.css'
import './assets/petTricks.css'
import './assets/content-page.css'
import './echo'                            // подключение к Reverb
```

Шрифты лежат локально (`@fontsource`), а не грузятся с Google Fonts — это убирает внешнюю зависимость и лишний DNS-запрос.

### 7.2. Структура папок

```
frontend/src/
├── assets/       9 глобальных CSS + logo.svg
├── components/   171 компонент (по папкам-разделам)
├── composables/  переиспользуемая реактивная логика
├── layouts/      AppLayout.css + каркас страниц
├── router/       index.js — все маршруты
├── services/     29 модулей запросов к API
├── stores/       Pinia: core/auth, core/counter, core/guide
├── utils/        seo, format, sound, ...
├── views/        45 экранов
├── test/         setup + 25 тестовых файлов
├── echo.js       вебсокеты
├── sentry.js     сбор ошибок
├── App.vue
└── main.js
```

Соотношение: **218 `.vue` файлов и 119 `.css`** — почти у каждого компонента есть сестринский файл стилей (см. 9.2).

# 8. Маршрутизация

## 8.1. Ленивая загрузка

Стартовый бандл содержит только главную страницу. Всё остальное — динамические импорты, каждый экран уезжает в свой чанк:

```
import HomeView from '@/views/players/HomeView.vue'      // сразу
const ProfileView = () => import('@/views/players/ProfileView.vue')  // по требованию
```

Причина прямая: если импортировать всё статически, профиль, магазин, кланы и форум попадают в стартовый бандл, и первая загрузка сайта растёт в разы.

## 8.2. Соглашения по `meta`

| Поле | Значение |
|---|---|
| `title` | заголовок вкладки (к нему добавляется `· APEX TIERS`) |
| `auth` | требуется авторизация |
| `guest` | только для неавторизованных (вход, регистрация) |
| `role` | массив ролей, которым доступен экран |

```
{ path: '/admin', component: () => import('@/views/admin/AdminView.vue'),
  meta: { title: 'Админ-панель', auth: true, role: ['moderator', 'admin'] } },
```

`meta.role` — это только удобство интерфейса: настоящая проверка всегда на бэкенде через middleware `role:`.

## 8.3. Прокрутка при переходах

`scrollBehavior` — место, куда обычно не заглядывают, но здесь он написан осознанно и с комментариями:

```
scrollBehavior(to, from, savedPosition) {
  if (savedPosition) return savedPosition              // назад/вперёд — где был
  if (to.hash) {                                       // якорь /rules#rating
    return new Promise((resolve) => {
      setTimeout(() => resolve({ el: to.hash, top: 80, behavior: 'smooth' }), 350)
    })
  }
  if (to.path === from.path) return false              // сменились только фильтры
  return { top: 0 }                                    // любой другой переход
}
```

Четыре разных случая, и каждый закрывает конкретную жалобу. Например, последний: раньше позиция сохранялась, и, открыв клан из середины списка, человек оказывался внизу страницы.

## 8.4. Полная карта экранов

| Путь | Экран | Доступ |
|---|---|---|
| `/` | Главная | всем |
| `/login`, `/register` | Вход, регистрация | гость |
| `/forgot-password`, `/reset-password`, `/verify-email` | Восстановление, подтверждение | — |
| `/profile` | Мой профиль | авторизован |

| | | |
|---|---|---|
| `/players`, `/players/:id` | Список игроков, профиль | авторизован |
| `/user/:id` | Каноничная ссылка на профиль (`/user/Ник`) | авторизован |
| `/compare?a=&b=` | Сравнение двух игроков | авторизован |
| `/achievements` | Достижения | авторизован |
| `/respects` | Топ «самых уважаемых» | всем |
| `/clans`, `/clans/:id`, `/clan/:id` | Кланы | всем |
| `/clans/create` | Создание клана | авторизован |
| `/my-clan` | Мой клан | авторизован |
| `/tournaments`, `/tournaments/:id` | Турниры | всем / авторизован |
| `/shop`, `/inventory`, `/cases`, `/auction`, `/wallet` | Магазин и экономика | часть — авторизован |
| `/pets` | Питомцы | авторизован |
| `/miner` | Мини-игра «Шахтёр» | авторизован |
| `/forum`, `/forum/new`, `/forum/:id` | Форум | всем / авторизован |
| `/messages/:id?` | Чат | авторизован |
| `/friends`, `/notifications` | Друзья, уведомления | авторизован |
| `/news`, `/news/:id` | Новости | всем |
| `/support/:id?` | Техподдержка | авторизован |
| `/rules`, `/about`, `/stats`, `/status` | Информационные | всем |
| `/changelog/:slug?`, `/roadmap` | Что нового, роадмап | всем |
| `/tester` | Пульт тестера | `tester`, `admin` |
| `/bridge-review` | Проверка бриджа | `bridge_tester`, `bridge_curator`, `admin` |
| `/bridge-curator` | Виды бриджа | `bridge_curator`, `admin` |
| `/admin` | Админ-панель | `moderator`, `admin` |
| `/:pathMatch(.*)*` | 404 | всем |

Обрати внимание на порядок: 404-маршрут объявлен **последним**, иначе он перехватил бы всё.

# 9. Стили и дизайн-система

## 9.1. Глобальные стили

В `assets/` лежит девять CSS-файлов. Восемь из них подключаются один раз в `main.js`:

| Файл | За что отвечает |
|---|---|
| `base.css` | сброс, типографика, базовые элементы |
| `main.css` | раскладка приложения, общие классы |
| `transitions.css` | анимации переходов между страницами |
| `backdrops.css` | фоны и подложки |
| `nameEffects.css` | эффекты ников (анимированные имена игроков) |
| `chatBubbles.css` | пузыри сообщений в чате |
| `petTricks.css` | анимации питомцев |
| `content-page.css` | общий макет контентных страниц |

Девятый, `newsProse.css`, оформляет текст новостей и подключается не глобально, а там, где рендерится содержимое новости — чтобы стили прозы не влияли на остальные страницы.

## 9.2. Правило сестринских CSS — главное соглашение фронтенда

**У каждого компонента стили лежат в файле рядом с ним, с тем же именем.** Не в общем файле, не в `<style>` внутри `.vue`.

```
components/clan/
├── ClanCard.vue
├── ClanCard.css
├── ClanMembers.vue
└── ClanMembers.css
```

Подключение — через `@import` внутри `<style scoped>`:

```
<style scoped>
@import "./ClanCard.css";
</style>
```

**Тут есть ловушка, о которой надо знать.** `@import` внутри `<style scoped>` создаёт **глобальные** стили — `scoped` на импортированный файл не действует, атрибут `data-v-*` к его селекторам не добавляется. Практическое следствие: классы вроде `.modal`, `.list`, `.item` из разных компонентов конфликтуют друг с другом.

Отсюда правило именования: **у модалок и крупных блоков должен быть уникальный префикс** (`.pet-modal`, `.clan-invite-modal`), а не общие имена.

## 9.3. Дизайн-система в цифрах

| Параметр | Значение |
|---|---|
| Шрифт интерфейса | Golos Text (400–800) |
| Шрифт акцентов и заголовков | Unbounded (500–800) |
| Основной акцент | фиолетовый (`#a78bfa` и оттенки) |
| Тиры | свои цвета на каждый (`tierColors`) |

| | |
|---|---|
| Графики | Chart.js через `vue-chartjs` |
| 3D-скины | `skinview3d` |

# 10. Слой запросов к API

## 10.1. Один клиент на всё

Все 29 модулей в `services/` используют общий клиент `services/core/api.js`. Он не использует axios — только `fetch`, и делает три вещи, которых обычно не хватает:

1. **CSRF.** Для всех методов, кроме `GET/HEAD/OPTIONS`, читает cookie `XSRF-TOKEN` и кладёт её в заголовок `X-XSRF-TOKEN`.
2. **Вложенные параметры.** Разворачивает объект в скобочную нотацию — Laravel читает её как массив:

```js
{ cursor: { score: 5 } }  →  cursor[score]=5
```

1. **Богатые ошибки.** Бросает объект со статусом, сообщением, ошибками полей и полным телом ответа:

```js
throw { status, message, errors, data }
```

Полное тело нужно не «на всякий случай»: в нём приходят подсказки — например, сколько монет не хватает до покупки.

```
// frontend/src/services/core/api.js
if (!response.ok) {
  throw {
    status: response.status,
    message: data.message || 'Произошла ошибка.',
    errors: data.errors || {},
    data,
  }
}
```

## 10.2. Модули по доменам

```
services/
├── core/          api.js, admin.js
├── auth/          auth.js
├── players/       players.js, home.js
├── tiers/         tierTests.js, tester.js
├── clan/          clans.js, myClan.js
├── shop/          shop.js, cases.js, gift.js, adminShop.js
├── wallet/        wallet.js
├── chat/          chat.js
├── forum/         forum.js
├── tournaments/   tournaments.js
├── bridge/        bridge.js
├── minecraft/     link.js
└── ...            activity, achievements, friends, news,
                   notifications, project, search, seasons,
                   site, support
```

Каждый модуль — тонкая обёртка: `api.get('/clans')`, `api.post('/clans', data)`. Бизнес-логики здесь нет.

## 11. Состояние: Pinia

Store-ов всего три, и это осознанно мало:

| Store | Что хранит |
|---|---|
| `stores/core/auth.js` | текущий пользователь, статус авторизации |
| `stores/core/counter.js` | счётчики для интерфейса (непрочитанные и т.п.) |
| `stores/core/guide.js` | состояние «путеводителя» для новичков |

Основная причина такой скромности: почти все данные приходят с сервера и нужны одному экрану. Класть их в глобальный store — лишний слой, который придётся синхронизировать.

# 12. Вебсокеты на клиенте

## 12.1. Зачем свой модуль

`frontend/src/echo.js` — не просто «подключить Echo». Это **сторож соединения**, и он написан из-за конкретной проблемы.

`pusher-js` из состояния `failed` не выходит: метод `connect()` отказывается работать, если состояние не `initialized`. После любого обрыва — перезапуск Reverb, потеря сети, сон ноутбука — соединение остаётся мёртвым до перезагрузки страницы, и новые сообщения просто не приходят.

## 12.2. Как устроен сторож

Раз в 15 секунд проверяется состояние соединения. Пересоздание запускается в четырёх случаях:

```
// frontend/src/echo.js
const stuck = state === 'connecting' && Date.now() - lastOkAt > STUCK_MS

if (state === 'failed' || state === 'unavailable'
    || state === 'disconnected' || stuck) {
  resetEcho()
  lastOkAt = Date.now()
}
```

| Состояние | Что значит | Реакция |
|---|---|---|
| `failed` | тупик, библиотека не восстановится | пересоздать |
| `unavailable` | соединение не поднялось | пересоздать |
| `disconnected` | закрыто и само не вернётся | пересоздать |
| `connecting` дольше 45 с | зависло на подключении | пересоздать |

Плюс два события браузера: возврат из сна (`visibilitychange`) и восстановление сети (`online`) — оба проверяют соединение немедленно, не дожидаясь тика сторожа.

## 12.3. Счётчик поколений

Ключевая деталь. После пересоздания соединения все подписки потеряны, и подписчики должны подписаться заново. Чтобы они об этом узнали, есть реактивный счётчик:

```
export const echoGeneration = ref(0)

export function onEchoReset(callback) {
  listeners.add(callback)
  return () => listeners.delete(callback)
}
```

Именно `ref`, а не обычный объект: подписки следят за ним через `watch`, и с обычным объектом после переподключения они молча терялись до перезагрузки страницы.

### 12.4. Адреса

```
const host = import.meta.env.VITE_REVERB_HOST || 'localhost'
const port = Number(import.meta.env.VITE_REVERB_PORT || 8080)
const scheme = import.meta.env.VITE_REVERB_SCHEME || 'http'
```

`wsPort` и `wssPort` задаются **одинаково намеренно**. Если оставить порт по умолчанию, pusher-js построит защищённый адрес с портом 80 — и получится `wss://хост:80`, то есть никуда.

Транспорты при этом **не перечисляются**: ограничение списка ломало проверку поддержки, и соединение не создавалось вовсе — состояние сразу становилось `failed`, без единого запроса к серверу.

# 13. Ошибки и тесты на фронтенде

## 13.1. Sentry

`frontend/src/sentry.js` инициализирует `@sentry/vue` и умеет привязывать пользователя (`setSentryUser`) — чтобы ошибка в интерфейсе была привязана к конкретному аккаунту, а не приходила безымянной.

## 13.2. Тесты

| Параметр | Значение |
|---|---|
| Раннер | Vitest |
| Окружение | jsdom |
| Тестовых файлов | 25 |
| Всего тестов | 437 (436 проходят, 1 пропущен) |
| Запуск | `npm test` → `vitest run` |
| Покрытие | `npm run test:coverage` |

Настройки в `vite.config.js`:

```
test: {
  environment: 'jsdom',
  include: ['src/**/*.test.js'],
  setupFiles: ['./src/test/setup.js'],
  globals: true,
}
```

Тесты двух видов: проверка маршрутов (`router/routes.test.js` — например, что все `component` существуют и `meta` корректны) и «монтирование» компонентов и экранов — Smoke-тесты, которые ловят синтаксические и импортные ошибки во всех 218 `.vue` файлах.

# 14. Сборка

```
npm run dev      → vite (прокси /api, /sanctum, /broadcasting на 127.0.0.1:8000)
npm run build    → vite build → dist/
npm run preview  → локальный просмотр собранного
```

Dev-сервер проксирует три префикса на Laravel, поэтому CORS в разработке не нужен — запросы идут с того же origin.

Размеры бандла на проде:

```
skinview3d            525 kB  (130 kB gzip)   ← 3D-скины
PlayerRatingBreakdown 216 kB  (65 kB gzip)   ← графики
AdminView             212 kB  (56 kB gzip)
dist                  173 kB  (60 kB gzip)
echo                   74 kB  (21 kB gzip)
index                 119 kB  (36 kB gzip)
```

Сборка выдаёт предупреждение про чанки больше 500 КБ. Самый крупный — `skinview3d`, и он нужен только на страницах профиля, поэтому грузится отдельным чанком.

# Часть IV. Справочник механик

Здесь разобраны все предметные области сайта по отдельности. В отличие от предыдущих частей, эти главы **не нумерованы** — это справочник, а не последовательное чтение. Нумерация остальных глав при этом непрерывна: 1–14 идут до справочника, 15–28 после.

Отдельно стоит глава про питомцев: там разобрана боевая система арены, экспедиции, перки и питомцедекс — с точными числами из кода.

## Домены `Players`, `Tiers`, `Seasons`, `Achievements`, `Games`

Пять доменов закрывают всю игровую вертикаль сайта: от анкеты и рейтинга до тир-теста, сезонного среза и мини-игры. Внутри каждого домена действует одно правило: контроллер только принимает запрос, валидация — в `Requests`, бизнес-логика — в `Services`, форма ответа — в `Resources`. Побочные эффекты, которые нельзя держать внутри одного домена (пересчёт рейтинга после правки аспектов, проверка ачивок после сообщения в чате), собраны в одном месте — `AppServiceProvider::boot()`.

| Домен | Что хранит | Ключевые классы |
|---|---|---|
| `Players` | профили, аспекты, рейтинг, топы, респекты, питомцы, гайд | `RatingService`, `PlayerProfileService`, `ProfileBlocksService` |
| `Tiers` | тир-тесты, результаты, апелляции, аспекты | `TierTestService`, `TierTestAppealService`, `GuardsActiveTierTest` |
| `Seasons` | сезоны рейтинга и снимки итогов | `SeasonService` |
| `Achievements` | каталог ачивок, выдача, условия | `AchievementService`, `AchievementRules` |
| `Games` | мини-игра «Шахтёр» | `MinerService`, `MinerProgress`, `MinerWorld` |

### Рейтинг игрока

#### Формула

Единственный источник истины — `RatingService::compute()`. Очки считаются не по «тир-тесту», а по аспектам, турнирам и войнам:

```
// backend/app/Domains/Players/Services/RatingService.php
$row['pvp'] = (int) ($pvp[$id] ?? 0);
$row['bedwars'] = (int) ($bedwars[$id] ?? 0);
$row['bridge'] = (int) round((float) ($bridge[$id] ?? 0));
// Бридж сильнее PvP — на сайте вместо тира показываем звание бриджера
$row['bridge_primary'] = $row['bridge'] > $row['pvp'];
$row['tiers_average'] = round(($row['pvp'] + $row['bedwars'] + $row['bridge']) / 3, 1);
```

```
...
// Вверх: игрок с 1% в одном режиме не должен обнуляться и выпадать из рейтинга
$row['tiers_points'] = (int) ceil(($row['pvp'] + $row['bedwars'] + $row['bridge']) / 3);

$row['total'] = $row['tiers_points']
    + $row['match_points'] + $row['tournament_points']
    + $row['war_points'] + $row['war_win_points'];
```

Слагаемые:

| Часть | Как считается | Откуда данные |
|---|---|---|
| `tiers_points` | `ceil((pvp + bedwars + bridge) / 3)`, делим **всегда на три** — несданный режим даёт 0 | `player_aspects_pvp`, `player_aspects_bedwars`, `user_bridge_techniques` |
| `match_points` | выигранные матчи турниров × `rating_match_win` (по умолчанию 5) | `tournament_matches.winner_id`, `tournament_participants` |
| `tournament_points` | сумма `tournaments.win_points` за выигранные финалы | `tournament_matches` с `next_match_id IS NULL` |
| `war_points` | участия в завершённых войнах × `rating_war_participation` (3) | `clan_war_participants` + `clan_wars.status = completed` |
| `war_win_points` | победы в войнах × `rating_war_win` (7) | `clan_wars.winner_clan_id = clan_war_participants.clan_id` |

Тонкости, которые видно только в коде:

- **Бридж-процент** — среднее трёх оценок по подтверждённым видам: `AVG((stability + speed + difficulty) / 3.0)`, только `status = 'confirmed'`.
- **Турнирные очки получают только одиночные участники**: в запросах стоит `whereNull('p.participant.clan_id')`, клановые турниры в рейтинг игрока не идут.
- **Пропуск (bye) — не победа**: `where('m.is_bye', false)`.
- **Победа в турнире** определяется как победа в матче без `next_match_id` — то есть в финале.

Веса читаются из `site_settings`, а `config('apex.php')` тут ни при чём:

```
// backend/app/Domains/Players/Services/RatingService.php
public const DEFAULTS = [
    'rating_match_win' => 5,
    'rating_war_participation' => 3,
    'rating_war_win' => 7,
];

public function weights(): array
{
    return [
        'match_win' => (int) SiteSetting::get('rating_match_win',
self::DEFAULTS['rating_match_win']),
        'war_participation' => (int) SiteSetting::get('rating_war_participation',
self::DEFAULTS['rating_war_participation']),
        'war_win' => (int) SiteSetting::get('rating_war_win', self::DEFAULTS['rating_war_win']),
    ];
}
```

Смена любого веса в админке (`PUT /api/admin/site-settings`) запускает полный пересчёт: `SiteSettingsController` сравнивает `weights()` до и после сохранения и вызывает `recalculateAll()`.

#### Где хранится

| Поле | Таблица | Тип | Комментарий |
|---|---|---|---|
| `rating_points` | `users` | `unsignedInteger`, индекс, default 0 | итог формулы |
| `bridge_primary` | `users` | `boolean` | `bridge > pvp`; от него зависит, что показывать вместо тира |
| `tier`, `tier_score` | `users` | `string`, `decimal(5,2)` | тир и процент, к рейтингу отношения не имеют |
| `power` | `clans` | signed integer | сила клана |

Сила клана — половина очков участников, с округлением вниз, и только по «ранжируемым» ролям:

```
// backend/app/Domains/Players/Services/RatingService.php
$sums = ClanMember::query()
    ->join('users', 'users.id', '=', 'clan_members.user_id')
    ->whereIn('clan_members.clan_id', $ids)
    ->whereNotIn('users.role', User::UNRANKED_ROLES)
    ->groupBy('clan_members.clan_id')
    ->selectRaw('clan_members.clan_id, SUM(users.rating_points) as total')
    ->pluck('total', 'clan_id');

foreach ($ids as $clanId) {
    $power = (int) floor(((int) ($sums[$clanId] ?? 0)) * self::CLAN_SHARE);
    Clan::whereKey($clanId)->where('power', '!=', $power)->update(['power' => $power]);
}
```

`CLAN_SHARE = 0.5`. Тот же коэффициент продублирован в `ClanMember::contribution()` — для показа вклада участника.

#### Как пересчитывается

Хранимое поле нельзя оставлять «на совести контроллеров», поэтому пересчёт подписан на модели в `AppServiceProvider`:

```
// backend/app/Providers/AppServiceProvider.php
$rating = fn () => app(RatingService::class);
$recalc = fn (...$ids) => $rating()->recalculate($ids);

foreach ([PlayerAspectPvp::class, PlayerAspectBedwars::class, UserBridgeTechnique::class] as
$model) {
    $model::saved(fn ($row) => $recalc($row->user_id));
    $model::deleted(fn ($row) => $recalc($row->user_id));
}
```

Дальше по списку: `TournamentMatch::saved` (победитель и прежний победитель), `Tournament::updated` при смене `win_points`, `ClanWarParticipant::created/deleted`, `ClanWar::updated` при смене `status`/`winner_clan_id`, `ClanMember::created/deleted` и `User::updated` при смене роли — они пересчитывают уже силу клана.

`recalculate()` пересчитывает только переданных игроков и кланы, в которых они состоят, — точечно, без прохода по всей базе. Полный проход — только по команде:

```
php artisan apex:rating-recalc
```

`RatingRecalcCommand` вызывает `recalculateAll()`: чанки по 500 игроков, затем `recalculateClans(Clan::pluck('id'))`. Команда нужна после миграции `2026_10_25_120000_add_rating_points.php` (в её docblock прямо написано «После миграции: `php artisan apex:rating-recalc`») и как страховка, если событие что-то пропустило.

#### Кто не участвует в рейтинге

```
// backend/app/Domains/Users/Concerns/HasRoles.php
public const UNRANKED_ROLES = ['admin', 'moderator'];
```

Тестеры, бридж-тестеры и кураторы бриджа в топах участвуют — так решено владельцем проекта, и это зафиксировано комментарием в трейте. Все выборки рейтинга используют `scopeExcludeStaff()`, который фильтрует именно `UNRANKED_ROLES`, а не `STAFF_ROLES`.

#### Что видит игрок

В профиле (`GET /api/players/{user}`) отдаётся блок `rating` — результат `RatingService::breakdown($user)`. Для админов и модераторов блок равен `null`, и фронтенд не рисует ни место, ни разбор:

```
// backend/app/Domains/Players/Services/PlayerProfileService.php
'rank' => [
    'position' => $position,
    'total' => $total,
],
// Из чего сложились очки рейтинга — для блока в профиле
'rating' => $player->isUnranked() ? null : app(RatingService::class)->breakdown($player),
```

`PlayerRatingBreakdown.vue` показывает четыре строки: «Тиры» (`tiers_points`, подсказка `PvP x% · BedWars y% · бридж z%`), «Матчи турниров» (`match_wins`), «Победы в турнирах» (`tournament_wins`), «Клановые войны» (`wars` + `wars_won`), и итог `total`.

#### Рейтинг-лист

`GET /api/ranking` — публичный, курсорная пагинация, `limit` до 100 (по умолчанию 30):

| `mode` | **По чему сортировка** |
|---|---|
| `overall` (по умолчанию) | `users.rating_points` |
| `pvp` | сумма пяти PvP-аспектов (`ROUND`) |
| `bedwars` | сумма пяти BedWars-аспектов (`ROUND`) |
| `bridge` | своя ветка: сначала число подтверждённых видов, затем сумма аспектов |

Курсор принимается в трёх форматах — строкой `score,id,offset`, массивом `cursor[score]`/`cursor[id]`/`cursor[offset]` и плоскими `cursor_score`/`cursor_id`/`offset`; приведение живёт в `RankingRequest::cursor()`. Позиция считается как `offset + i + 1`, то есть листание не ломает нумерацию. Есть фильтры `search` (по нику), `tier`, `clan_id`.

#### Расхождение «тир» и «рейтинг»

Это главная ловушка домена. `users.tier` — это **лучший** из двух режимов (PvP или BedWars), а `rating_points` — **среднее трёх** (плюс турниры и войны). Игрок с PvP 80% и несданным BedWars имеет тир A, но `tiers_points = ceil(80/3) = 27`. Поэтому тир нельзя использовать как «очки рейтинга» и наоборот; в коде они нигде не смешиваются, кроме отображения.

## Тиры

#### Шкала

Пороги заданы дважды — в `TierTestService` и в моделях аспектов — и совпадают:

```
// backend/app/Domains/Tiers/Services/TierTestService.php
private const TIER_THRESHOLDS = [
    'A' => 71,
    'B' => 56,
    'C' => 41,
    'D' => 21,
];
// ниже D — 'E'
```

| Тир | Как получается |
|---|---|
| `E` | сумма аспектов 0–20 |
| `D` | 21–40 |
| `C` | 41–55 |
| `B` | 56–70 |
| `A` | 71–100 |
| `S` | **не порогом**: админ вручную либо турнир уровня `A` при `tier_score >= 71` |
| `S+` | **не порогом**: админ вручную либо 5 побед в турнирах уровней `A..S` при `tier_score >= 71` |

`S` и `S+` в `TIER_THRESHOLDS` отсутствуют — тир-тест их не выдаёт. Продвижение за турниры:

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php
private static function promoteByLevel(User $user, ?string $level): void
{
    if ($level === 'A' && $user->tier_score >= 71 && ! in_array($user->tier, ['S', 'S+'], true)) {
        $user->update(['tier' => 'S']);
    }

    if ($level === 'S' && $user->tier_score >= 71) {
        $sWins = TournamentWin::where('user_id', $user->id)
            ->where('min_tier', 'A')
            ->where('max_tier', 'S')
            ->count();

        if ($sWins >= 5) {
            $user->update(['tier' => 'S+']);
        }
    }
}
```

Нигде нет автоматического понижения `S`/`S+`: во всех пересчётах стоит проверка `if (! in_array($this->tier, ['S', 'S+'], true))`.

#### Аспекты

Две таблицы, по одной записи на игрока (`unique('user_id')`), значения `unsignedTinyInteger` 0–20:

| **PvP** (`player_aspects_pvp`) | **BedWars** (`player_aspects_bedwars`) |
|---|---|
| `block_placing` — постановка блоков | `pvp` — бой |
| `rotka` — удочка | `game_sense` — игровое мышление |
| `movement` — передвижение | `bed_play` — игра у кроватей |

| | |
|---|---|
| `aim` — прицел | `teamplay` — командная игра |
| `game_sense` — игровое мышление | `building` — строительство |

`percent()` в обеих моделях — это просто сумма (0–100), `averageScore()` — сумма / 5, `tier()` — те же пороги 71/56/41/21. Пересчёт тира игрока:

```
// backend/app/Domains/Users/Models/User.php
public function recalcTierFromAspects(): void
{
    $pvp = $this->aspectPvp;
    $bw = $this->aspectBedwars;

    $best = max($pvp?->percent() ?? 0, $bw?->percent() ?? 0);

    $this->tier_score = $best;

    if (!in_array($this->tier, ['S', 'S+'], true)) {
        $this->tier = match (true) {
            $best >= 71 => 'A',
            $best >= 56 => 'B',
            $best >= 41 => 'C',
            $best >= 21 => 'D',
            default => 'E',
        };
    }

    $this->save();
}
```

`tier_score` участвует в проверке порогов турниров и в «настоящести» тира:

```
// backend/app/Domains/Users/Models/User.php
public function hasTier(): bool
{
    return (float) ($attributes['tier_score'] ?? 0) > 0
        || in_array($attributes['tier'] ?? null, ['S', 'S+'], true);
}

public function publicTier(): ?string
{
    return $this->hasTier() ? ($this->getAttributes()['tier'] ?? null) : null;
}
```

У новичка в базе `tier = 'E'`, но `tier_score = 0`, поэтому наружу отдаётся `null` — «Без тира», а не ложный E.

*Историческая деталь:* модель `Players\Models\PlayerAspect` (таблица `player_aspects`, `percent() = sum × 2`, пороги 90/80/70/60/50) осталась в коде, но таблица удалена миграцией `2026_09_24_191343_restructure_player_aspects.php`. Использовать её нельзя.

#### Заявка на тир-тест

`tier_tests` — центральная таблица домена.

| Поле | Тип | Смысл |
|---|---|---|
| `user_id` | FK → `users`, `cascadeOnDelete` | кто сдаёт |
| `tester_id` | FK → `users`, `nullOnDelete` | кто проводил |
| `claimed_by` | FK → `users`, `nullOnDelete` | кто взял в работу |
| `claimed_at` | timestamp | когда взял |
| `contact_type` | string(16) | `discord` или `telegram` |

| | | |
|---|---|---|
| `contact_value` | string(128) | сам контакт |
| `preferred_time` | string(128) | текстом, «сегодня 20:00–22:00» |
| `mode` | enum | `pvp` или `bedwars` |
| `expected_tier` | string(8) | ожидаемый тир (`S+…E`), заявка игрока |
| `status` | enum | `pending`, `in_progress`, `completed`, `cancelled` |
| `scheduled_at` | datetime | **нигде не заполняется** — поле есть в миграции, модели и `$fillable`, но ни один код в него не пишет |
| `completed_at` | datetime | когда завершён |
| `result_tier` | string | итоговый тир |
| `result_score` | `decimal(5,2)` | сумма аспектов 0–100 |
| `aspects` | json | снимок оценок на момент теста |
| `notes` | text | заметки тестера / причина отмены |
| `is_priority` | boolean | купленный буст |
| `priority_purchased_at` | timestamp | когда применён |
| `priority_price_paid` | unsigned int | сколько стоила покупка |
| `priority_weight` | unsigned int | вес в очереди |

Колонок под отдельные аспекты в `tier_tests` нет — только JSON `aspects`. В `$fillable` модели перечислены `block_placing`, `rotka`, `movement`, `building`, `ppl`, `bed_play`, но миграции, создающей эти колонки, нет (`2026_09_24_190414_add_bed_play_to_aspects_and_tests.php` содержит закомментированный `up()`). Это остаток от старой схемы, писать в эти поля нельзя.

Статусы и переходы:

| Статус | Кто ставит | Условие |
|---|---|---|
| `pending` | игрок при создании | одна активная заявка на игрока |
| `in_progress` | тестер, `claim` | заявка была `pending` и никем не занята |
| `completed` | тестер, `complete` | заявка `in_progress` и взята именно им |
| `cancelled` | игрок (`pending`), тестер (`in_progress`), админ при ручном тесте | — |

Ограничение «одна активная заявка» вынесено в трейт, потому что проверять нужно в двух местах — при создании и при ручном проведении теста админом:

```
// backend/app/Domains/Tiers/Services/GuardsActiveTierTest.php
public const ACTIVE_STATUSES = ['pending', 'in_progress'];

public function assertNoActiveTierTest(User $player): void
{
    $active = $this->activeTierTestFor($player->id);
    if (! $active) return;
```

```
    $status = $active->status === 'in_progress' ? 'уже в работе' : 'ожидает тестера';

    abort(422, sprintf(
        'У игрока уже есть заявка на тир-тест: она %s. '
            . 'Дождись её завершения или отмени её, прежде чем создавать новую.',
        $status,
    ));
}
```

#### Расчёт результата

```
// backend/app/Domains/Tiers/Services/TierTestService.php
$score = $this->scoreFrom($tierTest->mode, $scores);   // сумма пяти полей, максимум 100
$tier = $this->tierFor($score);                        // A/B/C/D/E по порогам

DB::transaction(function () use ($tierTest, $scores, $score, $tier) {
    $tierTest->update([
        'status' => 'completed',
        'completed_at' => now(),
        'result_tier' => $tier,
        'result_score' => $score,
        'aspects' => $scores,
        'notes' => $scores['notes'] ?? null,
    ]);

    $this->storeAspects($tierTest->user_id, $tierTest->mode, $scores);

    $user = $tierTest->user->fresh();
    $user->recalcTierFromAspects();

    // ApexCoin за пройденный тест — идемпотентно по id теста
    RewardService::forTierTest($tierTest->fresh());
    AchievementService::check($user);
});
```

Набор полей зависит от режима и валидируется динамически: `CompleteTierTestRequest` смотрит на `mode` из route-model binding и требует ровно пять полей режима, каждое `integer|min:0|max:20`.

Награда в ApexCoin — из `config/apex.php` (переопределяется в `shop_settings`):

| Тир | Монет |
|---|---|
| E | 60 |
| D | 90 |
| C | 140 |
| B | 220 |
| A | 350 |
| S | 600 |
| S+ | 1000 |

Плюс `first_test_bonus = 150` — раз в жизни, идемпотентность по ключу `tier_test_first:{user_id}`. Сама награда за тест идемпотентна по `tier_test:{id}`.

#### Очередь и работа тестера

`queue()` по умолчанию берёт `pending` + `in_progress` и сортирует так:

```
// backend/app/Domains/Tiers/Services/TierTestService.php
// Внутри одного приоритета сохраняется FIFO
return $query
    ->orderByDesc('is_priority')
```

```
    ->orderByDesc('priority_weight')
    ->orderBy('created_at')
    ->paginate(self::PER_PAGE);   // PER_PAGE = 30
```

Фильтры: `status`, `mode`, `mine=1` (взятые мной), `free=1` (никем не занятые). `TierTestResource` применяется к коллекции пагинатора, а не к пагинатору — иначе Laravel уводит `current_page`/`total` в `meta`, и контракт ломается.

Правила:

- `claim` — только из `pending`; если заявку уже взял другой тестер, 422.
- `unclaim` — только своя заявка в `in_progress`; чистит `claimed_by`, `claimed_at`, `tester_id` и возвращает `pending`.
- `complete` — только своя заявка в `in_progress` (проверка `assertClaimedBy` → `abort_if(..., 403)`).
- `cancel` — только своя заявка, причина в `notes` (до 500 символов).
- Игрок отменяет только `pending`: «Тест уже в работе. Отменить его может только тестер».
- `stats` — `total_completed`, `in_progress`, `pending_total`, `today` по `tester_id`.

Доступ к заявке: её автор или назначенный тестер (`assertCanAccess`, иначе 403). Роут `PUT /api/tier-tests/{tierTest}` доступен им же и позволяет править `status`, `result_tier`, `result_score`, `aspects`, `notes`; при `status = completed` проставляется `completed_at`, обновляется модель аспектов режима, `tier_score` перезаписывается всегда, а `tier` — только если он не `S`/`S+`.

Ручное проведение теста админом (`POST /api/admin/users/{user}/tier-test`) создаёт сразу `completed`-запись, отменяя активную заявку игрока (кроме случая апелляции: `cancelActive: false`):

```
// backend/app/Domains/Tiers/Services/TierTestService.php
if ($cancelActive) {
    $this->activeTierTestFor($player->id)?->update([
        'status' => 'cancelled',
        'notes' => 'Тест проведён вручную администратором.',
    ]);
}
```

#### Апелляции

`tier_test_appeals`: `tier_test_id` (уникальный — одна апелляция на тест, `cascadeOnDelete`), `user_id` (`cascadeOnDelete`), `reason` (20–1000 символов), `status` (`pending`/`approved`/`rejected`), `reviewer_id` (`nullOnDelete`), `resolution`, `revised_test_id` (`nullOnDelete` — новый тест, созданный при пересмотре), `resolved_at`.

Оспорить можно только при выполнении всех условий:

```
// backend/app/Domains/Tiers/Services/TierTestAppealService.php
private function appealError(TierTest $tierTest): ?string
{
    if ($tierTest->status !== 'completed' || ! $tierTest->completed_at) {
        return 'Оспорить можно только завершённый тест.';
    }

    if ($tierTest->completed_at->lt(now()->subDays(self::WINDOW_DAYS))) {
        return 'Срок апелляции — '.self::WINDOW_DAYS.' дней после теста.';
    }

    if (TierTestAppeal::where('tier_test_id', $tierTest->id)->exists()) {
        return 'Апелляция на этот тест уже подана.';
    }
    ...
```

```
    return $newer ? 'Оспорить можно только последний тест в режиме.' : null;
}
```

`WINDOW_DAYS = 14`. Решение принимает модератор или админ (`/api/admin/tier-test-appeals`), и **не тот, кто проводил тест**: `test->tester_id === reviewer->id || test->claimed_by === reviewer->id` → 403. При `approved` создаётся новый тест через `conductManually()` с новыми оценками, `notify: false`, `cancelActive: false`; `revised_test_id` связывает апелляцию с ним. Обе стороны получают уведомление `tier_test_appeal_resolved`.

#### Приоритетная заявка

Буст покупается как расходник в магазине (`ShopItem::TYPE_TIER_PRIORITY`) и применяется к конкретной заявке:

```
// backend/app/Domains/Shop/Services/ShopService.php
$tierTest->update([
    'is_priority' => true,
    'priority_purchased_at' => now(),
    'priority_price_paid' => $item->price,
    'priority_weight' => (int) config('apex.priority.default_weight', 100) + (int) $tierTest->priority_weight,
]);
```

`config/apex.php` → `priority.default_weight = 100`, `expires_after_days = null` (буст бессрочный — поле конфига есть, но код его не читает). Приоритетные заявки идут вне очереди, между собой — по весу, потом по времени создания. Если приоритет применили дважды, веса складываются (200, 300…).

#### Связь с игровым сервером

Тир уезжает в игру через эндпоинт для плагина `POST /api/minecraft/profile` (доступ по заголовку `X-Minecraft-Server-Key`, middleware `minecraft.server`):

```
// backend/app/Domains/Minecraft/Services/MinecraftLinkService.php
public function profilePayload(User $user): array
{
    $user->load(['bridgeRank', 'clanMember.clan:id,name,tag,banner_color']);

    return [
        'username' => $user->username,
        'nickname' => $user->minecraft_username,
        'tier' => $user->tier,
        'tier_score' => (int) $user->tier_score,
        'role' => $user->role,
        'clan_tag' => $user->clan_tag,
        'clan_color' => $user->clan_color,
        'avatar_url' => $user->avatar_url,
        'bridge_rank' => $user->bridgeRank
            ? ['label' => $user->bridgeRank->label, 'color' => $user->bridgeRank->color]
            : null,
    ];
}
```

Обратной связи «сайт → очередь тестов в игре» **в коде нет**. Плагин `minecraft-tier-plugin` держит свою очередь и явно оставил заглушку:

```
// minecraft-tier-plugin/src/main/java/ru/mcapex/tiers/service/QueueService.java
/** Заглушка вместо сайта: записей с сайта пока нет. */
public static final class StubQueueSource implements QueueSource {
    @Override
    public List<QueueEntry> fetch() {
        return List.of();
    }
}
```

То есть игровой тир-тест сейчас наполняется вручную командой `/tier add <ник>`, а заявки с сайта в плагин не попадают. Арены, киты и дисциплины плагина (`/tieradmin arena`, `/tieradmin kit`) живут отдельно от `tier_tests`.

#### Награды и ачивки за тир

Любое изменение `users.tier` проходит через `AchievementService::check($user)`, поэтому тиры `tier_e … tier_s_plus` выдаются автоматически; для `S`/`S+` в правилах заложено «включает все тиры ниже».

#### Шестая стихия — Свет

К огню, воде, природе, тени и зверю добавлен **свет** (юникорн). В данже — «Вспышка» и «Рассвет» (лечение отряда + очистка), на арене — «Рассвет» с лечением и арена «Храм света». Контра дополнен: зверь → свет → тень.

Связок стало **12** (было 10): «Глубина» и «Чаща» теперь гасят свет, новые «Сумерки» (тень+свет → зверь) и «Сияние» (зверь+свет → тень). Каждая стихия контрится ровно двумя связками — баланс сохранён.

#### Кабинет тестера: расписание, заработок и выплаты

Раньше тестер брал заявки из общей очереди без привязки ко времени и без оплаты. Кабинет добавляет две вещи: **расписание** (когда тестер готов) и **деньги** (сколько он за это получает и как выводит).

Домен отдельный — `App\Domains\Testers`. В `Tiers` остаётся результат теста (аспекты, тир, награды игроку), а кабинет отвечает за время и деньги. Смешивать их в одном сервисе значит получить класс, в котором нельзя поменять календарь, не тронув расчёт тира.

**Таблицы:**

| Таблица | Что хранит |
|---|---|
| `tester_slots` | свободное время тестера и занятость заявкой |
| `tester_earnings` | начисление за тест (рубли и ApexCoin), уникально по тесту |
| `tester_payouts` | запросы на выплату: ждёт, выплачено, отклонено |
| `tester_profiles` | блокировку календаря (ушёл в афк — доступ закрыт) |

**Правила календаря** живут в `TesterScheduleService`, а не в контроллере:

- слот нельзя открыть в прошлом, длительность от 30 минут до 4 часов, не дальше 30 дней вперёд;
- слоты одного тестера не пересекаются (проверка по пересечению интервалов);
- записаться можно только в открытый будущий слот и только на заявку в статусе `pending`;

- отмена заявки освобождает слот: если время не прошло — снова открыт, иначе закрыт (прошлое не должно попасть в запись);
- заблокированный тестер не открывает новые слоты и не показывается игрокам в списке свободного времени.

**Заработок.** Начисление создаётся при завершении теста: хук в `TierTestService::complete()` вызывает `TesterEarningsService::accrue()`. Уникальный индекс по `tier_test_id` гарантирует, что за один тест не начислят дважды. Ставка берётся из `site_settings` (группа `tester`):

| Настройка | По умолчанию |
|---|---|
| `pay_rub` | 50 ₽ |
| `pay_coins` | 500 ApexCoin |
| `tier_multipliers` | S+ ×2.0, S ×1.6, A ×1.3, B ×1.1, C ×1.0, D ×0.9, E ×0.8 |

**Долг сайта** считается как «начислено − выплачено − уже запрошено»: `summary()['available_rub']`. Запросить выплату можно только когда есть что выводить, и только если предыдущий запрос ещё не рассмотрен.

Сайт **не переводит деньги сам**: админ платит вручную и отмечает выплату. При отметке «выплачено» ApexCoin уходят на кошелёк тестера через `CoinService::credit()` с идемпотентным ключом `tester-payout-{id}`, а рубли фиксируются в `tester_payouts` и попадают в «выплачено».

**Эндпоинты:**

| Метод | Адрес | Кто |
|---|---|---|
| GET | `/api/tester/dashboard` | тестер |
| GET/POST/DELETE | `/api/tester/slots` | тестер |
| GET/POST | `/api/tester/payouts` | тестер |
| GET | `/api/tester-slots` | игрок |
| POST | `/api/tier-tests/{id}/book` | игрок |
| GET | `/api/admin/testers` | админ |
| GET | `/api/admin/testers/calendar` | админ |
| GET | `/api/admin/testers/tests` | админ |
| GET/PUT | `/api/admin/testers/settings` | админ |
| GET | `/api/admin/testers/payouts` | админ |
| POST | `/api/admin/testers/payouts/{id}/pay` | админ |
| POST | `/api/admin/testers/payouts/{id}/reject` | админ |
| POST | `/api/admin/testers/{user}/block` | админ |
| POST | `/api/admin/testers/{user}/unblock` | админ |

| | | |
|---|---|---|

**Интерфейс.** Кабинет — отдельная страница `/tester-dashboard` с вкладками «Календарь», «Заявки», «Заработок», «Профиль». Админ-раздел «Тестеры и тесты» встроен в общую админку (`?tab=testers`) и повторяет те же данные: список тестеров, выплаты, экономику и глобальный календарь. Игрок видит свободное время прямо в форме заявки на тир-тест (`TierTestForm.vue`) и может забронировать слот; если слот уже заняли — заявка всё равно создаётся, а игрок получает предупреждение.

### Сезоны

Сезон — это не сброс, а срез. Это прямо написано в docblock сервиса: «Закрытие не сбрасывает тиры — оно лишь фиксирует место каждого игрока в рейтинге».

```
// backend/app/Domains/Seasons/Services/SeasonService.php
public function current(): Season
{
    return Season::whereNull('ends_at')->latest('number')->first()
        ?? Season::create(['number' => 1, 'name' => 'Сезон 1', 'starts_at' => now()]);
}

public function close(?string $nextName = null): array
{
    return DB::transaction(function () use ($nextName) {
        $season = $this->current();

        $players = User::query()
            ->excludeStaff()
            ->where('rating_points', '>', 0)
            ->orderByDesc('rating_points')
            ->orderByDesc('id')
            ->get(['id', 'tier', 'tier_score', 'rating_points']);

        $now = now();
        $rows = $players->values()->map(fn (User $user, int $i) => [
            'season_id' => $season->id,
            'user_id' => $user->id,
            'rank' => $i + 1,
            'tier' => $user->tier ?? 'E',
            'tier_score' => (int) $user->tier_score,
            'rating_points' => (int) $user->rating_points,
            'created_at' => $now,
            'updated_at' => $now,
        ]);
        ...
```

Что важно:

- **Первый сезон создаётся сам** при первом обращении к `current()` — отдельной команды инициализации нет.
- **В срез попадают только игроки с `rating_points > 0`** и без роли `admin`/`moderator`. Порядок — по очкам, при равенстве выше тот, у кого больше `id` (как в рейтинге).
- **Снимок пишется пачками по 500** через `SeasonResult::insert`, затем `ends_at` закрывает сезон и создаётся следующий с номером `+1` и именем `Сезон N` (или переданным).

- **Рейтинг не обнуляется, тиры не сбрасываются, наград нет.** Ни `SeasonService`, ни команда, ни контроллер не начисляют ApexCoin, кейсы или ачивки; в коде нет ни одной сущности «награда за сезон». Подсказка блока профиля «Награды прошлых сезонов» — это текст интерфейса, а не реализованная механика.

Закрытие — только вручную:

```
php artisan apex:season-close --name="Зимний сезон" --force
```

Команды нет в планировщике (`routes/console.php` содержит только уборку, аукцион питомцев, sitemap, респекты и недельные призы «Шахтёра»).

Таблицы:

| `seasons` | `season_results` |
|---|---|
| `id`, `number` (unique), `name`, `starts_at`, `ends_at` (null = открыт) | `id`, `season_id` (cascade), `user_id` (cascade), `rank`, `tier` (string(2) — влезает `S+`), `tier_score`, `rating_points`, timestamps |
| | `unique(season_id, user_id)`, `index(season_id, rank)` |

API: `GET /api/seasons` (текущий + все закрытые с топ-3), `GET /api/seasons/{season}` (таблица топ-10), `GET /api/players/{user}/seasons` (места игрока, с числом участников сезона через `withCount('results')`). В интерфейсе это блок «Сезоны» в профиле (`PlayerSeasons.vue`: название, тир, `#место из N`) и секция на `/stats`.

## Достижения

#### Устройство

| `achievements` | `user_achievements` |
|---|---|
| `code` (unique) — по нему ищут правила | `user_id` (cascade) |
| `name`, `description`, `icon` (ключ пиксельной иконки), `color` | `achievement_id` (cascade) |
| `rarity` — `common/rare/epic/legendary` | `earned_at` (default `now()`) |
| `points` — очки коллекции | `unique(user_id, achievement_id)` |
| `coin_reward` — явная награда (nullable) | timestamps |
| `is_system`, `is_active` | |

`coin_reward` перекрывает расчёт: если он `null`, награда = `base + points × per_point` (по умолчанию 25 + 2 за очко). Системные ачивки нельзя удалить или деактивировать — админка вернёт 422.

Связь `User::featuredAchievements()` объявлена через пивот `user_featured_achievements`, но миграции, создающей эту таблицу, в репозитории нет; фактически «избранное» хранится в JSON-поле `users.featured_achievements` и аксессоре `featuredAchievementsList`.

#### Условия выдачи

`AchievementRules` — 13 групп, каждая проверяется отдельным методом (`groupClan`, `groupTiers`, …). Группа передаётся в `check($user, $groups)`, чтобы сообщение в чате не пересчитывало клан и рейтинг.

| Группа | Коды и пороги |
|---|---|
| `clan` | `clan_joined`, `clan_officer` (роль officer), `clan_leader` (leader), войны: 1/5/10 → `clan_war_win`/`clan_war_5`/`clan_war_10`, сила клана ≥ 1000 → `clan_power_1000`, ≥ 10 участников → `clan_squad` |
| `tiers` | `tier_e`…`tier_a` по текущему тиру; `S`/`S+` дают все младшие + `tier_s` (+ `tier_s_plus`) |
| `tests` | завершённых тестов 1/5/10 → `tier_test_first`/`tier_test_5`/`tier_test_10`; рост результата в том же режиме → `tier_comeback` |
| `aspects` | аспект PvP ≥ 18 → `block_master`/`rod_master`/`movement_master`/`aim_master`/`tactician`; максимум 20 → `aspect_perfect`; все пять ≥ 15 → `all_rounder`; есть BedWars → `bedwars_player`; оба режима → `dual_mode` |
| `social` | друзей 5/20/50 → `friends_5/20/50`; рекомендаций 1/5 → `recommended`/`recommended_5`; рефералов 1/5 → `referral_1`/`referral_5` |
| `forum` | тем ≥ 1 → `forum_topic_first`; ответов 10/50 → `forum_replies_10/50`; лайков ≥ 25 → `forum_liked_25` |
| `chat` | сообщений 100/1000 → `chat_100`/`chat_1000` |
| `shop` | предметов 1/10 → `shop_first`/`shop_collector`; потрачено ≥ 5000 → `big_spender`; подарено ≥ 1 → `gift_giver` |
| `tournaments` | участие ≥ 1 → `tournament_join`; побед 1/3 → `tournament_first_win`/`tournament_win_3` |
| `minecraft` | `minecraft_linked_at != null` → `minecraft_linked` |
| `profile` | аватар + био + обложка + 2 соцсети → `profile_complete`; 30/365 дней → `account_30d`/`account_365d` |
| `top` | место в рейтинге ≤ 10/3/1 → `top_10`/`top_3`/`top_1` (персонал и нулевой рейтинг отсекаются) |
| `bridge` | есть `bridge_rank_id` → `bridge_ranked`; подтверждённых видов ≥ 5 → `bridge_techniques_5` |

Каталог наполняется идемпотентным сидером `AchievementSeeder` — 50+ кодов, от `clan_joined` до `bridge_techniques_5`, с очками и редкостью.

#### Что триггерит выдачу

```
// backend/app/Domains/Achievements/Services/AchievementService.php
public static function check(User $user, ?array $groups = null): void
{
    $codes = app(AchievementRules::class)->eligible($user, $groups);
```

```
    if (! $codes) {
        return;
    }

    // Уже полученные пропускаем одним запросом, а не проверкой на каждый код
    $owned = $user->achievements()->pluck('code')->all();

    foreach (array_diff($codes, $owned) as $code) {
        self::grant($user, $code);
    }
}
```

Точки вызова:

| Событие | Где подписано | Группы |
|---|---|---|
| завершение тир-теста (тестером, вручную, по апелляции) | `TierTestService`, `TierTestController` | все |
| правка аспектов игроком / админом | `PlayerProfileService::updateAspects`, `AspectController::update` | все |
| сохранение профиля | `PlayerProfileService::updateMe` | все |
| новая тема / ответ на форуме | `AppServiceProvider` | `forum` |
| новое сообщение в чате | `AppServiceProvider` | `chat` |
| покупка/подарок в инвентаре | `AppServiceProvider` | `shop` |
| участие в турнире, победа в турнире | `AppServiceProvider` | `tournaments` |
| новая рекомендация в профиле | `AppServiceProvider` | `social` |
| новый игрок по реферальной ссылке | `AppServiceProvider` | `social` |
| смена статуса бридж-вида | `AppServiceProvider` | `bridge` |
| смена `minecraft_linked_at`, `bridge_rank_id`, `apex_coins_spent` | `AppServiceProvider` | `minecraft` / `bridge` / `shop` |

`grant()` идемпотентен, начисляет монеты, шлёт `AchievementGrantedNotification` (канал `database`) и пишет событие в ленту «Сейчас на APEX»:

```
// backend/app/Domains/Achievements/Services/AchievementService.php
$user->achievements()->attach($achievement->id, ['earned_at' => now()]);

// Награда в ApexCoin за ачивку (идемпотентно на стороне CoinService)
RewardService::forAchievement($user, $achievement);

$user->notify(new AchievementGrantedNotification($achievement));

self::announce($user, $achievement);
```

Важная деталь: **ручная выдача из админки не платит монеты**. `Admin\AchievementController::grant()` делает `attach()` + `notify()` + `announce()`, но не вызывает `RewardService` — в отличие от `AchievementService::grant()`. Доплату потом сделает только `apex:achievements`.

#### Награда и доплата

`RewardService::coinsForAchievement()` возвращает 0, если источник `achievement` выключен в настройках магазина; иначе `coin_reward` или `base + points × per_point`. Ключ идемпотентности

— `achievement:{user_id}:{achievement_id}`, уникальный индекс на `coin_transactions.idempotency_key`.

Массовый пересчёт и доплата за ачивки, выставленные без награды:

```
php artisan apex:achievements --user=ник --dry-run
php artisan apex:achievements --force
```

`AchievementService::syncAchievements()` идёт чанками по 200, для каждого игрока довыставляет ачивки и проверяет уже начисленные ключи одним запросом. `--dry-run` считает, что было бы выдано, ничего не записывая (`wouldGrant`).

#### Как показывается

`GET /api/achievements` (авторизованный) отдаёт весь каталог с флагом `earned`, датой получения, очками и «редкостью по факту»:

```
// backend/app/Domains/Achievements/Controllers/Api/AchievementController.php
$holders = DB::table('user_achievements')
    ->select('achievement_id', DB::raw('COUNT(*) as holders'))
    ->groupBy('achievement_id')
    ->pluck('holders', 'achievement_id');
...
'holders_percent' => round(((int) ($holders[$a->id] ?? 0)) / $players * 100, 1),
```

Фронтенд (`AchievementsView.vue`) группирует каталог по коду ачивки (`frontend/src/data/achievements/groups.js`: «Тиры и рейтинг», «Тир-тесты и навыки», «Кланы и войны», «Турниры», «Сообщество», «Магазин», «Профиль и бридж», «Особые»), показывает прогресс коллекции, разбивку по редкости и самую редкую полученную ачивку; `holders_percent` превращается в подписи «Пока ни у кого», «Меньше 1% игроков», «N% игроков».

В чужом профиле ачивки приходят внутри `PlayerProfileResource` как `all_achievements` (id, name, icon, color, description, points, `earned_at`), а для страницы игрока — отдельным публичным для авторизованных эндпоинтом `GET /api/players/{user}/achievements`.

## Игры

В домене `Games` одна игра — «Шахтёр» (`MinerService`). Это фарм ApexCoin без дневного лимита, но с перерывом между забегами.

#### Мир

Справочник данных — `Games\Support\MinerWorld`, правила — в сервисах. Сетка `8 × 7` (`COLS = 8`, `ROWS = 7`), энергия 24 удара, перерыв 600 секунд.

| Блок | Монет | Блок | Монет |
|---|---|---|---|
| `dirt` | 1 | `gold` | 54 |
| `stone` | 2 | `diamond` | 160 |
| `coal` | 9 | `emerald` | 250 |
| `copper` | 14 | `deepslate` | 4 |
| `iron` | 22 | `amethyst` | 65 |
| `redstone` | 27 | `quartz` | 32 |

| | | | |
|---|---|---|---|
| `lapis` | 36 | `nether_gold` | 50 |
| `netherrack` | 2 | `debris` | 400 |
| `end_stone` | 4 | `chorus` | 70 |
| `glowstone` | 40 | `purpur` | 100 |
| | | `shulker` | 270 |

`tnt`, `gravel`, `lava`, `chest`, `mimic`, `villager`, `relic`, `bedrock`, `obsidian` стоят 0 — у них поведение, а не цена. `ROCK` (порода) сбрасывает комбо, `WALLS` (`bedrock`, `obsidian`) не копаются без незеритовой кирки, `RARE` — список руды, на которую действуют «Удача» и золотая лихорадка.

Четыре яруса, у каждого своя порода и веса блоков по глубине:

| Ярус | Название | Порода | Особенности |
|---|---|---|---|
| 1 | Пещеры | `stone` | от грязи до алмазов и изумрудов |
| 2 | Глубинный сланец | `deepslate` | аметист, больше лавы |
| 3 | Незер | `netherrack` | кварц, незерское золото, `debris` (400) |
| 4 | Энд | `end_stone` | обсидиан, шалкеры, `purpur` |

Веса берутся по «первой границе, не меньшей ряда» (`MinerWorld::weights()`), поэтому глубина 1–2, 3–5 и 6–7 дают разные таблицы.

#### Забег

Состояние забега живёт в БД, а не на клиенте: `miner_runs.grid`, `opened`, `meta` — JSON. Клиент видит блок только после удара (и подсказки фонаря), поэтому подделать карту нельзя.

| Поле `miner_runs` | Смысл |
|---|---|
| `user_id` | FK → `users`, `cascadeOnDelete` |
| `grid` | json, 56 блоков |
| `opened` | json, индексы открытых клеток |
| `meta` | json: ярус, комбо, лучший комбо, динамит, уровни улучшений, `seen`, `chests`, `villagers`, `map`, `artifacts`, `last`, `rush` |
| `energy` | остаток ударов |
| `coins` | накоплено за забег (ещё не начислено) |
| `status` | `active` / `finished` |
| `finished_at` | когда закрыт |

Правила хода:

- Первый ряд открыт при старте — это «поверхность», копать можно только клетку по стороне от открытой (`reachable`).
- Рудный блок копит комбо: `comboMult = min(1.5, 1 + 0.05 × (combo − 1))`.

- Удар по `bedrock`/`obsidian` при кирке ниже 5 только «подсвечивает» блок и не тратит удар; незеритовая кирка ломает его за 3 удара.
- Лава — `-2` удара, гравий — `-1` и сброс комбо; с шансом 35% гравий открывает пещеру и бесплатно раскрывает соседей.
- `tnt` взрывает соседей по кругу (с диагоналями), соседний `tnt` — цепочкой; динамит из запаса бьёт крестом без диагоналей и без удара кирки.
- Сундук: 30–80 монет × множитель яруса (1.0 / 1.6 / 2.4 / 3.2), 25% — динамит, 15% — артефакт; с шансом 35% сундук оказывается мимиком и откусывает 30% накопленного.
- Житель: одна сделка на жителя, цена — доля добычи забега, но не меньше минимума (`strikes` 25%/30, `dynamite` 15%/20, `map` 10%/15).
- Спуск на ярус ниже — когда открыт нижний ряд, стоит 4 удара.
- Забег закрывается сам, когда ходов не осталось (`canAct` = нет энергии на достижимые цели и нет динамита), либо досрочно по `cash-out`.

#### Как считается результат

```
// backend/app/Domains/Games/Services/MinerService.php
private function finishRun(User $user, object $run, int $coins, array $meta): array
{
    DB::table('miner_runs')->where('id', $run->id)->update([
        'coins' => $coins,
        'status' => 'finished',
        'finished_at' => now(),
        'updated_at' => now(),
    ]);

    // Перк питомца: лучший из гуляющих по профилю добавляет процент
    $petBonus = (int) floor($coins * PetPerks::minerBonus($this->petLevel($user)));
    // Коллекция артефактов: +1% за каждый разный
    $artifactBonus = (int) floor($coins * $this->progress->artifactBonus($this->progress-
>profile($user)));
    $total = $coins + $petBonus + $artifactBonus;

    if ($total > 0) {
        CoinService::credit($user, $total, CoinTransaction::SOURCE_OTHER, 'Шахтёр: забег', "miner:
{$run->id}");
    }

    $this->progress->track($user->id, ['runs' => 1, 'coins' => $coins]);
    ...
```

Итого: `монеты = добыча + floor(добыча × бонус питомца) + floor(добыча × бонус коллекции)`. Ключ идемпотентности `miner:{run_id}` не даёт начислить забег дважды. Каждый блок при открытии даёт `value × comboMult × (2 если двойной дроп)`, двойной дроп — шанс `1% × уровень Удачи`.

#### Прогресс между забегами

`miner_profiles` — одна строка на игрока (`user_id` — первичный ключ, `cascadeOnDelete`):

| Поле | Смысл |
|---|---|
| `pickaxe` | +1 удар за уровень, 5 уровней; на 5 ломает бедрок |
| `fortune` | +4% веса редкой руды и +1% шанс двойного дропа за уровень |
| `lantern` | радиус и содержание подсказок: 1 — лава/гравий, 2 — бедрок, 3 — радиус 2, 4 — опасность по ярусу, 5 — сундуки, жители, артефакты |

| | |
|---|---|
| `cart` | −15 секунд перерыва за уровень |
| `dynamite` | заряд на каждый забег (один уровень, 6000) |
| `artifacts` | json `id => количество` |
| `quests` | json `{date, items: [{id, progress, claimed}]}` |

Цены уровней — в `MinerWorld::UPGRADES`, покупка списывает ApexCoin через `CoinService::debit(..., SOURCE_PURCHASE, ...)` в транзакции с `lockForUpdate`.

Ежедневные задания: пул из 10, игроку детерминированно выпадают три на дату (сортировка по `crc32("{userId}:{date}:{id}")`), прогресс копится в `track()`, для `combo` и `nether` — как максимум, а не сумма. Награда 200–400 монет, ключ `miner-quest:{user}:{date}:{quest}`.

Топ недели — по лучшему забегу за неделю (`MAX(coins)` по игрокам), место считается через `havingRaw('MAX(coins) > ?')`. Призы: 1 место — 6000 и кейс `royal`, 2 — 4000 и `seeker`, 3 — 2000 и `seeker`; выдача `awardWeek()` идёт по понедельникам в 00:15 (`routes/console.php`) с ключом `miner-week:{дата}:{место}`.

Золотая лихорадка — три часа в сутки, одинаковые для всех, без хранения:

```
// backend/app/Domains/Games/Services/MinerProgress.php
public static function rushHours(string $date): array
{
    $hours = range(12, 23);
    usort($hours, fn ($a, $b) => crc32("miner-rush:{$date}:{$a}") <=> crc32("miner-rush:{$date}:
{$b}"));
    $picked = array_slice($hours, 0, 3);
    sort($picked);

    return $picked;
}
```

Пока идёт лихорадка, вес редкой руды удваивается (`$meta['rush'] ? 2 : 1`). Артефактов восемь (`coin`, `fossil`, `compass`, `map`, `heart`, `totem`, `star`, `egg`), каждый разный даёт +1% к добыче, полная коллекция — ещё +2%; повторный артефакт продаётся за `40 × ярус`.

#### Что видит игрок

`MinerView.vue` (маршрут `/miner`, только для авторизованных) рисует сетку 8 колонок: открытые блоки спрайтами, скрытые — `null`, подсказки фонаря — отдельными классами, достижимые клетки подсвечены, режим динамита меняет курсор. Рядом — журнал последних событий, всплывающие «+N», тряска при взрыве, вспышка при спуске. Отдельные панели: мастерская (`MinerUpgrades`), задания дня (`MinerQuests`), топ недели (`MinerTop`), коллекция (`MinerCollection`), экран итога забега с разбивкой «добыто / бонус питомца / бонус коллекции».

## Эндпоинты доменов

Формат: `метод путь` — что делает; в колонке «Доступ» — `–` (публично), `auth` (Sanctum), роль.

#### Players (профиль, рейтинг, топы)

| Метод и путь | Что делает | Доступ | |
|---|---|---|---|
| `GET /api/home` | главная: hero, топ-10 игроков и кланов, | — | |

| | | | |
|---|---|---|---|
| | новости, счётчики | | |
| `GET /api/home/me` | панель игрока: кулдауны, питомцы, бонус, шахта | auth | |
| `GET /api/top` | топ-10 игроков и кланов | — | |
| `GET /api/ranking` | рейтинг с курсором, режимы `overall/pvp/bedwars/bridge` | — | |
| `GET /api/players/{user}` | профиль по id или нику | — | |
| `GET /api/users/{user}` | тот же профиль (алиас) | — | |
| `GET /api/players/{user}/skin` | скин Minecraft для 3D-модели | — | |
| `GET /api/players/teammates` | карточки тиммейтов для блока профиля | — | |
| `GET /api/players/{user}/guestbook` | гостевая книга | — | |
| `GET /api/players/{user}/respects` | респекты профиля | — | |
| `GET /api/respects/top` | топ самых уважаемых недели | — | |
| `GET /api/players/{user}/polls/{block}` | результаты опроса в профиле | — | |
| `GET /api/players/{user}/pet` | состояние питомца | — | |
| `GET /api/players/{user}/recommendations` | рекомендации профиля | — | |
| `GET /api/stats` | публичная статистика проекта (кэш 10 минут) | — | |
| `PUT\ | POST /api/players/me` | био, аватар, обложка, соцсети | auth |
| `PUT\ | POST /api/players/me/profile` | статус, цитата, цвета, фон карточки | auth |
| `PUT /api/players/me/blocks` | конструктор профиля | auth | |
| `PUT /api/players/me/aspects` | сохранить аспекты режима и пересчитать тир | auth | |
| `POST /api/players/me/avatar/remove` | удалить аватар | auth | |
| `POST /api/players/me/cover/remove` | удалить обложку | auth | |
| `POST /api/players/me/card-background/remove` | удалить фон карточки | auth | |

| | | | |
|---|---|---|---|
| `POST /api/players/{user}/guestbook` | записать в гостевую | auth | |
| `DELETE /api/guestbook/{entry}` | удалить запись | auth | |
| `POST /api/players/{user}/respects` | поставить респект (раз в день) | auth | |
| `DELETE /api/players/{user}/respects` | забрать респект | auth | |
| `POST /api/players/{user}/visit` | отметить визит | auth | |
| `GET /api/players/me/guests` | кто заходил в профиль | auth | |
| `POST /api/players/{user}/polls/{block}` | проголосовать | auth | |
| `POST /api/players/{user}/pet/{action}` | погладить/покормить/поиграть | auth | |
| `PUT /api/players/me/pet-name` | переименовать питомца | auth | |
| `POST /api/players/{user}/recommendations` | написать рекомендацию | auth | |
| `DELETE /api/players/{user}/recommendations` | удалить свою рекомендацию | auth | |
| `POST /api/recommendations/{recommendation}/hide` | скрыть рекомендацию | auth | |
| `GET /api/guide`, `POST /api/guide/start`, `POST /api/guide/steps/{step}/complete`, `POST /api/guide/steps/{step}/skip`, `POST /api/guide/hide` | путеводитель по сайту | auth | |
| `GET /api/pets` + `POST /api/pets/...` | питомцы: экспедиции, арена, трюки, питомцедекс | auth | |

#### Tiers

| Метод и путь | Что делает | Доступ |
|---|---|---|
| `GET /api/tier-tests` | мои заявки + заявки, где я тестер | auth |
| `POST /api/tier-tests` | создать заявку (`feature:tier_tests`) | auth |
| `GET /api/tier-tests/{tierTest}` | карточка заявки | auth (автор или тестер) |
| `POST /api/tier-tests/{tierTest}/cancel` | отменить свою заявку (`pending`) | auth |
| `PUT /api/tier-tests/{tierTest}` | правка | auth (автор или тестер) |

| | | |
|---|---|---|
| | статуса/результата/аспектов | |
| `POST /api/tier-tests/{tierTest}/appeal` | подать апелляцию (throttle 5/мин) | auth |
| `POST /api/tier-tests/{tierTest}/priority` | применить купленный приоритет | auth |
| `GET /api/tier-test-appeals` | мои апелляции | auth |
| `GET /api/players/{user}/tier-history` | история завершённых тестов | auth |
| `GET /api/tester/tier-tests` | очередь (`status`, `mode`, `mine`, `free`) | `tester`, `admin` |
| `GET /api/tester/tier-tests/stats` | счётчики тестера | `tester`, `admin` |
| `GET /api/tester/tier-tests/{tierTest}` | карточка для проведения | `tester`, `admin` |
| `POST /api/tester/tier-tests/{tierTest}/claim` | взять в работу | `tester`, `admin` |
| `POST /api/tester/tier-tests/{tierTest}/unclaim` | вернуть в очередь | `tester`, `admin` |
| `POST /api/tester/tier-tests/{tierTest}/complete` | завершить с оценками | `tester`, `admin` |
| `POST /api/tester/tier-tests/{tierTest}/cancel` | отменить с причиной | `tester`, `admin` |
| `GET /api/admin/tier-test-appeals` | очередь апелляций | `moderator`, `admin` |
| `POST /api/admin/tier-test-appeals/{appeal}/resolve` | решение по апелляции | `moderator`, `admin` |
| `PUT /api/admin/users/{user}/aspects` | прямая правка аспектов | `admin` |
| `POST /api/admin/users/{user}/tier-test` | провести тест вручную | `admin` |

#### Seasons

| Метод и путь | Что делает | Доступ |
|---|---|---|
| `GET /api/seasons` | текущий сезон + закрытые с топ-3 | — |
| `GET /api/seasons/{season}` | итоговая таблица сезона (топ-10) | — |
| `GET /api/players/{user}/seasons` | места игрока по сезонам | — |

#### Achievements

| Метод и путь | Что делает | Доступ |
|---|---|---|
| `GET /api/achievements` | каталог + прогресс игрока | auth |
| `GET /api/players/{user}/achievements` | ачивки конкретного игрока | auth |
| `GET /api/admin/achievements` | список для админки (поиск, фильтры, `granted_count`) | `moderator`, `admin` |
| `POST /api/admin/achievements` | создать ачивку | `moderator`, `admin` |
| `PUT` | изменить | `moderator`, `admin` |

| | | |
|---|---|---|
| `/api/admin/achievements/{achievement}` | | |
| `DELETE /api/admin/achievements/{achievement}` | удалить (системную — нельзя) | `moderator`, `admin` |
| `POST /api/admin/users/{user}/achievements/{achievement}` | выдать вручную | `admin` |
| `DELETE /api/admin/users/{user}/achievements/{achievement}` | отозвать | `admin` |

#### Games

| Метод и путь | Что делает | Доступ |
|---|---|---|
| `GET /api/games/miner` | полное состояние: забег, кулдаун, улучшения, артефакты, задания, топ, лихорадка | auth |
| `POST /api/games/miner/start` | начать забег | auth |
| `POST /api/games/miner/dig` | удар по клетке (`cell`) | auth |
| `POST /api/games/miner/blast` | динамит по клетке | auth |
| `POST /api/games/miner/chest` | открыть сундук | auth |
| `POST /api/games/miner/trade` | сделка с жителем (`offer`) | auth |
| `POST /api/games/miner/descend` | спуск на ярус ниже | auth |
| `POST /api/games/miner/cash-out` | забрать добытое досрочно | auth |
| `POST /api/games/miner/upgrade` | купить улучшение (`upgrade`) | auth |
| `POST /api/games/miner/quests/{quest}/claim` | забрать награду задания | auth |

Ошибки игровых правил (`RuntimeException`) контроллер превращает в `422` с текстом, а не в 500:

```
// backend/app/Domains/Games/Controllers/MinerController.php
private function run(callable $operation): JsonResponse
{
    try {
        return response()->json($operation());
    } catch (RuntimeException $e) {
        return response()->json(['message' => $e->getMessage()], 422);
    }
}
```

Отдельно стоит помнить про порядок объявления роутов: статические пути объявлены раньше параметрических (`/players/teammates` до `/players/{user}`, `/shop/priority-candidates` до `/shop/{shopItem}`), потому что Laravel берёт первое совпадение.

### Связи моделей и каскады

| Таблица | Ключи | Каскад |
|---|---|---|
| `users` | `id` | — |
| `player_aspects_pvp` | `user_id` → `users`, `unique(user_id)` | `cascadeOnDelete` |

| | | |
|---|---|---|
| player_aspects_bedwars | user_id → users, unique(user_id) | cascadeOnDelete |
| tier_tests | user_id → users | cascadeOnDelete |
| | tester_id → users | nullOnDelete |
| | claimed_by → users | nullOnDelete |
| tier_test_appeals | tier_test_id → tier_tests, unique | cascadeOnDelete |
| | user_id → users | cascadeOnDelete |
| | reviewer_id, revised_test_id → users / tier_tests | nullOnDelete |
| achievements | code unique | — |
| user_achievements | user_id, achievement_id, unique(user_id, achievement_id) | cascadeOnDelete с обеих сторон |
| seasons | number unique | — |
| season_results | season_id, user_id, unique(season_id, user_id), index(season_id, rank) | cascadeOnDelete |
| miner_runs | user_id → users, index(user_id, status), index(status, finished_at) | cascadeOnDelete |
| miner_profiles | user_id — первичный ключ | cascadeOnDelete |
| coin_transactions | user_id, actor_id (nullOnDelete), reference_type/reference_id (полиморф), idempotency_key unique | cascadeOnDelete по user_id |

Eloquent-связи доменов:

- User hasOne aspectPvp, hasOne aspectBedwars, hasMany tierTests, belongsToMany achievements (пивот user_achievements, withPivot('earned_at')), belongsTo bridgeRank, hasOne clanMember.
- TierTest belongsTo user / tester (tester_id) / claimer (claimed_by); TierTestAppeal belongsTo tierTest, revisedTest (revised_test_id), user, reviewer.
- Season hasMany results; SeasonResult belongsTo season, user.
- Achievement belongsToMany users (та же таблица); UserAchievement belongsTo user, achievement.
- Miner* — модели отсутствуют: домен работает через DB::table() и JSON-поля, поэтому связей Eloquent у него нет.

Полиморфные ссылки в coin_transactions хранят короткие псевдонимы (tier_test, achievement, user, clan), карта — в AppServiceProvider::boot() через Relation::morphMap. CoinService пишет reference_type напрямую и потому применяет карту сам через morphType().

### Команды и планировщик

| Команда | Что делает |
|---|---|
| `php artisan apex:rating-recalc` | пересчитать очки всех игроков и силу всех кланов |
| `php artisan apex:season-close --name=… --force` | закрыть сезон, сохранить места, открыть следующий |
| `php artisan apex:achievements --user=… --dry-run --force` | пересчитать ачивки и доплатить за неоплаченные |
| `php artisan apex:migrate-aspects` | разовый перенос аспектов на шкалу 0–20 |
| `php artisan apex:coins` | выдача/списание ApexCoin из консоли (используется и в наградах) |

В расписании (`routes/console.php`) из этих доменов есть только призы «Шахтёра»: `MinerProgress::awardWeek()` каждую неделю в понедельник 00:15. Пересчёт рейтинга, закрытие сезона и пересчёт ачивок автоматически не запускаются — это ручные операции.

### Что в коде отсутствует

Чтобы не искать это повторно:

- **Сброс или перенос рейтинга между сезонами** — нет. `SeasonService::close()` только пишет снимок и создаёт следующий сезон.
- **Награды за сезоны** (монеты, кейсы, ачивки, титулы) — нет ни в сервисе, ни в команде, ни в API.
- **Автоматическое понижение `S`/`S+`** — нет, все пересчёты явно сохраняют эти тиры.
- **Порог для `S`/`S+` в тир-тесте** — нет: `TIER_THRESHOLDS` заканчивается на `A`.
- **Связь очереди тир-тестов сайта с плагином Minecraft** — нет: `QueueSource` в плагине отдаёт пустой список, заявки заводятся вручную через `/tier add`.
- **Заполнение `tier_tests.scheduled_at`** — нет: поле есть, запись в него не производится.
- **Колонки аспектов в `tier_tests`** — нет: только JSON `aspects`, хотя в `$fillable` перечислены устаревшие имена.
- **Таблица `user_featured_achievements`** — нет в миграциях, хотя связь `User::featuredAchievements()` объявлена; избранное живёт в JSON `users.featured_achievements`.
- **Начисление ApexCoin при ручной выдаче ачивки из админки** — нет: `Admin\AchievementController::grant()` не вызывает `RewardService`, доплата возможна только через `apex:achievements`.
- **Модели для `miner_runs` и `miner_profiles`** — нет, домен работает через query builder.
- **Второй игры в домене `Games`** — нет: только «Шахтёр»; арена питомцев относится к домену `Players`.

# Кланы, форум и друзья: социальное ядро APEX

## 0. Как устроены эти три домена

Домен в APEX — это каталог `backend/app/Domains/<Домен>/<Вид>/`. У трёх разбираемых доменов разная «плотность»:

| Домен | Файлов | Состав |
|---|---|---|
| `Clan` | 60 | 8 моделей «поверх» клана, 8 сервисов, 12 контроллеров (включая 3 админских), 16 FormRequest, 3 политики, 2 ресурса, 4 уведомления, 3 support-класса |
| `Forum` | 26 | 5 моделей, 1 сервис, 2 контроллера, 8 FormRequest, 2 политики, 4 ресурса, 2 support-класса, 1 уведомление |
| `Friends` | 7 | 1 модель, 1 сервис, 1 контроллер, 1 FormRequest, 1 ресурс, 2 уведомления |

Распределение ответственности одинаково во всех трёх доменах:

- **контроллер** — принять запрос, дернуть `authorize()`, вернуть JSON; бизнес-логики нет (`ClanController` — 176 строк на 16 методов);
- **FormRequest** (`App\Http\Requests\BaseFormRequest`) — валидация; `authorize()` всегда `true`, доступ регулируют middleware и политики;
- **сервис** — правила, транзакции, уведомления (`ClanService`, `ClanWarService`, `ClanTreasuryService`, `ClanOverviewService`, `ClanLevelService`-функции в `Support\ClanLevel`, `ForumService`, `FriendService`);
- **политика** — права на объект, регистрируются явно в `backend/app/Providers/AuthServiceProvider.php`;
- **ресурс** — формат ответа (`ClanCardResource`, `ClanApplicationResource`, `ForumTopicResource`, `ForumReplyResource`, `FriendshipResource`).

Два сквозных механизма, без которых домен клана не читается:

1. `App\Domains\Clan\Support\ClanContext` — единая точка получения клана из запроса: сначала атрибут `clan`, который кладёт middleware `clan.member`, иначе — `$request->user()->clanMember->clan`. Нужен потому, что часть маршрутов объявлена вне группы `my-clan`, и раньше это давало 404 вместо 403.
2. `App\Support\Concerns\AbortsWithMessage` — бизнес-ошибки в двух форматах: плоский `{"message": "..."}` (`abortUnprocessable()` → 422, `abortForbidden()` → 403) и `ValidationException` с `errors` по полю (`invalid()`).

Middleware `EnsureClanMember` (`backend/app/Http/Middleware/EnsureClanMember.php`) обслуживает всю группу `/api/my-clan`:

```
$membership = $user->clanMember;

if (!$membership) {
    abort(403, 'Вы не в клане.');
}

// если нужно конкретное право
if ($permission && !$membership->can($permission)) {
    abort(403, 'Недостаточно прав.');
}

$request->attributes->set('clan_membership', $membership);
$request->attributes->set('clan', $membership->clan);
```

Права передаются параметром: `clan.member:news`, `clan.member:forum`, `clan.member:resources`.

# 1. Кланы

## 1.1 Схема таблиц

| Таблица | Ключевые поля | Внешние ключи и удаление |
|---|---|---|
| clans | name (unique), tag 8 (unique), description, avatar, cover_path, banner_color 16, leader_id, power (signed int), treasury bigint, wins, losses, is_open, entry_fee, is_highlighted, highlight_until, highlight_color, highlight_effect, max_members (default 30), is_banned, ban_reason, socials json, xp, level (default 1), motd, motd_updated_at, motd_by | leader_id → users **cascadeOnDelete** (удаление лидера удаляет клан), motd_by → users nullOnDelete; индекс по power |
| clan_members | role enum leader/officer/member, permissions json, title 32, contribution, joined_at, promoted_at, promoted_by | clan_id, user_id — cascade; promoted_by → users nullOnDelete; **unique(`user_id`)** — игрок только в одном клане (миграция 2026_10_05_195345), плюс исторический unique(clan_id,user_id) |
| clan_applications | message, status enum pending/accepted/declined | clan_id, user_id cascade; unique(clan_id,user_id) |
| clan_wars | challenger_clan_id, opponent_clan_id, created_by, status enum pending/accepted/declined/completed/cancelled, scheduled_at, challenger_score/opponent_score tinyint, winner_clan_id, outcome 16, notes | кланы — cascade, created_by → users cascade, winner_clan_id nullOnDelete |
| clan_war_participants | clan_war_id, user_id, clan_id, joined_at | все три — cascade; unique(clan_war_id,user_id) |
| clan_events | type enum announcement/event/training, title, body, starts_at | clan_id, author_id cascade |
| clan_event_comments | parent_id, body | clan_event_id, user_id, parent_id — cascade (все) |
| clan_forum_topics | title, body, is_pinned, is_locked, views, replies_count, last_reply_at, last_reply_user_id | clan_id, author_id cascade; last_reply_user_id nullOnDelete |
| clan_forum_replies | parent_id, body | topic_id, author_id, parent_id cascade |
| clan_resources | title, description, file_path, file_name, file_size, mime_type, category enum (resource_pack/screenshot/config/ | clan_id, author_id cascade |

| | | |
|---|---|---|
| | guide/other), downloads, is_public | |
| clan_checkins | day (date) | clan_id, user_id cascade; **unique(`user_id`,`day`)**, индекс (clan_id,day) |
| clan_treasury_transactions | amount (signed), balance_after, type 24 (donation/purchase/reward), description | clan_id cascade, user_id nullOnDelete |
| clan_inventory | shop_item_id, purchased_by, equipped_at | clan_id, shop_item_id cascade; purchased_by nullOnDelete; unique(clan_id,shop_item_id) |
| clan_stats_logs | wins_delta, losses_delta, reason | clan_id, user_id cascade; user_id **NOT NULL** |

Связь с игроком односторонняя: `users.clan_joined_at` (когда вступил) и `users.favorite_clan_id → clans` nullOnDelete (витрина «любимый клан», отдельно от членства).

### 1.2 Создание клана

`POST /api/clans`, middleware `feature:clan_creation` (глобальный выключатель `ControlService::FEATURES['clan_creation']`, значение по умолчанию — включено).

Валидация (`CreateClanRequest`): `name` 3–32 unique, `tag` 2–8 unique, `description` ≤1000, `banner_color` ≤16, `is_open` bool. Поля `max_members` в запросе нет — лимит приходит из default БД (30) и API его изменить нельзя.

`ClanService::create`:

```
if ($leader->clanMember) {
    $this->abortUnprocessable('Вы уже в клане.');
}

return DB::transaction(function () use ($leader, $data) {
    $clan = Clan::create([
        ...$data,
        'leader_id' => $leader->id,
        'banner_color' => $data['banner_color'] ?? '#7c3aed',
        'is_open' => $data['is_open'] ?? true,
    ]);

    ClanMember::create([
        'clan_id' => $clan->id, 'user_id' => $leader->id,
        'role' => 'leader', 'joined_at' => now(),
    ]);

    $leader->update(['clan_joined_at' => now()]);
    AchievementService::check($leader);
    return $clan;
});
```

Проверки «один клан на игрока» две: явная (`clanMember`) и уникальный индекс `clan_members.user_id`. Кланов-пустышек не бывает: лидер сразу становится участником.

### 1.3 Роли и права

Ролей ровно три: `leader`, `officer`, `member`. Отдельной таблицы ролей нет — enum + JSON `permissions` у членства.

```php
public function can(string $permission): bool
{
    if ($this->role === 'leader') return true;

    $rolePermissions = [
        'officer' => ['news', 'forum', 'applications', 'wars', 'resources'],
        'member' => [],
    ];

    $rolePerms = $rolePermissions[$this->role] ?? [];
    $customPerms = $this->permissions ?? [];

    return in_array($permission, $rolePerms)
        || in_array($permission, $customPerms);
}
```

`backend/app/Domains/Clan/Models/ClanMember.php:47`

Пять прав-ключей: `news` (объявление/MOTD), `forum` (темы, pin/lock), `applications` (заявки), `wars` (войны), `resources` (загрузка файлов).

Политики при этом смотрят **только на роль**:

```php
public function update(User $user, Clan $clan): bool
{
    return $clan->isLeader($user->id);
}

public function reviewApplications(User $user, Clan $clan): bool
{
    return $this->isStaff($user, $clan);
}

public function kick(User $user, Clan $clan): bool
{
    return $clan->isLeader($user->id);
}
```

`backend/app/Domains/Clan/Policies/ClanPolicy.php:17`

`isStaff` = членство с ролью `leader` или `officer`. Права из JSON `permissions` действуют только там, где вызывается `ClanMember::can` (middleware `clan.member:*`, флаги `my_permissions`, `can_edit_motd`), но не расширяют политики: выданное участнику право `applications` не даст доступа к `GET /api/clans/{clan}/applications`, потому что там `reviewApplications` → `isStaff` → проверка роли.

Сводная матрица:

| Действие | leader | officer | member | Где проверяется |
|---|---|---|---|---|
| Правка профиля, аватар, обложка | ✅ | ❌ | ❌ | `ClanPolicy::update` / `removeMedia` |
| Кик участника | ✅ | ❌ | ❌ | `ClanPolicy::kick` |
| Смена роли, титула, прав | ✅ | ❌ | ❌ | `ClanPolicy::manageRoles` |
| Передача лидерства | ✅ | ❌ | ❌ | `manageRoles` + `ClanService::transferLeadership` |
| Просмотр и приём заявок | ✅ | ✅ | ❌ | `reviewApplications` |
| Создание событий/новостей | ✅ | ✅ | ❌ | `createEvent` |

| | | | | |
|---|---|---|---|---|
| Удаление события | ✅ | ✅ (любое) | ✅ только своё | `deleteEvent($user, $clan, $authorId)` |
| Темы клан-форума, pin/lock | ✅ | ✅ | ✅ если `can('forum')` | middleware `clan.member:forum` |
| Ответ в теме | ✅ | ✅ | ✅ | только `clan.member` |
| Загрузка ресурсов | ✅ | ✅ | ✅ если `can('resources')` | `clan.member:resources` |
| Объявление (MOTD) | ✅ | ✅ | ✅ если `can('news')` | `clan.member:news` |
| Войны: вызов/принятие/результат/призыв | ✅ | ✅ | ❌ | `ClanWarService::assertCanManage` |
| Запись в состав войны / выход | ✅ | ✅ | ✅ | `assertParticipatingClan` |
| Отметка, взнос в казну | ✅ | ✅ | ✅ | членство |
| Покупка клановых предметов | ✅ | ❌ | ❌ | `ShopService` (`clan->isLeader`) |

### 1.4 Заявки на вступление

Прямых приглашений в код **нет**. Единственный путь попасть в клан — заявка, которую рассматривает руководство. Следствие: клан с `is_open = false` вступить нельзя вообще (никакого invite-эндпоинта не существует).

Правила `ClanService::apply`:

- уже в клане → 422 «Вы уже в клане.»;
- `is_open = false` → 422 «Клан закрыт для вступления.»;
- повторная заявка при `pending` → 422 «Заявка уже отправлена.»;
- иначе `updateOrCreate` по (`clan_id`,`user_id`) — заявка после отказа переиспользует ту же строку, статус снова `pending`;
- лидеру уходит `ClanApplicationNotification` (канал `database`), текст «{ник} подал заявку в клан [TAG] Name».

Список заявок: `pending`, `latest()`, с карточкой игрока (`tier`, `tier_score`, эффекты ника).

Приём (`acceptApplication`) — самый насыщенный транзакцией участок домена:

```
if ($clan->members()->count() >= \App\Domains\Clan\Support\ClanLevel::memberCap($clan)) {
    $this->abortUnprocessable('Клан заполнен.');
}

if ($clan->entry_fee > 0 && $applicant && ! CoinService::canAfford($applicant, $clan->entry_fee))
{
    $this->abortUnprocessable(
        "Для вступления нужна плата {$clan->entry_fee} ApexCoin. "
        . "У игрока на балансе {$applicant->apex_coins}."
    );
}

if (ClanMember::where('user_id', $application->user_id)->exists()) {
    $application->update(['status' => 'declined']);
```

```
        $this->abortUnprocessable('Игрок уже состоит в другом клане. Заявка отклонена.');
    }
```

`backend/app/Domains/Clan/Services/ClanService.php:330`

Дальше внутри `DB::transaction`:

1. повторная проверка членства `lockForUpdate()` → `abort(409)`, если гонка;
2. `CoinService::debit` заявителю на `entry_fee` (источник `CoinTransaction::SOURCE_CLAN_FEE`, ссылка на клан), затем `CoinService::credit` лидеру на ту же сумму (если лидер ≠ заявитель);
3. создание `ClanMember` с ролью `member`;
4. `AchievementService::check`, `users.clan_joined_at = now()`;
5. статус заявки `accepted`;
6. `$clan->recalculatePower()`.

После транзакции все прочие `pending`-заявки этого игрока переводятся в `declined` (он уже в клане), заявителю уходит `ClanApplicationAcceptedNotification`. Отказ — просто `status = declined` + `ClanApplicationDeclinedNotification`.

Плата берётся **в момент принятия**, а не подачи: иначе заявитель «замораживал» бы монеты, разослав заявки во все кланы (комментарий в коде и в миграции `2026_10_02_110000`). Диапазон `entry_fee`: 0–100000.

Защита от чужих заявок: `ClanController::acceptApplication` вызывает `ClanService::assertApplicationBelongsToClan` → `abort_if($application->clan_id !== $clan->id, 404)`.

### 1.5 Выход, кик, роспуск, передача лидерства

`leave`:

- лидер с участниками → 422 с подсказкой про кнопку с короной;
- лидер-одиночка → `dissolve($clan)`;
- обычный участник → удаление членства, `clan_joined_at = null`, пересчёт силы.

`dissolve` удаляет клан и чистит то, что не покрыто каскадом:

```
DB::transaction(function () use ($clan) {
    $memberIds = ClanMember::where('clan_id', $clan->id)->pluck('user_id');

    if ($memberIds->isNotEmpty()) {
        User::whereIn('id', $memberIds)->update(['clan_joined_at' => null]);
    }

    // Витрина профиля не должна ссылаться на удалённый клан
    User::where('favorite_clan_id', $clan->id)->update(['favorite_clan_id' => null]);

    $clan->delete();
});
```

`backend/app/Domains/Clan/Services/ClanService.php:475`

`kick` (leader-only по политике): кик лидера запрещён (422), отсутствие игрока в клане ошибкой не считается, `clan_joined_at` сбрасывается, сила пересчитывается.

`transferLeadership`: цель обязана быть участником (`firstOrFail`), старый лидер становится `officer`, новый — `leader`, `clans.leader_id` обновляется; уведомления в коде нет.

`updateMemberRole` (leader-only): нельзя менять свою роль; `role` в запросе — только `officer|member` (лидера назначить нельзя, только передать); `permissions` — подмножество пяти ключей; `title` ≤32; пишутся `promoted_at` и `promoted_by`.

### 1.6 Сила клана

Сила — не сумма очков, а половина:

```
$sums = ClanMember::query()
    ->join('users', 'users.id', '=', 'clan_members.user_id')
    ->whereIn('clan_members.clan_id', $ids)
    ->whereNotIn('users.role', User::UNRANKED_ROLES)
    ->groupBy('clan_members.clan_id')
    ->selectRaw('clan_members.clan_id, SUM(users.rating_points) as total')
    ->pluck('total', 'clan_id');

foreach ($ids as $clanId) {
    $power = (int) floor(((int) ($sums[$clanId] ?? 0)) * self::CLAN_SHARE);
    Clan::whereKey($clanId)->where('power', '!=', $power)->update(['power' => $power]);
}
```

`backend/app/Domains/Players/Services/RatingService.php:101`

`CLAN_SHARE = 0.5`; `User::UNRANKED_ROLES = ['admin', 'moderator']` (`HasRoles`), то есть персонал в силу не входит.

Очки игрока (`RatingService::compute`) складываются из:

- среднее трёх тиров (`pvp`, `bedwars`, `bridge`), округлённое вверх — `tiers_points`;
- победы в матчах турниров × `rating_match_win` (default 5);
- сумма `win_points` выигранных турниров;
- участие в завершённых КВ × `rating_war_participation` (default 3);
- победы в завершённых КВ × `rating_war_win` (default 7).

Веса читаются из `SiteSetting` с дефолтами. Участие/победы считаются по `clan_war_participants` при `clan_wars.status = 'completed'`; победа — `winner_clan_id = participant.clan_id`.

Вклад участника (`ClanMember::contribution`) — вычисляемый атрибут, та же доля: `floor(rating_points * 0.5)`; колонка `contribution` в БД никем не пишется.

`Clan::recalculatePower()` дергает `RatingService::recalculateClans([$this->id])` и перечитывает `power` из БД (`syncOriginalAttribute`). Вызывается после: принятия заявки, выхода, кика, завершения войны, админской правки статистики.

### 1.7 Рейтинг кланов и список

`GET /api/clans` — публичный, пагинация 20. Фильтры: `search` (по `name`/`tag`, с экранированием `%`, `_`, `!` через `ESCAPE '!'`), `open=true`, `has_slots=true`. Сортировки (`ClanService::SORTS`): `power` (default), `members`, `wins`, `winrate`, `rating`, `new`, `name`.

«Рейтинг» клана в списке — это средние очки участников, считаемые подзапросом:

```
$avgRating = '(SELECT COALESCE(AVG(u.rating_points), 0) FROM clan_members cm
    JOIN users u ON u.id = cm.user_id
    WHERE cm.clan_id = clans.id AND u.role NOT IN (...))';

$winrate = 'CASE WHEN (clans.wins + clans.losses) = 0 THEN 0
    ELSE clans.wins * 1.0 / (clans.wins + clans.losses) END';
```

`backend/app/Domains/Clan/Services/ClanService.php:57`

Свободные места считаются той же формулой, что `ClanLevel::memberCap`, но одним SQL:

```
'(SELECT COUNT(*) FROM clan_members cm WHERE cm.clan_id = clans.id) < clans.max_members + ? *
(clans.level - 1)'
```

с параметром `ClanLevel::SLOTS_PER_LEVEL = 2`.

Каждая строка отдаётся через `ClanCardResource` плюс два добавленных поля: `position` (место в текущей сортировке: `offset + i + 1`) и `avg_rating`. Ресурс считает `is_highlighted` через `isHighlightActive()` (учёт срока), `max_members` — через `memberCap`, `my_clan_id` — id своего клана для клика во вкладку «Мой клан».

`GET /api/clans/top` — топ-10 по `power` без пагинации, число участников добирается одним группированным запросом (без N+1). Забаненные (`is_banned = true`) в список не попадают, но `show` их не фильтрует — забаненный клан остаётся доступен по прямой ссылке.

Фронтенд: `frontend/src/views/clan/ClansView.vue` (фильтры в URL, «показать ещё», эффекты подсветки), `frontend/src/components/clan/ClanRanking.vue` (тройка лидеров + таблица, сортировки `power/rating/winrate/wins`).

### 1.8 Лимиты и уровень клана

Лимит участников = `max_members` (30 по умолчанию) + 2 места за каждый уровень сверх первого:

```
public static function xpFor(int $level): int
{
    return 100 * $level * ($level - 1);
}

public static function sync(Clan $clan): Clan
{
    $xp = ClanCheckin::where('clan_id', $clan->id)->count() * self::XP_PER_CHECKIN
        + (int) $clan->wins * self::XP_PER_WIN;

    $level = self::levelFor($xp);

    if ((int) $clan->xp !== $xp || (int) $clan->level !== $level) {
        $clan->forceFill(['xp' => $xp, 'level' => $level])->saveQuietly();
    }

    return $clan;
}
```

`backend/app/Domains/Clan/Support/ClanLevel.php:27`

| Уровень | Порог XP | Мест сверх `max_members` | ApexCoin в казну за отметку | Скидка из казны |
|---|---|---|---|---|
| 1 | 0 | 0 | 1 | — |
| 2 | 200 | +2 | 2 | — |
| 3 | 600 | +4 | 3 | 5% |
| 4 | 1200 | +6 | 4 | 5% |
| 5 | 2000 | +8 | 5 | 10% |
| 7 | 4200 | +12 | 7 | 15% |
| 10 | 9000 | +18 | 10 | 20% |

`XP_PER_CHECKIN = 10`, `XP_PER_WIN = 100`, `SLOTS_PER_LEVEL = 2`. `checkinCoins($level) = max(1, $level)`. Скидка — ступенчатая (`DISCOUNTS = [3=>5, 5=>10, 7=>15, 10=>20]`), применяется как `ceil(price * (100 - d) / 100)`.

Уровень хранится в `clans.xp`/`clans.level`, чтобы список кланов считал свободные места одним запросом, и пересчитывается (`ClanLevel::sync`) после отметки и после итога войны. Лестница бонусов для UI (`perks`) строится по уровням `[2,3,4,5,7,10]` с флагом `unlocked`.

### 1.9 Ежедневные отметки

`POST /api/my-clan/checkin` (throttle 10/мин).

- Одна отметка в день на игрока **глобально**, не на клан: unique(`user_id`,`day`) — сознательно, чтобы нельзя было накручивать переходом между кланами.
- Повтор → 422 «Сегодня вы уже отмечались.»; двойной клик ловится `QueryException` на уникальном индексе.
- Опыт клану: +10 XP всегда.
- Монеты в казну — только если игрок в клане больше суток:

```
$coins = 0;
$membership = $user->clanMember;
$joinedAt = $membership?->joined_at ?? $membership?->created_at;

if ($joinedAt && $joinedAt->lte(now()->subDay())) {
    $coins = ClanLevel::checkinCoins($levelBefore);
    app(ClanTreasuryService::class)->reward($clan, $user, $coins, 'Отметка участника');
}
```

`backend/app/Domains/Clan/Services/ClanOverviewService.php:85`

Монеты считаются по уровню **до** отметки (иначе отметка сама себе повышала бы награду), уровень пересчитывается после. Ответ содержит `streak`, `xp`, `coins`, `level_up`, полный `level` (прогресс).

Серия (`streak`) считается по последним 400 дням: если сегодня отметки нет — серия не сгорает, отсчёт идёт со вчера.

### 1.10 Казна

Казна — общие ApexCoin клана (`clans.treasury`), не привязанные к игроку. Движения — только через `ClanTreasuryService`:

```
private function move(Clan $clan, User $user, int $amount, string $type, string $description):
Clan
{
    /** @var Clan $locked */
    $locked = Clan::whereKey($clan->id)->lockForUpdate()->firstOrFail();
    $balance = (int) $locked->treasury + $amount;

    if ($balance < 0) {
        throw new RuntimeException('В казне клана недостаточно ApexCoin. Не хватает ' .
abs($balance) . '.');
    }

    $locked->forceFill(['treasury' => $balance])->save();
    ClanTreasuryTransaction::create([
        'clan_id' => $locked->id, 'user_id' => $user->id, 'amount' => $amount,
        'balance_after' => $balance, 'type' => $type, 'description' => $description,
    ]);
```

```
    ...
}
```

backend/app/Domains/Clan/Services/ClanTreasuryService.php:60

| Тип операции | Кто инициирует | Знак `amount` | Как попасть |
|---|---|---|---|
| `donation` | любой участник | + | `POST /api/my-clan/treasury/donate` |
| `reward` | система (отметка) | + | `ClanTreasuryService::reward` |
| `purchase` | лидер | – | покупка кланового предмета с `pay_from=treasury` в магазине |

Ограничения: взнос 1–100000 (`MAX_DONATION`), только участник клана, только при достаточном личном балансе (иначе 422 с точной нехваткой); вывод средств из казны невозможен — только трата на клановые предметы; отрицательный баланс запрещён на уровне `move`. Ответ `overview`: `balance`, `max_donation`, топ-10 вкладчиков (SUM по `donation`), последние 30 движений.

### 1.11 Инвентарь клана и оформление подсветки

Оформление принадлежит **клану**, а не лидеру (`clan_inventory`), поэтому переживает передачу лидерства. Купленный предмет: `shop_item_id`, `purchased_by`, `equipped_at`; unique(`clan_id`,`shop_item_id`).

Подсветка: `is_highlighted` + `highlight_until` (срок) + `highlight_color` + `highlight_effect`.

```
public function isHighlightActive(): bool
{
    if (! $this->is_highlighted) {
        return false;
    }

    return $this->highlight_until === null || $this->highlight_until->isFuture();
}
```

backend/app/Domains/Clan/Models/Clan.php:41

Наборы (`Support\ClanHighlight`): 14 цветов (`gold, crimson, cyan, violet, emerald, rose, midnight, sunset, frost, mint, blood, lavender, chrome, toxic`) и 15 эффектов (`glow, pulse, gradient, fire, ice, aurora, legendary, neon, holo, storm, matrix, galaxy, lava, glitch, shine`); по умолчанию цвет `gold`, эффект `frame` (статичная рамка без свечения). Ключи совпадают с `effect_value` предметов магазина и с `frontend/src/data/clan/clanHighlight.js`.

Покупка: `POST /api/shop/{item}/purchase` (middleware `feature:shop`), только лидер, клановый предмет; оформление нельзя купить без активной подсветки (422 «Сначала купите подсветку клана»). Применение: `POST /api/shop/{item}/equip|unequip` — пишет `highlight_color`/`highlight_effect` и `equipped_at`. Скидка уровня применяется при оплате из казны. `is_highlighted` намеренно **не принимается** в `UpdateClanRequest` — это платная услуга, а не настройка.

`MyClanController::index` отдаёт `highlight_styles` из `ShopService::clanHighlightStyles($user)` — раздельно `colors` и `effects` с флагом `equipped`.

### 1.12 Объявление, лента, цели, pulse

MOTD (объявление руководства): `motd` ≤500, `motd_updated_at`, `motd_by`. Поля скрыты в модели (`$hidden`) и не попадают в публичную карточку — только в `overview`. `can_edit_motd = membership->can('news')`.

`GET /api/my-clan/overview` собирает:

| Блок | Содержимое |
|---|---|
| `motd` | текст, дата, карточка автора |
| `level` | уровень, xp, текущий/следующий порог, `perks`, `bonuses` (slots, member_cap, checkin_coins, treasury_discount) |
| `checkin` | `done`, `streak`, `today_count`, `members`, до 12 отметившихся |
| `goals` | недельные цели + `resets_at` |
| `top_members` | 5 лучших по `rating_points` (роль, тир) |
| `wars` | до 3 активных войн (`pending`/`accepted`), `participants_count`, соперник |
| `treasury` | баланс |
| `feed` | до 15 событий |

Недельные цели считаются от начала недели: отметки (цель `max(10, members * 3)`), победы в КВ (1), взносы в казну (500), новые темы клан-форума (3). Сброс — `startOfWeek()->addWeek()`.

Лента не имеет отдельного журнала — она собирается из существующих таблиц: вступления (`clan_members`), взносы (`clan_treasury_transactions`), итоги войн (`clan_wars` completed), темы (`clan_forum_topics`), новости (`clan_events`); по 5–6 записей на источник, сортировка по времени, лимит 15. Типы: `join`, `donation`, `war_win`, `war_loss`, `war_draw`, `topic`, `news`.

`GET /api/my-clan/pulse` — короткая сводка для меню: клан, роль, уровень, `checked_in`, `wars_active`, `applications` (только если `can('applications')`, иначе 0).

### 1.13 Медиа, теги, цвета, бан

- Аватар: `image`, jpg/jpeg/png/webp/gif, ≤2 МБ, хранится в `clans/{id}` на диске `public`.
- Обложка: `image`, jpg/jpeg/png/webp, ≤5 МБ, `clans/{id}/covers`, колонка `cover_path` (ключа `cover` в таблице нет — сервис его `unset`).
- Удаление медиа — `POST /api/clans/{clan}/avatar/remove` и `/cover/remove`, политика `removeMedia` (лидер), файл удаляется с диска.
- `banner_color` ≤16 символов, дефолт `#7c3aed`; пресеты на фронте: 8 цветов.
- Тег: 2–8 символов, unique, отображается как `[TAG]`; на фронте вводится в верхнем регистре.
- `socials` json: `discord`, `telegram`, `youtube`, `vk` (≤255), `website` (валидный URL); `@канал` превращается в ссылку на фронте (`utils/socialLinks.js`).
- Бан: `is_banned`, `ban_reason`; API только админское (см. 1.17). Бан не удаляет клан и не мешает `show`.

### 1.14 События клана и комментарии

| Параметр | Правило |
|---|---|
| `type` | `announcement`, `event`, `training` |
| `title` | ≤120 |
| `body` | ≤5000 |
| `starts_at` | любая дата (в отличие от войны, `after:now` нет) |
| Создание | `createEvent` — лидер/офицер; 422-подобный `abort_unless(..., 403)` с текстом «Создавать события может руководство клана.» |
| Удаление | автор **или** staff (политика `deleteEvent`) |
| Список | `GET /api/clans/{clan}/events`, пагинация 20, сортировка `starts_at desc` |

Комментарии (`clan_event_comments`) — двухуровневые: `list` отдаёт только корневые (`parent_id IS NULL`) с вложенными `replies`. Смотреть и писать может только участник клана (`ClanEventCommentService::assertAccess` → 403 «Комментарии доступны участникам клана.»), тело ≤1000, родитель обязан быть из этого же события (иначе 422), удаляет автор или staff.

### 1.15 Клановый форум

`/api/my-clan/forum*`, весь раздел под `clan.member`.

| Операция | Правило |
|---|---|
| Список | пагинация 20, сортировка `is_pinned DESC, last_reply_at DESC`, с автором и последним ответившим |
| Создание темы | `title` ≤160, `body` ≤10000, право `forum` |
| Ответ | `body` ≤5000, `parent_id` существует и принадлежит **этой** теме; закрытая тема → 422 «Топик закрыт.» |
| Просмотр | `views++` на каждый запрос (дедупликации нет, в отличие от общего форума) |
| Закрепление/закрытие | toggle `is_pinned`/`is_locked`, право `forum` |
| Удаление | политика `ClanForumTopicPolicy::delete`: автор или роль `leader`/`officer` |
| Счётчики | `replies_count` пересчитывается после каждого ответа, `last_reply_at`/`last_reply_user_id` обновляются |

`ClanForumService::assertBelongsToClan` (`abort_if($topic->clan_id !== $clan->id, 404)`) защищает от подмены темы из чужого клана.

### 1.16 Ресурсы клана

`/api/my-clan/resources`. Категории: `resource_pack`, `screenshot`, `config`, `guide`, `other`. Файл ≤50 МБ (51200 КБ), хранится в `clans/{id}/resources`. Список — 20 на страницу с фильтром `category`.

Скачивание: `POST /api/my-clan/resources/{resource}/download` увеличивает `downloads` и возвращает `file_url`. Удаление — политика `ClanResourcePolicy::delete`: автор или лидер.

Тонкость: политика смотрит `$user->clanMember?->role === 'leader'`, то есть роль в **своём** клане, а не в клане ресурса; от межклановой подмены защищает предшествующий `assertBelongsToClan` в контроллере.

### 1.17 Админский контур кланов

| Метод и путь | Что делает | Группа |
|---|---|---|
| `GET /api/admin/clans` | список (поиск по name/tag, `banned=1`, 30 на страницу) | `role:moderator,admin` |
| `POST /api/admin/clans/{clan}/stats` | дельта побед/поражений (`max(0, …)`), запись в `clan_stats_logs`, пересчёт силы | `role:moderator,admin` |
| `PUT /api/admin/clans/{clan}/stats` | точные значения `wins`/`losses` | `role:moderator,admin` |
| `GET /api/admin/clans/{clan}/stats/logs` | последние 50 записей журнала | `role:moderator,admin` |
| `POST /api/admin/clans/{clan}/avatar/remove`, `/cover/remove` | удаление медиа | `role:moderator,admin` |
| `POST /api/admin/clans/{clan}/ban` / `unban` | бан с причиной (дефолт «Нарушение правил») | `role:admin` |
| `DELETE /api/admin/clans/{clan}` | удаление клана | `role:admin` |
| `GET /api/admin/clan-events`, `DELETE /api/admin/clan-events/{event}` | модерация новостей кланов | `role:moderator,admin` |

Замечание по коду: `ClanStatsService::adjust` создаёт `ClanStatsLog` без `user_id`, хотя колонка `user_id` в миграции `NOT NULL` и поле есть в `$fillable`. На MySQL это упадёт на strict-режиме; на SQLite тесты проходят. В книге это стоит подать как известный долг.

# 2. Клановые войны (КВ)

## 2.1 Модель

`clan_wars`: `challenger_clan_id` (кто вызвал), `opponent_clan_id`, `created_by`, `status`, `scheduled_at`, `challenger_score`/`opponent_score` (tinyint, nullable), `winner_clan_id` (nullable), `outcome` (nullable, `win`/`draw`), `notes`.

Статусы: `pending → accepted → completed`, либо `pending → declined`; `cancelled` объявлен в enum, но **нигде не выставляется** — эндпоинта отмены нет.

Состав — отдельная таблица `clan_war_participants` (`clan_war_id`, `user_id`, `clan_id`), unique(`clan_war_id`,`user_id`). `clan_id` здесь важен: по нему считается сторона и победные очки рейтинга.

## 2.2 Объявление войны

`POST /api/clans/{clan}/wars` (и дубль `POST /api/my-clan/clans/{clan}/wars`), тело: `scheduled_at` (`after:now`, nullable), `notes` ≤1000.

`ClanWarService::challenge`:

- нужен свой клан (`myClan`, иначе 403 «Вы не в клане.»);
- нельзя вызвать свой клан (422);
- нужна роль `leader`/`officer` (`assertCanManage`, 403 «Нет прав.»);
- создаётся `pending` с `created_by`;
- лидеру соперника сразу уходит `ClanWarNotification('clan_war_challenge', "Клан [TAG] Name вызывает вас на клановую войну", warId, notes)`.

## 2.3 Принятие и отклонение

Оба действия доступны **только клану-оппоненту** и только руководству:

```
public function decline(User $user, ClanWar $war): void
{
    // Права нужны и здесь: раньше любой участник клана-оппонента
    // мог отклонить войну за руководство.
    $this->assertOpponentSide($user, $war);
    $this->assertCanManage($this->myClan($user), $user);
    $this->assertPending($war);

    $war->update(['status' => 'declined']);
}
```

`backend/app/Domains/Clan/Services/ClanWarService.php:135`

`accept` ставит `accepted` и возвращает войну с загруженными связями. Оба требуют `status === 'pending'`, иначе 422 «Война уже не в статусе ожидания.».

## 2.4 Призыв бойцов

`POST /api/wars/{war}/call` (throttle 10/мин), тело: `user_ids` (1–50), `message` ≤300.

```
$joined = ClanWarParticipant::where('clan_war_id', $war->id)->pluck('user_id')->all();

$recipients = User::query()
    ->whereIn('id', array_slice(array_unique(array_map('intval', $userIds)), 0, self::MAX_CALLED))
```

```
    ->whereIn('id', ClanMember::where('clan_id', $myClan->id)->select('user_id'))
    ->whereNotIn('id', [...$joined, $user->id])
    ->get();
```

backend/app/Domains/Clan/Services/ClanWarService.php:95

MAX_CALLED = 50. Призыв доступен только руководству и только пока война не завершена; уведомление clan_war_call содержит ник зовущего, тег и имя врага, время (d.m H:i) и заметку ≤300. Если звать некого — 422 «Некого звать: выбранные игроки уже записались или не состоят в клане.».

### 2.5 Состав

join (POST /api/wars/{war}/join): игрок должен быть в одном из двух кланов (иначе 403), война не завершена (иначе 422), ещё не записан (иначе 422); создаётся запись с clan_id своего клана.

leave (POST /api/wars/{war}/leave): тоже только для участников войны (раньше метод отвечал «ок» всем), удаляет запись.

Запись возможна и в pending, и в accepted — ограничения «только после принятия» в коде нет.

### 2.6 Результат

POST /api/wars/{war}/complete, тело: challenger_score 0–100, opponent_score 0–100, notes (nullable string без лимита).

```
$isDraw = $challengerScore === $opponentScore;

$winnerId = match (true) {
    $isDraw => null,
    $challengerScore > $opponentScore => $war->challenger_clan_id,
    default => $war->opponent_clan_id,
};

$war->update([
    ...$data, 'status' => 'completed',
    'winner_clan_id' => $winnerId,
    'outcome' => $isDraw ? 'draw' : 'win',
]);
```

backend/app/Domains/Clan/Services/ClanWarService.php:167

Далее в транзакции: победитель increment('wins'), проигравший — increment('losses') (при ничьей — никому); recalculatePower() обоим; ClanLevel::sync() обоим (победа даёт 100 XP и может поднять уровень); AchievementService::check() обоим лидерам.

Защита от повторного отчёта: если статус в ['completed','cancelled','declined'] → 422 «Война уже завершена.». Права: любой из двух кланов, роль leader/officer. Подтверждения от второй стороны нет — результат вносит та сторона, которая открыла форму.

### 2.7 Награды

Кроме зафиксированных в коде последствий — ничего:

- клан: wins/losses, пересчёт power, опыт и возможный уровень;
- игроки: очки рейтинга за участие (+3) и победу (+7) в **завершённых** войнах — но только тем, кто записан в clan_war_participants;
- достижения лидерам (через AchievementService).

Монет, предметов, ресурсов и записей в казну за войну **в коде нет**.

### 2.8 Просмотр

`GET /api/wars/{war}` — только для участников войны (403 «Вы не участвуете в этой войне.»); отдаёт `war` (со связями `challenger`, `opponent`, `participants.user`, `participants.clan`), `my_clan_id`, `is_participant`.

Публичная страница клана показывает войны через `ClanService::view`: два отдельных запроса `incoming_wars` (`opponent_clan_id`) и `outgoing_wars` (`challenger_clan_id`), только `pending`/`accepted`. Отдельный запрос нужен потому, что связь `Clan::wars()` описана только по `challenger_clan_id`.

Фронтенд: `frontend/src/components/clan/tabs/ClanWarsTab.vue` (вызов, принятие, отклонение, форма результата, призыв), `frontend/src/components/clan/ClanWarCard.vue` (карточка со счётом и составом), `frontend/src/composables/clan/useClanWar.js` (разбор стороны, счёта, ничьей; подписи статусов: «Ожидает», «Активна», «Отклонена», «Завершена», «Отменена»).

### 2.9 Инварианты, зафиксированные в сервисе

В шапке `ClanWarService` перечислены три дыры, закрытые выносом логики из контроллера: `decline`/`leave` не проверяли права, `complete` можно было вызвать повторно (двойные победы), ничья отдавала победу оппоненту (сравнение было строго `>`). Отсюда: `assertCanManage` в `decline`, проверка `FINISHED_STATUSES` в `complete`, отдельная колонка `outcome` (миграция `2026_10_01_120000`).

# 3. Форум

## 3.1 Таблицы

| Таблица | Поля | Ключи |
|---|---|---|
| forum_categories | slug 64 unique, name 96, description 255, icon 32, color 16, sort_order, post_policy 16 (all/verified/staff), is_active | индекс (is_active,sort_order) |
| forum_topics | title 200, slug 220, body, is_pinned, is_locked, last_reply_user_id, last_reply_at, views, replies_count, likes_count, deleted_by, deleted_at | category_id, author_id cascade; last_reply_user_id, deleted_by nullOnDelete; индексы (category_id,is_pinned,last_reply_at), (last_reply_at) |
| forum_replies | parent_id, body, likes_count, edited_at, deleted_by, deleted_at | topic_id, author_id cascade; parent_id nullOnDelete; индекс (topic_id,created_at) |
| forum_attachments | attachable_type 32 (topic/reply), attachable_id, original_name, path, mime, size, is_image | user_id cascade; полиморфная связь **без FK**, индекс (attachable_type,attachable_id) |
| forum_likes | likeable_type 32 (topic/reply), likeable_id | user_id cascade; unique(user_id,likeable_type,likeable_id), без FK на объект |

Мягкое удаление — свои колонки deleted_at/deleted_by, без SoftDeletes; видимость задаёт scope visible() (whereNull('deleted_at')).

## 3.2 Разделы

GET /api/forum — публичный, отдаёт categories + stats. Активные разделы сортируются sort_order, затем name. Для каждого считается topics_count, replies_count (через whereHas('topic')), can_post, policy_label, и последняя тема раздела (last_reply_at или created_at, автор последнего ответа или автор темы).

Право писать в раздел:

```
public function allowsPosting(?User $user): bool
{
    if (! $user) {
        return false;
    }

    return match ($this->post_policy) {
        'verified' => (bool) $user->is_verified,
        'staff' => $user->isModerator() || $user->isTester(),
        default => true,
    };
}
```

backend/app/Domains/Forum/Models/ForumCategory.php:35

stats: число видимых тем, видимых ответов, всех игроков и последняя тема. Разделы создаются только из админки; в пользовательском API их CRUD отсутствует.

### 3.3 Темы

`GET /api/forum/topics` — фильтры `category` (slug, `firstOrFail` → 404 на неизвестный), `search` (по `title`/`body`), `sort` (`activity` default, `new`, `popular`, `unanswered`), `per_page` 5–50 (default 20). После сортировки всегда `orderByDesc('is_pinned')` — закреплённые сверху.

Создание (`POST /api/forum/topics`, middleware `feature:forum_posting`):

- `category_id` существует; раздел активен, иначе `ValidationException` «Раздел закрыт.»;
- `allowsPosting` иначе **403** «В этом разделе нельзя создавать темы.» (это отказ в доступе, а не ошибка данных);
- `title` 4–200, `body` 4–30000;
- `slug = Str::slug(title) . '-' . Str::lower(Str::random(5))`;
- сразу `last_reply_at = now()`, `last_reply_user_id = author`;
- `attachments` — до 10 id, привязываются `ForumAttachment::attach($author, 'topic', $id, $ids)`, что обновляет только чужие-непривязанные (`user_id = author` и `attachable_id IS NULL`) — подставить чужой файл нельзя.

Правка (`PUT /api/forum/topics/{topic}`): политика `update`, затем окно 24 часа (`assertWithinEditWindow`), поля `title`/`body` `sometimes`.

Удаление (`DELETE`): политика `delete`, мягкое удаление (`deleted_by`, `deleted_at: now()`).

### 3.4 Ответы и дерево

`POST /api/forum/topics/{topic}/reply` (`feature:forum_posting`), `body` 1–20000, `parent_id` nullable exist.

- тема удалена → 404;
- тема закрыта → `ValidationException` по полю `body`: «Тема закрыта — отвечать нельзя.» (422, не 403);
- родитель обязан быть из этой же темы и вложенность не выше предела:

```
if (! $parent || $parent->topic_id !== $topic->id) {
    throw ValidationException::withMessages([
        'parent_id' => ['Можно отвечать только на сообщения из этой темы.'],
    ]);
}

// Уровни считаются от 0, поэтому ответ на глубине MAX_DEPTH уже не помещается
if ($parent->depth() + 1 >= ForumReply::MAX_DEPTH) {
    throw ValidationException::withMessages([
        'parent_id' => ['Достигнута максимальная вложенность — ответь в начало ветки.'],
    ]);
}
```

`backend/app/Domains/Forum/Services/ForumService.php:422`

`MAX_DEPTH = 8`. `depth()` идёт по цепочке родителей с защитой от цикла (лимит 64 шага). После ответа пересчитывается `replies_count` (только видимые), обновляются `last_reply_at`/`last_reply_user_id`.

Просмотр темы (`GET /api/forum/topics/{topic}`) отдаёт `topic` (ресурс), `replies` (дерево), `replies_count`, `max_depth`, `can_reply`. Дерево строит `ForumReplyTree`: плоский список

превращается в `children`; узел, чей родитель не входит в выборку, считается корневым, чтобы ветка не терялась; `depth` проставляется при сборке.

Удаление ответа каскадно по ветке:

```
$ids = $reply->descendantIds();
$ids[] = $reply->id;

ForumReply::whereIn('id', $ids)->update([
    'deleted_by' => $user->id,
    'deleted_at' => now(),
]);

$topic = $reply->topic;
if ($topic) {
    $topic->update([
        'replies_count' => ForumReply::where('topic_id', $topic->id)->visible()->count(),
    ]);
}

return count($ids);
```

`backend/app/Domains/Forum/Services/ForumService.php:336`

`descendantIds()` обходит потомков стеком (без рекурсии). Ответ возвращает `removed` — сколько сообщений скрыто.

### 3.5 Права и правка

```
public function update(User $user, ForumTopic $topic): bool
{
    return $topic->author_id === $user->id || $user->isModerator();
}

public function delete(User $user, ForumTopic $topic): bool
{
    return $topic->author_id === $user->id || $user->isModerator();
}

/** Создавать темы может любой авторизованный: раздел решает сам. */
public function create(User $user): bool
{
    return true;
}
```

`backend/app/Domains/Forum/Policies/ForumTopicPolicy.php:16`

`ForumReplyPolicy` — зеркальный (`author || moderator`). `isModerator()` — роль `moderator` (админ проходит по `HasRoles`).

Окно правки — бизнес-правило, поэтому 422, а не 403:

```
if ($authorId !== $user->id || $user->isModerator()) {
    return;
}

abort_if(
    $createdAt && $createdAt->diffInHours(now()) > 24,
    422,
    "Редактировать {$what} можно в течение 24 часов."
);
```

`backend/app/Domains/Forum/Services/ForumService.php:288`

Ответ при правке получает `edited_at = now()` (у темы отдельного поля `edited_at` нет).

### 3.6 Модерация

Все маршруты — `/api/admin/forum/*`, группа `role:moderator,admin`:

| Метод и путь | Действие |
|---|---|
| `GET /api/admin/forum/stats` | разделы, видимые/удалённые темы, ответы, закрытые, закреплённые, вложения |
| `GET/POST /api/admin/forum/categories`, `PUT/DELETE .../{category}` | CRUD разделов; `slug` генерируется из `name`, если не задан |
| `DELETE /api/admin/forum/categories/{category}` | при наличии тем возвращает **409** со счётчиком и требует `force=1` |
| `GET /api/admin/forum/topics` | список с фильтрами `category_id`, `only_deleted=1`, `search`, 30 на страницу |
| `POST /api/admin/forum/topics/{topic}/pin` | toggle `is_pinned` |
| `POST /api/admin/forum/topics/{topic}/lock` | toggle `is_locked` |
| `DELETE /api/admin/forum/topics/{topic}` | мягкое удаление с `deleted_by` |
| `POST /api/admin/forum/topics/{topic}/restore` | снятие `deleted_at`/`deleted_by` |
| `DELETE /api/admin/forum/replies/{reply}` | мягкое удаление + пересчёт `replies_count` |

Важно: закрепление и закрытие тем **недоступны автору** — только модератору. Автор может править (24 ч) и удалять свою тему.

### 3.7 Счётчики

- `views` — уникальный просмотр раз в сутки с одного IP, через кэш (у публичного API нет сессии):

```
$key = 'forum_viewed:' . $topic->id . ':' . sha1((string) $ip);

if (! Cache::has($key)) {
    $topic->increment('views');
    Cache::put($key, true, now()->addDay());
}
```

`backend/app/Domains/Forum/Services/ForumService.php:444`

- `replies_count` — пересчитывается по видимым ответам после добавления/удаления;
- `likes_count` — инкремент/декремент в `ForumLike::toggle`, счётчик не может уйти ниже нуля (`max(0, …)` при чтении);
- `last_reply_at`/`last_reply_user_id` — обновляются при ответе; при создании темы ставятся на автора.

### 3.8 Вложения, лайки, поиск, уведомления

**Вложения.** Загрузка «про запас»: `POST /api/forum/upload` (throttle 30/мин), ≤10 МБ, mime-белый список: jpg/jpeg/png/webp/gif/pdf/txt/md/zip/rar/7z/json/log/cfg/yml/yaml. Путь — `forum/YYYY/MM` на диске `public`. `is_image` = mime начинается с `image/`. Скачивание — публичный `GET /api/forum/attachments/{attachment}/download` (проверка наличия файла на диске → 404). Максимум 10 вложений на тему/ответ (валидация `attachments` массива).

**Лайки.** `POST /api/forum/like/{type}/{id}`, `type ∈ {topic, reply}`; toggle возвращает `{liked, count}`; unique-индекс не даёт лайкнуть дважды.

**Поиск.** Публичный `GET /api/forum/search?q=` (2–120 символов), до 20 тем и 20 ответов по `LIKE`, темы — по `last_reply_at desc`, ответы — `latest()`.

**Уведомления.** При ответе уведомляются автор ответа-родителя (`forum_reply_to_you`) и автор темы (`forum_reply`), но не сам отвечающий и без дублей: если по этой теме уже есть непрочитанное уведомление того же типа — новое не отправляется.

# 4. Друзья

## 4.1 Таблица

`friendships`: `user_id`, `friend_id` (оба → `users`, cascade), `status` enum `pending/accepted/blocked`, default `pending`; unique(`user_id`,`friend_id`). Связь направленная: строка (`A → B`) означает «A отправил заявку B».

## 4.2 Заявки, принятие, удаление

`GET /api/friends` — три списка одним ответом:

| Поле | Выборка |
|---|---|
| `friends` | `status = accepted` и (`user_id = me` или `friend_id = me`), в ответе — **другая** сторона пары |
| `incoming_requests` | `friend_id = me`, `status = pending`, `latest()` |
| `outgoing_requests` | `user_id = me`, `status = pending`, `latest()` |

`POST /api/friends/{user}` — заявка: нельзя себе (422 «Нельзя добавить себя.»); если любая связь в любую сторону уже есть → 422 «Заявка уже существует.» (повторно отправить после отказа нельзя — строку надо удалить); создаётся `pending`; получателю уходит `FriendRequestNotification`.

Принятие — только получателем, и это выражено самой выборкой:

```
public function accept(User $me, User $sender): Friendship
{
    $friendship = Friendship::where('user_id', $sender->id)
        ->where('friend_id', $me->id)
        ->where('status', 'pending')
        ->firstOrFail();

    $friendship->update(['status' => 'accepted']);

    AchievementService::check($me);
    AchievementService::check($sender);

    $sender->notify(new FriendAcceptedNotification($me));

    return $friendship;
}
```

`backend/app/Domains/Friends/Services/FriendService.php:86`

Если входящей заявки нет — 404 (`firstOrFail`). Принять встречную «исходящую» нельзя — только удалить и отправить заново.

Удаление (`DELETE /api/friends/{user}`) одно на все случаи — удаление друга, отклонение входящей, отмена исходящей: `between($a,$b)->delete()`, где `between` ищет связь в обе стороны. Поэтому отклонение заявки на фронте — это тот же `DELETE`, что и «удалить из друзей».

## 4.3 Права, лимиты, блокировки

- Политик у домена нет; правки чужой связи нет по конструкции — все операции идут от текущего пользователя, а `accept` жёстко фильтрует по `friend_id = me`.

- Лимита друзей и лимита исходящих заявок в коде **нет** (ни в `FriendService`, ни в миграции, ни в маршрутах).
- Пагинации нет — списки отдаются целиком.
- Статус `blocked` объявлен в enum, но **нигде не выставляется**: кнопки/эндпоинта блокировки не существует.
- Уведомления — только `database`; у `FriendAcceptedNotification` в `toArray` стоит `'type' => 'friend_request'` (тот же тип, что у заявки) — фактическая неточность в коде.

### 4.4 Онлайн-статус и Reverb

Бэкенд в ответе `/api/friends` **не отдаёт** онлайн-статус. Он считается на клиенте композаблом `frontend/src/composables/chat/presence.js` из двух источников:

1. **Presence-канал Reverb** — основной: `window.Echo.join('online')`, события `here`/`joining`/`leaving` дают мгновенный вход и выход.
2. **Серверный fallback** — `GET /api/chat/presence`, `POST /api/chat/presence/online` (пульс раз в 30 с), `POST /api/chat/presence/offline`; полный список обновляется раз в 60 с.

```
channel = window.Echo.join('online')

channel.here((members) => {
  const ids = toSet(members.map((member) => member.id))
  channelMembers.value = ids
  channelReady.value = true
})

channel.leaving((member) => {
  const id = Number(member.id)
  ...
  withLeft((left) => left.set(id, Date.now()))
})
```

`frontend/src/composables/chat/presence.js:120`

Композиция источников: `onlineIds = channelMembers ∪ serverOnline`, но вышедший из канала не берётся из серверного списка ещё 90 с (`LEFT_SUPPRESS_MS`) — чтобы точка гасла мгновенно, а не ждала обновления сервера. Вход в канал делает всё приложение (`startSitePresence` из `AppLayout`), а не только страница чата — иначе игрок на главной не существовал бы для канала.

`FriendsView.vue` использует `isOnline(user.id)`: сортирует список «кто в сети — первыми» и показывает в карточке «В сети / Не в сети».

# 5. Эндпоинты

## 5.1 Публичные (без авторизации)

| Метод | Путь | Что делает |
|---|---|---|
| GET | `/api/clans` | список кланов: `search`, `sort`, `open`, `has_slots`, `page`; 20 на страницу |
| GET | `/api/clans/top` | топ-10 по силе |
| GET | `/api/clans/{clan}` | профиль клана по id **или** имени (пробелы/подчёркивания, регистр не важны); состав, события, войны, своя заявка |
| GET | `/api/forum` | разделы + статистика форума |
| GET | `/api/forum/topics` | темы: `category`, `search`, `sort`, `per_page`, `page` |
| GET | `/api/forum/search` | поиск `q` (2–120) по темам и ответам |
| GET | `/api/forum/topics/{topic}` | тема + дерево ответов, +1 просмотр (раз в сутки с IP) |
| GET | `/api/forum/attachments/{attachment}/download` | скачивание вложения |

## 5.2 Кланы и войны (auth:sanctum)

| Метод | Путь | Что делает | Право |
|---|---|---|---|
| POST | `/api/clans` | создание (`feature:clan_creation`) | любой без клана |
| PUT/POST | `/api/clans/{clan}` | правка профиля, аватар, обложка, соцсети, `entry_fee` | лидер |
| POST | `/api/clans/{clan}/apply` | заявка (`message` ≤500) | любой без клана, клан открыт |
| GET | `/api/clans/{clan}/applications` | очередь `pending` | лидер/офицер |
| POST | `/api/clans/{clan}/applications/{application}/accept` | приём (плата, лимит, транзакция) | лидер/офицер |
| POST | `/api/clans/{clan}/applications/{application}/decline` | отказ | лидер/офицер |
| POST | `/api/clans/{clan}/leave` | выход; лидер-одиночка распускает клан | участник |
| DELETE | `/api/clans/{clan}/members/{user}` | кик | лидер |
| POST | `/api/clans/{clan}/avatar/remove` | удалить аватар | лидер |
| POST | `/api/clans/{clan}/cover/remove` | удалить обложку | лидер |

| | | | |
|---|---|---|---|
| GET | `/api/clans/{clan}/events` | события клана, 20 на страницу | auth |
| POST | `/api/clans/{clan}/events` | создать событие | лидер/офицер |
| DELETE | `/api/clans/{clan}/events/{event}` | удалить событие | автор или staff |
| POST | `/api/clans/{clan}/wars` | вызвать клан на войну | лидер/офицер своего клана |
| GET | `/api/wars/{war}` | детали войны | участник войны |
| POST | `/api/wars/{war}/join` | записаться в состав | участник клана-стороны |
| POST | `/api/wars/{war}/leave` | выйти из состава | участник клана-стороны |
| POST | `/api/wars/{war}/call` | призыв до 50 бойцов (throttle 10/мин) | лидер/офицер |
| POST | `/api/wars/{war}/accept` | принять вызов | лидер/офицер клана-оппонента |
| POST | `/api/wars/{war}/decline` | отклонить вызов | лидер/офицер клана-оппонента |
| POST | `/api/wars/{war}/complete` | внести результат | лидер/офицер клана-стороны |
| GET | `/api/clans/{clan}/events/{event}/comments` | комментарии события | участник клана |
| POST | `/api/clans/{clan}/events/{event}/comments` | комментарий (`body` ≤1000, `parent_id`) | участник клана |
| DELETE | `/api/clans/{clan}/events/{event}/comments/{comment}` | удалить комментарий | автор/лидер/офицер |

## 5.3 «Мой клан» (auth:sanctum + clan.member)

| Метод | Путь | Что делает | Доп. право |
|---|---|---|---|
| GET | `/api/my-clan` | дашборд: клан, роль, `my_permissions`, статистика, `highlight_styles` | — |
| GET | `/api/my-clan/overview` | обзор: MOTD, отметки, уровень, цели, топ, войны, казна, лента | — |
| GET | `/api/my-clan/pulse` | сводка для меню | — |
| POST | `/api/my-clan/checkin` | отметка (throttle 10/мин) | — |
| PUT | `/api/my-clan/motd` | объявление ≤500 (throttle 20/мин) | `news` |
| GET | `/api/my-clan/treasury` | баланс, вкладчики, история | — |
| POST | `/api/my-clan/treasury/donate` | взнос 1–100000 (throttle 20/мин) | — |
| GET | `/api/my-clan/forum` | темы клан-форума (20) | — |
| POST | `/api/my-clan/forum` | создать тему | `forum` |

| | | | |
|---|---|---|---|
| GET | `/api/my-clan/forum/{topic}` | тема + ответы, +1 просмотр | — |
| POST | `/api/my-clan/forum/{topic}/reply` | ответ | — |
| POST | `/api/my-clan/forum/{topic}/pin` | закрепить/снять | `forum` |
| POST | `/api/my-clan/forum/{topic}/lock` | закрыть/открыть | `forum` |
| DELETE | `/api/my-clan/forum/{topic}` | удалить тему | автор/лидер/офицер |
| GET | `/api/my-clan/resources` | ресурсы (20, фильтр `category`) | — |
| POST | `/api/my-clan/resources` | загрузка ≤50 МБ | `resources` |
| POST | `/api/my-clan/resources/{resource}/download` | ссылка + `downloads++` | — |
| DELETE | `/api/my-clan/resources/{resource}` | удалить | автор/лидер |
| POST | `/api/my-clan/clans/{clan}/wars` | дубль вызова войны | лидер/офицер |
| POST | `/api/my-clan/wars/{war}/accept` | дубль принятия | лидер/офицер |
| POST | `/api/my-clan/wars/{war}/decline` | дубль отклонения | лидер/офицер |
| POST | `/api/my-clan/wars/{war}/complete` | дубль результата | лидер/офицер |
| POST | `/api/my-clan/roles/{user}` | роль/права/титул (`role ∈ officer,member`) | лидер |
| DELETE | `/api/my-clan/members/{user}` | кик | лидер |
| POST | `/api/my-clan/transfer/{user}` | передача лидерства | лидер |

## 5.4 Форум (auth:sanctum)

| Метод | Путь | Что делает | Право |
|---|---|---|---|
| POST | `/api/forum/topics` | создать тему (`feature:forum_posting`) | политика `create` + `allowsPosting` |
| PUT | `/api/forum/topics/{topic}` | правка (24 ч) | автор/модератор |
| DELETE | `/api/forum/topics/{topic}` | мягкое удаление | автор/модератор |
| POST | `/api/forum/topics/{topic}/reply` | ответ (`feature:forum_posting`) | auth, тема не закрыта |
| PUT | `/api/forum/replies/{reply}` | правка (24 ч, `edited_at`) | автор/модератор |
| DELETE | `/api/forum/replies/{reply}` | удаление ветки | автор/модератор |
| POST | `/api/forum/like/{type}/{id}` | лайк/анлайк (`topic`/`reply`) | auth |

| | | | |
|---|---|---|---|
| POST | `/api/forum/upload` | вложение ≤10 МБ (throttle 30/мин) | auth |

## 5.5 Друзья (auth:sanctum)

| Метод | Путь | Что делает |
|---|---|---|
| GET | `/api/friends` | друзья + входящие + исходящие |
| POST | `/api/friends/{user}` | заявка |
| POST | `/api/friends/{user}/accept` | принять входящую (только получатель) |
| DELETE | `/api/friends/{user}` | удалить/отклонить/отменить |

## 5.6 Админка

| Метод | Путь | Группа |
|---|---|---|
| GET | `/api/admin/clans` | `role:moderator,admin` |
| POST / PUT | `/api/admin/clans/{clan}/stats` | `role:moderator,admin` |
| GET | `/api/admin/clans/{clan}/stats/logs` | `role:moderator,admin` |
| POST | `/api/admin/clans/{clan}/avatar/remove`, `/cover/remove` | `role:moderator,admin` |
| POST | `/api/admin/clans/{clan}/ban`, `/unban` | `role:admin` |
| DELETE | `/api/admin/clans/{clan}` | `role:admin` |
| GET | `/api/admin/forum/stats` | `role:moderator,admin` |
| GET/POST/PUT/DELETE | `/api/admin/forum/categories…` | `role:moderator,admin` |
| GET | `/api/admin/forum/topics` | `role:moderator,admin` |
| POST | `/api/admin/forum/topics/{topic}/pin`, `/lock` | `role:moderator,admin` |
| DELETE | `/api/admin/forum/topics/{topic}`, `/api/admin/forum/replies/{reply}` | `role:moderator,admin` |
| POST | `/api/admin/forum/topics/{topic}/restore` | `role:moderator,admin` |
| GET/DELETE | `/api/admin/clan-events`, `/api/admin/clan-events/{event}` | `role:moderator,admin` |

# 6. Каскады и целостность

Что удаляется вместе с кланом (`clan->delete()`): `clan_members`, `clan_applications`, `clan_wars` (обе стороны), `clan_war_participants`, `clan_events`, `clan_event_comments` (через события), `clan_forum_topics`/`clan_forum_replies`, `clan_resources`, `clan_checkins`, `clan_treasury_transactions`, `clan_inventory`, `clan_stats_logs`. Руками добираются только `users.clan_joined_at` и `users.favorite_clan_id`.

Важные следствия схемы:

1. `clans.leader_id` — `cascadeOnDelete`: удаление пользователя-лидера удаляет клан целиком, со всем содержимым.
2. `clan_war_participants.user_id` — cascade: удаление игрока убирает его из составов и обнуляет его вклад в рейтинг войны.
3. `clan_wars.winner_clan_id` — `nullOnDelete`: после удаления клана-победителя война остаётся, `winner_clan_id = null`, и в ленте она читается как ничья (`winner_clan_id === null => 'draw'`).
4. `forum_attachments.attachable_*` — без внешнего ключа: удаление темы не удаляет файлы; строки остаются с висящими id.
5. `forum_likes` — без FK на лайкаемый объект: удаление темы оставляет лайки.
6. `forum_replies.parent_id` — `nullOnDelete`: удаление родителя физически (не мягко) сделало бы детей корневыми; поэтому удаление всегда мягкое и с обходом потомков.
7. `clan_checkins` unique(`user_id`,`day`) — межклановый: вступив в другой клан, за тот же день отметиться нельзя.
8. `clan_members` unique(`user_id`) — жёсткая гарантия «один клан на игрока»; `ClanService::create/apply/accept` дополнительно проверяют это в коде.

# 7. Все политики

| Политика | Метод | Правило | Где вызывается |
|---|---|---|---|
| `ClanPolicy` | `update` | `$clan->isLeader($user->id)` | `ClanController::update` |
| `ClanPolicy` | `removeMedia` | лидер клана | `removeAvatar`, `removeCover` |
| `ClanPolicy` | `reviewApplications` | членство с ролью `leader`/`officer` | список/приём/отказ заявок |
| `ClanPolicy` | `kick` | лидер клана | `ClanController::kick`, `ClanRoleController::kick` |
| `ClanPolicy` | `manageRoles` | лидер клана | смена роли, передача лидерства |
| `ClanPolicy` | `createEvent` | `leader`/`officer` | `ClanEventController::store` |
| `ClanPolicy` | `deleteEvent` | автор **или** `leader`/`officer` | `ClanEventController::destroy` |
| `ClanResourcePolicy` | `delete` | `author_id === user->id` **или** `user->clanMember?->role === 'leader'` | `ClanResourceController::destroy` |
| `ClanForumTopicPolicy` | `delete` | автор **или** роль `leader`/`officer` в своём клане | `ClanForumController::destroy` |
| `ForumTopicPolicy` | `update`, `delete` | автор **или** `isModerator()` | правка/удаление темы |
| `ForumTopicPolicy` | `create` | `true` (авторизован) | создание темы (раздел решает `post_policy`) |
| `ForumReplyPolicy` | `update`, `delete` | автор **или** `isModerator()` | правка/удаление ответа |

Всё остальное в доменах закрыто не политиками, а сервисными проверками: `ClanWarService::assertCanManage`/`assertOpponentSide`/`assertParticipatingClan`, `ClanEventCommentService::assertAccess`/`isStaff`, `ClanService::assertApplicationBelongsToClan`, `ClanForumService::assertBelongsToClan`, `ClanResourceService::assertBelongsToClan`, middleware `EnsureClanMember` и группы `role:*`.

## 8. Чего в коде нет

- **Приглашений в клан** — только заявки; закрытый клан (`is_open = false`) вступить нельзя никак.
- **Наград за войны** — ни монет, ни предметов, ни ресурсов; только `wins/losses`, сила, опыт/уровень, очки рейтинга участникам, достижения лидерам.
- **Отмены войны** — статус `cancelled` объявлен, но не выставляется.
- **Лимита друзей** и пагинации друзей.
- **Блокировки пользователей** — статус `blocked` в enum есть, логики нет.
- **Вывода средств из казны** — только трата на клановые предметы лидером.
- **Настройки `max_members`** — лимит только через уровень клана; поле не принимается API.
- **Отдельного журнала ленты клана** — лента собирается из существующих таблиц на лету.
- **Дедупликации просмотров клан-форума** — в отличие от общего форума, `views++` на каждый запрос.
- **Публичного списка войн клана** — войны видны только участникам через `GET /api/wars/{war}` и на странице клана в блоках `incoming_wars`/`outgoing_wars` (только `pending`/`accepted`).

# Экономика ApexCoin и управление функциями: домены Shop, Wallet, Control

## 1. Валюта и два места хранения баланса

ApexCoin — единственная внутренняя валюта проекта. Метаданные валюты лежат в конфиге и не редактируются из админки:

```
// backend/config/apex.php
'currency' => [
    'code' => 'APEX',
    'name' => 'ApexCoin',
    'symbol' => '₳',
],
```

Баланс хранится **денормализованно в таблице `users`** — двумя счётчиками:

| Поле | Тип | Назначение |
|---|---|---|
| `users.apex_coins` | `unsignedBigInteger`, default `0` | Текущий доступный баланс |
| `users.apex_coins_spent` | `unsignedBigInteger`, default `0` | Накопительная сумма всех списаний |

```
// backend/database/migrations/2026_09_27_100000_add_apex_coins_to_users_table.php
Schema::table('users', function (Blueprint $table) {
    // Баланс ApexCoin. Хранится денормализовано ради быстрых выборок;
    // истина — сумма ledger-а coin_transactions (см. App\Domains\Wallet\Services\CoinService).
    $table->unsignedBigInteger('apex_coins')->default(0);
    $table->unsignedBigInteger('apex_coins_spent')->default(0);
});
```

Дублирование нужно потому, что баланс читается на каждом запросе (витрина, профиль, `auth.user.apex_coins` на фронтенде), а леджер растёт неограниченно: `SUM(amount)` по `coin_transactions` на каждое чтение — это full scan по истории игрока. Поэтому быстрый ответ даёт колонка, а правильность — запись в леджер в той же транзакции.

Колонки `unsigned` — это нижняя граница защиты: отрицательный баланс нельзя записать даже прямым SQL, если не отключить строгий режим БД.

#### 1.1 Леджер `coin_transactions`

| Поле | Тип | Смысл |
|---|---|---|
| `id` | bigint PK | |
| `user_id` | FK `users`, `cascadeOnDelete` | Владелец операции |
| `amount` | `bigInteger`, со знаком | `+` начисление, `-` списание |
| `balance_after` | `bigInteger`, nullable | Баланс после операции (цепочка) |
| `source` | `string(32)` | Источник: `tier_test`, `purchase`, `admin`, … |
| `description` | `string(191)`, nullable | Человекочитаемая причина |
| `reference_type` / `reference_id` | `string(64)` / `unsignedBigInteger` | Полиморфная привязка (`shop_item`, `tier_test`, …) |
| `idempotency_key` | `string(191)`, nullable, **unique** | Ключ однократности начисления |
| `actor_id` | FK `users`, `nullOnDelete` | Кто инициировал (админ, |

| | | |
|---|---|---|
| | | даритель) |
| `meta` | `json`, nullable | Доп. данные: `slug`, `quantity`, `fee`, `multiplier` |
| `created_at` / `updated_at` | timestamps | |

Индексы: `(user_id, created_at)` — история игрока, `(source, created_at)` — админская аналитика по источникам.

`balance_after` — не декоративное поле: оно позволяет восстановить цепочку и проверить, что денормализованный баланс не разошёлся с леджером. Тест `WalletTest::test_ledger_balance_after_chain_matches_user_balance` проходит по строкам в порядке `id` и требует, чтобы накопленная сумма совпадала и с `balance_after`, и с `users.apex_coins`.

#### 1.2 Источники начисления

Константы объявлены в модели:

```
// backend/app/Domains/Wallet/Models/CoinTransaction.php
public const SOURCE_TIER_TEST = 'tier_test';
public const SOURCE_ACHIEVEMENT = 'achievement';
public const SOURCE_DAILY_BONUS = 'daily_bonus';
public const SOURCE_GIFT_IN = 'gift_in';
public const SOURCE_GIFT_OUT = 'gift_out';
public const SOURCE_PURCHASE = 'purchase';
public const SOURCE_ADMIN = 'admin';
public const SOURCE_REFERRAL = 'referral';
public const SOURCE_CLAN_FEE = 'clan_fee';
public const SOURCE_CLAN_TREASURY = 'clan_treasury';
public const SOURCE_OTHER = 'other';
```

Плюс `SOURCE_CASE = 'case'`. Фактические точки записи (по вызовам `CoinService::credit/debit` в коде):

| Источник | Кто пишет | Что именно |
|---|---|---|
| `tier_test` | `RewardService::forTierTest()` | Награда за пройденный тир-тест |
| `achievement` | `RewardService::forAchievement()` | Награда за ачивку |
| `daily_bonus` | `CoinService::claimDailyBonus()` | Ежедневный вход |
| `referral` | `RewardService::forReferral()`, `welcomeBonus()` | Пригласившему и приглашённому |
| `gift_in` / `gift_out` | `CoinService::gift()` | Подарки монет друзьям |
| `purchase` | `ShopService::purchase()`, `BundleService::purchase()`, `MinerProgress` | Покупки |
| `case` | `LootCaseService::open()` | Открытие сундука и монеты-утешители |
| `admin` | `Admin\ShopController::grantCoins()`, `GiftService::giveaway()`, `redeem()`, `apex:coins` | Ручные начисления, раздачи, промокоды |
| `clan_fee` / `clan_treasury` | `ClanService`, `ClanTreasuryService` | Вступление в клан, взносы в казну |
| `other` | `InventoryTradeService`, `PetAuctionService`, `GuideService`, `LootCaseService` (набор монет), | Продажа предметов, аукцион, путеводитель, питомцы, шахта, недельные награды |

| | | |
|---|---|---|
| | `PetArenaService`, `PetExpeditionService`, `PetHubService`, `RespectWeeklyRewardsCommand`, `MinerService` | |

В `config/apex.php` перечислены «переключаемые» источники: `tier_test`, `achievement`, `daily_bonus`, `admin`, `tournament`, `clan_war`, `referral`. Ключи `tournament` и `clan_war` **в коде отсутствуют** — ни одна ветка не пишет такие `source`; это заготовка под будущие награды турниров и клановых войн.

## 2. `CoinService` — единственная точка изменения баланса

Сервис объявляет контракт в шапке: любая операция идёт в транзакции с блокировкой строки пользователя, каждая пишет запись в леджер, ключ идемпотентности отсекает повтор.

#### 2.1 `credit()`: порядок шагов

1. `$amount === 0` → `return 0` (нулевых записей в леджере не бывает).
2. Если передан `idempotency_key` и строка с ним уже есть → `return 0`.
3. Если начисление положительное и источник входит в `BOOSTED_SOURCES` → умножение на действующий множитель события.
4. `DB::transaction`: `SELECT ... FOR UPDATE` (только mysql/pgsql/sqlsrv) → проверка неотрицательности → запись `users` → запись `coin_transactions` → синхронизация переданного инстанса `User`.

```
// backend/app/Domains/Wallet/Services/CoinService.php, строки 77–124
return DB::transaction(function () use ($user, $amount, $source, $description, $idempotencyKey,
$options) {
    $query = User::whereKey($user->id);

    // SQLite не умеет SELECT ... FOR UPDATE, поэтому блокировку строки
    // применяем только на драйверах, которые её поддерживают
    if (self::supportsRowLock()) {
        $query->lockForUpdate();
    }

    /** @var User $locked */
    $locked = $query->firstOrFail();
    $balance = (int) $locked->apex_coins;

    if ($amount < 0 && $balance + $amount < 0) {
        throw new RuntimeException('Недостаточно ApexCoin на балансе.');
    }

    $balance += $amount;
    $locked->apex_coins = $balance;
    if ($amount < 0) {
        $locked->apex_coins_spent = (int) $locked->apex_coins_spent + abs($amount);
    }
    $locked->save();
```

```
// backend/app/Domains/Wallet/Services/CoinService.php, строки 105–121
    CoinTransaction::create([
        'user_id' => $locked->id,
        'amount' => $amount,
        'balance_after' => $balance,
        'source' => $source,
        'description' => $description,
        'reference_type' => self::morphType($options['reference_type'] ?? null),
        'reference_id' => $options['reference_id'] ?? null,
```

```
            'idempotency_key' => $idempotencyKey,
            'actor_id' => $options['actor_id'] ?? null,
            'meta' => $options['meta'] ?? null,
        ]);

        // Синхронизируем переданный извне инстанс, чтобы вызывающий код видел новый баланс
        $user->setAttribute('apex_coins', $balance);
        $user->setAttribute('apex_coins_spent', $locked->apex_coins_spent);
        $user->syncOriginalAttributes(['apex_coins', 'apex_coins_spent']);

        return $amount;
    });
```

`morphType()` вручную приводит класс к псевдониму `morphMap` (`ShopItem::class` → `shop_item`), потому что поле пишется напрямую и Eloquent карту не применяет. Без этого в базе оказались бы FQCN, ломающиеся при переносе моделей.

`debit()` — обёртка: `credit($user, -abs($amount), ...)`. То есть списание идёт тем же путём, с той же проверкой баланса и той же записью в леджер, только со знаком минус.

`supportsRowLock()` возвращает `true` только для `mysql`, `pgsql`, `sqlsrv`. В `.env` проекта `DB_CONNECTION=sqlite`, то есть **в текущей конфигурации блокировка строки не выполняется** — остаются транзакция и уникальные индексы.

#### 2.2 Идемпотентность

Механика двухслойная:

- **Предпроверка** до транзакции: `CoinTransaction::where('idempotency_key', $key)->exists()` → `return 0`.
- **Уникальный индекс** `coin_transactions.idempotency_key` как последний барьер.

Ключи, которые реально формирует код:

| Ключ | Где | Смысл |
|---|---|---|
| `tier_test:{tierTestId}` | `RewardService::forTierTest` | Одна награда на тест |
| `tier_test_first:{userId}` | `RewardService::forTierTest` | Бонус за первую заявку в жизни |
| `achievement:{userId}:{achievementId}` | `RewardService::forAchievement` | Одна награда на ачивку |
| `referral:{invitedId}` | `RewardService::forReferral` | Одна награда за приглашённого |
| `referral_welcome:{invitedId}` | `RewardService::welcomeBonus` | Приветственный бонус |
| `daily_bonus:{userId}:{Y-m-d}` | `CoinService::claimDailyBonus` | Бонус раз в день |
| `promo:{promoId}:{userId}` | `GiftService::redeem` | Промокод на игрока |
| `guide:{userId}:{step}` | `GuideService::mark` | Шаг путеводителя |
| `miner:…`, `pet-expedition:…`, `pet-arena:…`, `pet-dex:…`, `respect-week:…` | соответствующие сервисы | Прочие разовые начисления |

Важная деталь: предпроверка стоит **вне** транзакции. При двух одновременных вызовах с одним ключом оба могут пройти `exists()`, после чего второй `INSERT` упадёт на уникальном индексе с `QueryException` (в коде нет `catch` на этот случай — клиент получит 500, а не аккуратный 422). Гарантия «не начислить дважды» при этом сохраняется, страдает только форма ответа.

#### 2.3 Идемпотентность на уровне бизнес-правил

Помимо ключей используются проверки по леджеру:

```
// backend/app/Domains/Wallet/Services/CoinService.php, строки 233–245
public static function dailyBonusAvailable(User $user): bool
{
    // Флаг читаем через настройки, а не напрямую из config:
    // иначе переключатель в админке ни на что не влиял.
    if (! ShopSettingService::getBool('daily_bonus.enabled')) {
        return false;
    }

    return ! CoinTransaction::where('user_id', $user->id)
        ->where('source', CoinTransaction::SOURCE_DAILY_BONUS)
        ->whereDate('created_at', today())
        ->exists();
}
```

То есть ежедневный бонус защищён трижды: флагом настройки, проверкой «за сегодня записи нет» и ключом `daily_bonus:{id}:{date}`.

#### 2.4 Множитель события

```
// backend/app/Domains/Wallet/Services/CoinService.php, строки 65–75
// Событие «xN ApexCoin» с «Пульта»: умножаем только заработок
// (тесты, ачивки, бонус, рефералы), а не покупки и подарки
if ($amount > 0 && in_array($source,
\App\Domains\Control\Services\ControlService::BOOSTED_SOURCES, true)) {
    $multiplier = \App\Domains\Control\Services\ControlService::multiplier();

    if ($multiplier > 1) {
        $amount *= $multiplier;
        $options['meta'] = [...($options['meta'] ?? []), 'multiplier' => $multiplier];
        $description = trim(($description ?? '').' · событие x'.$multiplier);
    }
}
```

`BOOSTED_SOURCES = ['tier_test', 'achievement', 'daily_bonus', 'referral']`. Множитель применяется только к положительным суммам, поэтому `admin`-раздачи и `gift_in` не удваиваются, а списания не масштабируются. Факт умножения фиксируется в `meta.multiplier` и в описании — по леджеру видно, откуда взялась сумма.

#### 2.5 Подарки монет

`CoinService::gift()` — единственный перевод между игроками:

- `GuideService::assertCanTrade()` — первые 7 дней после старта путеводителя переводить монеты нельзя;
- настройки читаются через `ShopSettingService` (`gift.enabled`, `gift.min`, `gift.max`, `gift.daily_limit`, `gift.fee_percent`), то есть правки из админки действуют;
- запрет на подарок самому себе и не-другу (`$from->isFriendsWith($to->id)`);
- дневной лимит считается как `SUM(abs(amount))` по `gift_out` за сегодня;
- комиссия `floor(amount * fee_percent / 100)`, округление вниз (при 5% подарок 19 монет даёт комиссию 0);
- обе проводки — в одной транзакции: при нехватке средств откатываются обе.

```
// backend/app/Domains/Wallet/Services/CoinService.php, строки 199–220
$fee = (int) floor($amount * $gift['fee_percent'] / 100);
$received = $amount - $fee;

DB::transaction(function () use ($from, $to, $amount, $fee, $received) {
    self::credit(
        $from,
        -$amount,
        CoinTransaction::SOURCE_GIFT_OUT,
        "Подарок для {$to->username}",
```

```
            null,
            ['actor_id' => $from->id, 'meta' => ['to_user_id' => $to->id, 'fee' => $fee]]
        );

        self::credit(
            $to,
            $received,
            CoinTransaction::SOURCE_GIFT_IN,
            "Подарок от {$from->username}",
            null,
            ['actor_id' => $from->id, 'meta' => ['from_user_id' => $from->id, 'fee' => $fee]]
        );
    });
```

#### 2.6 Ежедневный бонус

Сумма берётся из `shop_settings` (`daily_bonus.amount`, дефолт 40 из конфига), флаг — `daily_bonus.enabled`. Повторная попытка возвращает `0` из сервиса, контроллер превращает это в 422 с текстом «Ежедневный бонус уже получен. Возвращайтесь завтра!» и текущим балансом.

## 3. Настройки экономики: `shop_settings`

Все числа экономики правятся из админки и хранятся в key/value таблице, конфиг — только дефолт.

| Поле | Тип | Смысл |
|---|---|---|
| `key` | `string(96)`, unique | Плоский ключ с точкой: `daily_bonus.amount`, `tier_test.per_tier` |
| `value` | `json`, nullable | JSON-значение (число, bool, массив) |

Чтение идёт через `ShopSettingService`, где важна обработка «явного дефолта»:

```
// backend/app/Domains/Shop/Services/ShopSettingService.php, строки 26–33
// Дефолт: явный аргумент, иначе значение из config/apex.php.
// Проверяем именно передан ли аргумент: явный 0 (или false) — это
// осмысленный дефолт, но раньше он через ?? перекрывал config,
// из-за чего first_test_bonus из конфига не начислялся никогда.
return func_num_args() > 1
    ? $default
    : config('apex.coins.' . $key, config('apex.' . $key));
```

`sourceEnabled()` читает словарь `sources` (из БД, иначе из `config('apex.coins.sources')`) и возвращает `false`, если ключа нет. Поэтому выключение источника в админке действительно останавливает начисление, а не только скрывает кнопку.

## 4. `RewardService`: расчёт наград

| Метод | Формула | Идемпотентность |
|---|---|---|
| `coinsForTier($tier)` | `tier_test.per_tier[$tier]`, `0` если источник выключен | — |
| `forTierTest($test)` | награда за тир + `first_test_bonus` | `tier_test:{id}`, `tier_test_first:{user}` |
| `coinsForAchievement($a)` | `$a->coin_reward` либо `base + points * per_point` | — |
| `forAchievement($user, $a)` | то же | `achievement:{user}:{a}` |

| | | |
|---|---|---|
| `forReferral($referrer, $invited)` | `referral.amount` | `referral:{invited}` |
| `welcomeBonus($invited)` | `referral.welcome_bonus` | `referral_welcome:{invited}` |

Дефолтная таблица наград по тирам: `E=60, D=90, C=140, B=220, A=350, S=600, S+=1000`; ачивка — `25 + points * 2`; реферал — 200 пригласившему и 50 новичку.

Расхождение, зафиксированное тестом: `forTierTest()` вызывает `ShopSettingService::getInt('tier_test.first_test_bonus', 0)` — с явным нулём. Поэтому, пока админ не сохранит значение в `shop_settings`, разовый бонус за первую заявку **не выплачивается**, хотя в конфиге стоит 150. Это поведение описано в `ShopPurchaseTest::test_first_test_bonus_from_config_is_never_paid_documents_bug`.

Стартовые монеты путеводителя — 6000 + 2000 + 2000 за первые три шага (`GuideService::STEPS`), они же причина семидневного запрета на переводы (`TRADE_LOCK_DAYS = 7`).

## 5. Кошелёк: API и интерфейс

#### 5.1 `GET /api/wallet`

Отдаёт: блок `referral` (код, ссылка `{frontend_url}/register?ref=CODE`, число приглашённых, до 10 последних, ставка за приглашение), `balance`, `spent`, `earned`, `earned_by_source`, `gifted_today`, `daily_bonus_available`, `daily_bonus_amount`, `gift_limits`, `currency`.

`earned` — не отдельный счётчик, а тождество `apex_coins + apex_coins_spent`: сумма всех начислений за всё время (любое списание увеличивает `spent`, любой возврат — кредит). Код приглашения генерируется лениво при первом обращении: `Str::random(8)` в верхнем регистре с проверкой уникальности в цикле.

Два поля отдают конфиг, а не настройки:

- `daily_bonus_amount` → `config('apex.coins.daily_bonus.amount')`, тогда как начисляет `ShopSettingService::getInt('daily_bonus.amount')`;
- `gift_limits` → `config('apex.coins.gift')`, тогда как `CoinService::gift()` читает `shop_settings`.

Тест `WalletTest::test_wallet_preview_ignores_daily_bonus_shop_setting_documents_bug` фиксирует следствие: кошелёк показывает 40, а начисляется 123.

#### 5.2 `GET /api/wallet/transactions`

Пагинация Laravel, `per_page` зажимается в `min(100, max(5, ...))`, дефолт 30, сортировка `latest()`. Отдаёт полные строки леджера, включая `balance_after`, `reference_type`, `idempotency_key`, `meta`.

#### 5.3 `POST /api/wallet/daily-bonus`

Успех — 200 `{message, amount, balance}`; повтор или выключенный бонус — 422 `{message, balance}`.

#### 5.4 Фронтенд

`frontend/src/views/shop/WalletView.vue` (`/wallet`, `meta.auth`) показывает: карточку баланса с кнопкой «Забрать» (или плашку «Ежедневный бонус уже получен»), метрики «Всего

заработано» / «Потрачено» / «Подарено сегодня» с прогрессом дневного лимита, блок «Откуда монеты» (горизонтальные бары по `earned_by_source`), блок пригласительных ссылок, историю операций и форму промокода. Ошибки API кладутся в общий `error`, успех — в `notice`.

## 6. Магазин

#### 6.1 Модель товара

`shop_items`:

| Поле | Тип | Смысл |
|---|---|---|
| `slug` | `string(64)`, unique | Человеческий идентификатор (`frame-gold`) |
| `name` / `description` | `string(128)` / `text` | Витрина |
| `type` | `string(32)` | Категория (см. ниже) |
| `rarity` | `string(16)`, default `common` | `common/rare/epic/legendary` |
| `effect_value` | `string(128)`, nullable | Что пишется в поле профиля: `gold`, `#14b8a6`, путь к PNG |
| `price` | `unsignedInteger`, default 0 | Цена в ApexCoin |
| `is_active` | `boolean`, default true | Продаётся ли |
| `sort_order` | `unsignedInteger` | Порядок витрины |
| `is_consumable` / `is_repeatable` | `boolean` | Можно ли покупать повторно |
| `max_quantity` | `unsignedInteger`, nullable | Потолок накопленного количества |
| `metadata` | `json` | `icon`, `color`, `charges`, `amount`, `days`, `kind`, `items`, `stickers`, `sprite`, `anim`, `style` |

Индекс `(type, is_active)` — под фильтр вкладок.

Типы (21, включая набор):

| Тип | Что это | Надевается |
|---|---|---|
| `avatar_frame` | Рамка аватара | да |
| `profile_effect` | Эффект профиля | да |
| `accent_color` | Акцентный цвет (hex) | да |
| `skin_backdrop` | Сцена за 3D-скином | да |
| `card_theme` | Оформление карточки игрока | да |
| `profile_layout` | Макет страницы профиля | да |
| `badge` | Бейдж в профиле (до 3) | да |
| `name_effect` | Эффект ника по всему сайту | да |
| `profile_entrance` | Появление профиля | да |
| `cursor_trail` | След курсора | да |
| `profile_pet` | Питомец | да (до 3) |
| `chat_bubble` | Оформление сообщений в чате | да |
| `profile_title` | Титул под ником | да |

| | | |
|---|---|---|
| `name_icon` | Значок у ника | да |
| `sticker_pack` | Набор стикеров (коллекция) | нет |
| `tier_priority` | Приоритет заявки на тир-тест | нет (расходник) |
| `coin_bundle` | Набор монет | нет |
| `clan_highlight` | Подсветка клана на срок | нет |
| `clan_highlight_style` | Цвет/эффект подсветки | в клан-инвентарь |
| `bundle` | Набор косметики со скидкой | нет |
| `card_background` | Фон карточки | **нет** — см. 6.8 |

`ShopItem::EQUIPPABLE` содержит 16 типов; `card_background`, `tier_priority`, `coin_bundle`, `clan_highlight`, `bundle`, `sticker_pack` в него не входят.

#### 6.2 Каталог

`catalogQuery()` — активные товары, **без питомцев**, сортировка `sort_order → price → id`; клановые предметы скрыты, если игрок не лидер клана. Питомцы исключены намеренно: это `isCaseOnly()`, их выбивают из сундука.

`GET /api/shop` без `limit` отдаёт весь каталог плюс `balance`, `clan` (лидеру — казна), `tier_rewards`, `currency`. С `?limit=` включается курсорная пагинация:

- `catalogPage()` берёт `limit + 1` строку, чтобы понять, есть ли следующая страница;
- курсор — base64url от `[sort_order, price, id]`, условие «строго после» без `OFFSET`: вставка новых товаров между запросами не даёт дублей и пропусков;
- битый курсор (`decodeCursor()`) трактуется как «с начала»;
- шапка (`type_counts`, `bundles`, `balance`, `clan`, `tier_rewards`) отдаётся только с первой страницей — при непустом `cursor` ответ содержит лишь `items` и `next_cursor`.

`catalogTypeCounts()` считает количество по типам одним `GROUP BY`, чтобы вкладки не тянули весь каталог. Наборы из витрины-списка исключены и показываются отдельным блоком `bundles`.

#### 6.3 Флаги товара для игрока

`presentItem()` всегда отдаёт одинаковый набор полей — и гостю, и авторизованному: `owned`, `quantity`, `equipped`, `can_afford` (у гостя все `false`), `is_repeatable` (для питомцев принудительно `true`), `is_equippable`, `metadata`.

`presentCatalog()` добавляет:

- `owned` / `quantity` / `equipped` — из `user_inventory` (для `clan_highlight_style` — из `clan_inventory`);
- `can_afford` — хватает баланса **или** хватает казны клана по цене со скидкой уровня;
- `treasury_price` — цена из казны со скидкой (`ClanLevel::discounted`);
- админу личная косметика помечается как `owned` без записи в инвентаре.

#### 6.4 Инвентарь `user_inventory`

| Поле | Тип | Смысл |
|---|---|---|
| `user_id` | FK `users`, cascade | Владелец |

| | | |
|---|---|---|
| `shop_item_id` | FK `shop_items`, cascade | Предмет |
| `quantity` | `unsignedInteger`, default 1 | Остаток применений / копий |
| `equipped_at` | timestamp, nullable | Надет ли |
| `acquired_at` | timestamp, nullable | Когда получен |
| `gifted_by` | FK `users`, `nullOnDelete` | Кто подарил |
| `paid_price` | `unsignedInteger`, nullable | Сколько заплачено (для продажи магазину) |
| `pet_traits` | `json`, nullable | Окрас/фон/аксессуар экземпляра |
| `pet_serial` | `unsignedInteger`, nullable | Номер экземпляра вида |
| `pet_name` | `string(20)`, nullable | Имя питомца |

Индексы: `(user_id, equipped_at)`, `(user_id, shop_item_id)`. Изначальный `unique(user_id, shop_item_id)` **снят** миграцией `2026_10_29_150000_multiple_pets` — питомцы хранятся отдельными строками; для остальных предметов одна строка поддерживается кодом (`firstOrNew`). Для нескольких питомцев на профиле появилось `users.profile_pets` (json).

Модель `UserInventory` при создании «чеканит» питомца: если `pet_traits`/`pet_serial` не заданы, кидает черты и ставит `pet_serial = max(shop_item_id) + 1`; при сохранении пишет запись в «питомцедекс» и сбрасывает кэш топа.

#### 6.5 Покупка: порядок проверок

`POST /api/shop/{shopItem}/purchase` (`PurchaseRequest`: `tier_test_id`, `quantity` 1–10, `pay_from` = `balance|treasury`). Проверки до транзакции:

1. `is_active` → иначе «Этот предмет больше не продаётся.»
2. `isCaseOnly()` → «Питомцы не продаются — их можно выбить в «Сундуке питомцев».»
3. `isBundle()` → делегирование в `BundleService::purchase()`.
4. Админ и `isFreeForAdmin()` → «Вам вся косметика доступна бесплатно...».
5. `quantity > 1` для не-stackable → «Этот предмет можно купить только один раз.»
6. Клановый предмет: нужен клан, нужен лидер, для оформления — уже активная подсветка.
7. Уже куплено (не stackable, не питомец) → 422 с разными текстами для клана и личного инвентаря.
8. `max_quantity` превышен → «Достигнут максимум по этому предмету.»
9. Цена `price * quantity`; для казны — со скидкой уровня и проверкой `treasury`; иначе `CoinService::canAfford` → 422 с точной нехваткой.
10. Для `tier_priority` с `tier_test_id` — заявка должна быть своя, активная и без приоритета.

Затем одна транзакция:

```
// backend/app/Domains/Shop/Services/ShopService.php, строки 576–597
DB::transaction(function () use ($user, $item, $quantity, $price, $options, &$tierTest, &$clan,
$isStyle, $fromTreasury) {
    $title = "Покупка: {$item->name}" . ($quantity > 1 ? " ×{$quantity}" : '');
```

```
    if ($fromTreasury) {
        app(ClanTreasuryService::class)->spend($clan, $user, $price, $title);
    } else {
        CoinService::debit(
            $user,
            $price,
            CoinTransaction::SOURCE_PURCHASE,
            $title,
            [
                'reference_type' => ShopItem::class,
                'reference_id' => $item->id,
                'meta' => array_filter([
                    'slug' => $item->slug,
                    'quantity' => $quantity,
                    'tier_test_id' => $tierTest?->id,
                ], fn ($v) => $v !== null),
            ]
        );
    }
```

Ошибки бизнес-правил — это `RuntimeException`. Обёртка `toHttpError()` ловит её и делает `abort(response()->json(['message' => ...], 422))`; контроллер о правилах не знает. Успех — 201 `{message, balance, item, tier_test, clan_highlight, granted, paid, inventory}`.

При ошибке не меняется ничего: тест `test_purchase_fails_without_enough_coins_and_changes_nothing` проверяет, что после 422 баланс, `apex_coins_spent`, леджер и инвентарь остались прежними.

#### 6.6 Как выдаётся купленное

| Тип | Что делает покупка |
|---|---|
| Косметика | `firstOrNew` по `(user_id, shop_item_id)`, `quantity += metadata.charges * quantity`, `paid_price = item.price` |
| `profile_pet` | Всегда новая строка (новый экземпляр: черты + номер) |
| `tier_priority` | Строка-заряд; при указании заявки сразу списывается (`applyPriorityCharge`) и пишет `is_priority`, `priority_purchased_at`, `priority_price_paid`, `priority_weight = default_weight + текущий` |
| `coin_bundle` | Строка в инвентаре **и** `credit` на `metadata.amount * quantity` (источник `other`) |
| `clan_highlight` | Продлевает `clan.highlight_until` на `metadata.days` (по умолчанию 30) × quantity |
| `clan_highlight_style` | Пишет `highlight_color`/`highlight_effect` клана и строку в `clan_inventory` с `equipped_at` |
| `bundle` | См. 6.7 |

Набор монет — единственный товар, который одновременно списывает и начисляет: две строки леджера (`-price` с `source=purchase`, `+amount` с `source=other`). Запись в инвентарь при этом обязательна, иначе не работают проверки «уже куплено» и `max_quantity`, и бесплатный набор с `is_repeatable = false` покупался бы бесконечно (это отдельный тест).

Приоритет можно купить заранее: заряд лежит в инвентаре, применяется позже через `POST /api/tier-tests/{tierTest}/priority`. При применении запись удаляется, если заряд был последним; неудачные попытки (чужая заявка, завершённый тест) заряд не тратят.

#### 6.7 Наборы

`BundleService` собирает состав из `metadata.items` (slug'и), отбрасывая питомцев. Цена считается пропорционально недостающему:

```
// backend/app/Domains/Shop/Services/BundleService.php, строки 43–57
$value = (int) $items->sum('price');
$missing = $items->reject(fn (ShopItem $item) => in_array($item->id, $ownedIds, true));
$missingValue = (int) $missing->sum('price');

$price = $value > 0 && $missingValue > 0
    ? (int) ceil($bundle->price * $missingValue / $value)
    : 0;

return [
    'full' => (int) $bundle->price,
    'value' => $value,
    'missing_value' => $missingValue,
    'price' => $missing->isEmpty() ? 0 : max(1, $price),
    'owned_all' => $missing->isEmpty(),
];
```

Правила: если есть всё — 422; если пусто — 422; админу набор не продаётся; выдача — все недостающие предметы в одной транзакции вместе со списанием. В наборах бывают эксклюзивы с `is_active = false`, которые по отдельности не продаются. Кнопка «Надеть набор» (`POST /api/shop/{shopItem}/equip-bundle`) надевает по одному предмету на слот (бейджи — все, пока есть место) и молча пропускает ошибки занятых слотов.

#### 6.8 Надевание и снятие

`POST /api/shop/{shopItem}/equip`:

- не `EQUIPPABLE` → «Этот предмет нельзя надеть.»;
- `clan_highlight_style` → отдельная ветка: проверка лидера и активной подсветки, запись в поля клана, снятие предыдущего оформления того же `kind`;
- нет строки в инвентаре → 422 «Сначала нужно купить этот предмет.» (админу строка создаётся на месте);
- питомец → надевается до трёх, при переполнении снимается надетый раньше всех;
- бейдж → лимит `apex.shop.max_equipped_badges` (3), затем пересборка `users.equipped_badges`;
- одиночный слот → снимаются все предметы того же типа, в поле профиля пишется `effect_value` (или `slug`).

`unequip` симметричен; для `avatar_frame` поле сбрасывается в `'default'`, для остальных — в `null`; клановое оформление возвращается к `ClanHighlight::DEFAULT_COLOR` / `DEFAULT_EFFECT`.

Расхождение: `card_background` не входит в `EQUIPPABLE`, хотя `profileFieldFor()` умеет писать в `card_background`. Купленный фон карточки надеть нельзя — это зафиксировано тестом `test_card_background_cannot_be_equipped_documents_bug`.

`randomize` перемешивает надетое по слотам (`RANDOM_SLOTS`: рамка, эффект, акцент, фон скина, тема карточки, макет, эффект ника, появление, след, питомец, пузырь чата, титул, значок) и

надевает случайные бейджи до лимита. Админу доступна вся косметика, остальным — только купленная.

#### 6.9 Продажа магазину и подарок предмета

`InventoryTradeService`:

- `tradeable()` — не расходник, не `coin_bundle`/`tier_priority`, цена > 0;
- `sellValue()` — половина `paid_price`, а если предмет выпал из кейса или подарен (`paid_price = null`) — 15% цены (`LootCases::DUPLICATE_SHARE`), минимум 1;
- для старых строк без `paid_price` работает эвристика `legacyPaid()`: есть покупка в леджере и нет `gifted_by` → считаем купленным;
- `sell()` — снять (если надет), удалить/уменьшить строку, начислить `source=other`;
- `gift()` — только другу и не себе; питомец переезжает той же строкой (сохраняя черты и номер), при этом `paid_price` обнуляется, `gifted_by` проставляется. Это закрывает «отмывание»: подаренное продаётся за 15%, а не за 50%.

## 7. Кейсы (сундуки)

Кейсы описаны не в БД, а в `LootCases` (константы), история открытий — в `case_openings`.

| Slug | Цена | Особенность | Шансы (`nothing/coins/common/rare/epic/legendary`) |
|---|---|---|---|
| `daily` | 0 | Бесплатно раз в сутки | 25 / 30 / 30 / 11 / 3.4 / 0.6 |
| `seeker` | 750 | Гарантия эпика каждые 10 | 5 / 15 / 25 / 33 / 18 / 4 |
| `royal` | 2500 | Без пустоты и обычных | 0 / 10 / 0 / 40 / 35 / 15 |
| `pets` | 10000 | Только питомцы | 0 / 0 / 45 / 33 / 17 / 5 |

Правила:

- исход решает сервер; лента рулетки (`REEL_LENGTH = 50`, выигрыш на `REEL_WIN_INDEX = 42`) — декорация, но выигрыш стоит именно там, что проверяется тестом;
- бесплатный кейс — не чаще одного раза в календарный день (`CaseOpening` с `created_at >= startOfDay`);
- платный можно открыть пачкой 3/5/10 (`BATCHES`), сначала списываются ключи (`loot_case_keys`), остаток — монетами; достаточность проверяется заранее, чтобы не остановиться на середине;
- гарантия: если до эпика осталось ≤ 1 открытие, исход перебрасывается по шансам только `epic/legendary`, ответ помечает `pity = true`;
- повтор не-stackable предмета превращается в монеты: `max(5, round(price * 0.15))`; питомец всегда новый экземпляр;
- пустой исход → монеты из диапазона `coins`, округлённые до 5.

```
// backend/app/Domains/Shop/Services/LootCaseService.php, строки 104–111
$outcome = $this->roll($case['chances']);
$pity = false;
```

```
// Гарантия: в платном кейсе после серии без эпика выпадает эпик+
if (! $case['free_daily'] && $this->pityLeft($user, $slug) <= 1 && ! in_array($outcome, ['epic',
'legendary'], true)) {
    $outcome = $this->roll(array_intersect_key($case['chances'], array_flip(['epic',
'legendary'])));
    $pity = true;
}
```

Нехватка монет бросает `NotEnoughCoins` — единственное место в проекте, где используется это исключение; контроллер отдаёт `{message, missing, earn: '/miner'}`, чтобы клиент предложил, где заработать.

## 8. Аукцион питомцев

| Параметр | Значение |
|---|---|
| Длительность | 12 / 24 / 48 часов |
| Стартовая цена | от 10 ApexCoin |
| Шаг ставки | 5% от текущей, минимум 10 |
| Продление | ставка в последние 120 секунд продлевает торги на 120 секунд |
| Активных лотов у игрока | не больше 3 |
| Неустойка за снятие лота со ставками | 10% от текущей ставки, минимум 1 |
| Чат лота | 300 символов, пауза 3 секунды, дубль в течение минуты запрещён |

Механика ставки: монеты **списываются сразу** (заморозка), прошлому лидеру возвращаются, всё в одной транзакции с `lockForUpdate` по лоту. Закрытие торгов — ленивое (`closeExpired()` при любом обращении) и по расписанию; победителю переходит та же строка инвентаря с чертами и уровнем, `paid_price` становится ценой продажи, продавцу начисляются монеты. Изменения транслируются событием `AuctionChanged` (`ShouldBroadcastNow`, каналы `auctions` и `auction.{id}`).

## 9. Промокоды

#### 9.1 Структура

| promo_codes | Тип | Смысл |
|---|---|---|
| code | string(32), unique | Код (хранится в верхнем регистре) |
| coins | unsignedInteger, default 0 | Сколько монет |
| shop_item_id | FK shop_items, nullOnDelete | Предмет (необязательно) |
| max_uses | unsignedInteger, nullable | null — без ограничения |
| uses | unsignedInteger, default 0 | Сколько активаций |
| expires_at | timestamp, nullable | Срок |
| is_active | boolean, default true | Быстрое выключение |
| note | string, nullable | Комментарий для админов |
| created_by | FK users, nullOnDelete | Автор |

| promo_redemptions | Тип | Смысл |
|---|---|---|
| promo_code_id | FK, cascadeOnDelete | Код |
| user_id | FK, cascadeOnDelete | Игрок |
| created_at | timestamp, useCurrent | Момент (UPDATED_AT = null) |
| — | unique(promo_code_id, user_id) | Один код — одна активация на игрока |

#### 9.2 Создание (админ)

POST /api/admin/control/promo-codes: код [A-Za-z0-9_-]{3,32} и unique, монеты 0–100000, существующий shop_item_id, max_uses 1–100000, expires_at строго в будущем. Требуется хотя бы монеты или предмет. Код нормализуется в верхний регистр; если не передан — генерируется APEX- + 6 случайных символов. Факт создания пишется в ModerationLog (control.promo).

#### 9.3 Погашение

```
// backend/app/Domains/Control/Services/GiftService.php, строки 99–115
$promo = PromoCode::with('item')->whereRaw('UPPER(code) = ?', [mb_strtoupper(trim($code))])-
>first();

if (! $promo || ! $promo->is_active) {
    $this->abortUnprocessable('Такого промокода нет.');
}

if ($promo->expires_at && $promo->expires_at->isPast()) {
    $this->abortUnprocessable('Срок действия промокода истёк.');
}

if ($promo->max_uses !== null && $promo->uses >= $promo->max_uses) {
    $this->abortUnprocessable('Промокод уже закончился.');
}

if (PromoRedemption::where('promo_code_id', $promo->id)->where('user_id', $user->id)->exists()) {
    $this->abortUnprocessable('Вы уже активировали этот промокод.');
}
```

Поиск регистронезависимый (UPPER(code)), поэтому apex2026 и APEX2026 — один код. В транзакции: запись в promo_redemptions, increment('uses'), начисление монет (source=admin, ключ promo:{id}:{user}), выдача предмета — при этом уже имеющийся не-расходник не дублируется, но монеты всё равно начисляются. Ответ: {coins, item, balance}.

## 10. Control: «функции сайта»

#### 10.1 Что такое функция

Функция — это именованный рубильник, которым админ закрывает часть сайта, не трогая код. Список фиксирован в константе:

```
// backend/app/Domains/Control/Services/ControlService.php, строки 22–34
/** Что можно выключить одной кнопкой — ключ и подпись для админки. */
public const FEATURES = [
    'registration' => 'Регистрация',
    'shop' => 'Покупки в магазине',
    'tier_tests' => 'Заявки на тир-тест',
    'clan_creation' => 'Создание кланов',
    'forum_posting' => 'Новые темы и ответы на форуме',
];
```

```
/** Источники, на которые действует множитель события. */
public const BOOSTED_SOURCES = ['tier_test', 'achievement', 'daily_bonus', 'referral'];
```

#### 10.2 Где хранятся флаги

В общей таблице `site_settings` (`key` unique, `value` text, `type` string/json/bool/int, `group`). Группа — `control`.

| Ключ | Тип | Значение |
|---|---|---|
| `feature_registration` | bool | Регистрация |
| `feature_shop` | bool | Покупки |
| `feature_tier_tests` | bool | Заявки на тир-тест |
| `feature_clan_creation` | bool | Создание кланов |
| `feature_forum_posting` | bool | Посты на форуме |
| `announcement` | json | `{id, text, tone, link, link_label, until}` |
| `coin_multiplier` | int | 1–5 |
| `coin_multiplier_until` | string | До какого момента |

Чтение кешируется на 60 секунд:

```
// backend/app/Domains/Control/Services/ControlService.php, строки 36–51
public static function state(): array
{
    return Cache::remember(self::CACHE_KEY, self::CACHE_TTL, function () {
        $features = [];

        foreach (array_keys(self::FEATURES) as $key) {
            $features[$key] = (bool) SiteSetting::get("feature_{$key}", true);
        }

        return [
            'features' => $features,
            'announcement' => SiteSetting::get('announcement', null),
            'multiplier' => (int) SiteSetting::get('coin_multiplier', 1),
            'multiplier_until' => SiteSetting::get('coin_multiplier_until', null),
        ];
    });
}
```

`update()` пишет настройки и делает `Cache::forget('control:state')`, поэтому переключатель срабатывает сразу, а не через минуту. Дефолт для флага — `true`: отсутствующая настройка означает «включено».

#### 10.3 Middleware

```
// backend/app/Http/Middleware/EnsureFeatureEnabled.php
public function handle(Request $request, Closure $next, string $feature): Response
{
    if (! ControlService::enabled($feature) && ! $request->user()?->isAdmin()) {
        $label = ControlService::FEATURES[$feature] ?? 'Эта функция';

        return response()->json([
            'message' => "{$label}: временно отключено администрацией. Загляните чуть позже.",
            'feature' => $feature,
        ], 423);
    }

    return $next($request);
}
```

Алиас зарегистрирован в `bootstrap/app.php` как `'feature' => EnsureFeatureEnabled::class`, применяется в маршрутах: `->middleware('feature:shop')`.

Правила:

- **423 Locked**, не 403 и не 503 — клиент отличает «выключено временно» от «нет прав»;
- сообщение берётся из `FEATURES`, поэтому текст человеческий, а не код;
- администратор проходит всегда — чтобы проверить работоспособность;
- проверка идёт после аутентификации (`auth:sanctum` стоит на группе выше), поэтому `user()` доступен;
- ответ содержит `feature` — фронтенд может точечно обработать.

Какие маршруты закрыты:

| Функция | Маршруты |
|---|---|
| `registration` | `POST /api/auth/register/send-code`, `POST /api/auth/register` |
| `shop` | покупка, открытие кейса, создание/ставка/снятие лота, продажа и подарок предмета |
| `tier_tests` | `POST /api/tier-tests` (создание заявки) |
| `clan_creation` | `POST /api/clans` |
| `forum_posting` | `POST /api/forum/topics`, `POST /api/forum/topics/{topic}/reply` |

Не закрыты (фактическое поведение): чтение каталога `GET /api/shop`, надевание/снятие, `randomize`, стикеры, переключение питомца, сообщения в чате лота, погашение промокода, кошелёк. То есть `feature:shop` отключает **траты**, а не просмотр и не надевание уже купленного.

#### 10.4 Что видит пользователь при выключенной функции

- Сервер: 423 + `{message, feature}`.
- Публичный баннер и состояние — через `GET /api/site`: `ControlService::publicState()` отдаёт `announcement`, `features`, `event`.
- Фронтенд: `SiteAnnouncement.vue` показывает баннер объявления (закрытие запоминается в `localStorage` по `announcement.id`, поэтому новый анонс всплывает снова) и плашку события «xN ApexCoin».
- Специальной обработки 423 в клиенте **в коде отсутствует**: `api.js` бросает `{status, message, errors, data}`, и страница выводит `message` как обычную ошибку (например, в окне покупки — `purchaseError`). Отдельного экрана «функция отключена» нет.

#### 10.5 Пульт в админке

`frontend/src/components/admin/AdminControl.vue` (вкладка «Пульт», `adminOnly`, `AdminView.vue`):

- объявление: текст до 300 символов, тон `info/success/warning/danger`, ссылка и её подпись, срок «до»;
- рубильники: каждый клик — отдельный `PUT /api/admin/control` с `{features: {key: bool}}`, ответ сразу гидрирует состояние;
- событие: чипы x1…x5 (`max_multiplier`), поле «до», кнопки «Запустить»/«Остановить»;

- раздача: аудитория `online / active_week / all / clan / tier`, монеты 0–100000, сообщение до 300 символов; сначала `dry_run` для подсчёта, затем подтверждение и отправка;
- промокоды: таблица «код / что даёт / активации / до», копирование кода, выключение и удаление;
- «Онлайн сейчас»: `last_seen_at` за последние 5 минут, до 100 игроков, переход в досье.

Каждое изменение пишется в `ModerationLog`: `control.update` (текст анонса, флаги, множитель), `control.giveaway`, `control.promo`, `user.item_gift`.

Раздача (`GiftService::giveaway`) идёт `chunkById(200)`, каждому — `CoinService::credit` (source `admin`, `actor_id` админа, `meta.giveaway = true`) и уведомление; забаненные исключены на уровне запроса аудитории. Аудитория `tier` дополнительно требует `tier_score > 0`.

Ещё два инструмента пульта: `GET /api/admin/control/online` — кто сейчас на сайте, и `POST /api/admin/users/{user}/give-item` — подарок предмета (`GiftService::giveItem()` кладёт предмет в инвентарь с `gifted_by`, повтор не-расходника отклоняется 422).

## 11. Эндпоинты

#### 11.1 Публичные (без авторизации)

| Метод | Путь | Что делает |
|---|---|---|
| GET | `/api/shop` | Каталог; с `?limit&type&cursor` — страница по курсору |
| GET | `/api/shop/{shopItem}` | Карточка активного товара (404 для выключенного) |
| GET | `/api/cases` | Витрина кейсов, шансы, `pity_left`, ключи, история |
| GET | `/api/cases/status` | `{daily_available}` для меню |
| GET | `/api/auctions` | Активные лоты (до 60, по `ends_at`) |
| GET | `/api/auctions/{auction}` | Лот + последние 20 ставок |
| GET | `/api/auctions/{auction}/messages` | Чат лота (до 60 сообщений) |
| GET | `/api/site` | Подвал, соцсети, `control` (баннер, флаги, событие) |

#### 11.2 Авторизованные (`auth:sanctum`)

| Метод | Путь | Middleware | Что делает |
|---|---|---|---|
| GET | `/api/wallet` | — | Баланс, сводка, реферalка, лимиты |
| GET | `/api/wallet/transactions` | — | Леджер игрока (пагинация) |
| POST | `/api/wallet/daily-bonus` | — | Забрать ежедневный бонус (422, если уже) |
| GET | `/api/shop/inventory` | — | Инвентарь; с `?limit&tab&q&sort&cursor` — страница |

| | | | |
|---|---|---|---|
| GET | `/api/shop/priority-candidates` | — | Активные заявки + число зарядов |
| POST | `/api/shop/{shopItem}/purchase` | `feature:shop` | Покупка (201) |
| POST | `/api/shop/cases/{slug}/open` | `feature:shop`, `throttle:20,1` | Открытие кейса, `count=1/3/5/10` |
| POST | `/api/shop/{shopItem}/equip` | — | Надеть |
| POST | `/api/shop/{shopItem}/unequip` | — | Снять |
| POST | `/api/shop/{shopItem}/equip-bundle` | — | Надеть набор целиком |
| POST | `/api/shop/randomize` | `throttle:30,1` | Перемешать образ |
| POST | `/api/shop/pets/{row}/{action}` | — | Надеть/снять экземпляр питомца |
| GET | `/api/shop/stickers` | — | Стикерпаки игрока |
| POST | `/api/shop/auctions` | `feature:shop`, `throttle:10,1` | Выставить питомца (201) |
| POST | `/api/shop/auctions/{a}/bid` | `feature:shop`, `throttle:30,1` | Ставка |
| POST | `/api/shop/auctions/{a}/cancel` | `feature:shop`, `throttle:10,1` | Снять лот (с неустойкой) |
| POST | `/api/shop/auctions/{a}/messages` | `throttle:12,1` | Сообщение в чат лота (201) |
| DELETE | `/api/shop/auctions/{a}/messages/{m}` | `throttle:30,1` | Удалить сообщение (автор/модератор) |
| POST | `/api/shop/inventory/{row}/sell` | `feature:shop`, `throttle:30,1` | Продать магазину |
| POST | `/api/shop/inventory/{row}/gift` | `feature:shop`, `throttle:20,1` | Подарить другу |
| POST | `/api/tier-tests/{tierTest}/priority` | — | Применить купленный приоритет |
| GET | `/api/gifts/limits` | — | Баланс, `gifted_today`, лимиты подарков |
| POST | `/api/gifts/{user}` | — | Подарить монеты (201) |
| POST | `/api/promo/redeem` | `throttle:10,1` | Погасить промокод |

`throttle` здесь — кастомный `ThrottlePerRoute`: сигнатура включает метод и URI маршрута, поэтому лимит копания в шахте не съедает лимит ставок на аукционе.

#### 11.3 Админские: каталог и экономика (`role:moderator,admin`)

| Метод | Путь | Что делает |
|---|---|---|
| GET | `/api/admin/shop-items` | Каталог, включая выключенные; `types`, `rarities` |
| POST | `/api/admin/shop-items` | Создать товар (201) |
| PUT | `/api/admin/shop-items/{shopItem}` | Изменить |
| DELETE | `/api/admin/shop-items/{shopItem}` | Удалить (у купивших предмет |

| | | |
|---|---|---|
| | | остаётся) |
| POST | `/api/admin/shop-items/{shopItem}/toggle` | Вкл/выкл продажу |
| GET | `/api/admin/shop-rewards` | Настройки наград + сохранённые значения |
| PUT | `/api/admin/shop-rewards` | Сохранить настройки экономики |

#### 11.4 Админские: только `role:admin`

| Метод | Путь | Что делает |
|---|---|---|
| POST | `/api/admin/users/{user}/coins` | Ручное начисление/списание (source `admin`, 422 при нехватке) |
| GET | `/api/admin/coin-transactions` | Леджер всех игроков, фильтры `source`, `user_id`, 50 на страницу |
| GET | `/api/admin/control` | Состояние пульта + подписи + список товаров |
| PUT | `/api/admin/control` | Объявление, флаги, множитель |
| POST | `/api/admin/control/giveaway` | Раздача/рассылка, `dry_run` |
| GET | `/api/admin/control/online` | Кто онлайн (5 минут, до 100) |
| GET | `/api/admin/control/promo-codes` | Список кодов (100 последних) |
| POST | `/api/admin/control/promo-codes` | Создать код (201) |
| POST | `/api/admin/control/promo-codes/{promo}/toggle` | Вкл/выкл |
| DELETE | `/api/admin/control/promo-codes/{promo}` | Удалить |
| POST | `/api/admin/users/{user}/give-item` | Подарить предмет |
| GET | `/api/admin/users/{user}/inventory` | Инвентарь игрока |
| POST | `/api/admin/users/{user}/inventory/items/{shopItem}/equip` | Надеть за игрока |
| POST | `/api/admin/users/{user}/inventory/items/{shopItem}/unequip` | Снять за игрока |
| PUT | `/api/admin/users/{user}/inventory/{row}` | Задать количество (1–999) |
| DELETE | `/api/admin/users/{user}/inventory/{row}` | Забрать предмет |
| POST | `/api/admin/users/{user}/inventory/{row}/reroll-pet` | Перебросить черты питомца |

Каждое действие `AdminInventoryController` пишет `ModerationLog` (`user.item_equip`, `user.item_remove`, `user.item_quantity`, `user.pet_reroll`).

## 12. Схема БД и каскады

```
users (apex_coins, apex_coins_spent, referral_code, referred_by, profile_pets, equipped_badges,
pet_traits)
   │
   ├── coin_transactions.user_id        → cascade
   │       actor_id                     → nullOnDelete
   ├── user_inventory.user_id           → cascade
```

```
    │       shop_item_id → shop_items.id → cascade
    │       gifted_by                    → nullOnDelete
    ├── case_openings.user_id            → cascade
    │       shop_item_id                 → nullOnDelete
    ├── loot_case_keys.user_id           → cascade   unique(user_id, case_slug)
    ├── promo_codes.created_by           → nullOnDelete
    ├── promo_redemptions.user_id        → cascade   unique(promo_code_id, user_id)
    ├── pet_auctions.seller_id           → cascade
    │       inventory_id → user_inventory.id → cascade
    │       top_bidder_id                → nullOnDelete
    └── pet_auction_bids.user_id         → cascade

shop_items
    ├── user_inventory.shop_item_id      → cascade
    ├── promo_codes.shop_item_id         → nullOnDelete
    └── case_openings.shop_item_id       → nullOnDelete

pet_auctions
    ├── pet_auction_bids.auction_id      → cascade
    ├── pet_auction_messages.auction_id  → cascade
    └── (inventory_id → user_inventory)  → cascade
```

Смысл каскадов:

- **cascade от `users`** — удаление игрока уносит его историю, инвентарь, награды; осиротевших строк не остаётся.
- **cascade от `shop_items` на `user_inventory`** — удаление товара забирает его из инвентарей (в админке предупреждают: «У купивших предмет останется», но фактически FK `cascadeOnDelete` его удалит — это расхождение текста подтверждения и схемы).
- **`nullOnDelete` для ссылок «кто»** — `actor_id`, `gifted_by`, `created_by`, `top_bidder_id`: удаление актора не должно уничтожать экономическую историю.
- **`nullOnDelete` для `shop_item_id` в `promo_codes`/`case_openings`** — промокод и история открытия остаются, просто без ссылки на товар.

Уникальные индексы: `shop_items.slug`, `shop_settings.key`, `site_settings.key`, `promo_codes.code`, `coin_transactions.idempotency_key`, `promo_redemptions(promo_code_id, user_id)`, `loot_case_keys(user_id, case_slug)`, `respect_week_rewards(week_start, place)`.

## 13. Защита от накрутки и двойного списания

| Механизм | Где | Что даёт |
|---|---|---|
| Транзакция БД | `CoinService::credit`, `ShopService::purchase`, `BundleService::purchase`, `LootCaseService::open`, `GiftService::redeem`, `InventoryTradeService::sell/gift`, `PetAuctionService` | Списание и выдача не расходятся |
| `SELECT ... FOR UPDATE` | `CoinService`, `ClanTreasuryService::move`, `PetAuctionService` (лот), `GuideService` | Сериализация конкурентных изменений на mysql/pgsql/sqlsrv |
| Проверка баланса внутри транзакции | `CoinService` | Нельзя уйти в минус даже при гонке |
| `unsignedBigInteger` для баланса | `users` | Отрицательный баланс невозможен на уровне схемы |

| | | |
|---|---|---|
| Уникальный `idempotency_key` | `coin_transactions` | Повторное начисление за тот же тест/ачивку/день/промокод |
| Уникальность `(promo_code_id, user_id)` | `promo_redemptions` | Двойная активация кода одним игроком невозможна |
| Уникальность `(user_id, case_slug)` | `loot_case_keys` | Ключи не размножаются |
| `insertOrIgnore` + `unique(week_start, place)` | `RespectWeeklyRewardsCommand` | Недельная награда не выдаётся дважды |
| Проверки «уже куплено» / `max_quantity` | `ShopService::purchase`, `GiftService::giveItem` | Повторная покупка не-расходника и переполнение стека |
| `CaseOpening` за сегодня | `LootCaseService::freeAvailable` | Бесплатный сундук раз в день |
| Дневной лимит подарков | `CoinService::giftedToday` | Ограничение перекачки монет |
| Только друзьям | `isFriendsWith()` | Отсечение перевода случайным аккаунтам |
| Заморозка ставки | `PetAuctionService::bid` | Нельзя ставить больше, чем есть, и нельзя «отменить» ставку |
| Неустойка 10% | `cancel()` | Снятие лота со ставками невыгодно |
| `paid_price` + сброс при подарке | `InventoryTradeService` | Купленное продаётся за 50%, подаренное/выпавшее — за 15% |
| Семидневный запрет переводов | `GuideService::assertCanTrade` | Стартовые монеты путеводителя нельзя перегнать на другой аккаунт |
| Лимит 3 активных лотов | `PetAuctionService::MAX_ACTIVE` | Нельзя завалить аукцион |
| Проверка прав на клановые траты | `ShopService::purchase`, `ClanTreasuryService` | Тратит только лидер, из казны — по уровню со скидкой |
| `throttle` на маршрутах | `ThrottlePerRoute` | Ограничение частоты покупок, ставок, кейсов, промокодов |
| Роли и middleware | `role:admin`, `role:moderator,admin`, `feature:*` | Разделение админских и игровых действий |

Чего **в коде отсутствует**:

- БД-гарантия уникальности строки инвентаря для не-питомцев: `unique(user_id, shop_item_id)` снят, осталась проверка в PHP (`firstOrNew`) — при одновременных запросах две строки возможны.
- БД-гарантия «один бесплатный кейс в день»: проверка только по `case_openings`, без уникального индекса (`user_id, case_slug, дата`).
- Обработка `QueryException` от уникального `idempotency_key`: гонка даёт 500 вместо 422.
- Возврат средств за открытый кейс, отмена покупки, откат промокода.
- Аудит изменения `shop_settings` (пишутся только действия пульта и ручные начисления).
- Отдельный экран/состояние «функция отключена» на фронтенде: обрабатывается как обычная ошибка 423.
- Отдельный источник `tournament`/`clan_war` в леджере: ключи в конфиге есть, записей нет.

## 14. Известные расхождения, зафиксированные тестами

| Что | Где | Суть |
|---|---|---|
| `daily_bonus_amount` в кошельке | `WalletController::index` | Берётся из `config`, а начисляется из `shop_settings` — превью расходится с фактом |
| `gift_limits` | `WalletController::index`, `GiftController::limits` | Конфиг вместо настроек, комиссия/лимит в админке на превью не влияют |
| `first_test_bonus` | `RewardService::forTierTest` | `getInt(..., 0)` не даёт прочитать конфиг: бонус не платится, пока не сохранён в БД |
| `card_background` | `ShopItem::EQUIPPABLE` | Тип не в списке надеваемого, хотя `profileFieldFor()` его поддерживает |
| Флаги покупки в ответе | `ShopService::purchase` → `presentItem($item)` | Без `extra`: сразу после покупки `item.owned = false`, `can_afford = false` |
| Текст подтверждения удаления товара | `AdminShop.vue` | Обещает, что предмет останется у купивших, но FK `cascadeOnDelete` его удалит |

# Глава. Турниры, модерация, поддержка, новости и ядро сайта

Пять доменов, о которых идёт речь, устроены одинаково: `backend/app/Domains/<Домен>/<Вид>/`, где «вид» — это `Controllers/Api`, `Controllers/Api/Admin`, `Models`, `Requests/Admin`, `Services`, `Notifications`. Регистрация маршрутов — единый `backend/routes/api.php`; модели подключаются к полиморфной карте в `AppServiceProvider` (`'tournament' => Tournament::class`, `'news' => News::class` и т.д.), что позволяет жалобам и журналу ссылаться на объекты по строковому типу.

## 1. Домен Tournaments

Состав домена (12 файлов, ~38 КБ):

| Файл | Роль |
|---|---|
| `Models/Tournament.php` | турнир, статусы, `isRegistrationOpen()` |
| `Models/TournamentParticipant.php` | заявка/участник (игрок или клан), сид |
| `Models/TournamentMatch.php` | матч сетки, ссылка на следующий матч |
| `Models/TournamentWin.php` | факт победы в турнире (для ачивок и тиров) |
| `Services/TournamentBracketService.php` | генерация сетки, связывание, продвижение, финиш |
| `Controllers/Api/TournamentController.php` | публичный список/страница/регистрация |
| `Controllers/Api/Admin/TournamentController.php` | CRUD, участники, сиды, сетка, результаты |
| `Requests/Admin/*` | `Store`/`Update`/`SetSeeds`/`UpdateMatch` |
| `Notifications/TournamentNotification.php` | database-уведомления |

#### 1.1 Модель данных турнира

Таблица `tournaments` (`2026_09_24_101932_create_tournaments_table.php` + `win_points` из `2026_10_25_120000_add_rating_points.php`):

| Поле | Тип | Комментарий |
|---|---|---|
| `name`, `slug` | string, unique | slug = `Str::slug(name).'-'.Str::random(6)` |
| `description`, `banner` | text, string | баннер — файл в `storage/public/tournaments` |
| `type` | enum `solo`,`clan` | кого ставим в слот: игрока или клан |
| `format` | enum `single_elim`,`double_elim`,`round_robin` | используется только `single_elim` |
| `status` | enum `draft`,`registration`,`ongoing`,`completed`,`cancelled` | default `draft` в БД, но `store()` пишет `registration` |
| `prize_pool` | decimal(12,2) | только отображение |
| `prize_currency` | string(8) | default `RUB` |

| | | |
|---|---|---|
| `prize_description` | string | «50/30/20» и подобное |
| `min_tier`, `max_tier` | string(2) | `E..S+`, фильтр допуска |
| `max_participants` | int | 2..128, default 16 |
| `win_points` | int | очки рейтинга победителю, default 50 |
| `registration_starts_at`, `registration_ends_at`, `starts_at`, `ends_at` | timestamp | окно регистрации и проведения |
| `created_by` | FK users, cascade | автор |

Модель добавляет два вычисляемых поля: `level` (`B..A` → `A`, `A..S` → `S`, иначе `null`) и `banner_url` (`asset('storage/'.$banner)`), плюс `isRegistrationOpen()`:

```
// backend/app/Domains/Tournaments/Models/Tournament.php
public function isRegistrationOpen(): bool
{
    if ($this->status !== 'registration') return false;

    $now = now();
    if ($this->registration_starts_at && $now->lt($this->registration_starts_at)) return false;
    if ($this->registration_ends_at && $now->gt($this->registration_ends_at)) return false;

    return true;
}
```

Ключевая деталь: регистрация не «закрывается» планировщиком. Она открыта ровно тогда, когда статус `registration` и `now()` внутри окна; никакой задачи, переводящей турнир из `registration` в `ongoing` по времени, в коде нет — это делает админ кнопкой «Сформировать сетку».

#### 1.2 Жизненный цикл

| Статус | Как выставляется | Что означает |
|---|---|---|
| `draft` | вручную из формы админки (только при редактировании) | черновик, регистрация закрыта |
| `registration` | `store()` — жёстко; либо `removeParticipant()` при сбросе сетки; либо вручную | приём заявок |
| `ongoing` | только `TournamentBracketService::generateSingleElim()` | сетка построена |
| `completed` | только `finishTournament()` (победа в финале); либо вручную | есть чемпион |
| `cancelled` | только вручную | турнир не состоялся |

Переходы не валидируются: `UpdateTournamentRequest` принимает любое из пяти значений (`'status' => ['sometimes','in:draft,registration,ongoing,completed,cancelled']`), поэтому админ может вернуть `completed` обратно в `registration`, и сетка при этом останется. Единственная защита от разрушения истории — проверки в сервисе и в `removeParticipant()`.

#### 1.3 Регистрация участников

`POST /api/tournaments/{id}/register` → `TournamentController::register()`. Порядок проверок жёсткий и важен:

1. `isRegistrationOpen()`, иначе `422 «Регистрация закрыта.»`.
2. Тир-фильтр. Порядок тиров задан массивом `['E','D','C','B','A','S','S+']`; индекс игрока ищется через `array_search(..., true)`. Неизвестный тир — `422`. В комментарии к коду зафиксирован исторический баг: пока `S+` не было в массиве, `array_search` возвращал `false`, и `S+`-игрок проходил любые ограничения.
3. Клановый тип: `abort_if(! $user->clanMember, 422, 'Нужно состоять в клане.')` — иначе появлялся «фантомный» участник с `user_id = null` и `clan_id = null`.
4. Дубликат: ищется по `user_id` **или** по `clan_id` текущего клана игрока — независимо от типа турнира.
5. Лимит: `approvedParticipants()->count() >= max_participants` → `422 «Турнир заполнен.»`. Считаются только одобренные, поэтому заявок может висеть больше лимита.
6. Создание: `status = 'pending'`, `user_id` только для `solo`, `clan_id` только для `clan`.

Отзыв заявки — `withdraw()`: находит участие по `user_id`/`clan_id`, запрещает отзыв при `status = 'approved'` (`422`) и **удаляет** строку. Статус `withdrawn` из enum не используется.

Одобрение/отклонение — в админке: `approveParticipant()` ставит `approved` и шлёт `tournament_approved`, `rejectParticipant()` ставит `rejected` и шлёт `tournament_rejected`. Уведомление уходит игроку, а для клана — лидеру:

```
// backend/app/Domains/Tournaments/Controllers/Api/Admin/TournamentController.php
private function notifyParticipant(TournamentParticipant $participant, string $type, string
$message): void
{
    $recipient = $participant->user ?? $participant->clan?->leader;

    $recipient?->notify(new TournamentNotification($type, $message, $participant->tournament_id));
}
```

Удаление участника (`removeParticipant()`) — отдельная операция с побочным эффектом: если участник уже вписан в сетку, но ни одного «настоящего» матча не сыграно (`completed` и `is_bye = false`), сетка удаляется целиком, а турнир возвращается в `registration`. Если матчи уже играли — `422` с подсказкой «отметьте поражение в матче». Ответ содержит `bracket_reset`, и фронтенд просит сформировать сетку заново.

#### 1.4 Формат и геометрия сетки

Поддерживается ровно один формат — single elimination. Проверка стоит в двух местах: `StoreTournamentRequest`/`UpdateTournamentRequest` (`'format' => ['required','in:single_elim']`) и `generateBracket()` (`abort_if($tournament->format !== 'single_elim', 422, 'Пока поддерживается только single_elim.')`). Двойная сетка и круговой турнир в коде отсутствуют: колонка `tournament_matches.bracket` заполняется константой `'main'`, ветвей `upper`/`lower` нет.

Размер сетки — ближайшая сверху степень двойки:

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php
$count = $participants->count();
if ($count < 2) {
    throw new \Exception('Нужно минимум 2 одобренных участника.');
}

$bracketSize = 1;
while ($bracketSize < $count) {
```

```
    $bracketSize *= 2;
}

$totalRounds = (int) log($bracketSize, 2);
```

| Участников | Размер сетки | Раундов | Матчей | BYE |
|---|---|---|---|---|
| 2 | 2 | 1 | 1 | 0 |
| 3 | 4 | 2 | 3 | 1 |
| 5 | 8 | 3 | 7 | 3 |
| 8 | 8 | 3 | 7 | 0 |
| 9 | 16 | 4 | 15 | 7 |
| 16 | 16 | 4 | 15 | 0 |

Число матчей всегда `bracketSize - 1`, число BYE — `bracketSize - count`. В финале всегда один матч, потому что `bracketSize` — степень двойки.

#### 1.5 Генерация сетки

Полностью `generateSingleElim()`:

1. Защита от потери результатов: если есть хотя бы один сыгранный не-BYE матч — исключение «Уже есть сыгранные матчи — пересоздание сетки сотрёт результаты.» Контроллер ловит его и отдаёт `422` с текстом.
2. Выборка участников — только `approvedParticipants()`, порядок: сначала с заполненным сидом (`orderByRaw('seed IS NULL')`), затем по `seed`, затем по `id`.
3. Расчёт `bracketSize` и `totalRounds`.
4. Транзакция: удаление старых матчей, вставка одним `insert()`, связывание, обработка BYE, перевод турнира в `ongoing` (`starts_at` подставляется `now()`, если пуст).

Раскладка первого раунда — «складка»: сид `k` против сида `size-1-k`.

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php, строки 59-77
// Первый раунд: посев k играет с посевом (size-1-k). Участников
// больше половины сетки, поэтому в каждой паре есть хотя бы один,
// а «пустые» соперники достаются сильнейшим посевам
for ($k = 0; $k < $bracketSize / 2; $k++) {
    $p1 = $participants[$k] ?? null;
    $p2 = $participants[$bracketSize - 1 - $k] ?? null;

    $rows[] = self::row($tournament, 1, $k, $p1?->id, $p2?->id, ($p1 && $p2) ? 'ready' :
'pending', $now);
}

for ($r = 2; $r <= $totalRounds; $r++) {
    for ($i = 0; $i < $bracketSize / (2 ** $r); $i++) {
        $rows[] = self::row($tournament, $r, $i, null, null, 'pending', $now);
    }
}

TournamentMatch::insert($rows);
```

Свойства этой раскладки:

- в первом раунде ровно `bracketSize/2` матчей, в раунде `r` — `bracketSize / 2^r`, в финале — один;
- матчи всех раундов, кроме первого, создаются пустыми (`pending`, участники `null`);

- `position` — сквозной номер внутри раунда, начиная с 0; именно по нему считается, в какой слот следующего матча пойдёт победитель;
- участник с `p1 != null, p2 == null` получает BYE; поскольку сиды отсортированы по возрастанию, BYE достаётся сильнейшим;
- при `count`, равном степени двойки, BYE не возникает вовсе — все пары заполнены, потому что `count > bracketSize/2` выполняется всегда.

Каждый ряд строится хелпером `row()`: `bracket = 'main'`, `is_bye = false`, `created_at/updated_at` из одного `$now`.

#### 1.6 Связывание матчей

`linkNextMatches()` проставляет `next_match_id` и `next_slot` во всей сетке, группируя матчи по раунду:

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php, строки 121-145
$matches = $tournament->matches()->get();
$byRound = $matches->groupBy('round');

foreach ($byRound as $round => $roundMatches) {
    $nextRoundMatches = $byRound[$round + 1] ?? null;
    if (! $nextRoundMatches) {
        continue;
    }

    foreach ($roundMatches->sortBy('position')->values() as $i => $match) {
        $nextMatch = $nextRoundMatches->firstWhere('position', intdiv($i, 2));
        if (! $nextMatch) {
            continue;
        }

        $match->update([
            'next_match_id' => $nextMatch->id,
            'next_slot' => $i % 2 === 0 ? 'p1' : 'p2',
        ]);
    }
}
```

Правило: матч с чётным `position` отдаёт победителя в `participant1_id` следующего, с нечётным — в `participant2_id`; сам следующий матч — `position div 2`. Матч без `next_match_id` — финал; это и есть единственный признак финала (`TournamentMatch::isFinal()` и `AdminTournamentBracketModal` используют именно `!next_match_id`).

#### 1.7 BYE-матчи

BYE — это матч с ровно одним участником. Определяется исключающим ИЛИ по булевым приведениям, а не сравнением с `null`:

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php, строки 79-93
// Пропуски: одиночка в паре сразу проходит дальше
$tournament->matches()
    ->where('round', 1)
    ->get()
    ->filter(fn (TournamentMatch $m) => (bool) $m->participant1_id !== (bool) $m->participant2_id)
    ->each(function (TournamentMatch $m) {
        $m->update([
            'winner_id' => $m->participant1_id ?? $m->participant2_id,
            'status' => 'completed',
            'is_bye' => true,
            'completed_at' => now(),
        ]);

        self::advanceWinner($m->fresh());
    });
```

Что важно в этом блоке:

- фильтр работает только по первому раунду — во всех остальных раундах пустые матчи остаются `pending` и ждут победителей;
- BYE-матч сразу получает `status = 'completed'`, `winner_id` и `is_bye = true`, то есть выглядит как сыгранный, но помечен флагом;
- победитель BYE немедленно продвигается в следующий матч, и если у следующего уже есть оба участника — тот становится `ready`;
- BYE не считается победой в рейтинге: и `RatingService::matchWins()`, и `RatingService::tournamentWinPoints()` фильтруют `m.is_bye = false`, а в финальный подсчёт попадают только матчи с `next_match_id IS NULL`;
- фронтенд показывает такой матч подписью «Пропуск — без соперника» и исключает его из прогресса «сыграно N из M» (`AdminTournamentBracketModal.vue`, `progress`).

Флаг `is_bye` появился отдельной миграцией `2026_10_25_160000_add_is_bye_to_tournament_matches.php` с прямым комментарием: «Такой матч не считается победой в рейтинге».

#### 1.8 Продвижение победителя и завершение турнира

`advanceWinner()` — единственная точка, через которую результат попадает в сетку дальше:

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php, строки 154-178
if (! $match->winner_id) {
    return;
}

if ($match->next_match_id) {
    $next = $match->nextMatch;
    if (! $next) {
        return;
    }

    $field = $match->next_slot === 'p2' ? 'participant2_id' : 'participant1_id';
    $next->update([$field => $match->winner_id]);

    // Оба участника на месте — матч готов к игре
    if ($next->participant1_id && $next->participant2_id && $next->status === 'pending') {
        $next->update(['status' => 'ready']);
    }

    return;
}

self::finishTournament($match);
```

Историческая ремарка из кода: раньше метод выходил сразу, если следующего матча нет, — то есть ровно на финале, и чемпион нигде не фиксировался. Сейчас отсутствие `next_match_id` трактуется как финал.

`finishTournament()` делает пять вещей:

1. `status = 'completed'`, `ends_at = ends_at ?? now()`.
2. Удаляет `tournament_wins` прежнего чемпиона, если результат финала исправили: `where('user_id','!=',$winner->user_id ?? 0)->delete()`.
3. Для `type = 'solo'` создаёт `TournamentWin` (`firstOrCreate` по `user_id + tournament_id`, сохраняя `min_tier`/`max_tier` на момент победы), повышает тир, гоняет ачивки и шлёт уведомление:

```
// backend/app/Domains/Tournaments/Services/TournamentBracketService.php, строки 206-220
TournamentWin::firstOrCreate(
    ['user_id' => $user->id, 'tournament_id' => $tournament->id],
    ['min_tier' => $tournament->min_tier, 'max_tier' => $tournament->max_tier],
);

self::promoteByLevel($user, $tournament->level);

AchievementService::check($user);
AchievementService::grant($user, 'tournament_first_win');

$user->notify(new TournamentNotification(
    'tournament_won',
    "Вы выиграли турнир «{$tournament->name}»! +{$tournament->win_points} очков рейтинга.",
    $tournament->id,
));
```

1. Для кланового турнира `TournamentWin` **не** создаётся, уведомление получает только лидер клана (`$winner->clan?->leader`).
2. Повышение тира (`promoteByLevel()`): уровень `A` (диапазон `B..A`) при `tier_score >= 71` и текущем тире ниже `S` даёт `S`; уровень `S` (диапазон `A..S`) при `tier_score >= 71` и пяти победах в турнирах диапазона `A..S` даёт `S+`.

Победитель турнира в интерфейсе вычисляется на чтение, а не хранится отдельным полем: в `show()` берётся первый матч с `next_match_id === null` и непустым `winner_id`, а из `winner_id` достаётся участник и его игрок/клан. Это же значение видит фронтенд как `tournament.champion`.

Очки рейтинга не «начисляются» в момент финиша — они пересчитываются. Триггеры висят на событиях моделей:

```
// backend/app/Providers/AppServiceProvider.php, строки 113-127
TournamentMatch::saved(function (TournamentMatch $match) use ($recalc) {
    if (! $match->wasChanged(['winner_id', 'next_match_id']) && ! $match->wasRecentlyCreated) {
        return;
    }
    $participantIds = array_filter([$match->winner_id, $match->getOriginal('winner_id')]);
    $recalc(...TournamentParticipant::whereIn('id', $participantIds)->pluck('user_id')->all());
});

Tournament::updated(function (Tournament $tournament) use ($recalc) {
    if ($tournament->wasChanged('win_points')) {
        $recalc(...$tournament->participants()->pluck('user_id')->all());
    }
});
```

Формула (`RatingService`): среднее трёх тиров + победы в матчах турниров × `rating_match_win` (default 5) + сумма `win_points` выигранных турниров + клан-войны. Турнирные очки получают только одиночные участники (`whereNull('p.clan_id')`).

#### 1.9 Ввод результатов матчей

`PUT /api/admin/tournaments/{id}/matches/{match}` → `updateMatch()`. Логика:

- `abort_if($match->tournament_id !== $tournament->id, 404)` — матч обязан принадлежать турниру из пути;
- если пришёл `winner_id`, он обязан совпадать с одним из двух участников этого матча (`422 «Победитель должен быть одним из участников матча.»`);
- если следующий матч уже `completed` и победитель меняется — `422 «Следующий матч уже сыгран — сначала исправьте его.»`;
- при наличии `winner_id` принудительно ставится `status = 'completed'` и `completed_at = now()`;

- после `update()` вызывается `TournamentBracketService::advanceWinner($match->fresh())`.

Валидация (`UpdateTournamentMatchRequest`): `score1`/`score2` — `nullable|integer|min:0` (`score1`/`score2` в БД — `unsignedTinyInteger`), `winner_id` — `exists:tournament_participants,id`, `status` — `in:pending,ready,live,completed,cancelled`, `scheduled_at` — дата. Счёт не влияет на победителя: сервер не сравнивает `score1` и `score2`. Автоподстановка победителя по счёту есть только в браузере:

```
// frontend/src/components/admin/AdminTournamentBracketModal.vue
// Счёт подсказывает победителя: у кого больше — тот и выиграл
function autoWinner() {
  const s1 = Number(form.score1)
  const s2 = Number(form.score2)

  if (Number.isFinite(s1) && Number.isFinite(s2) && s1 !== s2 && form.score1 !== null &&
form.score2 !== null) {
    form.winner_id = s1 > s2 ? selected.value.participant1_id : selected.value.participant2_id
  }
}
```

Кнопка «Идёт сейчас» отправляет `{ status: 'live', winner_id: null }` — так матч помечается как играющийся, не завершаясь. Кнопка сохранения заблокирована, пока не выбран `winner_id`.

#### 1.10 Сиды

`POST /api/admin/tournaments/{id}/seeds` принимает `seeds: [{id, seed}]` и покомпонентно обновляет `tournament_participants.seed` с проверкой принадлежности турниру. Сид влияет только на порядок в `generateSingleElim()`: меньшее число = выше в списке = сильнее, и именно эти участники получают BYE. Ограничения на уникальность сида нет — дубликаты допустимы, тогда порядок добьётся `id`.

#### 1.11 Награды и призовой фонд

| Награда | Где реализована | Условие |
|---|---|---|
| `tournament_wins` (строка) | `finishTournament()` | победа в финале, `type = solo` |
| Повышение тира `A → S`, `S → S+` | `promoteByLevel()` | `tier_score >= 71`, уровень турнира, 5 побед для `S+` |
| Ачивка `tournament_first_win` («Чемпион») | `AchievementService::grant()` | первая победа |
| Ачивка `tournament_win_3` | `AchievementRules::groupTournaments()` | 3 победы |
| Ачивка `tournament_join` | `AchievementRules::groupTournaments()` | 1 участие |
| Очки рейтинга `win_points` | `RatingService::tournamentWinPoints()` | финальный матч, solo |
| Уведомление `tournament_won` | `TournamentNotification` | всегда |

Призовой фонд (`prize_pool`, `prize_currency`, `prize_description`) — только поля отображения: выплат, транзакций ApexCoin или начислений за призовые места в коде нет.

#### 1.12 Промокоды

В домене `Tournaments` промокодов нет. Они реализованы в домене `Control` (`backend/app/Domains/Control/`), но тематически примыкают к турнирам и раздачам, поэтому описываются здесь.

Таблицы (`2026_10_25_150000_create_promo_codes.php`):

`promo_codes`: `id`, `code` (string 32, unique), `coins` (uint, default 0), `shop_item_id` (FK `shop_items`, nullOnDelete), `max_uses` (uint, nullable — без лимита), `uses` (uint, default 0), `expires_at` (timestamp, nullable), `is_active` (bool, default true), `note` (string), `created_by` (FK users, nullOnDelete).

`promo_redemptions`: `id`, `promo_code_id` (cascade), `user_id` (cascade), `created_at`; уникальный индекс `(promo_code_id, user_id)` — один код на игрока один раз.

Создание (`promoStore()`): код либо задаётся (`regex:/^[A-Za-z0-9_-]+$/`, 3..32, unique), либо генерируется как `'APEX-'.Str::upper(Str::random(6))`; код всегда приводится к верхнему регистру. Требуется хотя бы одно из `coins`/`shop_item_id` (`422 «Промокод должен что-то давать: монеты или предмет.»`). Каждое создание пишется в журнал (`control.promo`).

Активация (`GiftService::redeem()`), порядок проверок: код ищется регистронезависимо (`whereRaw('UPPER(code) = ?')`), затем `is_active`, затем `expires_at`, затем `max_uses`, затем факт повторной активации. В транзакции: запись в `promo_redemptions`, `increment('uses')`, начисление монет через `CoinService::credit()` с источником `admin` и ключом идемпотентности `promo:{promo}:{user}`, выдача предмета в `user_inventory` (неповторяемый предмет, который уже есть, второй раз не кладётся — монеты при этом начисляются). Ответ: `{coins, item, balance}`.

#### 1.13 Эндпоинты

| Метод | Путь | Действие | Права |
|---|---|---|---|
| GET | `/api/tournaments` | список, фильтры `search`/`type`/`status`, 12 на страницу, порядок: `registration` → `ongoing` → `completed` → остальное, затем `created_at desc` | публично |
| GET | `/api/tournaments/{id}` | турнир + участники + матчи + `champion` + `my_participation` + `registration_open` | публично |
| POST | `/api/tournaments/{id}/register` | заявка (`pending`) | auth |
| POST | `/api/tournaments/{id}/withdraw` | удалить заявку, если не одобрена | auth |
| GET | `/api/admin/tournaments` | список для админки, фильтр `status`, 20 на страницу | moderator, admin |
| POST | `/api/admin/tournaments` | создание (multipart, баннер) | moderator, admin |
| PUT/POST | `/api/admin/tournaments/{id}` | обновление (баннер — файлом) | moderator, admin |
| DELETE | `/api/admin/tournaments/{id}` | удаление (участники и матчи каскадом) | moderator, admin |
| GET | `/api/admin/tournaments/{id}/participants` | участники с игроком/кланом | moderator, admin |
| POST | `/api/admin/tournaments/{id}/participants/{p}/` | одобрить + уведомление | moderator, admin |

| | | | |
|---|---|---|---|
| | `approve` | | |
| POST | `/api/admin/tournaments/{id}/participants/{p}/reject` | отклонить + уведомление | moderator, admin |
| DELETE | `/api/admin/tournaments/{id}/participants/{p}` | удалить, при необходимости сбросить сетку | moderator, admin |
| POST | `/api/admin/tournaments/{id}/seeds` | сохранить сиды | moderator, admin |
| GET | `/api/admin/tournaments/{id}/matches` | матчи с участниками и победителем | moderator, admin |
| POST | `/api/admin/tournaments/{id}/bracket/generate` | сгенерировать сетку | moderator, admin |
| PUT | `/api/admin/tournaments/{id}/matches/{m}` | счёт/победитель/статус | moderator, admin |

#### 1.14 Фронтенд

- `frontend/src/views/tournaments/TournamentsView.vue` — витрина: hero, «главный» турнир (первый со статусом `registration`/`ongoing` на первой странице без фильтров), обратный отсчёт до конца регистрации или старта (таймер обновляется раз в секунду), фильтры-чипы по типу и статусу, поиск с дебаунсом 300 мс, заполненность считается как `approved_count / max_participants`.
- `frontend/src/views/tournaments/TournamentView.vue` — страница турнира: баннер, чипы статуса/типа/диапазона тиров, отсчёт, кнопка участия, блок приза, четыре цифры (участники, старт, конец регистрации, заявки), блок чемпиона, сетка через тот же `AdminTournamentBracket` (в режиме просмотра), список участников со стопкой аватарок и тиром. Загрузка различает `404` (`PageState`) и прочие ошибки.
- `frontend/src/components/admin/AdminTournamentBracket.vue` — рендер сетки: колонки по раундам, заголовки «Финал/Полуфинал/Четвертьфинал/1/8 финала» и бейджи `F/SF/QF/R16`, слоты с аватаром, ником, счётом, подсветкой победителя, подписями «Пропуск — без соперника», «LIVE», «Завершён», «Ожидание». Кликабельность матча — только при `editable && !is_bye && participant1 && participant2`.
- `frontend/src/components/admin/AdminTournamentBracketModal.vue` — проведение турнира: загрузка матчей, выбор матча, форма счёта и радио победителя, кнопка «Идёт сейчас», прогресс по не-BYE матчам, определение чемпиона по матчу без `next_match_id`. Статус турнира после сохранения перечитывается из списка, потому что его меняет сервер.
- `frontend/src/components/admin/AdminTournamentParticipants.vue` — одобрение/отклонение/удаление, редактирование сидов, кнопка «Сформировать сетку» (доступна при `approvedCount >= 2` и `format === 'single_elim'`), сообщения об успехе и об сбросе сетки.
- `frontend/src/components/admin/AdminTournamentForm.vue` — создание/редактирование: название, описание, баннер, тип, формат (в селекте единственный вариант), статус (только при редактировании), приз, тиры, лимит, `win_points`, четыре даты. Пустые строки перед отправкой заменяются на `null`.

- `frontend/src/components/admin/AdminTournaments.vue` — список с фильтром по статусу, создание, редактирование, «Участники», «Сетка», удаление с двойным подтверждением.

Маршруты фронтенда: `/tournaments` (публичный), `/tournaments/:id` (`meta.auth = true` — страница требует входа, хотя API отдаёт данные и гостю).

## 2. Домен Moderation

Состав: `Models/Report.php`, `Models/ModerationLog.php`, `Services/ReportService.php`, `Controllers/ReportController.php`, `Controllers/DashboardController.php`, `Controllers/PlayerAdminController.php`, `Notifications/ModerationNotification.php`.

#### 2.1 Жалобы: объекты, причины, статусы

Константы живут в модели и являются единственным источником правды и для валидации, и для подписей:

```
// backend/app/Domains/Moderation/Models/Report.php
public const STATUS_OPEN = 'open';
public const STATUS_RESOLVED = 'resolved';
public const STATUS_DISMISSED = 'dismissed';

/** На что можно пожаловаться. */
public const SUBJECTS = ['user', 'message', 'forum_topic', 'forum_reply'];

public const REASONS = [
    'cheating' => 'Читы',
    'insult' => 'Оскорбления',
    'spam' => 'Спам или реклама',
    'scam' => 'Мошенничество',
    'nickname' => 'Ник или аватар',
    'other' => 'Другое',
];

/** Решения модератора. */
public const ACTIONS = ['dismiss', 'warn', 'ban', 'remove_content'];
```

| Объект (`subject_type`) | Что проверяется при подаче | Куда попадает удаление |
|---|---|---|
| `user` | пользователь существует | удалить нельзя — только бан/предупреждение |
| `message` (чат) | автор жалобы обязан быть участником диалога (`ConversationParticipant`), иначе `403` | жёсткое удаление строки `messages` |
| `forum_topic` | тема не удалена (`deleted_at IS NULL`) | soft delete + `deleted_by` |
| `forum_reply` | ответ не удалён | soft delete + `deleted_by` |

Статусы жалобы: `open` → `resolved` (меры приняты) или `dismissed` (нарушений не найдено). Третьего состояния нет; повторное рассмотрение невозможно — `resolve()` отдаёт `422 «Жалоба уже рассмотрена.»`.

#### 2.2 Подача жалобы

`POST /api/reports` (любой авторизованный, throttle 10 запросов в минуту на маршрут). Валидация: `type` ∈ `SUBJECTS`, `id` — целое ≥ 1, `reason` ∈ ключи `REASONS`, `comment` ≤ 1000.

```
// backend/app/Domains/Moderation/Services/ReportService.php, строки 27-55
public function submit(User $reporter, string $type, int $id, string $reason, ?string $comment):
```

```
Report
{
    [$targetId, $snapshot] = $this->resolveSubject($reporter, $type, $id);

    if ($targetId === $reporter->id) {
        $this->abortUnprocessable('Нельзя пожаловаться на самого себя.');
    }

    $duplicate = Report::where('reporter_id', $reporter->id)
        ->where('subject_type', $type)
        ->where('subject_id', $id)
        ->where('status', Report::STATUS_OPEN)
        ->exists();

    if ($duplicate) {
        $this->abortUnprocessable('Вы уже пожаловались — модераторы скоро рассмотрят жалобу.');
    }

    return Report::create([
        'reporter_id' => $reporter->id,
        'target_user_id' => $targetId,
        'subject_type' => $type,
        'subject_id' => $id,
        'reason' => $reason,
        'comment' => $comment,
        'snapshot' => $snapshot !== null ? Str::limit($snapshot, 2000) : null,
        'status' => Report::STATUS_OPEN,
    ]);
}
```

`snapshot` — копия текста на момент жалобы, обрезанная до 2000 символов: автор мог удалить или отредактировать сообщение, и модератор должен видеть, на что жаловались. Для `user` в снапшот попадает ник, для темы — `title` + `body`, для ответа и сообщения — тело.

Идентификатор автора объекта (`target_user_id`) определяется в `resolveSubject()`; `null` в этой колонке возможен только если игрока удалили позже (`nullOnDelete`).

Ответ на подачу — `201 {report: {id, status}}`. Эндпоинта «мои жалобы» в коде отсутствует: игрок узнаёт о решении только из уведомления.

#### 2.3 Очередь модератора

`GET /api/admin/reports` (`role:moderator,admin`). По умолчанию `status=open`, по 30 на страницу. Сортировка: открытые — `created_at asc` (сначала самые старые), остальные — `resolved_at desc`. Фильтры: `status` (`open|resolved|dismissed|all`), `target` (id игрока).

К каждой жалобе добавляется `reason_label` и `reports_on_target` — сколько всего жалоб на этого игрока (отдельный `groupBy` по `target_user_id` пачкой, без N+1). В ответе также отдаётся словарь `reasons` целиком.

#### 2.4 Решения

`POST /api/admin/reports/{report}/resolve`, тело: `action` ∈ `ACTIONS`, `note` ≤ 500, `ban_days` 1..365 (пусто — бессрочно).

| Действие | Что происходит | Итоговый статус |
|---|---|---|
| `dismiss` | ничего, кроме фиксации | `dismissed` |
| `warn` | уведомление нарушителю `moderation_warning` + журнал `user.warn` | `resolved` |
| `ban` | бан аккаунта + журнал `user.ban` | `resolved` |

| | | |
|---|---|---|
| `remove_content` | удаление сообщения/темы/ответа + журнал `content.remove` | `resolved` |

Ключевая особенность — «пакетное» закрытие: одно решение закрывает **все** открытые жалобы на тот же объект, и каждый заявитель получает уведомление.

```
// backend/app/Domains/Moderation/Services/ReportService.php, строки 73-106
$status = $action === 'dismiss' ? Report::STATUS_DISMISSED : Report::STATUS_RESOLVED;

// Закрываем все открытые жалобы на этот же объект одним решением
$related = Report::where('subject_type', $report->subject_type)
    ->where('subject_id', $report->subject_id)
    ->where('status', Report::STATUS_OPEN)
    ->get();

foreach ($related as $item) {
    $item->update([
        'status' => $status,
        'resolution' => $action,
        'moderator_id' => $moderator->id,
        'moderator_note' => $note,
        'resolved_at' => now(),
    ]);

    $item->reporter?->notify(new ModerationNotification(
        'report_resolved',
        $status === Report::STATUS_DISMISSED
            ? 'Жалоба рассмотрена: нарушений не найдено.'
            : 'Жалоба рассмотрена: меры приняты. Спасибо, что помогаете APEX!',
        ['report_id' => $item->id],
    ));
}

ModerationLog::record(
    $moderator,
    $status === Report::STATUS_DISMISSED ? 'report.dismiss' : 'report.resolve',
    $report->target_user_id,
    ['reports' => $related->count(), 'action' => $action, 'note' => $note],
    'report',
    $report->id,
);
```

Всё в `DB::transaction`. Запись в журнал одна на всю пачку, с числом закрытых жалоб в `details.reports`.

#### 2.5 Баны и ограничения

Отдельной таблицы банов нет — состояние лежит в `users` (`2026_09_24_093413_add_role_and_ban_to_users_table.php`): `is_banned` (bool), `ban_reason` (string), `banned_until` (timestamp, null = бессрочно), `banned_by` (FK users, nullOnDelete).

Правила из `ReportService::ban()`:

```
// Персонал банит только администратор, администратора — никто
if ($target->isAdmin() || ($target->isStaff() && ! $moderator->isAdmin())) {
    $this->abortForbidden('Этого пользователя нельзя забанить.');
}

$reason = trim(Report::REASONS[$report->reason].($note ? ": {$note}" : ''));

$target->update([
    'is_banned' => true,
    'ban_reason' => $reason,
    'banned_until' => $days ? now()->addDays($days) : null,
    'banned_by' => $moderator->id,
]);
```

То есть причина бана всегда собирается из причины жалобы, к ней добавляется заметка модератора. Срок — либо N дней, либо `null` (бессрочно).

Срок проверяется на чтение: `User::isBanned()` возвращает `false`, если `banned_until` уже в прошлом, — то есть бан «истекает» сам, без задачи и без изменения `is_banned` в БД. Блокировка обеспечивается middleware `EnsureUserIsNotBanned`, навешенным на всю группу `api`:

```
// backend/app/Http/Middleware/EnsureUserIsNotBanned.php
if ($user && $user->isBanned()) {
    $token = $user->currentAccessToken();

    // Вход по токену API — удаляем токен. Вход через сайт (сессия) даёт
    // TransientToken без delete(): там завершаем сессию, иначе был 500
    if ($token instanceof PersonalAccessToken) {
        $token->delete();
    } elseif ($request->hasSession()) {
        Auth::guard('web')->logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
    }

    return response()->json(['message' => 'Ваш аккаунт забанен.', 'reason' => $user->ban_reason],
403);
}
```

Прямой бан/разбан из карточки игрока — `POST /api/admin/users/{user}/ban` и `/unban` (только `role:admin`): `BanUserRequest` требует `reason` (≤ 500) и опциональный `until` (`after:now`), запрещает банить себя и администратора; разбан обнуляет все четыре поля. Оба действия пишутся в журнал.

Ограничений мягче бана в коде нет: ни мутов, ни счётчика предупреждений, ни «только чтение». `warn` — это уведомление и запись в журнал, без сохранения состояния на аккаунте.

#### 2.6 Журнал модерации

Таблица `moderation_logs` (`2026_10_25_140000_create_reports_and_moderation_logs.php`): `id`, `actor_id` (FK users, nullOnDelete), `action` (string 48, индекс), `target_user_id` (FK users, nullOnDelete), `subject_type` (string 32, nullable), `subject_id` (bigint, nullable), `details` (json), `created_at` (только created_at — `public const UPDATED_AT = null`). Индексы: `action`, `created_at`, `(target_user_id, created_at)`.

Запись — единая точка:

```
// backend/app/Domains/Moderation/Models/ModerationLog.php
public static function record(
    ?User $actor,
    string $action,
    ?int $targetUserId = null,
    array $details = [],
    ?string $subjectType = null,
    ?int $subjectId = null,
): self {
    return static::create([
        'actor_id' => $actor?->id,
        'action' => $action,
        'target_user_id' => $targetUserId,
        'subject_type' => $subjectType,
        'subject_id' => $subjectId,
        'details' => $details ?: null,
    ]);
}
```

Что логируется (карта `LABELS`, 27 действий):

| Группа | action |
|---|---|
| Баны | user.ban, user.unban, user.warn |
| Аккаунт | user.role, user.verify, user.unverify, user.rename, user.password_reset_sent, user.password_set, user.email_verified, user.sessions_revoked, user.minecraft_unlinked |
| Профиль | user.avatar_removed, user.cover_removed, user.profile_cleared, user.note |
| Экономика | user.coins, user.item_gift |
| Жалобы и контент | report.dismiss, report.resolve, content.remove |
| «Пульт» | control.update, control.giveaway, control.promo |
| Прочее | bridge.submission_updated, bridge.submission_deleted, changelog.create, changelog.delete |

Чтение — GET /api/admin/moderation-log: фильтры action (поиск по префиксу, LIKE 'action%') и target, 40 записей на страницу, сортировка created_at desc, id desc, к каждой записи добавляется label из LABELS (fallback — сам action).

Заметка о игроке (note) — не отдельная сущность, а запись журнала user.note с details.note: «внутренняя заметка персонала об игроке — видна только в досье и журнале».

#### 2.7 Дашборд, счётчики, досье

GET /api/admin/counters — четыре числа для бейджей в меню, опрашивается раз в минуту: открытые жалобы, апелляции тир-тестов, тир-тесты в pending, открытые обращения поддержки.

GET /api/admin/dashboard — блоки: attention (+ активные турниры и клан-войны), queue (тир-тесты: сколько ждут, самый старый, среднее время взятия за 30 дней, завершено за неделю), players (всего, новые за день/неделю, онлайн за 5 минут, активные за неделю, забаненные), testers (по роли tester/admin, тесты за неделю, последний визит), tests_by_day (14 дней), economy (только админу), recent_actions (8 последних записей журнала).

GET /api/admin/users/{user}/dossier — карточка игрока: профиль, форумная активность (включая удалённые темы/ответы), рефералы, место в рейтинге, разбивка очков, последние 10 тир-тестов, счётчики жалоб «на него» и «от него», 15 записей журнала по нему. Три поля скрыты от модератора и отдаются только админу: email, apex_coins, referral_code (и блок transactions). Логика — тернарники по $isAdmin = $request->user()->isAdmin().

GET /api/admin/players/search?q= — быстрый поиск игрока по нику для перехода в досье: минимум 2 символа, экранирование %, _, ! с ESCAPE '!', сортировка по длине ника, лимит 8, в выдаче нет почты.

#### 2.8 Действия с аккаунтом из досье

PlayerAdminController разделён по правам: чистка профиля и заметки — модератору и админу, всё остальное — только админу (маршруты в разных группах).

| Метод | Действие | Права |
|---|---|---|
| removeAvatar / removeCover | удалить файл и обнулить поле | moderator, admin |

| | | |
|---|---|---|
| `clearProfile` | стереть `bio`, `status`, `quote` | moderator, admin |
| `note` | запись в журнал `user.note`, `text` ≤ 1000 | moderator, admin |
| `rename` | ник 3..32, `^[a-zA-Z0-9_]+$`, unique | admin |
| `sendPasswordReset` | письмо-код через `PasswordResetService`, throttle 5/мин | admin |
| `temporaryPassword` | пароль `xxxx-xxxx-xxxx` (нижний регистр), сброс сессий, показывается один раз | admin |
| `verifyEmail` | проставить `email_verified_at`, если ещё не подтверждена | admin |
| `logoutEverywhere` | обнулить `remember_token`, снести сессии и токены | admin |
| `unlinkMinecraft` | обнулить `minecraft_uuid/username/linked_at` | admin |
| `referrals` | сводка рефералов: итоги, топ-20, последние 30 | admin |

Общий предохранитель:

```
// backend/app/Domains/Moderation/Controllers/PlayerAdminController.php
/** Себя и администратора так трогать нельзя; персонал — только админу. */
private function guardTarget(Request $request, User $user): void
{
    $actor = $request->user();

    abort_if($user->id === $actor->id && ! $actor->isAdmin(), 422, 'Это действие нельзя применить к себе.');
    abort_if($user->isAdmin() && $user->id !== $actor->id, 403, 'Нельзя менять аккаунт администратора.');
    abort_if($user->isStaff() && ! $actor->isAdmin(), 403, 'Аккаунт персонала меняет только администратор.');
}
```

Снятие сессий учитывает драйвер: `database` — `delete from sessions where user_id`, `file` — обход файлов сессий с поиском ключа `login_web_` и нужного id (на проде `SESSION_DRIVER=file`), в конце — удаление Sanctum-токенов, если у модели есть `tokens()`.

#### 2.9 Эндпоинты

| Метод | Путь | Действие | Права |
|---|---|---|---|
| GET | `/api/reports/reasons` | список причин для формы | auth |
| POST | `/api/reports` | подать жалобу, throttle 10/мин | auth |
| GET | `/api/admin/reports` | очередь жалоб, 30 на страницу | moderator, admin |
| POST | `/api/admin/reports/{r}/resolve` | решение по жалобе | moderator, admin |
| GET | `/api/admin/moderation-log` | журнал, 40 на страницу | moderator, admin |
| GET | `/api/admin/dashboard` | дашборд | moderator, admin |

| | | | |
|---|---|---|---|
| GET | `/api/admin/counters` | бейджи меню | moderator, admin |
| GET | `/api/admin/users/{u}/dossier` | досье игрока | moderator, admin |
| GET | `/api/admin/players/search` | поиск игрока по нику | moderator, admin |
| POST | `/api/admin/users/{u}/remove-avatar` | удалить аватар | moderator, admin |
| POST | `/api/admin/users/{u}/remove-cover` | удалить обложку | moderator, admin |
| POST | `/api/admin/users/{u}/clear-profile` | очистить «о себе» | moderator, admin |
| POST | `/api/admin/users/{u}/note` | заметка в журнал | moderator, admin |
| POST | `/api/admin/users/{u}/ban` | бан (причина + срок) | admin |
| POST | `/api/admin/users/{u}/unban` | разбан | admin |
| POST | `/api/admin/users/{u}/rename` | сменить ник | admin |
| POST | `/api/admin/users/{u}/send-password-reset` | письмо сброса, throttle 5/мин | admin |
| POST | `/api/admin/users/{u}/temporary-password` | временный пароль | admin |
| POST | `/api/admin/users/{u}/verify-email` | подтвердить почту | admin |
| POST | `/api/admin/users/{u}/logout-everywhere` | разлогинить везде | admin |
| POST | `/api/admin/users/{u}/unlink-minecraft` | отвязать Minecraft | admin |
| GET | `/api/admin/referrals` | реферальная сводка | admin |

#### 2.10 Фронтенд

- `frontend/src/components/core/ReportDialog.vue` — окно «Пожаловаться»: заголовок зависит от `subject_type`, шесть причин-кнопок, комментарий ≤ 1000, состояние «Жалоба отправлена». Состояние окна живёт в `composables/core/useReport.js` (`reportState`, `closeReport`), сам запрос — `api.post('/reports', {type, id, reason, comment})`.
- `frontend/src/components/admin/AdminReports.vue` — очередь: чипы `open/resolved/dismissed/all`, карточка с причиной, типом объекта, счётчиком «N жалоб на игрока», цитатой `snapshot`, кнопками «Отклонить/Предупредить/Удалить контент/Бан» и селектом срока (1, 3, 7, 30 дней, «Навсегда»). Кнопка «Удалить контент» скрыта для жалоб на профиль — это совпадает с сервером, который отвечает `422`.
- `frontend/src/components/admin/AdminModerationLog.vue` — журнал: чипы по префиксам (`user.ban`, `user.unban`, `user.warn`, `report`, `content`, `user.role`, `user.coins`), человеческое описание `details` для банов, ролей, монет и решений по жалобам.
- `frontend/src/components/admin/AdminDashboard.vue`, `AdminDossier.vue`, `AdminUserActions.vue`, `AdminBanModal.vue` — дашборд, досье, кнопки действий с аккаунтом и модалка бана.
- `frontend/src/views/admin/AdminView.vue` — меню: разделы «Модерация» (`support`, `reports`, `appeals`, `log`) видны модератору, `adminOnly`-пункты (`control`, `users`, `verification`,

`referrals`, `news`, `achievements`, `home`, `project`) — только админу; активный раздел хранится в query-параметре `?tab=`.

## 3. Домен Support

Состав: `Models/SupportTicket.php`, `Models/SupportMessage.php`, `Services/SupportService.php`, `Controllers/SupportController.php`, `Controllers/AdminSupportController.php`, `Notifications/SupportNotification.php`.

#### 3.1 Схема

`support_tickets` (`2026_10_25_190000_create_support_tickets.php`): `id`, `user_id` (FK, cascade), `subject` (string 120), `category` (string 24, default `other`), `status` (string 16, default `open`), `assigned_to` (FK users, nullOnDelete), `last_message_at` (timestamp), `timestamps`. Индексы: `(status, last_message_at)`, `(user_id, last_message_at)`.

`support_messages`: `id`, `ticket_id` (FK, cascade), `user_id` (FK, nullOnDelete — сотрудника могли удалить), `is_staff` (bool), `body` (text), `timestamps`; индекс `(ticket_id, id)`.

Категории — константа модели, дублированная на фронте в `components/support/supportFormat.js`:

| Ключ | Подпись |
|---|---|
| `account` | Аккаунт и вход |
| `tier_test` | Тир-тесты |
| `shop` | Магазин и ApexCoin |
| `clan` | Кланы |
| `bug` | Ошибка на сайте |
| `other` | Другое |

#### 3.2 Создание обращения

`POST /api/support/tickets` (throttle `5,10` — 5 запросов за 10 минут на маршрут). Валидация: `subject` 3..120, `category` ∈ ключи `CATEGORIES` (nullable, по умолчанию `other`), `body` 5..4000.

```
// backend/app/Domains/Support/Services/SupportService.php, строки 24-45
public function open(User $user, string $subject, string $category, string $body): SupportTicket
{
    $active = SupportTicket::where('user_id', $user->id)
        ->where('status', '!=', SupportTicket::STATUS_CLOSED)
        ->count();

    abort_if($active >= self::MAX_OPEN, 422, 'У вас уже ' . self::MAX_OPEN . ' открытых обращения
— дождитесь ответа или закройте решённые.');

    return DB::transaction(function () use ($user, $subject, $category, $body) {
        $ticket = SupportTicket::create([
            'user_id' => $user->id,
            'subject' => $subject,
            'category' => array_key_exists($category, SupportTicket::CATEGORIES) ? $category :
'other',
            'status' => SupportTicket::STATUS_OPEN,
            'last_message_at' => now(),
        ]);

        $ticket->messages()->create(['user_id' => $user->id, 'is_staff' => false, 'body' =>
$body]);
```

```
        return $ticket;
    });
}
```

Лимит `MAX_OPEN = 3` считается по незакрытым обращениям (и `open`, и `answered`). Неизвестная категория молча превращается в `other` — валидация контроллера, впрочем, до этого не доводит.

#### 3.3 Статусы и переходы

| Статус | Значение | Кто ставит |
|---|---|---|
| `open` | ждёт ответа персонала | создание обращения; любой ответ игрока (в том числе в закрытое) |
| `answered` | персонал ответил, ждёт игрока | первый и последующие ответы персонала |
| `closed` | вопрос решён | игрок или персонал, вручную |

Переходов-ограничений нет: закрытый тикет можно закрыть повторно, а ответ игрока в закрытый тикет молча переоткрывает его (`playerReply()` безусловно ставит `open`). В интерфейсе это поддержано подсказкой в `SupportThread.vue`: «Обращение закрыто — напишите, чтобы открыть его снова».

При первом ответе персонала тикет «закрепляется» за сотрудником: `assigned_to = assigned_to ?? staff->id` — то есть первый ответивший становится ответственным, дальше поле не меняется.

#### 3.4 Ответы и уведомления

Два почти симметричных метода: `playerReply()` (статус → `open`, уведомление ответственному сотруднику) и `staffReply()` (статус → `answered`, уведомление игроку). Оба обновляют `last_message_at` — по нему строится сортировка в обеих очередях.

Уведомления дедуплицируются, чтобы не заваливать человека: новое уведомление создаётся, только если по этому тикету уже нет непрочитанного.

```
// backend/app/Domains/Support/Services/SupportService.php, строки 119-133
private function notifyOnce(?User $recipient, string $type, string $message, SupportTicket
$ticket): void
{
    if (! $recipient) {
        return;
    }

    $pending = $recipient->unreadNotifications()
        ->where('data->type', $type)
        ->where('data->ticket_id', $ticket->id)
        ->exists();

    if (! $pending) {
        $recipient->notify(new SupportNotification($type, $message, $ticket->id));
    }
}
```

Типы уведомлений: `support_reply` (игроку), `support_player_reply` (сотруднику, который ведёт тикет). Канал — только `database`.

#### 3.5 Кто что видит

- Игрок: `GET /api/support/tickets` — только свои, сортировка по `last_message_at desc`, лимит 50; в ответе словарь `categories`. Любое обращение по id проверяется на владельца, и чужое отдаёт **404** (не 403) — `authorizeOwner()`.
- Персонал (`role:moderator,admin`): `GET /api/admin/support` — все тикеты, фильтр `status` (`open|answered|closed|all`), по умолчанию `open`; при `open` сортировка `last_message_at asc` (дольше всех ждущие сверху), иначе `id desc`; лимит 100. Плюс `counts` по `open` и `answered`.
- Детали тикета отдаются через `present($ticket, true)`: шапка (тема, категория + подпись, статус, заявитель, ответственный, `last_message_at`, `created_at`) и массив сообщений с автором и флагом `is_staff`.
- Приватность обеспечивается только `user_id` тикета; внутренних заметок персонала в переписке нет — всё, что пишет сотрудник, видит игрок.

#### 3.6 Эндпоинты

| Метод | Путь | Действие | Права | | | |
|---|---|---|---|---|---|---|
| GET | `/api/support/tickets` | свои обращения + категории | auth | | | |
| POST | `/api/support/tickets` | создать обращение, throttle 5 за 10 мин | auth | | | |
| GET | `/api/support/tickets/{id}` | переписка, только своё (иначе 404) | auth | | | |
| POST | `/api/support/tickets/{id}/messages` | ответ игрока, throttle 20/мин | auth | | | |
| POST | `/api/support/tickets/{id}/close` | закрыть | auth | | | |
| GET | `/api/admin/support` | очередь, `?status=open\` | answered\ | closed\ | all` | moderator, admin |
| GET | `/api/admin/support/{id}` | переписка | moderator, admin | | | |
| POST | `/api/admin/support/{id}/messages` | ответ персонала | moderator, admin | | | |
| POST | `/api/admin/support/{id}/close` | закрыть | moderator, admin | | | |

#### 3.7 Фронтенд

- `frontend/src/views/support/SupportView.vue` — двухпанельный экран: слева список обращений и кнопка «Новое обращение», справа форма создания (тема, раздел, сообщение) или переписка. Открытая переписка обновляется каждые 15 секунд, но только

если вкладка видима и нет незавершённого действия. Категории инициализируются локальной копией `SUPPORT_CATEGORIES`, чтобы форма работала до загрузки списка.

- `frontend/src/components/support/SupportThread.vue` — общий для игрока и персонала компонент переписки; проп `staff` меняет сторону «своих» сообщений и подпись («Поддержка APEX» вместо ника), Enter с Ctrl/Cmd отправляет, кнопка закрытия скрыта для закрытого тикета.
- `frontend/src/components/admin/AdminSupport.vue` — очередь с чипами «Ждут ответа / Ответили / Закрытые / Все», счётчиками, выбором тикета и кнопкой перехода в досье заявителя.
- Маршрут `/support/:id?` требует авторизации.

## 4. Домен News

Состав: `Models/News.php`, `Controllers/Api/NewsController.php`, `Controllers/Api/Admin/NewsController.php`, `Requests/Admin/{Store,Update}NewsRequest.php` и неиспользуемая пара `Requests/News/{Store,Update}NewsRequest.php` (в контроллерах не встречается — мёртвый код).

#### 4.1 Схема

Таблица `news` (`2026_09_24_114501_create_news_table.php`):

| Поле | Тип | Комментарий |
|---|---|---|
| `title` | string | ≤ 120 в валидации |
| `slug` | string, unique | `Str::slug(title).'-'.Str::random(5)`; для маршрутизации не используется |
| `excerpt` | text | ≤ 255, идёт в карточку |
| `body` | longText | HTML визуального редактора либо старый plain text |
| `cover` | string | файл в `storage/public/news` |
| `type` | enum `news`,`update`,`event`,`announcement` | «категория» публикации |
| `is_pinned` | bool | закрепление (первым в списках) |
| `is_published` | bool, default true | `false` = черновик |
| `author_id` | FK users, cascade | автор |
| `published_at` | timestamp, nullable | дата публикации |

Индекс `(is_published, published_at)`. Модель добавляет `cover_url` (`asset('storage/'.$cover)`) в `$appends`.

Тегов, отдельной таблицы категорий и связи многие-ко-многим в коде отсутствуют — роль категории играет поле `type`.

#### 4.2 Публикация, черновики, планирование

- Список (`GET /api/news`): `is_published = true` **и** (`published_at IS NULL` или `published_at <= now()`), с автором, сортировка `is_pinned desc, published_at desc, created_at desc`, по 12 на страницу.
- Деталь (`GET /api/news/{id}`): если новость не опубликована — `404`, но администратору отдаётся (проверка `auth()->user()?->isAdmin()`, маршрут при этом публичный). Дополнительно отдаются `prev` и `next` — ближайшие опубликованные по `id` (только `id` и `title`).
- Админский список (`GET /api/admin/news`): все новости, включая черновики, `is_pinned desc, created_at desc`, по 20.
- Создание: `published_at` подставляется `now()`, если не передан, — то есть «опубликовать сейчас». Чтобы сделать черновик, ставят `is_published = false`; `published_at` при этом всё равно заполнится, но публичный список отфильтрует запись по флагу. Полноценного планирования «опубликовать в будущем» нет: `is_published` не переключается по времени, а запись с `published_at` в будущем просто не попадёт в список до наступления даты — при этом она уже помечена как опубликованная.
- Обновление и удаление работают с файлом обложки: при замене старый файл удаляется, при удалении новости — тоже.

#### 4.3 Санитайзер HTML

Текст из визуального редактора чистится мутатором модели при каждой записи:

```
// backend/app/Domains/News/Models/News.php
public function setBodyAttribute(?string $value): void
{
    $this->attributes['body'] = $value !== null && preg_match('/<[a-z][\s\S]*>/i', $value)
        ? HtmlSanitizer::clean($value)
        : $value;
}
```

`App\Support\HtmlSanitizer` — белый список: `p, br, hr, h2, h3, strong, b, em, i, u, s, ul, ol, li, blockquote, a[href,class], img[src,alt]`. Теги из `DROP` (`script, style, iframe, object, embed, form, input, button, textarea, select, svg, math, template, noscript`) вырезаются вместе с содержимым; незнакомые теги «разворачиваются» (текст остаётся), а `div/section/article` превращаются в `<p>`. Ссылки — только `http(s)`, `mailto` и относительные `/...`; картинки — только `https`. Единственный разрешённый класс ссылки — `news-btn`; всем ссылкам добавляется `target="_blank" rel="noopener noreferrer nofollow"`, картинкам — `loading="lazy"`.

На фронте признак «это HTML» определяется тем же регулярным выражением (`NewsItemView.vue`), поэтому старые новости с обычным текстом рендерятся абзацами, а не как разметка.

#### 4.4 Эндпоинты

| Метод | Путь | Действие | Права |
|---|---|---|---|
| GET | `/api/news` | опубликованные, 12 на страницу | публично |
| GET | `/api/news/{id}` | новость + `prev`/`next`; черновик — только админу | публично |

| | | | |
|---|---|---|---|
| GET | `/api/admin/news` | все, включая черновики, 20 на страницу | admin |
| POST | `/api/admin/news` | создать (multipart, обложка) | admin |
| PUT/POST | `/api/admin/news/{id}` | обновить (POST — для multipart) | admin |
| DELETE | `/api/admin/news/{id}` | удалить вместе с обложкой | admin |

#### 4.5 Фронтенд

- `frontend/src/views/news/NewsView.vue` — сетка карточек: обложка или первая буква заголовка, бейдж закрепления 📌, бейдж типа («Новость/Обновление/Событие/Анонс»), дата в формате `dd MMM yyyy`.
- `frontend/src/views/news/NewsItemView.vue` — статья: обложка, бейджи, дата, тело (HTML или plain text), навигация «назад к новостям»; различает 404 и прочие ошибки.
- Главная показывает 6 последних новостей тем же фильтром (`HomeController::index`).
- Админка: `AdminNews.vue` (список с типами и статусами), `AdminNewsForm.vue` (заголовок со счётчиком 120, короткое описание 255, тело в `RichTextEditor` со счётчиком «буквы без HTML», тип, закрепление, публикация, дата, обложка с превью и удалением). Раздел скрыт для модератора (`adminOnly`).

## 5. Домен Core

Состав домена минимален: `Models/SiteSetting.php`, `Controllers/Api/SiteController.php`, `Controllers/Api/Admin/SiteSettingsController.php`, `Requests/Admin/UpdateSiteSettingsRequest.php`. Главная страница физически живёт в `Players\Controllers\HomeController`, но целиком построена на настройках Core; объявления — в `Control`.

#### 5.1 SiteSetting: key-value с типами

Таблица `site_settings` (`2026_09_24_114457_create_site_settings_table.php`): `key` (string, unique), `value` (text, nullable), `type` (string 16, default `string`), `group` (string 32, default `general`), timestamps, индекс по `group`.

Модель — статический репозиторий с приведением типов:

```
// backend/app/Domains/Core/Models/SiteSetting.php
public static function get(string $key, $default = null)
{
    $setting = static::where('key', $key)->first();
    if (!$setting) return $default;

    return static::cast($setting->value, $setting->type, $default);
}

protected static function cast($value, string $type, $default = null)
{
    return match ($type) {
        'bool' => (bool) $value,
        'int' => (int) $value,
        'json' => $value ? json_decode($value, true) : $default,
        default => $value,
```

```
    };
}
```

`set()` делает `updateOrCreate` по ключу и кодирует значение (`bool` → `'1'/'0'`, `json` → `json_encode`, остальное → строка). `group($name)` возвращает ассоциативный массив всей группы с приведением типов — именно этим пользуются контроллеры.

Кеша на уровне модели нет: каждая настройка — отдельный `SELECT`. Кешируются только агрегаты: `site:footer` (300 с) в `SiteController`, `home:stats` (300 с) в `HomeController`, `control:state` (60 с) в `ControlService`.

#### 5.2 Группы настроек

| Группа | Ключи | Где читается |
|---|---|---|
| `hero` | `hero_title`, `hero_subtitle`, `hero_badge`, `hero_primary_text`, `hero_primary_url`, `hero_secondary_text`, `hero_secondary_url` | `GET /api/home` |
| `socials` | `social_discord`, `social_telegram`, `social_youtube`, `social_vk`, `social_twitch`, `social_twitter` | `/api/home`, `/api/site` (подвал) |
| `footer` | `footer_text` | `/api/home`, `/api/site` |
| `stats` | `stats_show` (bool) | `/api/home` |
| `rating` | `rating_match_win` (5), `rating_war_participation` (3), `rating_war_win` (7) | `RatingService::weights()` |
| `control` | `feature_*` (bool), `announcement` (json), `coin_multiplier` (int), `coin_multiplier_until` | `ControlService` |

Значения по умолчанию засеяны `SiteSettingsSeeder`: `hero_title = 'APEX TIERS'`, `hero_subtitle = 'Рейтинг игроков и кланов Minecraft PvP'`, `hero_badge = 'Season 1 · Live'`, кнопки `/players` и `/clans`, `footer_text = '© 2026 APEX TIERS. Все права защищены.'`, `stats_show = true`, соцсети пустые.

#### 5.3 Главная страница

`GET /api/home` (`HomeController::index`) собирает один ответ из восьми блоков:

| Блок | Источник | Особенности |
|---|---|---|
| `hero`, `socials`, `footer`, `stats` | `SiteSetting::group(...)` | сырые группы настроек |
| `stats_data` | кеш `home:stats`, 300 с | `players` (все), `clans` (не забаненные), `tournaments` (`registration` + `ongoing`), `matches` (`completed`) |
| `players` | топ-10 по `rating_points` | `excludeStaff()` — персонал в топ не попадает; отдаётся `UserCardResource` (рамка, эффект, статус) |
| `clans` | топ-10 по `power` | только `is_banned = false`, |

| | | |
|---|---|---|
| | | `withCount('members')`, отдаётся `ClanCardResource` (подсветка с учётом срока) |
| `news` | 6 последних опубликованных | тот же фильтр, что в `NewsController::index` |

Отдельный метод `HomeController::top()` (топ игроков/кланов без ресурсов) в маршрутах не используется — за `/api/top` отвечает `Players\Controllers\TopController`. Панель вошедшего игрока — `GET /api/home/me` (`HomeController::dashboard`, `HomeDashboardService`: кулдауны, питомцы, надетое).

#### 5.4 `/api/site`: подвал и состояние сайта

```
// backend/app/Domains/Core/Controllers/Api/SiteController.php, строки 33-48
public function index(): JsonResponse
{
    $data = Cache::remember('site:footer', 300, fn () => [
        'socials' => array_filter(SiteSetting::group('socials')),
        'footer_text' => SiteSetting::group('footer')['footer_text'] ?? null,
    ]);

    return response()->json([
        ...$data,
        'server_address' => config('services.minecraft.address') ?:
config('services.minecraft.host'),
        // Очки за турниры и войны — для страницы правил (настраиваются в админке)
        'rating' => app(\App\Domains\Players\Services\RatingService::class)->weights(),
        // «Пульт»: баннер на весь сайт, выключенные функции, событие xN
        'control' => \App\Domains\Control\Services\ControlService::publicState(),
    ]);
}
```

Тонкость: пустые соцсети отсекаются `array_filter`, поэтому в ответе остаются только заполненные ссылки; адрес сервера берётся из конфига, а не из БД; блок `rating` нужен странице правил, чтобы показать актуальные веса очков. Кешируется только часть про подвал — `control` читается на каждом запросе (внутри у него свой кеш на 60 с).

`GET /api/site/server-status` — статус Minecraft-сервера через `MinecraftStatusService::status()`, throttle 60/мин. `GET /api/site/team` — персонал по ролям в фиксированном порядке `admin → moderator → tester → bridge_curator → bridge_tester`, внутри — по числу завершённых тир-тестов; в выдаче `role_label`, `tests`, `since`. Почта и прочие приватные поля не отдаются.

#### 5.5 Объявления (SiteAnnouncement)

Модели `SiteAnnouncement` не существует. Объявление — это одна строка `site_settings` с `key = 'announcement'`, `type = 'json'`, `group = 'control'`. Структура: `{id, text, tone, link, link_label, until}`.

```
// backend/app/Domains/Control/Services/ControlService.php, строки 59-73
/** Активное объявление или null — истёкшее не показываем. */
public static function announcement(): ?array
{
    $announcement = self::state()['announcement'];

    if (! is_array($announcement) || empty($announcement['text'])) {
        return null;
    }

    if (! empty($announcement['until']) && Carbon::parse($announcement['until'])->isPast()) {
        return null;
```

```
    }

    return $announcement;
}
```

- Тон (`tone`): `info`, `success`, `warning`, `danger` — четыре цвета баннера.
- `link` + `link_label` — внутренняя ссылка (`/tournaments`) рендерится `RouterLink`, внешняя — `<a target="_blank">`.
- `until` — дата, после которой баннер не показывается; истёкшее объявление остаётся в БД, но `announcement()` вернёт `null`.
- `id` генерируется заново при каждом сохранении (`Str::random(8)`), чтобы «закрытый» игроком баннер не возвращался, а новое объявление показывалось всем. Закрытие хранится на клиенте: `localStorage['apex:announcement-dismissed']`.
- Публикация пишется в журнал модерации как `control.update` с текстом объявления, переключателями и множителем.

Тот же «Пульт» отдаёт переключатели функций (`registration`, `shop`, `tier_tests`, `clan_creation`, `forum_posting`) и событие-множитель ApexCoin `xN` (1..5, с датой окончания). Проверка переключателя — middleware `feature:<ключ>`, который возвращает **423** с ключом функции (админ проходит всегда):

```
// backend/app/Http/Middleware/EnsureFeatureEnabled.php
if (! ControlService::enabled($feature) && ! $request->user()?->isAdmin()) {
    $label = ControlService::FEATURES[$feature] ?? 'Эта функция';

    return response()->json([
        'message' => "{$label}: временно отключено администрацией. Загляните чуть позже.",
        'feature' => $feature,
    ], 423);
}
```

#### 5.6 Админка настроек

`GET /api/admin/site-settings` (только `role:admin`) отдаёт четыре группы (`hero`, `socials`, `footer`, `stats`) плюс `rating`, смёрженный с `RatingService::DEFAULTS` — чтобы форма показывала значения, даже если админ их ни разу не сохранял.

`PUT /api/admin/site-settings` принимает те же группы. Тип значения выводится из PHP-типа: `bool → 'bool'`, `int → 'int'`, `array → 'json'`, иначе `'string'`. Валидация строгая только для рейтинга: `rating` — `array` с разрешёнными ключами `rating_match_win`, `rating_war_participation`, `rating_war_win`, каждое значение `integer|min:0|max:1000`; остальные группы — просто `array`, то есть набор ключей не ограничен и любой новый ключ сохранится.

Побочный эффект сохранения: если веса рейтинга изменились, выполняется `recalculateAll()` — пересчёт очков всех игроков и силы кланов. Сравнение идёт по `weights()` до и после записи, поэтому лишних пересчётов не бывает.

#### 5.7 Эндпоинты

| Метод | Путь | Действие | Права |
|---|---|---|---|
| GET | `/api/home` | главная: hero, соцсети, подвал, статистика, топ-10 игроков и кланов, 6 новостей | публично |
| GET | `/api/home/me` | панель вошедшего | auth |

| | | | |
|---|---|---|---|
| | | игрока | |
| GET | `/api/site` | подвал, адрес сервера, веса рейтинга, состояние «Пульта» | публично |
| GET | `/api/site/server-status` | статус Minecraft, throttle 60/мин | публично |
| GET | `/api/site/team` | команда проекта | публично |
| GET | `/api/admin/site-settings` | группы настроек + `rating` по умолчанию | admin |
| PUT | `/api/admin/site-settings` | сохранить; при смене весов — пересчёт рейтинга | admin |
| GET/PUT | `/api/admin/control` | «Пульт»: объявление, переключатели, множитель | admin |
| POST | `/api/admin/control/giveaway` | раздача монет/рассылка (есть `dry_run`) | admin |
| GET | `/api/admin/control/online` | кто онлайн (5 минут) | admin |
| GET/POST | `/api/admin/control/promo-codes` | список / создание промокода | admin |
| POST | `/api/admin/control/promo-codes/{id}/toggle` | включить/выключить | admin |
| DELETE | `/api/admin/control/promo-codes/{id}` | удалить | admin |
| POST | `/api/promo/redeem` | активировать промокод, throttle 10/мин | auth |

#### 5.8 Фронтенд

- `frontend/src/components/core/SiteFooter.vue` — подвал на всех страницах: соцсети (пустые отфильтрованы, «@канал» превращается в ссылку через `socialLinks.js`), текст подвала с фолбэком, адрес сервера.
- `frontend/src/components/core/SiteAnnouncement.vue` — баннер объявления и полоса события `xN ApexCoin`; данные берутся из `siteApi.info()` (промис мемоизирован в `services/site/site.js`, чтобы подвал и баннер не делали два запроса).
- `frontend/src/views/players/HomeView.vue` — главная: hero с двумя кнопками, статистика, топ игроков и кланов, блок новостей.
- `frontend/src/components/admin/AdminHome.vue` — редактор настроек с вкладками `Hero / Соцсети / Футер / Очки рейтинга`, счётчики длины и подсказка про формулу очков; сохранение отправляет весь объект настроек целиком.
- `frontend/src/components/admin/AdminControl.vue` — «Пульт»: превью баннера, выбор тона, ссылка и её текст, дата окончания, переключатели функций, событие-множитель, раздача по аудиториям (`online`, `active_week`, `all`, `clan`, `tier`), промокоды, список онлайна.

## 6. Схема данных и связи

```
users ──1:N── tournaments.created_by
users ──1:N── tournaments (через participants.user_id)
clans ──1:N── tournament_participants.clan_id
tournaments ──1:N── tournament_participants
tournaments ──1:N── tournament_matches
tournament_participants ──1:N── tournament_matches.participant1_id / participant2_id / winner_id
tournament_matches ──1:1── tournament_matches.next_match_id (self-FK, nullOnDelete)
users ──1:N── tournament_wins ──N:1── tournaments

users ──1:N── reports.reporter_id / target_user_id / moderator_id
users ──1:N── moderation_logs.actor_id / target_user_id

users ──1:N── support_tickets ──1:N── support_messages
users ──1:1── support_tickets.assigned_to

users ──1:N── news.author_id
site_settings — изолированная key-value таблица без FK
promo_codes ──1:N── promo_redemptions ──N:1── users
promo_codes ──N:1── shop_items
```

Сводка таблиц:

| Таблица | Ключевые колонки | Каскады |
|---|---|---|
| `tournaments` | `status`, `type`, `format`, `created_by` | `created_by` — cascade |
| `tournament_participants` | `tournament_id`, `user_id`, `clan_id`, `seed`, `status` | турнир — cascade; unique `(tournament_id, user_id)`, `(tournament_id, clan_id)` |
| `tournament_matches` | `round`, `position`, `bracket`, `participant1_id`, `participant2_id`, `winner_id`, `score1`, `score2`, `status`, `next_match_id`, `next_slot`, `is_bye` | участники и `next_match_id` — `nullOnDelete`, матчи турнира — cascade |
| `tournament_wins` | `user_id`, `tournament_id`, `min_tier`, `max_tier` | cascade; unique `(user_id, tournament_id)` |
| `reports` | `reporter_id`, `target_user_id`, `subject_type`, `subject_id`, `reason`, `snapshot`, `status`, `resolution`, `moderator_id`, `resolved_at` | `reporter_id` — cascade, остальные — `nullOnDelete` |
| `moderation_logs` | `actor_id`, `action`, `target_user_id`, `subject_type`, `subject_id`, `details`, `created_at` | `nullOnDelete` |
| `support_tickets` | `user_id`, `subject`, `category`, `status`, `assigned_to`, `last_message_at` | заявитель — cascade, `assigned_to` — `nullOnDelete` |
| `support_messages` | `ticket_id`, `user_id`, `is_staff`, `body` | тикет — cascade, автор — `nullOnDelete` |
| `news` | `title`, `slug`, `excerpt`, `body`, `cover`, `type`, `is_pinned`, `is_published`, `author_id`, `published_at` | автор — cascade |
| `site_settings` | `key` (unique), `value`, `type`, `group` | — |
| `promo_codes` | `code` (unique), `coins`, `shop_item_id`, `max_uses`, `uses`, `expires_at`, `is_active` | `shop_item_id` — `nullOnDelete`, `created_by` — `nullOnDelete` |
| `promo_redemptions` | `promo_code_id`, `user_id` | cascade; unique `(promo_code_id, user_id)` |

Общее для всех доменов: уведомления — стандартная Laravel-таблица `notifications` (uuid, `notifiable` morph, `data` text, `read_at`), канал только `database`; все четыре домена шлют уведомления через собственные классы (`TournamentNotification`, `ModerationNotification`, `SupportNotification`), у каждого в `toArray()` есть `type` и профильный идентификатор (`tournament_id`, `report_id`, `ticket_id`).

## 7. Права и общая инфраструктура

| Механизм | Файл | Поведение |
|---|---|---|
| `role:a,b` | `EnsureUserHasRole` | строгое сравнение `users.role` с перечнем, иначе `403 «Недостаточно прав.»`. Роли: `user`, `media`, `tester`, `moderator`, `admin`, `bridge_tester`, `bridge_curator` |
| `not.banned` | `EnsureUserIsNotBanned` | навешен на всю группу `api`; для гостя — no-op, для забаненного — `403` с причиной и удаление токена/сессии |
| `feature:<ключ>` | `EnsureFeatureEnabled` | `423`, если функция выключена на «Пульте»; админ проходит всегда |
| `throttle:N,M` | `ThrottlePerRoute` | счётчик на связку «пользователь + метод + URI», а не общий на все маршруты |
| `auth:sanctum` | группа со строки 245 `api.php` | всё, что ниже, требует входа; публичными остаются только явно вынесенные выше (`/home`, `/tournaments`, `/news`, `/site`) |

Группировка админских маршрутов: `role:moderator,admin` — модерация, поддержка, турниры, форум, комментарии, кланы, магазин, ачивки; `role:admin` — пользователи и баны, рефералы, «Пульт», настройки сайта, новости, чейнджлог/роадмап, верификация.

Единый формат ошибок бизнес-правил — `App\Support\Concerns\AbortsWithMessage`: `abortUnprocessable()` отдаёт плоский `{"message": "..."}` с кодом 422, `abortForbidden()` — 403, `invalid()` — `ValidationException` с `errors` по полю. Этот трейт используют `ReportService`, `GiftService` и другие сервисы, поэтому фронтенд везде читает `e.message`.

## 8. Чего в коде нет

| Ожидание | Фактическое состояние |
|---|---|
| Double elimination, round-robin | формат в enum есть, проведение отсутствует: `generateBracket` пропускает только `single_elim` |
| Автоматическое закрытие регистрации по времени | нет планировщика; `isRegistrationOpen()` — вычисляемая проверка |
| Выплата призового фонда | нет; `prize_*` — только поля отображения |
| Определение победителя по счёту | нет; `winner_id` вводит админ, счёт — витрина |
| Промокоды в домене Tournaments | отсутствуют; реализованы в `Control` |

| | |
|---|---|
| Модель `SiteAnnouncement` | отсутствует; объявление — ключ `announcement` в `site_settings` |
| «Мои жалобы» для игрока | нет эндпоинта; только уведомление о решении |
| Счётчик предупреждений / муты / ограничения мягче бана | отсутствуют; `warn` — уведомление + запись в журнал |
| Теги и категории новостей | отсутствуют; есть только поле `type` (4 значения) |
| Планирование публикации новости | нет; есть `is_published` + `published_at`, но переключения по времени нет |
| Вложения к обращениям поддержки | отсутствуют; только текст |
| Внутренние заметки в тикете | отсутствуют; всё, что пишет персонал, видно игроку |
| `TournamentParticipant.status = withdrawn` | объявлен в enum, нигде не выставляется — `withdraw()` удаляет строку |
| `TournamentMatch.status = cancelled` | объявлен в enum и валидации, но нигде не выставляется |
| `HomeController::top()` | мёртвый метод, маршрута нет |
| `News\Requests\News\*` | неиспользуемые дубликаты `Requests\Admin\*` |

# Реальное время: чат, каналы и мост к Minecraft-серверу

Транспорт — Laravel Reverb (протокол Pusher). Внутри доменов нет ни одной очереди на рассылку: все события реализуют `ShouldBroadcastNow`, поэтому HTTP-запрос отправки сообщения не завершается, пока событие не отдано в Reverb. Ниже — только то, что есть в коде; где чего-то нет, это сказано прямо.

## 17.1. Что в проекте называется «бриджем»

Первое, что нужно зафиксировать, потому что название обманывает: `backend/app/Domains/Bridge/` (26 файлов) — это не сетевой мост к игровым серверам. Это домен **бридж-тестов**: каталог видов бриджа (speedbridge, godbridge, telly), заявки игроков с видео, проверка бридж-тестером, звания бриджера и очки в топ.

Мост «сайт ↔ Minecraft-сервер» — это два других места:

| Что | Где | Роль |
|---|---|---|
| Приём запросов от плагина сервера | `backend/app/Domains/Minecraft/` + `EnsureMinecraftServer` | единственный вход с игрового сервера |
| Общий секрет сервера | `config/services.php` → `minecraft.server_key`, заголовок `X-Minecraft-Server-Key` | аутентификация плагина |
| Статус сервера (онлайн, игроки, версия) | `MinecraftStatusService` (Server List Ping) | сайт сам опрашивает сервер по TCP |
| Скин игрока | `MinecraftSkinService` (Mojang API) | сайт сам ходит к Mojang |

Соответственно в домене `Bridge` **в коде отсутствует** идентификация серверов, ключи, heartbeat, синхронизация игроков и статус онлайн. Всё это — в `Minecraft`, раздел 17.14.

## 17.2. Транспорт: конфигурация Reverb и подключение фронтенда

`config/broadcasting.php` — стандартная заготовка Laravel, из неё реально используется одно соединение:

| Ключ | Значение в коде | Комментарий |
|---|---|---|
| `default` | `env('BROADCAST_CONNECTION', 'null')` | в `backend/.env.example` стоит `BROADCAST_CONNECTION=log`, в проде — `reverb` |
| `connections.reverb.driver` | `reverb` | |
| `key` / `secret` / `app_id` | `REVERB_APP_KEY` / `REVERB_APP_SECRET` / `REVERB_APP_ID` | те же значения, что в приложении Reverb |
| `options.host` | `REVERB_HOST` | |
| `options.port` | `REVERB_PORT`, по умолчанию `443` | |
| `options.scheme` / `useTLS` | `REVERB_SCHEME` (по умолчанию `https`), `useTLS = scheme === 'https'` | |

`config/reverb.php` — сервер и приложение:

| Параметр | Значение | Смысл в проекте |
|---|---|---|
| `servers.reverb.host` / `port` | `REVERB_SERVER_HOST` (0.0.0.0) / `REVERB_SERVER_PORT` (8080) | куда биндится процесс `reverb:start` |
| `servers.reverb.max_request_size` | `REVERB_MAX_REQUEST_SIZE`, 10 000 байт | лимит входящего сообщения |
| `servers.reverb.scaling.enabled` | `false` по умолчанию, Redis при включении | горизонтальное масштабирование не включено |
| `apps.provider` | `config` | одно приложение из переменных окружения |
| `apps[0].allowed_origins` | `['*']` | CORS сокета открыт |
| `apps[0].ping_interval` / `activity_timeout` | 60 / 30 секунд | |
| `apps[0].max_message_size` | `REVERB_APP_MAX_MESSAGE_SIZE`, 10 000 байт | **меньше**, чем лимит текста сообщения (5 000 символов), см. 17.11 |
| `apps[0].accept_client_events_from` | `members` | клиентские события разрешены, но фронтенд их не шлёт: `whisper` в `frontend/src` отсутствует |
| `apps[0].rate_limiting.enabled` | `false` по умолчанию | rate limit сокета выключен |

Фронтенд подключается в `frontend/src/echo.js`. Ключевое: адрес и порт задаются всегда, включая https, иначе pusher-js строит `wss://host:80`; транспорт не перечисляется вручную (иначе соединение сразу падало в `failed`):

```
// frontend/src/echo.js, фрагмент options()
return {
  broadcaster: 'reverb',
  key: KEY,
  wsHost: host,
  wsPort: port,
  wssPort: port,          // оба порта одинаковые: иначе wss уйдёт на порт 80
  forceTLS: secure,
  authEndpoint: '/broadcasting/auth',
  activityTimeout: 30000,
}
```

Сверху — сторож соединения (`startWatchdog`, интервал 15 с): состояния `failed`, `unavailable`, `disconnected` и «залипший» `connecting` дольше 45 с приводят к `resetEcho()` — полному пересозданию Echo. Пересоздание увеличивает реактивный счётчик `echoGeneration`, на который подписаны все подписки в приложении, поэтому они восстанавливаются сами, без перезагрузки страницы. Дополнительно `resetEcho()` вызывается по `online` и по возврату вкладки из фона.

Роут `/broadcasting/auth` framework регистрирует сам (`withRouting(channels: ...)` в `bootstrap/app.php`), CSRF для него исключён явно, API работает в `statefulApi()` — авторизация канала идёт по Sanctum-cookie.

### 17.3. Каналы вебсокетов

`backend/routes/channels.php` — 21 строка, два канала. Это весь список:

```php
// backend/routes/channels.php
Broadcast::channel('App.Models.User.{id}', function ($user, $id) {
    return (int) $user->id === (int) $id;
});

Broadcast::channel('online', function ($user) {
    return [
        'id' => (int) $user->id,
        'username' => $user->username,
    ];
});
```

| Канал | Тип | Кто может подписаться | Что приходит |
|---|---|---|---|
| `App.Models.User.{id}` | private | только сам пользователь (`(int) $user->id === (int) $id`) | все события чата и уведомлений: `message.sent`, `messages.read`, `user.typing`, `conversation.changed`, `notification.created` |
| `online` | presence | любой авторизованный | список участников канала: `{id, username}`; вход/выход видны всем подписчикам |
| `activity` | public | кто угодно, авторизация не нужна | `activity.recorded` — лента «Сейчас на APEX» |

Три следствия, важных для архитектуры:

1. **Канала на диалог нет.** Ни `conversation.{id}`, ни `chat.{id}` в `channels.php` не объявлено. Приватность обеспечивается тем, что событие копируется в личный канал каждого участника (см. 17.6), а не подпиской на общий канал диалога.
2. **`activity` в `channels.php` отсутствует** — публичные каналы не проходят авторизацию, имя задано константой `ActivityRecorded::CHANNEL = 'activity'`.
3. Канал `online` отдаёт `username` всем подписчикам — это осознанно (ник и так виден в профилях), но список онлайна фактически публичен внутри авторизованной части сайта.

## 17.4. Модель чата: три вида бесед

`Conversation` (`backend/app/Domains/Chat/Models/Conversation.php`) различает типы строкой, а не enum: миграция `2026_11_06_100000_chat_groups_gifts_voice.php` перевела `enum('direct','clan_message')` в `string(20)`, потому что значение `group` в enum не влезало.

| Тип | Как создаётся | Участники | Особенности |
|---|---|---|---|
| `direct` | `POST /api/chat/start-direct` → `ChatService::startDirect()` → `Conversation::findOrCreateDirect()` | двое, оба `ConversationParticipant` через `firstOrCreate` | пара нормализуется: `user_min_id = min(a,b)`, `user_max_id = max(a,b)`, на пару стоит `unique(['user_min_id','user_max_id'])` — второго диалога между теми же людьми не будет |
| `clan_message` | `POST /api/chat/start-` | автор обращения + | `unique(['clan_id','autho` |

| | | | |
|---|---|---|---|
| | `clan` → `ChatService::startClan()` | `leader`/`officer` клана на момент создания | `r_id'])` — одно обращение на пару «клан + автор»; состав участников фиксируется при создании и позже не пересчитывается |
| `group` | `POST /api/chat/groups` → `ChatGroupService::create()` | создатель + приглашённые друзья, максимум 20 | `owner_id`, `title` (2–48 символов после схлопывания пробелов) |

Правила групповых бесед (`ChatGroupService`) — это единственное место, где чат ограничивает социальный граф:

- приглашать можно **только принятых друзей**: `friendship.status = 'accepted'`; любой чужой id в списке → 422 «В беседу можно звать только друзей»;
- `MAX_MEMBERS = 20`, проверка и при создании (`count($userIds) + 1 > 20`), и при добавлении (`count($present) + count($new) > 20`);
- добавлять участников может любой участник беседы (в контроллере только `authorize('send')` = «я участник»);
- исключать — только `owner_id`; выйти самому может любой;
- переименовывать — только `owner_id`;
- если создатель ушёл, `owner_id` переходит к участнику с минимальным `id` (кто дольше в беседе); если ушёл последний — беседа удаляется вместе с сообщениями (`cascadeOnDelete`), контроллер возвращает `{'ok': true, 'conversation': null}`;
- каждое изменение пишет служебное сообщение `kind = 'system'` с `meta.action`: `created`, `added`, `left`, `removed`, `renamed`, `meta.target_ids`.

Личный диалог не показывается в списке, пока в нём нет ни одного сообщения: `listConversations()` фильтрует `type != 'direct' OR has('messages')`. Это лечит «Новый диалог» без единой строки у собеседника, который создавался при каждом открытии чата с игроком.

### 17.5. Путь сообщения: от нажатия до доставки

Ниже — полная цепочка для обычного текстового сообщения, с файлами и функциями.

1. **Ввод.** `frontend/src/views/chat/Messages.vue`: `body` (ref), `pendingAttachments`, `replyTo`. Ввод в поле вызывает `onBodyInput()` → `notifyTyping()` (не чаще `TYPING_SEND_INTERVAL_MS = 2500` мс) и отложенное сохранение черновика (400 мс) в `localStorage` по каждому диалогу.
2. **Отправка.** `sendMessage()` оптимистично очищает поле, обнуляет `replyTo` и `pendingAttachments`, сохраняет бэкап для отката при ошибке, вызывает `chatApi.send(id, text, replyId, attachmentIds)`.
3. **HTTP.** `POST /api/chat/conversations/{conversation}/messages`, тело `{body, reply_to_id, attachments: [id…]}`. Маршрут только под `auth:sanctum`, **без throttle**.
4. **Валидация** (`SendMessageRequest`): `body` — `nullable|string|max:5000`, `reply_to_id` — `exists:messages,id`, `attachments` — массив до 10, каждый `exists:message_attachments,id`.

5. **Право.** `$this->authorize('send', $conversation)` → `ConversationPolicy::send` → существует ли строка в `conversation_participants` для этой пары.

6. **Сервис.** `ChatService::send()` режет пробелы, оставляет только свои и ещё не привязанные вложения (`where('user_id',$me->id)->whereNull('message_id')`), падает 422 на пустое тело без вложений, проверяет, что цель ответа — из этого же диалога, и вызывает `StickerService::guard()` (нельзя отправить стикер из некупленного набора, обход через `forwarded_from_user_id` закрыт).

7. **Запись.** Одна транзакция: `insert` в `messages`, `update message_attachments.message_id`, `touchAfterWrite()` — `conversations.last_message_at = now()`, автору сразу `message_reads(read_at = now())`, `conversation_participants.last_read_at = now()`.

8. **Перечитывание.** `reload()`: `fresh()` + `MESSAGE_RELATIONS` (автор с косметикой, `reads`, `replyTo.user`, `replyTo.attachments`, `forwardedFrom`, `attachments`, `reactions`) + `ChatCardService::attach()` — ссылки на игрока/клан/питомца/лот превращаются в карточки (максимум 3 на сообщение).

9. **Ответ и рассылка.** Контроллер отдаёт `201 {message: MessageResource}` и сразу зовёт `SafeBroadcast::send(new MessageSent($message), toOthers: true)`.

```
// backend/app/Domains/Chat/Controllers/Api/ChatController.php, 98–110
public function send(SendMessageRequest $request, Conversation $conversation): JsonResponse
{
    $this->authorize('send', $conversation);

    $message = $this->chat->send(
        conversation: $conversation,
        me: $request->user(),
        body: $request->input('body'),
        replyToId: $request->filled('reply_to_id') ? $request->integer('reply_to_id') : null,
        attachmentIds: $request->input('attachments', []),
    );

    SafeBroadcast::send(new MessageSent($message), toOthers: true);
```

```
// backend/app/Domains/Chat/Services/ChatService.php, 297–310
$message = DB::transaction(function () use ($conversation, $me, $body, $replyToId, $attachmentIds)
{
    $message = Message::create([
        'conversation_id' => $conversation->id,
        'user_id' => $me->id,
        'reply_to_id' => $replyToId,
        'body' => $body,
    ]);

    $this->attachFiles($message, $attachmentIds);

    $this->touchAfterWrite($conversation, $message, $me);

    return $message;
});
```

1. **Reverb.** `MessageSent::broadcastOn()` строит по приватному каналу на каждого участника, кроме автора. Автор своё сообщение по сокету не получает — его интерфейс обновляется ответом HTTP.

2. **Другие вкладки автора.** `toOthers()` добавляет заголовок `X-Socket-ID`, и Reverb не доставляет событие той вкладке, которая инициировала запрос; остальные вкладки того же игрока событие получают.

3. **Клиент.** `useRealtimeMessages` (один на приложение) ловит `.message.sent`, кладёт сообщение в `latestMessage`; страница через `watch(latestMessage)` вызывает

`mergeMessages()`, играет звук, если автор не я, гасит «печатает», поднимает диалог наверх списка, а если это другой диалог — увеличивает `unread_count` и общий `chatUnread`.

```
// backend/app/Domains/Chat/Events/MessageSent.php, 25–38
public function broadcastOn(): array
{
    $channels = [];

    $this->message->conversation
        ->participants()
        ->where('user_id', '!=', $this->message->user_id)
        ->pluck('user_id')
        ->each(function ($userId) use (&$channels) {
            $channels[] = new PrivateChannel('App.Models.User.' . $userId);
        });

    return $channels;
}
```

Событие — `message.sent`, payload — `{message: <MessageResource>}`. Важная деталь: `broadcastWith()` собирает payload **тем же** `MessageResource`, что и API, и принудительно добавляет `reads: []`, потому что страница ждёт список, а не отсутствие ключа (раньше массив собирался вручную и расходился с ресурсом — галочка прочтения на сообщении из сокета не вставала).

**Три пути одного сообщения.** На клиенте одно и то же сообщение может прийти ответом на отправку, по сокету и запасным опросом. Все три пути идут через `mergeMessages()` (`frontend/src/utils/chatMessages.js`): дедупликация по `id`, сортировка по возрастанию `id` только если новое вставилось не в конец, известное сообщение не перезаписывается (у него могли появиться локальные отметки прочтения).

**Запасной опрос.** `useMessagePolling` работает, пока открыт диалог: `GET /chat/conversations/{id}?after_id={newest}&limit=50&mark_read=0`. Интервал — 2 с, но при живом сокете круг пропускается, если с последнего опроса прошло меньше `LIVE_INTERVAL_MS = 15000`; проверка живости — `window.Echo.connector.pusher.connection.state === 'connected'`.

### 17.6. Четыре события чата

| Событие | Класс | Канал | Кому | Payload |
|---|---|---|---|---|
| `message.sent` | `MessageSent` | private `App.Models.User.{id}` | все участники, кроме автора | `{message}` — полный `MessageResource` + `reads` |
| `messages.read` | `MessagesRead` | private, по каналу на участника | все участники, кроме читателя | `{conversation_id, reader: {id,username,avatar_url}, read_at}` |
| `user.typing` | `UserTyping` | private, по каналу на участника | все участники, кроме печатающего | `{conversation_id, user: {id,username,avatar_url}}` |
| `conversation.changed` | `ConversationChanged` | private, по каналу на участника + `extraUserIds` | все участники и явно указанные | `{conversation_id, kind, …data}` |

`ConversationChanged.kind` — пять значений, и каждое обрабатывается отдельной веткой `switch` в `Messages.vue`: `message.updated` (заменить сообщение), `message.deleted` (убрать и подставить новый `last_message` в превью), `reactions` (перерисовать реакции), `pinned` (обновить плашку закрепа), `conversation` (новый состав/название; если меня в списке участников больше нет — убрать беседу).

`extraUserIds` нужен ровно для одного случая: исключённый участник уже удалён из `conversation_participants`, поэтому обычная выборка его не вернёт, а сообщить ему надо — иначе беседа висит в списке до перезагрузки.

**Рассылка не роняет запрос.** Все вызовы идут через `SafeBroadcast`:

```
// backend/app/Support/Broadcasting/SafeBroadcast.php, 17–31
public static function send(object $event, bool $toOthers = false): void
{
    try {
        $pending = broadcast($event);

        if ($toOthers) {
            $pending->toOthers();
        }

        // PendingBroadcast рассылает в деструкторе — вызываем его внутри try
        unset($pending);
    } catch (Throwable $e) {
        Log::warning('Broadcast failed: '.$e->getMessage(), ['event' => $event::class]);
    }
}
```

Логика: данные уже сохранены, и если Reverb лежит, клиент всё равно получает `201`, а собеседник подтянет сообщение опросом. Без этого пользователь видел 500 и при повторной отправке плодил дубли.

**Почему `ShouldBroadcastNow`, а не очередь.** Прямое следствие в тестах `MessageBroadcastTest`: раньше рассылка уходила в `jobs`, без воркера задачи копились (в базе накопилось 348), и собеседник видел сообщение только после обновления страницы. Тест `test_sending_message_does_not_queue_a_job` проверяет, что после отправки `count(jobs) === 0`.

### 17.7. Непрочитанные и прочтения

Прочтение — отдельная таблица `message_reads` (по строке на пару «сообщение + читатель»), а не флаг на сообщении. Это позволяет считать непрочитанные для каждого участника независимо и отдавать список читателей для галочек в групповой беседе.

| Действие | Где | Механика |
|---|---|---|
| Список диалогов с непрочитанными | `ChatService::listConversations()` | один агрегирующий запрос на все диалоги: `count(*)` по сообщениям, где `user_id != me` и нет `reads` с моим `user_id`; раньше был N+1 |
| Общий счётчик | `ChatService::unreadCount()` | тот же `unreadQuery()` + `whereIn` по моим диалогам |
| Открытие диалога | `GET /chat/conversations/{id}` → `history(markRead: true)` | перед отметкой запоминается `first_unread_id` (минимум `id` среди непрочитанных) — по нему страница рисует разделитель «Новые сообщения» |

| | | |
|---|---|---|
| Отметить диалог | `POST /chat/conversations/{id}/read` | `insertOrIgnore` пачкой по всем непрочитанным + `participants.last_read_at = now()` |
| Отметить одно | `POST /chat/messages/{id}/read` | `firstOrCreate`, своё сообщение игнорируется |

После любой отметки сервер шлёт `MessagesRead::forConversation()` — всем остальным участникам. Событие уходит **всем**, а не только авторам прочитанных сообщений: иначе в диалоге без сообщений от собеседника события не было бы вовсе. Клиент сам решает, что пометить, и складывает `read_at` в `lastReadAt` по паре «диалог + читатель».

Клиентская отметка отложенная: `markActiveRead()` ставит таймер 250 мс; если вкладка скрыта, отметка откладывается до `visibilitychange`. Это защита от «прочитал, проскроллив на бегу» и от лишних запросов при быстром переключении диалогов. При открытии диалога дополнительно запрашивается `GET /chat/unread-count`, чтобы бейдж в шапке совпал с реальностью.

### 17.8. Правка, удаление, пересылка, реакции, закреп

| Операция | Право | Ограничения в коде |
|---|---|---|
| Редактирование | `MessagePolicy::update` — только автор | не позже `abs(created_at->diffInHours(now())) > 24` → 422; нельзя править `kind != 'text'`; нельзя, если тело совпадает с меткой стикера `[sticker:…]` (иначе правка текста обходила бы покупку набора); пустое тело допустимо только при наличии вложений; стикер-паттерн снова проверяется `StickerService::guard`; ставится `edited_at` |
| Удаление | `MessagePolicy::delete` — автор **или** `isModerator()` (роли `moderator`/`admin`) | удаление физическое (soft deletes у модели нет); после удаления `last_message_at` пересчитывается по `max(created_at)` оставшихся, иначе диалог подскакивал наверх; собеседникам уходит `conversation.changed` с `kind: message.deleted` и новым `last_message` |
| Пересылка | `MessagePolicy::forward` — участник диалога; плюс `authorize('send', $target)` | нельзя в тот же диалог; только `kind = 'text'` (подарок и служебная отметка — события своего диалога); вложения копируются физически новыми строками `message_attachments` (файл на диске переиспользуется по тому же `path`); `forwarded_from_user_id` = |

| | | |
|---|---|---|
| | | исходный автор или уже существующий источник пересылки |
| Реакции | `authorize('view', $message->conversation)` — любой участник | эмодзи только из `MessageReaction::EMOJI`: ❤ 👍 😂 😮 😳 🔥 💀 👑; повторный клик снимает; `unique(message_id, user_id, emoji)`; throttle 60/мин |
| Закреп | `authorize('send', $conversation)` — любой участник | `message_id = null` открепляет; сообщение обязано быть из этого диалога (`findOrFail` с `where conversation_id`); хранится в `conversations.pinned_message_id` |

Реакции отдаются сгруппированно и **без привязки к получателю**: `{emoji, count, mine, user_ids}`. В событие кладётся `user_ids`, а «моя ли реакция» каждый клиент считает сам — иначе одно событие на всех врало бы про `mine`.

### 17.9. Вложения и голосовые

Загрузка двухшаговая: файл сначала живёт «про запас» (`message_id = null`) и привязывается при отправке.

| Параметр | Значение | Где |
|---|---|---|
| Сколько файлов за раз | `MAX_BATCH = 10` | `UploadAttachmentRequest` |
| Размер файла | 12 МБ (`MAX_KILOBYTES = 12288`) | там же |
| Разрешённые типы | `jpg,jpeg,png,webp,gif,mp4,webm,pdf,txt,md,zip,rar,7z,json` | там же |
| Куда кладётся | диск `public`, путь `chat/YYYY/MM/…` | `ChatService::uploadAttachment()` |
| Как отдаётся | `asset('storage/' . path)` — прямой URL | `MessageAttachment::getUrlAttribute()` |
| Голосовое | до 120 с (`VOICE_MAX_SECONDS`), файл до 4 МБ, `mimetypes`: webm/ogg/mp4/m4a/aac/mpeg, путь `chat/voice/YYYY/MM/` | `ChatController::uploadVoice()`, `ChatService::uploadVoice()` |
| Проверка скачивания | `MessageAttachmentPolicy::download`: участник диалога; неотправленный файл — только загрузивший | политика применяется на `GET /chat/attachments/{id}/download` |
| Очистка | при удалении сообщения строки уходят каскадом, файлы на диске остаются | в коде нет задачи очистки |

Практическое следствие, которое видно из кода: вложения лежат на **публичном** диске и имеют прямой URL `/storage/...`, поэтому политика `MessageAttachmentPolicy` защищает только маршрут `download`; сама ссылка из `MessageResource` работает без авторизации. Это факт реализации, а не вывод.

Порядок загрузки пачки: `POST /chat/attachments` с `files[0..n]` (индексы обязательны — без них PHP собирает вложенные массивы и `files.*` не видит файлы), ответ содержит и `attachments` (весь список), и `attachment` (первый) для обратной совместимости.

### 17.10. Присутствие: два независимых источника

`PresenceService` складывает онлайн из двух источников и дополнительно помнит явно вышедших:

| Источник | Как работает | Окно |
|---|---|---|
| Presence-канал Reverb `online` | `markOnline()` при подписке, отметка живёт в кэше `presence:members` вместе с timestamp | запись считается устаревшей через `PRESENCE_TTL_SECONDS = 90` |
| `users.last_seen_at` | middleware `TouchLastSeen` пишет время не чаще раза в 20 с, без событий модели | `ONLINE_WINDOW_SECONDS = 60` |
| Явный выход | `markLeft()` убирает из `presence:members` и пишет в `presence:left` | из `presence:left` игрока исключают 60 с |

`markLeft()` существует потому, что одной активности мало: игрок, нажавший «Выйти», ещё минуту оставался онлайн, так как страница продолжала слать запросы от его имени. Теперь активность не возвращает его в список, пока запись о выходе свежая; при следующем `markOnline()` запись стирается.

Эндпоинты:

| Метод | Путь | Что делает |
|---|---|---|
| `GET` | `/api/chat/presence` | без параметров — все онлайн-ids; с `user_ids` (массив или строка через запятую) — только про запрошенных |
| `POST` | `/api/chat/presence/online` | `markOnline` + отдаёт свежий список |
| `POST` | `/api/chat/presence/offline` | `markLeft` |

На клиенте (`frontend/src/composables/chat/presence.js`) присутствие стартует на всё приложение сразу после авторизации (`startSitePresence` в `AppLayout`), а не только на странице чата — иначе собеседник на главной для канала не существовал. Ритмы: пульс `POST /presence/online` каждые 30 с (пропускается при скрытой вкладке), полный список `GET /presence` раз в 60 с, о собеседниках открытого диалога — раз в 15 с. Выход из канала (`leaving`) сразу убирает игрока из локального множества, а серверному списку про него не верят `LEFT_SUPPRESS_MS = 90_000` — ровно столько сервер в худшем случае держит отметку.

### 17.11. Права, лимиты и защита от флуда

Политики (`backend/app/Domains/Chat/Policies/`):

| Политика | Метод | Правило |
|---|---|---|
| `ConversationPolicy` | `view`, `send`, `markRead` | только участник (exists в `conversation_participants`) |
| `MessagePolicy` | `view`, `forward` | участник диалога сообщения |
| `MessagePolicy` | `update` | только автор |
| `MessagePolicy` | `delete` | автор или модератор |
| `MessageAttachmentPolicy` | `download` | участник диалога; неотправленный файл — только владелец |

Ограничения уровней валидации и сервисов:

| Что | Значение |
|---|---|
| Длина сообщения | 5 000 символов (`SendMessageRequest`, `UpdateMessageRequest`) |
| Вложений на сообщение | 10 |
| Страница истории | 40 по умолчанию, максимум 100 |
| Диалогов в списке | 50 |
| Участников в беседе | 20 |
| Название беседы | 2–48 символов |
| Голосовое | 120 с |
| Поиск по людям/кланам | от 2 символов, по 10 результатов |
| Поиск по сообщениям | от 2 символов, 40 результатов, `LIKE` с экранированием `!`, `%`, `_` |

Throttle по маршрутам (счётчик у каждого маршрута свой — `ThrottlePerRoute` подмешивает в ключ метод и URI):

| Маршрут | Лимит |
|---|---|
| `POST /chat/attachments` | 40/мин |
| `POST /chat/messages/{id}/reactions` | 60/мин |
| `POST /chat/conversations/{id}/gift` | 10/мин |
| `POST /chat/voice` | 30/мин |
| `POST /chat/groups` | 10/мин |
| `POST /chat/groups/{id}/members` | 30/мин |
| `PUT /chat/groups/{id}` | 20/мин |
| `DELETE /chat/groups/{id}/members/{user}` | без лимита |
| `POST /chat/conversations/{id}/messages` | **без лимита** |
| `PUT`/`DELETE /chat/messages/{id}` | без лимита |
| `POST /chat/conversations/{id}/typing` | без лимита на сервере; клиент шлёт не чаще 2,5 с |

То есть серверного антифлуда на отправку сообщений в коде нет: защита сводится к авторизации, лимиту длины, проверке вложений и клиентскому интервалу на «печатает». Отдельно стоит держать в голове несоответствие: `max_message_size` приложения Reverb — 10 000 байт, а текст сообщения — до 5 000 символов; для кириллицы это уже ~10 000 байт без JSON-обёртки и объекта автора. Проверки, что сериализованное событие укладывается в лимит, в коде нет — при превышении сработает только `SafeBroadcast` (запись в лог, сообщение сохранено, но не доставлено).

### 17.12. Уведомления

Хранилище — штатная таблица Laravel `notifications`: `uuid` первичный ключ, `type`, `morphs('notifiable')`, `data` (text, сериализованный массив), `read_at`, timestamps.

| Метод | Путь | Действие |
|---|---|---|
| `GET` | `/api/notifications` | `paginate(20)` + `unread_count` |
| `POST` | `/api/notifications/{id}/read` | `findOrFail` в пределах своих, `markAsRead()`, возвращает свежий `unread_count` |
| `POST` | `/api/notifications/read-all` | массовый `update(['read_at' => now()])` + `unsetRelation` для `notifications` и `unreadNotifications` |

`markAllAsRead` сделан запросом, а не коллекцией, специально: ленивая коллекция могла быть уже загружена, часть уведомлений оставалась непрочитанной, и бейдж не пропадал до перезагрузки.

Типы уведомлений (все — `via() = ['database']`, то есть только база, без почты и сокета):

| Домен | Класс | `data.type` |
|---|---|---|
| Друзья | `FriendRequestNotification` | `friend_request` |
| Друзья | `FriendAcceptedNotification` | `friend_accepted` |
| Тир-тесты | `TierTestRequestNotification` | `tier_test_request` |
| Тир-тесты | `TierTestCompletedNotification` | `tier_test_completed` |
| Тир-тесты | `TierTestAppealResolvedNotification` | `tier_test_appeal_resolved` |
| Кланы | `ClanApplicationNotification` | `clan_application` |
| Кланы | `ClanApplicationAcceptedNotification` | `clan_application_accepted` |
| Кланы | `ClanApplicationDeclinedNotification` | `clan_application_declined` |
| Кланы | `ClanWarNotification` | `clan_war_call`, `clan_war_challenge` |
| Турниры | `TournamentNotification` | `tournament_approved`, `tournament_rejected`, `tournament_won` |
| Форум | `ForumReplyNotification` | `forum_reply`, `forum_reply_to_you` |
| Бридж | `BridgeSubmissionNotification` | `bridge_submission` |
| Бридж | `BridgeTechniqueReviewedNotificati` | `bridge_technique_reviewed` |

| | on | |
|---|---|---|
| Ачивки | AchievementGrantedNotification | achievement_granted |
| Модерация | ModerationNotification | moderation_warning, report_resolved, announcement |
| Поддержка | SupportNotification | support_reply, support_player_reply |
| Аккаунт | VerifiedNotification | verified |
| Почта | VerifyEmailNotification, ResetPasswordNotification | письма, не database |

Доставка в реальном времени: класс NotificationCreated (ShouldBroadcastNow, private App.Models.User.{id}, событие notification.created) **объявлен**, фронтенд на него подписан (useRealtimeNotifications, звук notification, инкремент счётчика), но в рабочем коде приложения событие **никто не диспатчит** — new NotificationCreated встречается только в tests/Feature/BroadcastTest.php и tests/Feature/MessageBroadcastTest.php. То есть в текущем коде уведомления доставляются только в базу, а счётчик в шапке обновляется запросом при загрузке страницы и ответами unread_count; мгновенного прихода по сокету нет.

Счётчик на клиенте один на приложение (module-level ref в useRealtimeNotifications): раньше ref создавался внутри функции, у шапки и страницы были разные счётчики, и «прочитать всё» на странице не сбрасывало бейдж.

### 17.13. Activity: лента «Сейчас на APEX»

Таблица activities: id, type (string 40, индекс), actor_id (nullable, nullOnDelete), payload (json), timestamps + индекс по created_at. payload — снимок на момент события: переименованный позже клан в ленте остаётся под старым именем.

| Тип | Источник события | payload |
|---|---|---|
| tier.assigned | TierTest created (ручной тест админа сразу completed) или updated с wasChanged('status') и result_tier | {tier, score, mode} |
| clan.created | Clan created | {clan:{id,name,tag,color}} |
| clan_war.finished | ClanWar updated, status = completed и wasChanged('status') | {challenger, opponent, challenger_score, opponent_score, winner_clan_id, outcome}, actor = null |
| tournament.won | TournamentWin created | {tournament:{id,name}} |
| minecraft.linked | User updated, wasChanged('minecraft_linked_at') и значение не null | {nickname} |
| achievement.earned | AchievementService::grant() вручную (ачивки выдаются через attach() без событий модели) | по домену ачивки |

```
// backend/app/Domains/Activity/Services/ActivityFeed.php, 31–42
public function record(string $type, ?User $actor, array $payload = []): Activity
{
```

```
    $activity = Activity::create([
        'type' => $type,
        'actor_id' => $actor?->id,
        'payload' => $payload,
    ]);

    DB::afterCommit(fn () => $this->broadcast($activity));

    return $activity;
}
```

`afterCommit` — принципиально: запись идёт в транзакции действия, и если действие откатилось, зрители не должны увидеть событие, которого не было. Ошибка рассылки только логируется (`Log::warning`) — лента не бизнес-логика.

Событие `ActivityRecorded` уходит в публичный канал:

```
// backend/app/Domains/Activity/Events/ActivityRecorded.php, 29–37
public function broadcastOn(): array
{
    return [new Channel(self::CHANNEL)];
}

public function broadcastAs(): string
{
    return 'activity.recorded';
}
```

Подписка моделей — в `AppServiceProvider::registerActivityFeed()`: `TierTest::created/updated`, `Clan::created`, `ClanWar::updated`, `TournamentWin::created`, `User::updated`. Модели о ленте не знают — удалить ленту значит удалить один метод, а не править пять моделей.

HTTP: `GET /api/activity` — публичный, `limit` 1–50 (по умолчанию 20), курсор `before` (id), новые сверху. Фронтенд (`useActivityFeed`) берёт 8 событий, держит в памяти не больше 60, новые по сокету подсвечивает на 4 секунды, при пересоздании соединения (`onEchoReset`) переподписывается и делает `catchUp()` — добирает пропущенное. Индикатор «в эфире» — не состояние подписки, а `window.Echo.connector.pusher.connection.state`, опрашивается раз в 3 с.

## 17.14. Мост к Minecraft-серверу: привязка ника и вход

Это единственная часть системы, где сайт и игровой сервер действительно разговаривают.

**Идентификация сервера.** Один общий секрет: `config/services.php` → `minecraft.server_key` ← `MINECRAFT_SERVER_KEY`; плагин присылает его в заголовке `X-Minecraft-Server-Key`. Сравнение — постоянного времени, пустой ожидаемый ключ закрывает доступ полностью:

```
// backend/app/Http/Middleware/EnsureMinecraftServer.php, 17–27
public function handle(Request $request, Closure $next): Response
{
    $expected = (string) config('services.minecraft.server_key');
    $given = (string) $request->header('X-Minecraft-Server-Key', '');

    if ($expected === '' || ! hash_equals($expected, $given)) {
        abort(403, 'Недоступно.');
    }

    return $next($request);
}
```

Группа `Route::middleware('minecraft.server')->prefix('minecraft')` вынесена из-под сессий: ходит плагин, не браузер. Отдельных ключей на сервер, списка серверов, ротации и подписи тела запроса в коде **нет** — один секрет на всё.

**Как привязывается ник.** История решения видна в миграциях: сначала (`2026_10_08_161952`) был одноразовый код из игры в таблице `minecraft_link_codes`, но код ничего не доказывал — его вводил кто угодно, и ник занимал тот, кто успел первым. Миграция `2026_10_08_162912` удалила таблицу кодов и сделала ник ключом привязки: игрок заявляет ник в профиле под своей сессией, ник уникален на сайте (`unique('minecraft_username')`), а плагин не пускает того, чей ник в игре не совпадает с заявленным.

| Поле `users` | Тип | Смысл |
|---|---|---|
| `minecraft_username` | string(32), unique, nullable | заявленный ник, ключ привязки |
| `minecraft_uuid` | string(36), unique, nullable | UUID; доверяем только при `online-mode=true`, поэтому при заявке ника сбрасывается в null и заполняется при первом входе |
| `minecraft_linked_at` | timestamp, nullable | когда заявлен; изменение этого поля — триггер события `minecraft.linked` в ленте |

Правила ника: `^[A-Za-z0-9_]{3,16}$` (константа `NICKNAME_PATTERN`), поиск аккаунта — `LOWER(minecraft_username) = ?`, потому что в Minecraft регистр не важен. Заявка чужого ника → 422 «Этот ник уже заявлен другим аккаунтом».

**Вход на сервер.** Плагин делает два запроса: сначала `resolve` (заявлен ли ник и чей), потом `login` (ник + пароль сайта). Пароль приходит по HTTPS, сверяется с хешем и нигде не сохраняется и не логируется.

```
// backend/app/Domains/Minecraft/Services/MinecraftLinkService.php, фрагмент login()
$key = 'minecraft-auth:' . mb_strtolower($nickname);

if (RateLimiter::tooManyAttempts($key, self::MAX_ATTEMPTS)) {
    return ['ok' => false, 'reason' => 'throttled', 'attempts_left' => 0,
            'retry_after' => RateLimiter::availableIn($key), 'user' => null];
}

$user = $this->userByNickname($nickname);

// Хеш сверяем всегда, даже когда ник свободен: иначе по времени ответа
// можно было бы узнать, заявлен ник или нет.
$hash = $user?->password ?? $this->dummyHash();

if (! Hash::check($password, $hash) || ! $user) {
    RateLimiter::hit($key, self::DECAY_SECONDS);
    // …
}
```

Защита от подбора: `MAX_ATTEMPTS = 5`, `DECAY_SECONDS = 600`, ключ лимитера — нижний регистр ника. Защита от утечки через тайминг: при свободном нике всё равно сверяется bcrypt-хеш-заглушка (`Hash::make('apex-timing-equalizer')`, кэшируется в статике). Успешный вход очищает лимитер и, если плагин прислал `uuid` и он отличается, записывает его.

Контракт ответа `login`: `{ok, reason, attempts_left, retry_after, player?}`; `reason` ∈ `throttled` | `invalid` | `not_claimed`; HTTP 429 при блокировке, 401 при неверном пароле, 200 при успехе.

`player` — `profilePayload`: `username`, `nickname`, `tier`, `tier_score`, `role`, `clan_tag`, `clan_color`, `avatar_url`, `bridge_rank{label,color}`.

**Админские операции** (тоже под секретом сервера, вызываются плагином): `admin/lookup` — на какой аккаунт заявлен ник (`account_id`, `uuid`, `claimed_at`), `admin/unlink` — освободить ник. Это нужно потому, что в offline-режиме доказать принадлежность ника нельзя и решение принимает админ.

Синхронизации игроков в обе стороны в коде **нет**: сервер не присылает список онлайн-игроков, сайт не пушит события в игру, периодической сверки ников нет. Всё общение — три запроса плагина к сайту (`resolve`, `login`, `profile`) и два админских.

Отдельно — скин. `MinecraftSkinService` не относится к плагину: сайт сам спрашивает Mojang (`api.mojang.com/users/profiles/minecraft/{nick}` → UUID, `sessionserver.mojang.com/session/minecraft/profile/{uuid}` → подписанный `textures`), скачивает PNG **только** с `textures.minecraft.net`, проверяет, что это PNG 64×64 или 64×32, и кладёт на диск `local` в `minecraft-skins/{uuid}.png`. Кэш: метаданные скина 6 ч, UUID ника 24 ч, «такого ника нет» 1 ч. Свой скин игрока (`custom-skins/{id}.png`) важнее скина из Minecraft. Отдача — `GET /api/players/{user}/skin` с заголовком `X-Skin-Model: classic|slim` (и `Access-Control-Expose-Headers`), кэш 3600 с.

### 17.15. Статус сервера: Server List Ping вместо плагина

Онлайн, число игроков и версия берутся не от плагина, а тем же запросом, который делает клиент игры в список серверов. Плагин для этого не нужен:

```
// backend/app/Domains/Minecraft/Services/MinecraftStatusService.php, 66–72
// Handshake: версия протокола -1 (любая), адрес, порт, next state = 1 (status)
$handshake = "\x00" . $this->varInt(-1) . $this->string($host) . pack('n', $port) . $this-
>varInt(1);
fwrite($socket, $this->varInt(strlen($handshake)) . $handshake);

// Status request
fwrite($socket, "\x01\x00");
```

| Параметр | Значение |
|---|---|
| Таймаут соединения | `TIMEOUT = 2.0` с, `fsockopen` |
| Кэш успеха / отказа | 30 с / 15 с, ключ `minecraft:server-status` |
| Адрес в ответе | `minecraft.address` или `minecraft.host` |
| Поля ответа | `configured`, `online`, `address`, `players`, `max`, `version`, `motd` |
| MOTD | строка или дерево компонентов (`text` + `extra`), коды цвета `§x` вырезаются |
| Конфиг | `MINECRAFT_SERVER_HOST`, `MINECRAFT_SERVER_PORT` (25565), `MINECRAFT_SERVER_ADDRESS` |
| Эндпоинт | `GET /api/site/server-status`, публичный, throttle 60/мин |

Если `host` не задан — `configured: false`, `online: false`, без сетевых попыток. Список конкретных игроков онлайн сервер не отдаёт: только числа `players`/`max`.

### 17.16. Домен Bridge: заявки на виды бриджа

Раз уж название обманывает, разберём его полностью — это вторая по объёму часть «реального времени вокруг игры», хотя сокетов здесь нет вообще.

**Каталог.** `bridge_techniques` (`key` unique, `label`, `description`, `sort_order`, `is_active`) и `bridge_technique_variants` (`technique_id`, `key`, `label`, `is_special`, `sort_order`, `is_active`, `unique(technique_id,key)`). Каталог живёт в базе, а не в коде: виды добавляются куратором без выкладки. `key` генерируется из названия через `Str::slug($label,'_')` с суффиксом при коллизии.

**Заявка.** `user_bridge_techniques`: один вид — одна запись на игрока (`unique(user_id, technique_id)`), повторная подача перезаписывает её. Статусы: `declared` → `confirmed` | `rejected`. Оценки тестера: `stability`, `speed`, `difficulty` (0–100 каждая), `score` («владение видом», 0–10), `review_notes`, `reviewed_by`, `reviewed_at`, `variants` (json).

```
// backend/app/Domains/Bridge/Services/BridgeService.php, фрагмент declare()
if ($uploadId !== null) {
    $upload = BridgeVideoUpload::query()
        ->where('uuid', $uploadId)
        ->where('user_id', $user->id)   // чужая загрузка не подойдёт
        ->first();

    if (! $upload) {
        $this->abortUnprocessable('Загрузка видео не найдена.');
    }

    if (! $upload->isCompleted()) {
        $this->abortUnprocessable('Видео ещё не догружено до конца.');
    }

    $videoPath = $upload->path;
}
```

Подтверждённую заявку нельзя ни переподать, ни отозвать: `withdraw()` и `declare()` падают 422. Отклонённую можно переподать — прошлые оценки при этом обнуляются, потому что видео новое. Набор подвидов нормализуется: остаются только id, принадлежащие этому виду (`normalizeVariants`), дубли убираются — чужой подвид из другого вида в заявку не попадёт.

**Видео.** Внешних хостингов больше нет: ролик грузится частями в закрытое хранилище `local`.

| Параметр | Значение | Где |
|---|---|---|
| Размер части | 4 МБ (`CHUNK_SIZE = 4194304`), переопределяется `BRIDGE_CHUNK_SIZE` | `BridgeVideoUpload`, `config/bridge.php` |
| Максимальный ролик | 300 МБ (`MAX_SIZE = 314572800`), `BRIDGE_MAX_VIDEO_SIZE` | там же |
| Разрешённые MIME | mp4, webm, quicktime, x-matroska, x-msvideo | `BridgeVideoService::ALLOWED_MIME` |
| Части | `bridge-chunks/{uuid}/{index}`, список принятых — в `received_chunks` (json) | повторная часть перезаписывается, это не ломает докачку |
| Сборка | потоком, `stream_copy_to_stream`, затем **сверка размера** с заявленным; при расхождении файл удаляется и 422 | `complete()` |

| | | |
|---|---|---|
| Отдача | только по подписанной ссылке `URL::temporarySignedRoute('bridge.video', now()->addHours(3))`, TTL `BRIDGE_VIDEO_TTL_HOURS` | `BridgeSubmissionResource` |
| Удаление | после проверки тестером, при отзыве заявки, при удалении админом | `deleteFor()` |
| Уборка | `pruneStale()` — незавершённые загрузки старше суток | вызывается вне HTTP (команда/крон в коде не зарегистрирован) |

```
// backend/app/Domains/Bridge/Services/BridgeVideoService.php, 161–172
for ($index = 0; $index < $upload->total_chunks; $index++) {
    $chunkPath = $this->disk()->path(self::CHUNK_DIR . '/' . $upload->uuid . '/' . $index);

    if (! is_file($chunkPath)) {
        $this->abortUnprocessable("Часть {$index} потерялась, загрузи ролик заново.");
    }

    $in = fopen($chunkPath, 'rb');

    stream_copy_to_stream($in, $out);
    fclose($in);
}
```

**Проверка.** Очередь — все заявки в статусе `declared`, свежие сверху по `updated_at`. При подтверждении обязательны все четыре оценки (`required_if:confirm,true`), тестер может поправить набор подвидов. При отклонении достаточно причины. В обоих случаях игроку уходит `BridgeTechniqueReviewedNotification`, а ролик удаляется. История проверок — `confirmed` + `rejected`, лимит 200, по `reviewed_at`.

**Звания и топ.** `bridge_ranks` (`key`, `label`, `color`, `sort_order`, `is_active`) — звание выдаёт тестер вручную, это не тир по очкам; хранится в `users.bridge_rank_id` + `bridge_rank_by` + `bridge_rank_at`, кэшируется навсегда (`bridge_ranks:by_id`, сброс по `saved`/`deleted`). Место в бридже считается запросом, без денормализации: `score = aspects_total * 1000 + confirmed_count`, где `aspects_total` — сумма трёх аспектов по подтверждённым видам, а `1000` — вес, заведомо больший любого реального числа видов. Раньше было наоборот, и игрок с 169 очками стоял выше игрока с 210. Из выборок исключаются `admin` и `moderator` (`scopeExcludeStaff`), тестеры и кураторы участвуют наравне с игроками. Очки рейтинга пересчитывает `RatingService` (в админских правках — сразу после изменения).

Флаг `users.bridge_primary` означает «бридж — основной режим игрока»: тогда на сайте вместо тира показывается звание бриджера. Его пересчитывает `RatingService` вместе с очками.

Замечание по маршрутам: метод `BridgeController::user()` (публичный профиль бриджера с подтверждёнными видами, сводкой и местом) в `routes/api.php` **не зарегистрирован** — публичного эндпоинта этого профиля в коде нет.

### 17.17. Схема таблиц и связей

```
users ──┬─< conversation_participants >── conversations ──< messages ──< message_reads
        │                                       │   │              ├──< message_attachments
        │                                       │   └─ pinned_message_id ──> messages
        │                                       └─ clan_id ──> clans
        ├─< message_reactions >── messages
        ├─< notifications (morphs notifiable)
```

```
        ├─< activities (actor_id, nullOnDelete)
        ├─< bridge_rank_id ──> bridge_ranks
        └─< user_bridge_techniques >── bridge_techniques ──< bridge_technique_variants
                                       bridge_video_uploads (uuid, user_id)
```

### conversations

| Поле | Тип | Примечание |
|---|---|---|
| `id` | bigint PK | |
| `type` | string(20), default `direct` | было enum, расширено миграцией |
| `title` | string(64), nullable | только для `group` |
| `user_min_id`, `user_max_id` | FK users, nullable, cascade | `unique(min,max)` для `direct` |
| `clan_id`, `author_id` | FK clans/users, nullable, cascade | `unique(clan_id,author_id)` для `clan_message` |
| `owner_id` | FK users, nullable, `nullOnDelete` | создатель беседы |
| `pinned_message_id` | FK messages, nullable, `nullOnDelete` | закреп |
| `last_message_at` | timestamp, индекс | сортировка списка |

### conversation_participants

| Поле | Тип | Примечание |
|---|---|---|
| `conversation_id`, `user_id` | FK, cascade | `unique(conversation_id,user_id)` |
| `last_read_at` | timestamp, nullable | индекс `(user_id,last_read_at)` |

### messages

| Поле | Тип | Примечание |
|---|---|---|
| `conversation_id`, `user_id` | FK, cascade | |
| `reply_to_id` | bigint, nullable | без FK-констрейнта |
| `forwarded_from_user_id` | FK users, nullable, `nullOnDelete` | |
| `kind` | string(16), default `text` | `text` / `gift` / `system` |
| `body` | text, nullable | nullable с миграции вложений |
| `meta` | json, nullable | сумма подарка, `action`/`target_ids` у system |
| `edited_at` | timestamp, nullable | |
| индексы | `(conversation_id,created_at)`, `(conversation_id,id)`, `(user_id,created_at)` | |

**message_reads**: `message_id`, `user_id` (оба cascade), `read_at` (`useCurrent`), `unique(message_id,user_id)`, индекс `(user_id,read_at)`; `$timestamps = false`.

**message_attachments**: `message_id` (nullable — файл ещё не отправлен, cascade), `user_id` (cascade), `original_name`, `path`, `mime`, `size`, `is_image`, `is_voice`, `duration` (smallint, nullable), индексы `message_id` и `(user_id,message_id)`.

**message_reactions**: `message_id`, `user_id`, `emoji` (string 16), `unique(message_id,user_id,emoji)`.

**notifications**: `id` uuid PK, `type`, `morphs('notifiable')`, `data` text, `read_at`, timestamps.

**activities**: `type` (string 40, индекс), `actor_id` (nullable, `nullOnDelete`), `payload` json, индексы `type` и `created_at`.

**bridge_techniques**: `key` unique, `label`, `description`, `sort_order`, `is_active`.

**bridge_technique_variants**: `technique_id` (cascade), `key`, `label`, `is_special`, `sort_order`, `is_active`, `unique(technique_id,key)`.

**user_bridge_techniques**: `user_id`, `technique_id`, `status` enum(`declared`,`confirmed`,`rejected`), `video_url` (512, legacy), `video_path` (512), `variants` json, четыре оценки, `review_notes`, `reviewed_by` (`nullOnDelete`), `reviewed_at`, `unique(user_id,technique_id)`, индекс `(status,created_at)`.

**bridge_video_uploads**: `uuid` unique, `user_id` (cascade), `original_name`, `mime` (96), `size`, `total_chunks`, `received_chunks` json, `path` (512), `status` enum(`pending`,`completed`), индекс `(user_id,status)`.

**bridge_ranks**: `key` unique, `label`, `color` (default `#8b5cf6`), `sort_order`, `is_active`.

### 17.18. Все эндпоинты

**Чат** (все под `auth:sanctum`):

| Метод | Путь | Действие | Права | Лимит |
|---|---|---|---|---|
| GET | `/api/chat/conversations` | список диалогов + непрочитанные + онлайн участников | свои | — |
| GET | `/api/chat/conversations/{conversation}` | история, курсоры `before_id`/`after_id`, `limit`, `mark_read` | участник (`view`) | limit ≤ 100 |
| GET | `/api/chat/search` | поиск игроков и кланов для нового диалога | авторизован | — |
| POST | `/api/chat/start-direct` | найти/создать личный диалог | авторизован | — |
| POST | `/api/chat/start-clan` | обращение к клану (автор + лидер/офицер) | авторизован | — |
| GET | `/api/chat/unread-count` | общий счётчик непрочитанных | свои | — |
| GET | `/api/chat/conversations/{conversation}/search` | поиск по тексту сообщений диалога | участник | — |
| POST | `/api/chat/conversations/{conversation}/messages` | отправить сообщение | участник (`send`) | нет |
| PUT | `/api/chat/messages/{message}` | правка своего сообщения (≤ 24 ч) | автор | нет |
| DELETE | `/api/chat/messages/{message}` | удалить сообщение | автор или | нет |

| | | | | |
|---|---|---|---|---|
| | | | модератор | |
| POST | `/api/chat/messages/{message}/forward` | переслать в другой диалог | участник источника + `send` в цель | нет |
| POST | `/api/chat/messages/{message}/reactions` | поставить/снять реакцию | участник диалога | 60/мин |
| POST | `/api/chat/messages/{message}/read` | отметить одно сообщение | участник | нет |
| POST | `/api/chat/conversations/{conversation}/read` | отметить диалог | участник | нет |
| POST | `/api/chat/conversations/{conversation}/typing` | «печатает» | участник | нет |
| POST | `/api/chat/conversations/{conversation}/pin` | закрепить/открепить | участник | нет |
| POST | `/api/chat/conversations/{conversation}/gift` | подарить монеты (`1…1 000 000`, подпись ≤ 200) | участник личного диалога | 10/мин |
| POST | `/api/chat/attachments` | загрузить файл(ы) «про запас» | авторизован | 40/мин |
| GET | `/api/chat/attachments/{attachment}/download` | скачать вложение | участник диалога | — |
| POST | `/api/chat/voice` | загрузить голосовое (≤ 120 с) | авторизован | 30/мин |
| POST | `/api/chat/groups` | создать беседу (≤ 20 человек, только друзья) | авторизован | 10/мин |
| POST | `/api/chat/groups/{conversation}/members` | добавить друзей | участник | 30/мин |
| DELETE | `/api/chat/groups/{conversation}/members/{user}` | исключить / выйти | создатель или сам | — |
| PUT | `/api/chat/groups/{conversation}` | переименовать | создатель | 20/мин |
| GET | `/api/chat/presence` | кто онлайн (все или `user_ids`) | авторизован | — |
| POST | `/api/chat/presence/online` | отметка входа / пульс | авторизован | — |
| POST | `/api/chat/presence/offline` | отметка выхода | авторизован | — |

**Уведомления и лента:**

| Метод | Путь | Действие | Права |
|---|---|---|---|

| | | | |
|---|---|---|---|
| GET | `/api/notifications` | список по 20 + `unread_count` | свои |
| POST | `/api/notifications/read-all` | прочитать всё | свои |
| POST | `/api/notifications/{id}/read` | прочитать одно | свои |
| GET | `/api/activity` | лента, `limit` 1–50, курсор `before` | публичный |

**Мост к Minecraft-серверу** (заголовок `X-Minecraft-Server-Key`):

| Метод | Путь | Действие | Ответ |
|---|---|---|---|
| POST | `/api/minecraft/nickname/resolve` | заявлен ли ник и чей | `{claimed, nickname, account_username}` |
| POST | `/api/minecraft/auth/login` | вход по нику и паролю сайта | `{ok, reason, attempts_left, retry_after, player?}`, 401/429 |
| POST | `/api/minecraft/profile` | данные игрока для игры | `{claimed, player}` или 404 |
| POST | `/api/minecraft/admin/lookup` | на какой аккаунт заявлен ник | `{claimed, nickname, account_username, account_id, uuid, claimed_at}` |
| POST | `/api/minecraft/admin/unlink` | освободить ник | `{ok, nickname}` |

**Привязка со стороны игрока** (под `auth:sanctum`):

| Метод | Путь | Действие |
|---|---|---|
| GET | `/api/players/me/minecraft/link` | состояние заявки |
| POST | `/api/players/me/minecraft/link` | заявить ник (3–16, `[A-Za-z0-9_]`) |
| DELETE | `/api/players/me/minecraft/link` | снять заявку |
| POST | `/api/players/me/skin` | свой скин PNG 64×64/64×32, ≤ 64 КБ, `model` = `classic`/`slim` (10/мин) |
| DELETE | `/api/players/me/skin` | вернуть скин из Minecraft (10/мин) |
| GET | `/api/players/{user}/skin` | PNG скина + `X-Skin-Model` (публичный, 60/мин) |
| GET | `/api/site/server-status` | статус сервера по SLP (публичный, 60/мин) |

**Bridge — игрок:**

| Метод | Путь | Действие | Права |
|---|---|---|---|
| GET | `/api/bridge/techniques` | каталог + мои заявки + сводка | авторизован |
| POST | `/api/bridge/techniques` | подать вид (`technique_id`, | авторизован |

| | | | |
|---|---|---|---|
| | | `upload_id`, `variants`) | |
| DELETE | `/api/bridge/submissions/{submission}` | отозвать заявку (кроме подтверждённой) | владелец |
| PUT | `/api/bridge/submissions/{submission}/variants` | включить/выключить подвиды | владелец или тестер |
| POST | `/api/bridge/uploads` | начать загрузку видео | авторизован |
| POST | `/api/bridge/uploads/{upload:uuid}/chunks` | принять часть | владелец загрузки |
| POST | `/api/bridge/uploads/{upload:uuid}/complete` | собрать файл | владелец загрузки |
| GET | `/api/bridge/videos/{submission}` | отдать видео | только подписанная ссылка (`signed`) |

**Bridge — тестер** (`role:bridge_tester,bridge_curator,admin`):

| Метод | Путь | Действие |
|---|---|---|
| GET | `/api/bridge-review/submissions` | очередь `declared` |
| POST | `/api/bridge-review/submissions/{submission}` | подтвердить (все оценки) или отклонить |
| GET | `/api/bridge-review/history` | история проверок (200) |
| GET | `/api/bridge-review/ranks` | звания |
| GET | `/api/bridge-review/players` | бриджеры с подтверждёнными видами (100) |
| POST | `/api/bridge-review/players/{user}/rank` | выдать/снять звание |

**Bridge — куратор** (`role:bridge_curator,admin`): `GET|POST /api/bridge-curator/techniques`, `PUT|DELETE /api/bridge-curator/techniques/{technique}`, `POST /api/bridge-curator/techniques/{technique}/variants`, `PUT|DELETE /api/bridge-curator/variants/{variant}`.

**Bridge — админ** (`role:admin`): `GET /api/admin/users/{user}/bridge`, `PUT /api/admin/bridge/submissions/{submission}`, `DELETE /api/admin/bridge/submissions/{submission}`. Правки пишутся в `ModerationLog` (`bridge.submission_updated`, `bridge.submission_deleted`) и пересчитывают рейтинг.

### 17.19. Чего в коде нет

- Каналов на диалог/беседу (`conversation.{id}`) — вся адресация через личные каналы.
- Подписки гостей на что-либо, кроме публичного `activity`; авторизация канала у гостя всегда провалится.
- Серверного throttle на отправку сообщений, правку и удаление.
- Rate limiting самого Reverb (по умолчанию `false`), ограничения числа соединений (`max_connections` пустой).
- Очереди для рассылки — все события `ShouldBroadcastNow`, синхронно в запросе.

- Диспатча `NotificationCreated` в рабочем коде — только тесты; realtime-уведомления фактически не доставляются.
- Клиентских событий (`whisper`): «печатает» идёт HTTP-запросом, хотя Reverb разрешает `accept_client_events_from: members`.
- Множественных Minecraft-серверов, отдельных ключей, ротации секрета, подписи тела запроса.
- Синхронизации игроков между сайтом и сервером в любую сторону; списка игроков онлайн (только числа из SLP).
- Периодической проверки, что заявленный ник всё ещё принадлежит игроку.
- E2E-шифрования, редактирования истории, мягкого удаления сообщений, восстановления удалённых.
- Задачи очистки файлов вложений с диска (строки удаляются каскадом, файлы остаются) и зарегистрированного расписания для `BridgeVideoService::pruneStale()`.

# Мои питомцы: полная механика

Питомцы — отдельная подсистема внутри домена `Players`, разложенная по семи классам. Вкладка «Мои питомцы» (`/pets`) собирает всё сразу: сводку по всем экземплярам, арену, экспедиции, питомцедекс и топ сервера.

| Файл | За что отвечает |
|---|---|
| `Support/PetTraits.php` | черты экземпляра: окрас, фон, аксессуар, редкость, номер |
| `Support/PetPerks.php` | перки по уровню |
| `Support/PetBattleEngine.php` | правила раунда боя — без базы и денег |
| `Support/PetLocks.php` | занятость: питомец в бою, в походе или на продаже |
| `Services/PetService.php` | уход, опыт, уровень, переименование |
| `Services/PetHubService.php` | сводка, трюки, питомцедекс, топ |
| `Services/PetArenaService.php` | арена: бои, соперники, рейтинг, награды |
| `Services/PetExpeditionService.php` | экспедиции: зоны, добыча, ключи |
| `Services/PetBulkService.php` | массовые операции с питомцами |

### Где живут питомцы

Питомец — **не отдельная таблица**, а строка в `user_inventory` с заполненным `pet_traits`. Поэтому у питомца общая судьба с остальным инвентарём: он продаётся, дарится и продаётся на аукционе той же строкой.

| Поле `user_inventory` | Что хранит |
|---|---|
| `pet_traits` | json: окрас, фон, аксессуар, редкость |
| `pet_serial` | номер экземпляра среди всех питомцев этого вида |
| `pet_name` | имя, до 20 символов |
| `pet_xp` | опыт от экспедиций и боёв |
| `pet_rating` | рейтинг на арене, старт — 1000 |
| `pet_wins` / `pet_losses` | счёт боёв |

На профиле одновременно гуляют **до трёх** питомцев — список лежит в `users.profile_pets` как json с `inventory_id`. Перковый бонус «Шахтёра» берётся от **лучшего** из гуляющих.

Опыт ухода хранится отдельно — в `pet_interactions`, по строке на действие. Итого опыт питомца = `pet_xp` + сумма действий по весам.

### Виды и стихии

**21 вид** питомцев, каждый привязан к стихии. Стихия определяет умение и ультимейт в бою.

| Стихия | Виды |
|---|---|
| `fire` | дракон, феникс |
| `water` | аксолотль, пингвин, осьминог, медуза |

| | |
|---|---|
| `nature` | лягушка, пчела, гриб, заяц, слизень |
| `shadow` | призрак, сова, робот, ворона |
| `beast` | кот, пёс, лис, панда, капибара, единорог |

Ворона — особый случай: она не выпадает и в питомцедекс не входит, её выдаёт только администрация.

### Черты и номера

Каждый питомец при получении «чеканится»: получает номер экземпляра и три случайные черты. Двух одинаковых не бывает.

| Черта | Сколько вариантов | Самый обычный | Самый редкий |
|---|---|---|---|
| Окрас (`model`) | 35 | Классический — 3000 | Звёздный — 20 |
| Фон (`backdrop`) | 27 | Без фона — 3500 | Край — 35 |
| Аксессуар (`accessory`) | 33 | Без аксессуара — 4000 | Ледяная корона — 15 |

Веса заданы **в долях процента**, то есть вероятность = вес / сумма весов. По этим трём чертам выводится **редкость** экземпляра: `common`, `rare`, `epic`, `legendary`, `mythic`. Редкость напрямую влияет на бой — см. таблицу бонусов ниже.

Номер (`pet_serial`) чеканится как `max(serial) + 1` среди всех питомцев этого вида, поэтому у каждого «Листёнка #17» номер уникален навсегда — он сохраняется при подарке и продаже.

### Уровни и опыт

Максимальный уровень — **30**. Кривая опыта:

```
// backend/app/Domains/Players/Services/PetService.php
/** Опыт на уровень растёт: 0, 5, 15, 30, 50… (5 · n(n−1)/2). */
private static function xpFor(int $level): int
{
    return intdiv(5 * $level * ($level - 1), 2);
}
```

| Уровень | Нужно опыта всего | Уровень | Нужно опыта всего |
|---|---|---|---|
| 2 | 5 | 15 | 525 |
| 3 | 15 | 20 | 950 |
| 4 | 30 | 22 | 1155 |
| 5 | 50 | 25 | 1500 |
| 6 | 75 | 26 | 1625 |
| 10 | 225 | 30 | 2175 |

Опыт даётся за:

| Источник | Сколько |
|---|---|
| Погладить | 1 |

| | |
|---|---|
| Покормить | 2 |
| Поиграть | 2 |
| Победа на арене | 4 |
| Поражение на арене | 1 |
| Экспедиция | 3 / 8 / 15 / 24 по зоне |

### Уход

Три действия, и каждое — **раз в день от каждого игрока**, включая хозяина. То есть чужой питомец может набрать опыт от нескольких visitors за сутки.

| Действие | Опыт | Что даёт |
|---|---|---|
| `pet` — погладить | 1 | настроение |
| `feed` — покормить | 2 | снимает голод |
| `play` — поиграть | 2 | настроение |

Настроение вычисляется, а не хранится:

```
// backend/app/Domains/Players/Services/PetService.php
$mood = match (true) {
    ! $lastFed || now()->diffInHours($lastFed, true) >= self::HUNGRY_AFTER_HOURS => 'hungry',
    $lastAny && now()->diffInHours($lastAny, true) < self::HAPPY_WITHIN_HOURS => 'happy',
    default => 'calm',
};
```

| Состояние | Условие |
|---|---|
| `hungry` — голодный | не кормили **24 часа** или вообще ни разу |
| `happy` — радостный | любое действие за последние **6 часов** |
| `calm` — спокойный | всё остальное |

Имя — до 20 символов, теги вырезаются.

### Перки

Открываются навсегда и копятся: питомец 25 уровня даёт всё, что открылось на 5, 10, 15 и 20.

| Уровень | Перк | Что даёт |
|---|---|---|
| 5 | `expedition_coins_10` | +10% монет из экспедиций |
| 10 | `miner_coins_10` | +10% монет в «Шахтёре», пока питомец гуляет по профилю |
| 15 | `key_chance` | +5% к шансу ключа в экспедиции, открыт Незер |
| 20 | `arena` | +1 бой на арене в день и +25% монет за победу |
| 25 | `expedition_coins_25` | +25% монет из экспедиций вместо 10% |
| 30 | `legend` | +20% монет в «Шахтёре» и 10% |

| | | |
|---|---|---|
| | | шанс двойной добычи в экспедиции |

Экспедиции и арена берут перки **того питомца**, который пошёл в поход или дерётся. «Шахтёр» — лучшего из гуляющих по профилю.

### Арена: сколько боёв и что за победу

| Параметр | Значение |
|---|---|
| Бесплатных боёв в день | **5** |
| Бонус за питомца 20 уровня | **+1** бой в день |
| Дополнительный бой | **300**, потом **400**, **500**… — +100 за каждый следующий за день |
| Раундов в бою максимум | **30** |
| Награда за победу | `round((40 + уровень × 3) × 3 × 1.25)` — множитель 3, а с перком 20 уровня ещё +25% |
| Опыт за победу / поражение | 4 / 1 |
| Рейтинг | Эло, старт 1000, K = 32 |

Награда по уровням: 1 → 129, 10 → 210, 20 → 300 (с перком 375), 30 → 390 (с перком 487).

Рейтинг считается по классическому Эло:

```
// backend/app/Domains/Players/Services/PetArenaService.php
$expected = 1 / (1 + 10 ** (($foeRating - $myRating) / 400));
$change = (int) round(self::K * (($won ? 1 : 0) - $expected));

// Защищался не хозяин, а компьютер — соперник меняется вдвое слабее
$foeChange = -intdiv($change, 2);
```

Соперников компьютер подбирает в окне **±250** рейтинга, до шести штук, и добирает ближайшими, если в окне пусто.

**Недельный сброс.** Раз в неделю рейтинг подтягивается к базе вдвое — `1000 + (rating − 1000) / 2`. Тогда же выплачиваются призы недели лучшим питомцам с уведомлением и ссылкой на арену.

### Как устроен бой

Бой пошаговый: игрок выбирает приём, соперник — компьютер с характером своей стихии. Пять приёмов:

| Приём | Что делает | Ограничение |
|---|---|---|
| `attack` | обычный удар; **третья подряд** — серия с силой 1.3 (1.45 на саванне) | — |
| `defend` | лечит 6% максимума (12% на океане) и даёт +15 ярости; урон по тебе ×0.4 (ультимейт ×0.5) | — |
| `skill` | умение стихии | перезарядка **3** раунда |
| `counter` | ловит атаку или умение: урон по | перезарядка **2** раунда |

| | | |
|---|---|---|
| | тебе ×0.4/×0.7, отражается 70%/40% | |
| `ultimate` | ультимейт стихии | нужна ярость **100** |

**Ярость** копится от полученного урона (`урон / максимум × 100`), плюс 6 за каждый свой удар, плюс 15 за защиту. Ультимейт обнуляет её.

**Характеры компьютера** — по стихии:

| Стихия | Характер | Умение | Контратака | Защита |
|---|---|---|---|---|
| `fire` | Вспыльчивый | 80% | 10% | 20% |
| `water` | Осторожный | 60% | 20% | 50% |
| `nature` | Терпеливый | 65% | 25% | 35% |
| `shadow` | Хитрый | 60% | 35% | 25% |
| `beast` | Безрассудный | 75% | 10% | 10% |

Компьютер **читает игрока**: если тот дважды подряд атаковал — растёт шанс контратаки; если у игрока полная ярость — с шансом 50% компьютер закрывается. Замысел выбирается заранее, поэтому мощные приёмы видны: «готовит Пламя!».

### Умения и ультимейты стихий

| Стихия | Умение | Сила | Особенность | Ультимейт | Сила | Особенность |
|---|---|---|---|---|---|---|
| `fire` | Пламя | 1.7 | 35% поджог | Метеор | 2.6 | сквозь защиту + гарантированный поджог |
| `water` | Волна | 1.35 | лечит 12%, смывает все статусы | Цунами | 1.8 | урон, лечение 25%, промокание |
| `nature` | Шипы | 1.3 | шипы возвращают 40% урона (55% в лесу) | Корни | 1.6 | оглушение — соперник пропустит ход |
| `shadow` | Тень | 1.45 | сквозь защиту, 50% страх | Пожиратель душ | 2.0 | сквозь защиту, крадёт 60% урона здоровьем |
| `beast` | Рывок | 1.5 | 30% крит, кровотечение при крите | Ярость стаи | 0.9 × 3 | три удара, каждый может крить |

### Статусы

| Статус | Держится | Эффект |
|---|---|---|
| Поджог (`burn`) | 3 раунда | −4% максимума здоровья за раунд |
| Кровотечение (`bleed`) | 3 раунда | −3% максимума за раунд |
| Страх (`fear`) | 2 раунда | атака слабее на 25% |
| Промок (`wet`) | 2 раунда | скорость ×0.6, то есть ходишь вторым |
| Оглушён (`stun`) | 1 раунд | пропуск хода |

Оглушение и промокание начинают работать со следующего раунда — конец текущего их не съедает.

### Арены

Арена выбирается случайно в каждом бою. Своя стихия на арене бьёт на **20% сильнее**.

| Арена | Стихия | Особенность |
|---|---|---|
| Колизей | — | ярость копится в **1.5 раза** быстрее |
| Вулкан | `fire` | каждый **4-й** раунд извержение бьёт обоих на 4% максимума |
| Океан глубин | `water` | защита лечит вдвое сильнее — 12% вместо 6% |
| Древний лес | `nature` | шипы острее (55% вместо 40%), с кустов падают ягоды |
| Склеп | `shadow` | крит чаще: **+7%** к шансу |
| Саванна | `beast` | серия из трёх атак бьёт на 1.45 вместо 1.3 |

### Формула урона

```
// backend/app/Domains/Players/Support/PetBattleEngine.php
$atk = $attacker['atk'] * (($attacker['fx']['fear'] ?? 0) > 0 ? 0.75 : 1);
$damage = $atk * $power * random_int(90, 110) / 100;
$damage -= $pierce ? 0 : $target['def'] * 0.6;
```

Дальше множители по очереди:

| Условие | Множитель |
|---|---|
| Стихия бьёт по слабой (`fire` > `nature` > `water` > `fire`, `shadow` > `beast`) | **×1.25** |
| Стихия бьёт по сильной | **×0.8** |
| Стихия совпадает с ареной | **×1.2** |
| Крит (база 8%, в склепе 15%, у «Рывка» 30%) | **×1.5** |

Итог всегда не меньше 1.

### Характеристики питомца

```
// backend/app/Domains/Players/Services/PetArenaService.php
$hp = 60 + $level * 6 + $r * 8;

'atk' => (int) round(10 + $level * 1.2 + $r * 1.5),
'def' => (int) round(5 + $level * 0.6 + $r),
'spd' => (int) round(10 + $level * 0.4 + ($element === 'shadow' ? 2 : 0) + ($element === 'beast' ?
1 : 0)),
```

`$r` — бонус редкости: `common` 0, `rare` 1, `epic` 2, `legendary` 3, `mythic` 4.

В бою здоровье умножается на **1.6** — чтобы бой успевал развернуться. Питомец 1 уровня: 66 HP, атака 11, защита 6, скорость 10. Тридцатого: 240 HP, атака 46, защита 23, скорость 22.

### Случайные события

С шансом **12%** за раунд происходит что-то само:

| Событие | Эффект |
|---|---|
| Зритель кидает яблоко | случайный боец лечится на 8% |
| Молния | оба теряют 5% максимума |
| Кот-судья подмигивает | отстающий получает **+25** ярости |
| С куста падают ягоды (только лес) | случайный боец лечится на 10% |

### Экспедиции

Питомец уходит в поход и приносит монеты, опыт и иногда ключи. Добыча **выкатывается при отправке** — переоткрытием её не перебросить, — но видна только по возвращении.

| Зона | Часов | Мин. уровень | Монеты | Опыт | Ключ «Искателя» | Ключ питомцев |
|---|---|---|---|---|---|---|
| Цветочный луг | 1 | 1 | 120–200 | 3 | 4% | 0.5% |
| Тёмный лес | 4 | 3 | 550–800 | 8 | 12% | 2% |
| Кристальные пещеры | 8 | 8 | 1200–1700 | 15 | 25% | 5% |
| Огненные земли | 12 | 15 | 2000–2800 | 24 | 40% | 10% |

Монеты растут на **2% за каждый уровень** питомца. Перки добавляют +10% (с 5 уровня) или +25% (с 25-го), с 30 уровня — ещё 10% шанс двойной добычи. Перк 15 уровня даёт +5% к шансу ключа.

Кроме награды питомец приносит «историю» — случайную фразу о походе, на награду она не влияет.

### Трюки

Открываются уровнем, всего восемь:

| Трюк | Уровень | Название |
|---|---|---|

| | | |
|---|---|---|
| `spin` | 3 | Волчок |
| `flip` | 6 | Сальто |
| `dance` | 10 | Танец |
| `wave` | 14 | Привет всем |
| `hearts` | 18 | Сердечки |
| `sparkle` | 22 | Звездопад |
| `rainbow` | 26 | Радуга |
| `firework` | 30 | Фейерверк |

### Питомцедекс

Коллекция собирается по четырём разделам: виды, окрасы, фоны, аксессуары. За вехи дают монеты и ключи — **11 вех**:

| Веха | Что нужно | Награда |
|---|---|---|
| 3 вида | 3 вида | 1000 |
| 6 видов | 6 видов | 2500 |
| 10 видов | 10 видов | 5000 |
| 15 видов | 15 видов | 10 000 |
| Все виды | все виды | 25 000 + 3 ключа «Сундука питомцев» |
| 10 окрасов | 10 окрасов | 2000 |
| Все окрасы | все 35 | 15 000 |
| 10 фонов | 10 фонов | 2000 |
| Все фоны | все 27 | 12 000 |
| 10 аксессуаров | 10 | 2000 |
| Все аксессуары | все 33 | 15 000 |

Награда берётся один раз: повторная попытка отбивается уникальным индексом.

### Занятость

Питомец не может быть в двух местах сразу. `PetLocks::reason()` не даёт начать бой или отправить в поход того, кто уже занят, и не даёт продать или подарить занятого питомца. Причина показывается текстом: «питомец в экспедиции», «питомец на арене».

### Скрещивание

Два свободных питомца дают потомка за **300 ApexCoin**. Вид — случайный

родитель, черты (окрас, фон, аксессуар) — случайно от каждого родителя.

Кулдаун — сутки на каждого родителя. Запись ведётся в таблице

pet_breedings, потомок чеканится как обычный экземпляр UserInventory (серийный номер по виду). Эндпоинт: POST /api/pets/breeding с parent1 и parent2. В интерфейсе — вкладка «Скрещивание» в «Моих питомцах».

### Досрочный возврат с экспедиции

POST /api/pets/expeditions/{id}/recall возвращает питомца из похода раньше срока, **без награды** — даже если до конца оставалась минута. Питомец сразу освобождается (снимается блокировка PetLocks), отзыв фиксируется отдельным полем recalled_at, чтобы интерфейс различал «забрал добычу» и «вернул».

### Крафт экипировки из материалов

Мобы и боссы подземелий роняют материал своего тира (коготь, чешуя, магма, иней, пустота): с босса всегда два, с моба — шанс 30%. Три одинаковых материала дают случайный предмет **того же тира** (слот и редкость случайны). Склад — таблица dungeon_materials, сервис DungeonCraftService, эндпоинты GET/POST /api/dungeons/craft.

### Подземелья

Подземелье — режим для отряда с **пошаговым боем**. Питомцы идут по залам, этаж закрывает босс, со следующего этажа мобы сильнее.

| Файл | За что отвечает |
|---|---|
| `Support/Dungeons.php` | залы, мобы, боссы, их особенности, экипировка |
| `Support/DungeonBattle.php` | правила раунда: приёмы, стихии, связки, статусы |
| `Services/DungeonService.php` | поход: зал, добыча, отдых, отступление |
| `Services/PetGearService.php` | экипировка: надеть, снять, бонусы |
| `Controllers/Api/DungeonController.php` | HTTP: `/api/dungeons/**` |
| `frontend/src/data/shop/pixelSprites.js` | 12 мобов и 3 босса, пиксель-арт |

Движок боя, как и `PetBattleEngine` на арене, **не трогает базу и деньги**: на вход характеристики, на выход состояние и лог. Поэтому его можно проверять без поднятия БД, а состояние целиком лежит в `dungeon_runs.state` и переживает перезагрузку.

#### Устройство этажа

Этаж — `per_floor` обычных мобов, последний зал этажа — босс. Босс закрывает этаж, после него привал и спуск ниже.

| Ключ | Название | Питомцы от | Этажей | Мобов на | Всего залов | Мобов в |
|---|---|---|---|---|---|---|

| | | | | этаже | | наборе |
|---|---|---|---|---|---|---|
| `catacombs` | Крысиные катакомбы | 1 | 2 | 2 | 6 | 12 |
| `sunken` | Затонувшая шахта | 8 | 3 | 2 | 9 | 12 |
| `forge` | Огненная кузня | 16 | 3 | 2 | 9 | 12 |
| `crypt` | Мёрзлый склеп | 24 | 4 | 2 | 12 | 12 |
| `void` | Пустотный разлом | 28 | 4 | 2 | 12 | 12 |

На каждое подземелье — **12 разных мобов** и по боссу на этаж (всего 60 мобов и 16 боссов — **у каждого босса свой спрайт**). Спрайтов мобов в общем наборе 24, поэтому часть мобов

### Поражение и отступление

Если отряд пал, поход заканчивается **штрафом**: списывается 80 / 200 / 450 / 800 / 1400 ApexCoin по ступени подземелья. Больше, чем есть на счету, не спишут — баланс в минус не уходит. Всё, что успели собрать до поражения, остаётся у игрока.

**Отступление бесплатно.** Игрок ничего не теряет, кроме самой попытки: заход уже потрачен, но монеты, экипировка и ключи остаются. Поэтому отступить перед сильным боссом — нормальная тактика, а не проигрыш.

Списание идёт через `CoinService::penalty()`: он не увеличивает `apex_coins_spent`, иначе штраф попал бы в «заработано» в кошельке.

делит рисунок: внутри одного подземелья один и тот же спрайт встречается не больше двух раз, а имя, стихия, особенность и характеристики у всех свои.

**Походы.** Три бесплатных в день, дальше докупаются за ApexCoin: первый стоит **800**, каждый следующий за день — на **400** дороже (800, 1200, 1600…). Счётчик покупок — `dungeon_extra_runs` по паре (игрок, день), поэтому лимит обнуляется сам. Деньги списываются через `CoinService::debit`, как за бои на арене.

Мощь мобов и боссов растёт на **18% за этаж**. Боссы по этажам свои: Крысиный король → Король падали; Утопленник-старейшина → Хозяин затопленных штолен → Утонувший шахтёр-гигант; Кузнец преисподней →

Мастер горна → Владыка лавы.

#### Пошаговый раунд

Каждый раунд игрок выбирает приём **каждому живому питомцу**, потом все ходят по скорости — и отряд, и враг. Четыре приёма:

| Приём | Что делает | Условие |
|---|---|---|
| Атака | обычный удар | всегда |
| Умение | приём стихии | перезарядка **3** раунда |
| Защита | входящий урон ×0.5, +6% здоровья, +15 ярости | всегда |
| Ультимейт | командный приём стихии | ярость **100** |

**Ярость** копится: +8 за свой удар, +15 за защиту и доля от полученного урона. Ультимейт её обнуляет. Враг бьёт **слабейшего** питомца, а на последних 30% здоровья с шансом 30% заслоняется.

Урон: `атака × сила приёма − защита × 0.5`, дальше стихии (сильная +25%, слабая −20%), связка, защита и крит (10%, ×1.5), потом разброс ±10%.

#### Стихии и командные способности

Стихия питомца — не украшение: она решает, чем он полезен отряду. Вода лечит отряд, тень крадёт здоровье, зверь баффает атаку.

| Стихия | Умение | Ультимейт | Что даёт отряду |
|---|---|---|---|
| Огонь | Пламя (1.7, 35% поджог) | Метеор (2.6, поджог) | урон и поджог |
| Вода | Волна (1.3, лечит слабейшего на 12%) | Цунами (0.6 + **лечит отряд на 18%** и снимает статусы) | лечение |
| Природа | Шипы (1.4, 40% оглушение) | Корни (1.6, оглушение, +6% отряду) | контроль |
| Тень | Тень (1.45, 50% страх) | Пожиратель душ (2.0, **делит украденное на отряд**) | вампиризм |
| Зверь | Рывок (1.5, +20% крит) | Ярость стаи (1.2, **+25% атаки отряду на 2 раунда**) | бафф |

Оглушение — пропуск хода, страх — атака слабее на 25%, поджог — 4% максимума в раунд три раунда.

#### Подбор отряда

В сводке у каждого питомца есть `power` — условная сила (`атака × 2 + защита × 1.5 + скорость + здоровье / 4`), а у подземелья

— `elements`: какие стихии в нём встречаются (собирается из мобов и боссов). `power` считается **вместе с надетой экипировкой** (`PetGearService::bonus()` входит в атаку, защиту, скорость и здоровье), поэтому подбор видит вещи.

Кнопка «Подобрать отряд» перебирает пары и тройки в браузере: наибольшая сумма `power` плюс **400** сверху, если связка контрит одну из стихий подземелья. Пятнадцать питомцев — это 455 вариантов, считается мгновенно. Выбранный отряд запоминается в `localStorage` и подставляется при следующем заходе.

#### Связки стихий

Пара стихий в отряде даёт **супер-контру** против третьей: урон по такой цели **×1.35**. Пара считается, если оба питомца живы.

| Связка | Пара | Контрит |
|---|---|---|
| Пар | вода + огонь | природу |
| Пепел | огонь + природа | тень |
| Тлен | природа + тень | зверя |
| Стая | тень + зверь | воду |
| Прилив | зверь + вода | огонь |
| Роса | вода + природа | огонь |
| Гарь | огонь + тень | природу |
| Охота | огонь + зверь | воду |
| Глубина | вода + тень | тень |
| Чаща | природа + зверь | зверя |

Всего **десять пар** — ровно по две контры на каждую стихию, поэтому отряд всегда можно собрать осмысленно.

В отряде из трёх питомцев пар сразу три, и активной становится та, что контрит **текущего врага** (`DungeonBattle::combo()` получает стихию противника и ищет совпадение по `counters`). Если ни одна не контрит — берётся первая подходящая по порядку в `COMBOS`.

Отсюда тактика: отряд собирают **под стихию врага**, а не «кого сильнее». Связка и уязвимость врага показаны в интерфейсе до боя.

#### Добыча

| Параметр | Ступень 1 | Ступень 2 | Ступень 3 |
|---|---|---|---|
| Монет за моба | 60 × этаж | 140 × этаж | 260 × этаж |
| Монет за босса | 400 × этаж | 900 × этаж | 1600 × этаж |

| | | | |
|---|---|---|---|
| Экипировка с моба | 14% | 22% | 30% |
| Ключ «Искателя» | 4% | 12% | 25% |

Добыча хранится в двух видах: `loot.gear` — только id строк инвентаря, а `loot.items` — то же со названием, спрайтом, слотом и редкостью. Второе нужно окну итогов: по одному id нарисовать нечего.

**Финальное состояние приходит в ответе на последний ход**, а не из сводки: закрытый поход в `overview()` не попадает (`run: null`), и если клиент перечитает сводку после вайпа, результат потеряется. Поэтому `DungeonView.vue` перехватывает ответ мутации, у которого `status` не `active`, и держит его в окне итогов, пока игрок не закроет.

С босса экипировка **всегда**, с шансом **25%** — верхней ступени. Опыт: **4** за моба, **20** за босса. Монеты идут с ключом `dungeon:{run}:{этаж}:{зал}`, поэтому повтор не заплатит дважды.

**Здоровье переносится между залами**: потрёпанный отряд рискует не дойти до босса. Привал после босса возвращает 15% максимума.

#### Пиксель-арт

Мобы и боссы — в формате косметики магазина: сетка символов и палитра в `pixelSprites.js`, рендер в SVG через `PixelSprite`. Мобы 12 px в ширину, боссы 14×13 — на десяти не помещаются корона, молот и клыки.

### Экипировка питомца

Экипировка — предметы магазина типа `pet_gear`, но выключенные (`is_active = false`): в витрине их нет, только найти в подземелье. Так они остаются строками инвентаря, и с ними работают продажа, подарок и аукцион.

Сидер `DungeonGearSeeder` создаёт **30 предметов**: три слота, пять ступеней, по два варианта на каждую пару «ступень × слот». Варианты различаются уклоном: «точный» против «тяжёлого», «лёгкий» против «кованого», «быстрый» против «живучего».

**Ступень 1 — катакомбы.** Оружие: Крысиный клык (+4 атаки), Обглоданная кость (+3 атаки, +1 скорости). Доспех: Крысиная шкура (+3 защиты, +12 здоровья), Рваная тряпка (+2 защиты, +8 здоровья, +1 скорости). Талисман: Крысиная косточка (+1 атаки, +6 здоровья, +1 скорости), Свечной огарок

(+5 здоровья, +2 скорости).

**Ступень 2 — шахта.** Оружие: Шахтёрская кирка (+9 атаки), Ржавый тесак (+8 атаки, +1 скорости). Доспех: Ржавая кольчуга (+7 защиты, +26 здоровья), Мокрый ватник (+5 защиты, +22 здоровья, +1 скорости). Талисман: Жемчужина глубин (+2/+2/+14/+2), Медный компас (+8 здоровья, +3 скорости).

**Ступень 3 — кузня.** Оружие: Кузнечный молот (+16 атаки, +2 защиты), Раскалённый клинок (+18 атаки, +6 здоровья, −1 скорости). Доспех: Лавовый панцирь (+13 защиты, +46 здоровья, −1 скорости), Пепельный плащ (+9 защиты, +34 здоровья, +1 скорости). Талисман: Тлеющий уголь (+4/+3/+22/+3), Огненный глаз (+6 атаки, +2 скорости).

**Ступень 4 — склеп.** Оружие: Ледяной клинок (+30 атаки, +1 скорости), Костяной молот (+34 атаки, +4 защиты, −2 скорости). Доспех: Мёрзлый доспех (+24 защиты, +90 здоровья, −1 скорости), Плащ инея (+18 защиты, +70 здоровья, +2 скорости). Талисман: Осколок льда (+8 атаки, +6 защиты, +44 здоровья, +5 скорости), Глаз вьюги (+10 атаки, +20 здоровья, +6 скорости).

**Ступень 5 — разлом.** Оружие: Клинок бездны (+50 атаки, +2 скорости), Коса жнеца (+58 атаки, +40 здоровья, −3 скорости). Доспех: Пустотная бронь (+40 защиты, +160 здоровья, −2 скорости), Плащ тени (+30 защиты, +120 здоровья, +4 скорости). Талисман: Око пустоты (+14 атаки, +10 защиты, +80 здоровья, +8 скорости), Кристалл разлома (+18 атаки, +50 здоровья, +10 скорости).

**Что падает.** `Dungeons::gearForStage()` отдаёт предметы своей ступени и одной предыдущей: в катакомбах — только первой, в разломе — четвёртой и пятой. Поэтому в глубоких подземельях не попадается «Крысиный клык».

Надевают экипировку во вкладке **«Мои питомцы» → «Экипировка»** (`components/pets/PetGear.vue`): три слота выбранного питомца и свободные предметы. На экране подземелья этого нет намеренно — надевание это уход за питомцем, а не часть похода. Эндпоинты общие: `POST /api/dungeons/gear/{pet}/equip` и `.../unequip/{slot}`.

#### Редкость и характеристики

Значения в `GEAR` — это **база** обычного предмета, а редкость их растягивает: один и тот же «Кузнечный молот» может выпасть обычным и легендарным. Редкость и числа лежат в `user_inventory.gear_stats`, то есть

**у каждого экземпляра свои**; metadata предмета магазина остаётся базой.

| Редкость | Множитель |
|---|---|
| Обычный | ×1.0 |
| Редкий | ×1.35 |
| Эпический | ×1.8 |
| Легендарный | ×2.4 |

Шансы зависят от ступени подземелья:

| Ступень | Обычный | Редкий | Эпический | Легендарный |
|---|---|---|---|---|
| 1 (катакомбы) | 70% | 25% | 5% | — |
| 2 (шахта) | 45% | 40% | 15% | — |
| 3 (кузня) | 25% | 45% | 25% | 5% |

С босса редкость бросается **дважды** и берётся лучший результат — `Dungeons::rollRarity($stage, boss: true)`. Скорость множится как есть, поэтому у «Лавового панциря» она так и остаётся −1 (или −2 у легендарного).

Что надето — json-поле `pet_gear` на строке питомца. Каждый экземпляр лежит **отдельной строкой**, поэтому два одинаковых меча можно надеть на двух питомцев; один и тот же предмет дважды надеть нельзя.

Бонусы прибавляются в `PetArenaService::fighter()`, а здоровье потом умножает движок арены (1.6) — доспех на +46 даёт в бою +74.

**На арене видно снаряжение обоих.** `PetArenaService::present()` кладёт в ответ `gear.me` и `gear.foe` — предметы по слотам (`slot`, `name`, `sprite`) и суммарный `bonus`. Интерфейс рисует три мини-слота в HUD каждого бойца, пустые — пунктиром, а в подсказке перечислены названия. Иначе было бы непонятно, почему питомец с виду слабее, а бьёт сильнее.

### Что в коде отсутствует

- **Отдельной таблицы питомцев** — нет, всё в `user_inventory`.
- **Ручного выбора соперника на арене** — соперников даёт компьютер.
- **Боёв между живыми игроками** — соперником всегда управляет компьютер.
- **Продажи питомца за монеты напрямую** — только через аукцион или магазин.
- **Понижения уровня** — опыт только растёт.
- **Перков выше 30 уровня** — 30 максимальный.
- **Уведомлений о добыче экспедиции** — забрать нужно вручную.
- **PvP-подземелий** — отряд всегда воюет против мобов и боссов, а не против игроков.

- **Общей экипировки на аккаунт** — предмет надевается на конкретного питомца, а не на всех.
- **Ремонта и износа экипировки** — предмет не портится и не теряется при гибели отряда.

# Часть V. Разработка

## 21. Локальный запуск

### 21.1. Что должно работать одновременно

| Порт | Что | Команда |
|---|---|---|
| 8000 | Laravel API | `php artisan serve` |
| 5173 | Vite dev-сервер | `npm run dev` (в `frontend/`) |
| 8082 | Reverb (вебсокеты) | `php artisan reverb:start --port=8082` |

Фронтенд в разработке проксирует `/api`, `/sanctum` и `/broadcasting` на `127.0.0.1:8000`, поэтому CORS не нужен.

### 21.2. База в тестах и в разработке

- **Тесты** используют SQLite `:memory:` — каждый тест получает чистую базу.
- **Разработка и прод** — MySQL.

**Это различие — источник отдельного класса багов.** SQLite мягче MySQL и прощает то, чего MySQL не прощает. Главный случай: `ORDER BY` по несуществующей или неоднозначной колонке. SQLite молча вернёт что-нибудь, MySQL бросит исключение — и тест зелёный, а на реальной базе запрос падает. Поэтому в проекте есть отдельный страж-тест `RankingOrderGuardTest`, который ловит такие места.

# 22. Тесты

## 22.1. Масштаб

| Часть | Файлов | Тестов | Команда |
|---|---|---|---|
| Бэкенд | 127 | ~1080 (5136 проверок) | `php artisan test` |
| Фронтенд | 25 | 437 (1 пропущен) | `npm test` |

Почти **полторы тысячи тестов** — это то, что позволяет рефакторить турниры и кошелёк, не боясь, что что-то тихо отвалится.

## 22.2. Полезные режимы

```
php artisan test                          # весь бэкенд
php artisan test --stop-on-failure     # остановиться на первой ошибке
php artisan test --filter=Tournament   # только совпадающие по имени
php artisan test tests/Feature/ClanTest.php   # один файл

npm test                                  # фронтенд один раз
npm run test:watch                        # в фоне
npm run test:coverage                     # с покрытием
```

## 22.3. Что именно проверяется на фронтенде

Два вида тестов, и второй важнее, чем кажется:

1. **Маршруты** — что у всех маршрутов `component` существует, `meta` корректна, 404 стоит последним.
2. **Монтирование** — smoke-тесты, которые поднимают все компоненты и экраны. Они не проверяют поведение, но ловят самое частое: сломанный импорт, синтаксическую ошибку, обращение к несуществующему свойству при инициализации. На 218 `.vue` файлах это дешёвая страховка.

# 23. Как добавить новую механику

## 23.1. Новый домен на бэкенде

```
1. backend/app/Domains/<Имя>/
   ├── Controllers/Api/<Имя>Controller.php
   ├── Models/<Имя>.php
   ├── Services/<Имя>Service.php
   ├── Requests/<Имя>Request.php
   ├── Resources/<Имя>Resource.php
   └── Notifications/  (если нужны)
2. Миграция:   php artisan make:migration create_<имя>s_table
3. Маршруты:   backend/routes/api.php, внутри auth:sanctum
4. Тест:       backend/tests/Feature/<Имя>Test.php
```

## 23.2. Новый экран на фронтенде

```
1. frontend/src/views/<раздел>/<Имя>View.vue  + <Имя>View.css
2. frontend/src/services/<раздел>/<имя>.js    (обёртки api.get/api.post)
3. frontend/src/router/index.js               (ленивый импорт + meta)
4. frontend/src/test/                         (smoke-тест монтирования)
```

Маршрут добавлять **перед** `pathMatch(.*)*`, иначе 404 перехватит его.

## 23.3. Чек-лист перед коммитом

```
[ ] php artisan test      — зелёный
[ ] npm test              — зелёный
[ ] npm run build         — собирается
[ ] в коммит не попали: .idea/, plugins/, *.jar, .env
```

# 24. Грабли, проверенные на практике

Собрано из реальных поломок этого проекта. Каждая стоила времени.

### 24.1. `@import` внутри `<style scoped>` даёт глобальные стили

```
<style scoped>
@import "./ClanCard.css";   /* ← scoped здесь НЕ работает */
</style>
```

Атрибут `data-v-*` к селекторам из импортированного файла не добавляется. Следствие: одинаковые классы (`.modal`, `.list`, `.item`) в разных компонентах конфликтуют. **Лечение:** уникальные префиксы у крупных блоков и модалок.

### 24.2. Точные маршруты объявляются раньше параметрических

```
Route::get('/shop/priority-candidates', ...);   // ← сначала точный
Route::get('/shop/{shopItem}', ...);            // ← потом параметрический
```

Laravel берёт первый совпавший маршрут. Наоборот — и `/shop/priority-candidates` уйдёт в `{shopItem}`.

### 24.3. SQLite прощает то, что MySQL не прощает

Любой запрос с `ORDER BY` по вычисляемому полю нужно проверять на MySQL. В тестах он пройдёт, на проде — 500.

### 24.4. Vue Router: регулярка в строке требует двойного экранирования

```
path: '/support/:id(\\d+)?'   // правильно: в JS-строке нужен \\
path: '/support/:id(\d+)?'    // ошибка: это литерал «d»
```

Именно поэтому в маршруте темы форума регулярки нет вообще — при обновлении страницы маршрут должен совпасть.

### 24.5. Reverb падает — сайт жив, но чат мёртв

`error.log` с `Connection refused ... /app/` — это не 503 сайта. Из браузера это выглядит одинаково: страницы открываются, а сообщения не приходят. Отличить легко по консоли — `window.Echo.connector.pusher.connection.state` не станет `connected`. На клиенте для этого есть собственный сторож (глава 12), который пересоздаёт соединение, но если сервер вебсокетов не отвечает вообще, он будет пробовать снова и снова, пока тот не вернётся.

# Приложения

## Приложение A. Схема базы данных

**91 таблица, 130 миграций.** Ниже — карта по доменам. Полные определения полей смотрите в `backend/database/migrations/`, здесь — назначение и связи.

### A.1. Пользователи и доступ

| Таблица | Назначение |
|---|---|
| `users` | аккаунт: логин, почта, пароль, `role`, флаги бана, реферальный код, `minecraft_nickname`, `profile_mode` |
| `player_aspects` | общие игровые показатели, связь с аспектами |
| `player_aspects_pvp` | рейтинг и статистика в PvP |
| `player_aspects_bedwars` | рейтинг и статистика в BedWars |
| `email_verifications` | коды подтверждения почты и одноразовые токены регистрации |
| `password_reset_tokens` | коды сброса пароля |
| `personal_access_tokens` | токены Sanctum (для не-SPA клиентов) |
| `sessions` | серверные сессии |

Особенность: **два аспекта игрока** — PvP и BedWars — живут в отдельных таблицах, а не в колонках `users`. Игрок имеет два независимых рейтинга.

### A.2. Профиль

| Таблица | Назначение |
|---|---|
| `profile_visits` | кто заходил в профиль, счётчик посещений |
| `profile_respects` | «респекты» — отметки уважения к игроку |
| `respect_week_rewards` | награды за попадание в недельный топ респектов |
| `profile_guestbook_entries` | гостевая книга профиля |
| `profile_recommendations` | рекомендации игроку |
| `profile_poll_votes` | голоса в опросах профиля |
| `user_guides` | пройденные шаги «путеводителя» для новичков |

### A.3. Тиры и бридж

| Таблица | Назначение |
|---|---|
| `tier_tests` | заявка/проведение тир-теста: игрок, тестер, статус, результат, тир |
| `tier_test_appeals` | обжалования результатов теста |

| | |
|---|---|
| bridge_ranks | ранги бриджа (справочник) |
| bridge_techniques | техники бриджа (справочник) |
| bridge_technique_variants | варианты техник |
| user_bridge_techniques | какие техники освоил игрок |
| bridge_video_upload | чанковая загрузка видео для проверки |

## A.4. Кланы (14 таблиц — самый крупный блок)

| Таблица | Назначение |
|---|---|
| clans | клан: имя, тег, цвет, описание, лидер, уровень |
| clan_members | участники и их роли (leader, officer, member) |
| clan_applications | заявки на вступление |
| clan_events | события клана |
| clan_event_comments | обсуждение событий |
| clan_forum_topics / clan_forum_replies | внутренний форум клана |
| clan_inventory | предметы клана |
| clan_resources | ресурсы клана |
| clan_treasury_transactions | движение средств казны |
| clan_stats_logs | история изменения силы и статистики |
| clan_wars / clan_war_participants | войны кланов и их участники |
| clan_checkins | отметки активности участников |

## A.5. Турниры

| Таблица | Назначение |
|---|---|
| tournaments | турнир: название, статус, формат, даты, приз |
| tournament_participants | зарегистрированные участники |
| tournament_matches | матчи сетки: раунд, пары, счёт, признак BYE |
| tournament_wins | история побед |

## A.6. Экономика

| Таблица | Назначение |
|---|---|
| coin_transactions | **леджер** — каждая операция с ApexCoin: сумма, источник, идемпотентный ключ |
| shop_items | товары магазина |
| shop_settings | настройки магазина и наград (перекрывают config/apex.php) |
| user_inventory | купленное и полученное: предметы, значки, |

| | |
|---|---|
| | косметика |
| `case_openings` | история открытий кейсов |
| `loot_case_keys` | ключи от кейсов |
| `promo_codes` / `promo_redemptions` | промокоды и их погашения |

## A.7. Питомцы

| Таблица | Назначение |
|---|---|
| `pet_dex` | справочник питомцев |
| `pet_dex_rewards` | награды за коллекцию |
| `pet_battles` | бои питомцев |
| `pet_expeditions` | экспедиции (пассивная добыча) |
| `pet_interactions` | взаимодействия с питомцем |
| `pet_arena_extra_fights` | дополнительные бои на арене |
| `pet_auctions`, `pet_auction_bids`, `pet_auction_messages` | аукцион питомцев |

## A.8. Общение

| Таблица | Назначение |
|---|---|
| `conversations` | беседа (личная или групповая) |
| `conversation_participants` | участники беседы |
| `messages` | сообщения |
| `message_attachments` | вложения |
| `message_reactions` | реакции на сообщения |
| `message_reads` | прочтения (для непрочитанных) |
| `forum_categories` | разделы форума |
| `forum_topics` | темы |
| `forum_replies` | ответы |
| `forum_likes` | лайки |
| `forum_attachments` | вложения форума |
| `friendships` | заявки и дружба |
| `notifications` | уведомления |
| `activities` | лента событий |

## A.9. Достижения, сезоны, игры

| Таблица | Назначение |
|---|---|
| `achievements` | справочник достижений |
| `user_achievements` | полученные достижения |
| `seasons` | сезоны |

| | |
|---|---|
| `season_results` | итоги сезона по игрокам |
| `miner_profiles`, `miner_runs`, `miner_upgrades` | мини-игра «Шахтёр» |

### A.10. Служебное

| Таблица | Назначение |
|---|---|
| `reports` | жалобы |
| `moderation_logs` | журнал действий модерации |
| `support_tickets` / `support_messages` | обращения в поддержку и переписка |
| `news` | новости |
| `site_settings` | настройки сайта, объявления |
| `changelog_entries`, `roadmap_items` | «Что нового» и роадмап |

### A.11. Инфраструктура Laravel

`cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `sessions`, `personal_access_tokens`.

### A.12. Правила, которые видны в схеме

Три вещи, заметные по структуре таблиц:

1. **Леджер вместо одного поля баланса.** `coin_transactions` — это журнал, а не «сумма». Баланс восстанавливается из истории, и любая операция имеет идемпотентный ключ, поэтому двойное начисление невозможно даже при повторе запроса.
2. **Настройки в двух местах.** `config/apex.php` — значения по умолчанию, `shop_settings` — то, что задано из админки. База главнее конфига.
3. **Клан — это отдельная мини-система.** Внутри клана есть свой форум, инвентарь, казна, ресурсы и войны — 14 таблиц, самая крупная часть схемы.

# Приложение Б. Остальные домены

Небольшие области, которые не попали в крупные главы.

## Б.1. Search — глобальный поиск

Два файла: `SearchController` и `GlobalSearchService`.

> `GET /api/search`

Поиск идёт по нескольким сущностям сразу (игроки, кланы, темы форума) и возвращает сгруппированный результат — фронтенд показывает его выпадающим списком в шапке.

Отдельно от глобального поиска существуют **частные поиски внутри разделов**, и это разные эндпоинты с разной логикой:

| Эндпоинт | Где |
|---|---|
| `GET /api/forum/search` | поиск по темам форума |
| `GET /api/chat/search` | поиск по диалогам |
| `GET /api/chat/conversations/{id}/search` | поиск по сообщениям конкретного диалога |
| `GET /api/admin/players/search` | поиск игрока в админке (для жалоб) |

Разделение осмысленно: у каждого поиска свои права доступа и своя область видимости. Глобальный поиск не должен показывать приватные сообщения, а поиск в админке работает по игрокам, которых обычный поиск не отдаст.

## Б.2. Og — картинки для превью ссылок

Два файла: `OgController` и `OgImageRenderer`.

Когда ссылку на сайт кидают в Discord или Telegram, мессенджер запрашивает Open Graph-картинку. Домен генерирует её **на сервере**: рисует изображение с ником игрока, тиром и статистикой, чтобы превью выглядело как карточка, а не как серая заглушка.

Это же отвечает за `og:image` для профилей и кланов — поэтому ссылка на игрока в чате разворачивается в картинку с его тиром.

## Б.3. Project — «Что нового» и роадмап

| Файл | Назначение |
|---|---|
| `ProjectController` | публичное чтение |
| `AdminProjectController` | управление из админки |
| `Models/ChangelogEntry` | запись в «Что нового» |
| `Models/RoadmapItem` | пункт роадмапа |
| `Services/ProjectService` | логика |

Публично:

```
GET /api/project/changelog
GET /api/project/changelog/{slug}
GET /api/project/roadmap
```

Админка (полный CRUD):

```
GET|POST        /api/admin/project/changelog
PUT|DELETE      /api/admin/project/changelog/{entry}
GET|POST        /api/admin/project/roadmap
PUT|DELETE      /api/admin/project/roadmap/{item}
```

Отдельный `slug` у записи changelog нужен для прямых ссылок: `/changelog/название-записи` на фронтенде.

### Б.4. Users — пользователь и роли

Самый «низкоуровневый» домен: здесь живёт модель `User` и всё, что касается аккаунта целиком.

| Файл | Назначение |
|---|---|
| `Models/User.php` | модель пользователя |
| `Concerns/HasRoles.php` | трейт проверки ролей |
| `Controllers/Api/Admin/UserController.php` | управление пользователями из админки |
| `Controllers/Api/Admin/RoleController.php` | выдача ролей |
| `Requests/Admin/BanUserRequest.php` | валидация бана |
| `Requests/Admin/UpdateRoleRequest.php` | валидация смены роли |
| `Requests/Admin/VerifyUserRequest.php` | ручное подтверждение почты |
| `Notifications/FriendRequest.php` | уведомление о заявке в друзья |

**Трейт `HasRoles`** — то, чем пользуются middleware и политики. Он отвечает на вопросы вида «есть ли у пользователя одна из этих ролей», и именно он позволяет маршрутам писать `role:moderator,admin`.

Админские возможности по пользователю: сменить роль, забанить с причиной и сроком, снять бан, вручную подтвердить почту, отвязать Minecraft-ник.

### Б.5. Core — главная и настройки сайта

Самый маленький домен (4 файла), но именно он собирает данные для главной страницы и отдаёт настройки сайта наружу.

Через него на фронтенд попадают:

- объявление сайта (`site_settings`), которое показывается на всех страницах;
- данные для главной: топ игроков, статистика проекта;
- публичные флаги функций — чтобы интерфейс знал, что скрыть.

**Связка с Control.** Флаги функций хранятся в `site_settings`, меняются через домен `Control`, а применяются в двух местах: на бэкенде — middleware `EnsureFeatureEnabled`, на фронтенде — скрытием разделов. Если функцию выключили, а интерфейс о ней не знает, пользователь получит ошибку вместо аккуратного скрытия — поэтому флаги и отдаются через `Core`.

# Приложение В. Шпаргалка

## В.1. Команды разработки

```
# --- Бэкенд ---
cd backend
php artisan serve                       # API на :8000
php artisan migrate                     # применить миграции
php artisan migrate:fresh --seed        # пересоздать базу с сидерами
php artisan test                        # все тесты
php artisan test --stop-on-failure      # остановиться на первой ошибке
php artisan test --filter=Clan          # только совпадающие по имени
php artisan test tests/Feature/ClanTest.php

# --- Пересчёты (ручные, автоматически не запускаются) ---
php artisan apex:rating-recalc                    # рейтинг игроков + сила кланов
php artisan apex:season-close --name="Сезон 2" --force
php artisan apex:achievements --dry-run           # что было бы выдано
php artisan apex:achievements --force             # выдать и доплатить

# --- Фронтенд ---
cd frontend
npm run dev                             # Vite на :5173
npm run build                           # сборка в dist/
npm test                                # тесты
npm run test:coverage

# --- Вебсокеты ---
php artisan reverb:start --port=8082    # локально
```

## В.2. Диагностика: что смотреть при симптоме

| Симптом | Первое подозрение | Проверка |
|---|---|---|
| 419 на любом POST | нет CSRF-cookie | запрос к `/sanctum/csrf-cookie`, заголовок `X-XSRF-TOKEN` |
| 423 на действии | функция выключена в «Пульте» | `GET /api/site` → `control.features` |
| 200 на сайте, но чат молчит | Reverb не отвечает | состояние `window.Echo` в консоли, `/broadcasting/auth` |
| Ошибка приходит без текста | не разобрали `errors` | `api.js` бросает `{status, message, errors, data}` |
| Тесты зелёные, а на MySQL падает | SQLite простил ошибку | проверить `ORDER BY` на реальной базе |
| Правило не срабатывает | забыт вызов пересчёта | `apex:rating-recalc`, `apex:achievements` |

## В.4. Ключевые числа

| Показатель | Значение |
|---|---|
| Доменов бэкенда | 26 |
| Таблиц | 91 |
| Миграций | 130 |
| Моделей | 64 |
| Маршрутов в `api.php` | ~387 |
| Тестов бэкенда | ~1080 |

| | |
|---|---|
| Экранов | 45 |
| Компонентов | 218 `.vue` + 119 `.css` |
| Тестов фронтенда | 437 |
| Тиров | E, D, C, B, A, S, S+ |
| Ролей | user, tester, bridge_tester, bridge_curator, moderator, admin |

## B.5. Где что искать

| **Нужно** | **Файл** |
|---|---|
| Добавить маршрут | `backend/routes/api.php` |
| Поменять награды в ApexCoin | `backend/config/apex.php` или админка «Магазин → Награды» |
| Поменять формулу рейтинга | `backend/app/Domains/Players/Services/RatingService.php` |
| Поменять пороги тиров | `TierTestService::TIER_THRESHOLDS` и `User::recalcTierFromAspects()` |
| Добавить экран | `frontend/src/views/` + `frontend/src/router/index.js` |
| Поменять стили | `.css` рядом с компонентом |
| Подключить канал вебсокетов | `backend/routes/channels.php` + `frontend/src/echo.js` |
| Настроить веса рейтинга | админка «Настройки сайта → Очки рейтинга» |
| Посмотреть журнал действий | админка «Модерация → Журнал» |

# Приложение Г. Все числа проекта

Сводка всех чисел из кода: наград, лимитов, шансов и порогов. Источник указан рядом, чтобы было видно, где значение живёт и можно ли его поменять без выкладки.

Обозначения: **конфиг** — `config/apex.php`, перекрывается из админки; **константа** — зашито в класс; **валидация** — правило формы; **БД** — значение по умолчанию в миграции.

### Г.1. Валюта и награды

| Что | Значение | Источник |
|---|---|---|
| Код валюты | `APEX` | конфиг |
| Название | ApexCoin | конфиг |
| Символ | `A` | конфиг |
| Награда за тир-тест: E / D / C / B / A / S / S+ | **60 / 90 / 140 / 220 / 350 / 600 / 1000** | конфиг |
| Бонус за первую заявку на тест | **150** | конфиг |
| Награда за достижение | `coin_reward`, иначе **25 + очки × 2** | конфиг |
| Ежедневный вход | **40** | конфиг |
| Пригласившему за друга | **200** | конфиг |
| Приветственный бонус новичку | **50** | конфиг |
| Подарок: минимум | **10** | конфиг |
| Подарок: максимум за раз | **100 000** | конфиг |
| Подарок: дневной лимит | **5 000** | конфиг |
| Подарок: комиссия | **5%** | конфиг |
| Вес приоритетной заявки | **100** | конфиг |
| Срок действия приоритета | бессрочно (`null`) | конфиг |
| Максимум надетых бейджей | **3** | конфиг |
| Стартовые монеты путеводителя | 6000 + 2000 + 2000 | константа |
| Запрет переводов после регистрации | **7 дней** | константа |

**Источники начисления** (включаются и выключаются из админки):

| Источник | По умолчанию |
|---|---|
| `tier_test` — тир-тесты | включён |
| `achievement` — достижения | включён |
| `daily_bonus` — ежедневный вход | включён |
| `admin` — ручные начисления | включён |
| `referral` — приглашения | включён |
| `tournament` — турниры | **выключен**, начислений нет |
| `clan_war` — войны кланов | **выключен**, начислений нет |

Множитель события «×N ApexCoin» действует только на `tier_test`, `achievement`, `daily_bonus` и `referral`. Максимум — **×5**.

### Г.2. Рейтинг и тиры

| Что | Значение | Источник |
|---|---|---|
| Очков за победу в матче турнира | **5** | настройка сайта |
| Очков за участие в завершённой войне | **3** | настройка сайта |
| Очков за победу в войне | **7** | настройка сайта |
| Доля рейтинга участников в силе клана | **0.5** (с округлением вниз) | константа |
| Навыковая часть рейтинга | `ceil((pvp + bedwars + bridge) / 3)` | константа |
| Не участвуют в рейтинге | роли `admin`, `moderator` | константа |

Пороги тиров по проценту владения:

| Тир | Процент |
|---|---|
| `E` | 0–20 |
| `D` | 21–40 |
| `C` | 41–55 |
| `B` | 56–70 |
| `A` | 71–100 |

`S` и `S+` порогом не выдаются: `S` — вручную или турнир уровня A при 71%+, `S+` — вручную или 5 побед в турнирах уровня S при 71%+.

Аспекты: каждый **0–20**, сумма пяти даёт **0–100**. Достижение «мастер аспекта» — **18** по аспекту, «универсал» — все пять **15**+.

### Г.3. Кейсы

| Кейс | Цена | Монеты при выпадении | Пусто | Монеты | Обычный | Редкий | Эпик | Легендарный |
|---|---|---|---|---|---|---|---|---|
| Ежедневный | 0 | 10–60 | 25% | 30% | 30% | 11% | 3.4% | 0.6% |
| Искателя | 750 | 150–600 | 5% | 15% | 25% | 33% | 18% | 4% |
| Королевский | 2500 | 800–2000 | 0% | 10% | 0% | 40% | 35% | 15% |
| Питомцев | 10 000 | — | 0% | 0% | 45% | 33% | 17% | 5% |

| Параметр | Значение |
|---|---|

| | |
|---|---|
| Гарантия эпика | каждые **10** открытий в платном кейсе |
| Пачки | **1, 3, 5, 10** |
| Компенсация за повтор | **15%** цены предмета |
| Бесплатный кейс | **1 раз в календарный день** |

## Г.4. Магазин

| **Что** | **Значение** |
|---|---|
| Продажа обратно (куплено самому) | **50%** от уплаченной цены |
| Продажа обратно (из кейса или подарок) | **15%** цены |
| Цена набора при частичном владении | пропорционально недостающему |
| Питомцев на профиле | **3** |
| Бейджей | **3** |

## Г.5. Аукцион питомцев

| **Параметр** | **Значение** |
|---|---|
| Длительность лота | **12 / 24 / 48** часов |
| Стартовая цена | от **10** |
| Шаг ставки | **5%** от текущей, минимум **10** |
| Активных лотов у игрока | **3** |
| Продление ставкой | последние **120** секунд → +**120** секунд |
| Неустойка за снятие лота со ставками | **10%** текущей ставки, минимум 1 |
| Сообщение в чат лота | до **300** символов |
| Пауза между сообщениями | **3** секунды |

## Г.6. «Шахтёр»

| **Параметр** | **Значение** |
|---|---|
| Поле | **8 × 7** клеток |
| Энергия | **24** удара |
| Перерыв между забегами | **600** секунд (вагонетка уменьшает на **15** за уровень) |
| Комбо | **+0.05** за руду, максимум **×1.5** |
| Лава | **−2** удара |
| Гравий | **−1** удар и сброс комбо |
| Спуск на ярус | **4** удара |
| Сундук: динамит / артефакт | **25% / 15%** |
| Мимик | **35%**, откусывает **30%** добычи забега |
| Гравий: обвал | **1** удар; **35%** — открывает пещеру и соседей |
| Сделки жителя | **25%** (мин 30), **15%** (мин 20), **10%** (мин 15) от добычи |

| | |
|---|---|
| Дубль артефакта | **40 × ярус** монет |
| Артефакт | **+1%** к добыче за каждый разный |
| Полная коллекция артефактов | **+2%** сверху |
| Бонус питомца в «Шахтёре» | **+10%** (уровень 10), **+20%** (уровень 30) |

**Цена блоков:**

| Блок | Монет | Блок | Монет | Блок | Монет |
|---|---|---|---|---|---|
| Грязь | 1 | Лазурит | 36 | Хорус | 70 |
| Камень | 2 | Золото | 54 | Пурпур | 100 |
| Уголь | 9 | Алмаз | 160 | Шалкер | 270 |
| Медь | 14 | Изумруд | 250 | Древние обломки | **400** |
| Железо | 22 | Глубинный сланец | 4 | Незерак | 2 |
| Редстоун | 27 | Аметист | 65 | Кварц | 32 |
| Светокамень | 40 | Незерское золото | 50 | Камень Края | 4 |

Особые блоки стоят **0** и что-то делают: динамит, гравий, лава, сундук, мимик, житель, артефакт, бедрок, обсидиан.

**Улучшения и цены по уровням:**

| Улучшение | Цены | Что даёт |
|---|---|---|
| Кирка | 800 / 2000 / 4500 / 9000 / 16 000 | +1 удар за уровень; 5-й ломает бедрок и обсидиан за 3 удара |
| Удача | 1000 / 2500 / 5000 / 10 000 / 18 000 | +4% редкой руды и +1% двойной добычи за уровень |
| Фонарь | 600 / 1500 / 3500 / 7000 / 12 000 | подсказки: 1 — лава и гравий, 2 — бедрок, 3 — радиус 2, 4 — опасность по ярусу, 5 — сундуки, жители, артефакты |
| Вагонетка | 1000 / 2500 / 5000 / 9000 / 15 000 | −15 секунд перерыва за уровень |
| Динамит | 6000 | заряд на каждый забег |

**Восемь артефактов** (с какого яруса попадаются):

| Артефакт | Ярус |
|---|---|
| Древняя монета | 1 |
| Окаменелость | 1 |
| Ржавый компас | 1 |
| Карта сокровищ | 2 |

| | |
|---|---|
| Сердце моря | 2 |
| Тотем бессмертия | 3 |
| Звезда Незера | 3 |
| Яйцо дракона | 4 |

**Задания дня** (выпадают по три):

| Задание | Цель | Награда |
|---|---|---|
| Добудь 3 алмаза | 3 | 300 |
| Собери комбо из 5 руд | 5 | 250 |
| Спустись на ярус ниже | 1 | 200 |
| Доберись до Незера | 1 | 400 |
| Открой 2 сундука | 2 | 200 |
| Устрой 3 взрыва | 3 | 200 |
| Добудь 1000 монет за день | 1000 | 300 |
| Сделай 4 забега | 4 | 250 |
| Поторгуй с жителем | 1 | 200 |
| Найди артефакт | 1 | 300 |

**Призы недели:** 1 место — **6000** и королевский сундук, 2 — **4000** и сундук искателя, 3 — **2000** и сундук искателя. Выдача по понедельникам.

**Золотая лихорадка: 3 часа** в сутки, одинаковые для всех; редкая руда весит **×2**.

## Г.7. Кланы

| Параметр | Значение |
|---|---|
| Название | 3–32 символа, уникальное |
| Тег | 2–8 символов, уникальный |
| Описание | до 1000 |
| Лимит участников | **30** + **2** за каждый уровень сверх первого |
| Вступительный взнос | 0–100 000 |
| Взнос в казну | 1–100 000 |
| Объявление (MOTD) | до 500 |
| Призыв бойцов | до **50** за раз, заметка до 300 |
| Счёт войны | 0–100 каждой стороне |
| Опыт за отметку | **10** |
| Опыт за победу в войне | **100** |
| Порог уровня | `100 × уровень × (уровень − 1)` |
| Скидка в казне | 5% (3 ур.), 10% (5), 15% (7), 20% (10) |

Пороги уровней: 2 → 200, 3 → 600, 4 → 1200, 5 → 2000, 7 → 4200, 10 → 9000 опыта.

## Г.8. Турниры

| Параметр | Значение |
|---|---|
| Формат | только `single_elim` |
| Участников | 2–128 |
| Очков рейтинга за победу | по умолчанию **50** (`win_points`) |
| Размер сетки | ближайшая сверху степень двойки |
| Матчей в сетке | размер − 1 |
| Пропусков (BYE) | размер − число участников |

## Г.9. Общение

### Чат:

| Параметр | Значение |
|---|---|
| Длина сообщения | до **5000** символов |
| Вложений на сообщение | до **10** |
| Размер файла | до **12 МБ** |
| Голосовое | до **120** секунд |
| Беседа: участников | до **20** |
| Беседа: название | 2–48 символов |
| Сообщений на странице | **40** (максимум 100) |
| Диалогов в списке | **50** |
| Реакций | **8** эмодзи |
| Правка сообщения | **24 часа** |

### Форум:

| Параметр | Значение |
|---|---|
| Заголовок темы | 4–200 |
| Текст темы | 4–30 000 |
| Ответ | до 20 000 |
| Вложений | до **10** |
| Размер файла | до **10 МБ** |
| Вложенность ответов | до **8** уровней |
| Правка | **24 часа** |
| Просмотр | +1 раз в сутки с одного IP |

**Друзья:** лимита нет, пагинации нет — список отдаётся целиком.

## Г.10. Служебное

| Параметр | Значение |
|---|---|

| | |
|---|---|
| Ник | 3–32 символа, `[a-zA-Z0-9_]` |
| Пароль | от **8** символов |
| Код подтверждения почты | живёт **15 минут** |
| Попыток входа в игру | **5**, потом блокировка на **600** секунд |
| Обращение в поддержку: тема | 3–120 |
| Обращение в поддержку: текст | 5–4000 |
| Открытых обращений | до **3** |
| Причина апелляции | 20–1000 символов |
| Окно апелляции | **14 дней** |
| Комментарий к жалобе | до **1000** |
| Загрузка видео бриджа: часть | **4 МБ** |
| Видео бриджа | до **300 МБ** |
| Ссылка на видео | живёт **3 часа** |
| Радиус подбора соперника на арене | **±250** рейтинга |

## Г.11. Что настраивается из админки

| **Раздел админки** | **Что меняет** |
|---|---|
| Магазин → Награды | награды за тиры, достижения, бонус, реферал, лимиты подарков, включение источников |
| Пульт | объявление, выключение функций сайта, множитель «×N», раздача, промокоды |
| Настройки сайта → Очки рейтинга | веса 5 / 3 / 7 (пересчёт запускается сам) |
| Настройки сайта → Hero, Соцсети, Футер | главная страница и подвал |

## Г.12. Кабинет тестера

| **Параметр** | **Значение** |
|---|---|
| Слот: минимум / максимум | **30 мин / 4 ч** |
| Открывать вперёд | не дальше **30 дней** |
| Оплата за тест | **50 ₽** и **500 ApexCoin** |
| Множители по тирам | S+ **2.0**, S 1.6, A 1.3, B 1.1, C 1.0, D 0.9, E 0.8 |
| Заработок за S+ | **100 ₽** и **1000 ApexCoin** |
| Заработок за E | **40 ₽** и **400 ApexCoin** |
| Выплата | вручную админом, монеты — через `CoinService` |
| Долг сайта | начислено − выплачено − запрошено |

## Г.13. Новые системы

| Система | Числа |
|---|---|
| Возврат с экспедиции | награды нет, питомец свободен сразу |
| Шахта: ярус 5 | «Руины Энда», руда 320 и 520, множитель сундуков ×4 |
| Шахта: выносливость | +2 удара за уровень, 5 уровней |
| Шахта: магнит | +3% монет за руду за уровень, 5 уровней |
| Стихия «Свет» | юникорн; связок 12, каждая стихия контрится ×2 |
| Крафт | 3 материала → предмет того же тира; босс — 2, моб — шанс 30% |
| Скрещивание | 300 ApexCoin, кулдаун 24 ч на родителя |

## Г.14. Подземелья и экипировка

| Параметр | Значение |
|---|---|
| Походов в день | **3** |
| Докупка похода | **800** + 400 × куплено (800, 1200, 1600…) |
| Питомцев в отряде | **2–3** |
| Мобов на этаже | **2**, третьим залом идёт босс |
| Подземелий | **5** |
| Мобов в наборе | **12** на подземелье, всего **60** |
| Боссов | по одному на этаж, всего **16** |
| Мощь мобов за этаж | **+18%** |
| Привал после босса | **15%** максимума |
| Крит | **10%**, урон ×1.5 |
| Стихия сильнее / слабее | **×1.25 / ×0.8** |
| Связка против своей цели | **×1.35** |
| Ярость за удар / защиту | **+8 / +15** |
| Умение: перезарядка | **3** раунда |
| Опыт: моб / босс | **4 / 20** |
| Штраф за поражение | **80 / 200 / 450 / 800 / 1400** по ступени 1–5 |
| Отступление | бесплатно, теряется только поход |

### Подземелья:

| Ключ | Питомцы от | Этажей | Всего залов | Зал босса | Мобов |
|---|---|---|---|---|---|
| `catacombs` | 1 | 2 | 6 | 3, 6 | 12 |
| `sunken` | 8 | 3 | 9 | 3, 6, 9 | 12 |
| `forge` | 16 | 3 | 9 | 3, 6, 9 | 12 |
| `crypt` | 24 | 4 | 12 | 3, 6, 9, 12 | 12 |

| | | | | | |
|---|---|---|---|---|---|
| `void` | 28 | 4 | 12 | 3, 6, 9, 12 | 12 |

**Стихии и способности:**

| Стихия | Умение | Ультимейт |
|---|---|---|
| Огонь | Пламя 1.7, поджог 35% | Метеор 2.6, поджог 100% |
| Вода | Волна 1.3, лечит слабейшего 12% | Цунами 0.6, лечение отряда 18%, снимает статусы |
| Природа | Шипы 1.4, оглушение 40% | Корни 1.6, оглушение 100%, лечение отряда 6% |
| Тень | Тень 1.45, страх 50% | Пожиратель душ 2.0, отряду 40% украденного |
| Зверь | Рывок 1.5, +20% крит | Ярость стаи 1.2, +25% атаки отряду на 2 раунда |

**Особенности новых подземелий:** «Мороз» — −1 бесплатный удар и урон моба ×1.05; «Проклятие» — защита отряда ×0.6 и урон моба ×1.15; «Пустота» — наш урон ×0.8 и здоровье моба ×1.1.

**Связки:**

| Связка | Пара | Контрит |
|---|---|---|
| Пар | вода + огонь | природа |
| Пепел | огонь + природа | тень |
| Тлен | природа + тень | зверь |
| Стая | тень + зверь | вода |
| Прилив | зверь + вода | огонь |
| Роса | вода + природа | огонь |
| Гарь | огонь + тень | природа |
| Охота | огонь + зверь | вода |
| Глубина | вода + тень | тень |
| Чаща | природа + зверь | зверь |

**Статусы:** поджог 3 раунда по 4% максимума, оглушение 1 раунд, страх 2 раунда (−25% атаки).

**Монеты и шансы:**

| Ступень | Моба | Босса | Экипировка | Ключ «Искателя» | Ключ питомцев |
|---|---|---|---|---|---|
| 1 | 60 × этаж | 400 × этаж | 14% | 4% | 0.5% |
| 2 | 140 × этаж | 900 × этаж | 22% | 12% | 2% |
| 3 | 260 × этаж | 1600 × этаж | 30% | 25% | 5% |
| 4 | 420 × этаж | 2600 × этаж | 38% | 35% | 8% |

| | | | | | |
|---|---|---|---|---|---|
| 5 | 640 × этаж | 4000 × этаж | 46% | 45% | 12% |

**Редкость выпавшего предмета:**

| Редкость | Множитель статов | Ступень 1 | Ступень 2 | Ступень 3 |
|---|---|---|---|---|
| Обычный | ×1.0 | 70% | 45% | 25% |
| Редкий | ×1.35 | 25% | 40% | 45% |
| Эпический | ×1.8 | 5% | 15% | 25% |
| Легендарный | ×2.4 | — | — | 5% |

Ступени 4 и 5 (мёрзлый склеп и пустотный разлом): шансы редкости

**15 / 40 / 35 / 10** и **10 / 30 / 40 / 20** процентов соответственно.

С босса редкость бросается дважды, берётся лучшая. Статы экземпляра лежат

в `user_inventory.gear_stats`, база — в `Dungeons::GEAR`.

**Экипировка (30 предметов, базовые значения):**

| Ступень | Слот | Вариант 1 | Вариант 2 |
|---|---|---|---|
| 1 | Оружие | Крысиный клык +4 атк | Обглоданная кость +3 атк, +1 скр |
| 1 | Доспех | Крысиная шкура +3 защ, +12 зр | Рваная тряпка +2 защ, +8 зр, +1 скр |
| 1 | Талисман | Крысиная косточка +1 атк, +6 зр, +1 скр | Свечной огарок +5 зр, +2 скр |
| 2 | Оружие | Шахтёрская кирка +9 атк | Ржавый тесак +8 атк, +1 скр |
| 2 | Доспех | Ржавая кольчуга +7 защ, +26 зр | Мокрый ватник +5 защ, +22 зр, +1 скр |
| 2 | Талисман | Жемчужина глубин +2 атк, +2 защ, +14 зр, +2 скр | Медный компас +8 зр, +3 скр |
| 3 | Оружие | Кузнечный молот +16 атк, +2 защ | Раскалённый клинок +18 атк, +6 зр, −1 скр |
| 3 | Доспех | Лавовый панцирь +13 защ, +46 зр, −1 скр | Пепельный плащ +9 защ, +34 зр, +1 скр |
| 3 | Талисман | Тлеющий уголь +4 атк, +3 защ, +22 зр, +3 скр | Огненный глаз +6 атк, +2 скр |
| 4 | Оружие | Ледяной клинок +30 атк, +1 скр | Костяной молот +34 атк, +4 защ, −2 скр |
| 4 | Доспех | Мёрзлый доспех +24 защ, +90 зр, −1 скр | Плащ инея +18 защ, +70 зр, +2 скр |
| 4 | Талисман | Осколок льда +8 атк, +6 защ, +44 зр, +5 скр | Глаз вьюги +10 атк, +20 зр, +6 скр |
| 5 | Оружие | Клинок бездны +50 атк, | Коса жнеца +58 атк, +40 |

| | | | |
|---|---|---|---|
| | | +2 скр | зр, −3 скр |
| 5 | Доспех | Пустотная бронь +40 защ, +160 зр, −2 скр | Плащ тени +30 защ, +120 зр, +4 скр |
| 5 | Талисман | Око пустоты +14 атк, +10 защ, +80 зр, +8 скр | Кристалл разлома +18 атк, +50 зр, +10 скр |

Спрайты предметов — по тиру на ступень: `gear-weapon`…`gear-weapon-5`,

`gear-armor-1`…`-5`, `gear-trinket`…`-5`.

**Спрайты:** 24 рисунка мобов (делятся на 60 мобов) и **16 уникальных боссов** — ключи `mob-*` и `boss-*`.

Всё остальное — константы и валидация. Их изменение требует выкладки, и они не подстраиваются под игрока.